A PRACTICAL HANDBOOK

Working with Claude
The Eleven Building Blocks

From doing to directing — the operating manual for the eleven capabilities that turn Claude from a chat window into a system you configure.

A standalone, beginner-friendly reference covering every building block end to end. For each one: what it is, when to use it, how to set it up, how to use it, how to maintain it, worked examples, and where the official documentation lives. No prior experience assumed.

Projects · Instructions · Skills · MCP Servers · Connectors · Artifacts · Marketplace · Plugins · Routines · Agents · Workflows

AUDIENCE	PLATFORM	VERSION	SCOPE
All roles — QA, Product/BA, Design, Development	Claude — web, desktop, Cowork, Claude Code CLI & VS Code	1.0 August 2026	Eleven building blocks, 23 chapters

CREATED AND COMPILED BY Mahmood Ahmad

CONTENTS

What's in this guide

GETTING ORIENTED

1 Cover

2 How to Use This Guide

3 The Big Picture

4 Understanding the Core Concepts

THE ELEVEN BUILDING BLOCKS — EACH WITH THE SAME NINE SECTIONS

5 Projects START

8 MCP Servers SETUP

11 Marketplace PREREQ

14 Agents ADV

6 Instructions START

9 Connectors START

12 Plugins ADV

15 Workflows ADV

7 Skills START

10 Artifacts START

13 Routines ADV

PUTTING IT TOGETHER

16 How Everything Fits Together

18 Practical Examples by Role

20 Best Practices

22 Official Documentation & Resources

17 "Which One Should I Use?" Decision Guide

19 Troubleshooting

21 Security & Organizational Considerations

23 Quick Reference Cheat Sheet

§ The structure of every building-block chapter

Chapters 5–15 all follow the same nine-part structure, so you can jump straight to the part you need without reading the chapter end to end.

#	SECTION	ANSWERS THE QUESTION
1	**What is it?**	What is this thing, what problem does it solve, how does it work, and what is it *not*?
2	**When should I use it?**	When to reach for it, when to avoid it, and a "Use this when…" decision panel.
3	**How do I set it up?**	Step-by-step from zero: where to find it, what to click, what to type, recommended settings.
4	**How do I use it?**	Practical end-to-end usage, what to expect back, and how to verify the result.
5	**How do I manage it?**	The full lifecycle: create, view, edit, disable, delete, duplicate, troubleshoot.
6	**Examples**	Concrete role-based scenarios — QA, Product/BA, Design, Development.
7	**Best practices**	What to do, what goes wrong, naming, versioning, security, team considerations.
8	**Official resources**	Verified links to Anthropic's own documentation. Nothing invented.
9	**Visuals**	Screenshots and diagrams placed beside the step they illustrate.

CHAPTER 2

How to Use This Guide

Read this page first. It explains the labels, the conventions, and the four reading paths through the book.

This guide is written for someone who has never used any of these features. It assumes you can open a browser and, for a few chapters, a terminal. Everything else is explained. You do not need to read it in order, and you almost certainly should not read it in one sitting.

2.1 Four ways to read this book

IF YOU ARE…	READ THIS PATH	WHY
Brand new to all of this	Chapters 3 → 4 → 5 → 6 → 10, then stop	Projects, Instructions and Artifacts give you most of the day-one value with no setup and no terminal.
Wiring Claude into your real tools	Chapters 5, 6, 7, 8, 10 — then the worked walkthrough in Chapter 16	These five chain together into one working pipeline: Project → Instructions → an MCP server for your tracker → a Skill for your format → an analysis Artifact. Chapter 16 traces that chain end to end.
Trying to pick the right tool	Chapter 4, then Chapter 16, then Chapter 17	Concepts, then the relationship map, then the decision tables. Chapter 17 is the one to bookmark.
Rolling this out to a team	Chapters 20, 21, then the "Best practices" section of each block you plan to standardise	Naming, versioning, review, admin controls and the security boundary are all collected there.

2.2 The effort labels

Every building-block chapter opens with a single coloured label answering one question: *what do I have to do before this is useful to me?* Not how powerful it is — how much stands between you and using it. Read them as a suggested order, not a ranking.

LABEL	MEANING	CHAPTERS
START HERE	**Usable in minutes, in a browser.** No terminal, no configuration files, nothing to install — at most a switch to turn on. If you read only one group, read this one.	5 Projects, 6 Instructions, 7 Skills, 9 Connectors, 10 Artifacts
NEEDS SETUP	**Real configuration first.** A server to point at, a transport to choose, credentials to supply. Very high value once running, but it is a task rather than a click.	8 MCP Servers
PREREQUISITE	**Not an end in itself.** You need it to make another chapter work. Only Chapter 11 carries this — you cannot install a plugin without the catalogue it lives in.	11 Marketplace
ADVANCED	**Automation and scale.** These assume you already have the earlier blocks working, because they mostly *package or schedule* them. Powerful, and the wrong place to start.	12 Plugins, 13 Routines, 14 Agents, 15 Workflows

THE HONEST READING ORDER

Five START HERE chapters cover the capabilities most people get most of their value from, and none of them require a terminal. The four ADVANCED chapters are the ones that sound most impressive and help least on day one — a plugin that distributes a skill you have not written yet solves nothing. Build something small that works, then package it.

2.3 Surface and plan labels

Not every capability exists everywhere. The second row of labels on each chapter tells you which Claude surface it lives on, and which plans include it.

Surfaces

claude.ai	Claude in a browser at `claude.ai`.
Desktop app	The installable Claude app for Windows and macOS.
Claude Code CLI	The `claude` command in a terminal.
VS Code extension	Claude Code as a panel inside VS Code, Cursor and other forks.
Claude Code on the web	Cloud sessions at `claude.ai/code`.
Cowork	Agentic mode inside claude.ai and the desktop app — no terminal.

Reading the plan label

A label such as `PRO / MAX / TEAM / ENTERPRISE` means the capability requires at least one of those plans. `ALL PLANS` includes Free.

VERIFY BEFORE YOU PROMISE

Plan availability and beta status change frequently, and several capabilities in this guide are explicitly labelled *research preview* or *beta* by Anthropic. Before you build a team process on one, open the official link in that chapter's Section 8 and confirm the current state.

HOW CURRENT IS THIS, AND WHAT TO DO WHEN A SCREEN DOES NOT MATCH

Every click path, setting name and screenshot here was checked against Anthropic's own documentation and help centre while writing, and the screenshots are Anthropic's own. Two things follow from that, and both are worth knowing before you rely on a step:

- **The product moves faster than its documentation.** Where the two disagreed, this guide describes *what is in the product* and says so explicitly — the settings step in Section 10.3 is the clearest example, and it names both layouts so either one works.
- **Settings get reorganised more often than they get renamed.** If a path in this guide does not match your screen, the switch has almost always moved rather than disappeared. Look in the same tab under a different section heading before concluding you do not have the feature — and if you are on Team or Enterprise, check whether an Owner has enabled it at organisation level, because many of these settings are invisible to members until they do.

Anything genuinely ambiguous is flagged in place rather than smoothed over. Corrections are welcome; this is a living document.

2.4 Typographic conventions

CONVENTION	MEANS
`monospace`	Something you type literally, or an exact file name, path, command or setting key.
Bold Sentence Case	A button, menu item, tab or field label exactly as it appears in the interface.
Settings › Connectors	A click path. Follow the items left to right.
Ctrl + Shift + P	A keyboard shortcut. macOS equivalents are given where they differ.
DIAGRAM — NOT A SCREENSHOT	A drawn illustration of an interface or a concept. Labels match the real product, but it is a diagram, not a capture.

2.5 About the screenshots in this guide

Every photographic screenshot in this document was taken from Anthropic's own published documentation and help centre, and each one carries a caption naming its source. Where Anthropic does not publish a screenshot of a particular screen, this guide supplies a labelled interface diagram instead of a substitute image from an unofficial source, and pairs it with the exact click path and the exact terminal output so you can still follow along precisely.

WHY SOME STEPS HAVE A DIAGRAM INSTEAD OF A PHOTO

> Interfaces change. A diagram that names the button correctly stays useful after a redesign, and it cannot mislead you into thinking a screen looked a particular way on a particular day. Where you want a current capture of your own environment, take one and paste it beside the step — this guide is meant to be extended.

2.6 What this guide does not cover

- **Prompt writing as a craft.** There is guidance throughout on how to structure input for each capability, but this is not a prompt-engineering course.
- **The Claude Developer Platform / API.** Several building blocks also exist as API features for engineers building applications. This guide covers the product surfaces your team actually uses day to day; API equivalents are noted in passing with a link, not documented.
- **Other AI platforms.** This edition is Claude-only by design. ChatGPT and Microsoft Copilot have partially overlapping features under different names; mapping them accurately would double the length and date faster. Slide 9 of the deck is the reference for how the three tools divide up.
- **Your organisation's specific policies.** Chapter 21 explains the security boundary and the admin controls that exist. What your organisation permits is a separate document.

CHAPTER 3

The Big Picture

The shift this guide is built around, the three ways to work with Claude, and what each of the eleven blocks actually asks of you.

3.1 The shift: from doing to directing

The premise of this guide is not "AI writes your code" or "AI answers your questions." It is a change in where your effort goes. You stop performing every step of a task by hand and start describing the outcome clearly enough that Claude can perform the steps — while you keep ownership of the result.

ROLE	DOING — EVERY STEP BY HAND	DIRECTING — YOU DESCRIBE, CLAUDE EXECUTES
Development	Write unit tests function by function.	"Here's my service class — write full test coverage in our style."
QA	Map test cases from the spec by hand.	"Here are the acceptance criteria — draft the test suite in our template."
Design	Write annotations for every component.	"Here's the component — write the handoff spec."
Product / BA	Write the sprint retro from memory.	"Here's the ADO board export — generate the retro summary."

A before-and-after worth trying on your own work.

The value moves from *how fast you can do* to *how clearly you can direct*. Every one of the eleven building blocks in this guide exists to make directing more reliable: by giving Claude better context, reusable know-how, reach into your real systems, or the ability to run without you watching.

> **THE ONE-LINE SUMMARY OF THIS WHOLE GUIDE**
> You are the mind; AI is the hands. Chapters 5–15 are eleven ways to make the hands more capable, more consistent, and more connected to the systems where your work actually lives.

3.2 Three ways to work with Claude

Before the building blocks, know which room you are standing in. Claude offers three working modes, and several of the eleven capabilities behave differently — or do not exist — depending on which one you are using.

MODE 1
Chats

Ask, analyse, generate. Quick question-and-answer and one-off analysis. The fastest way to think out loud or get a draft.

Where: claude.ai, desktop app, mobile.

MODE 2
Cowork

Work a task together. Claude works alongside you on a multi-step task and produces real deliverables — documents, organised files, synthesised research — rather than answering one prompt at a time. Same agentic engine as Claude Code, no terminal.

Where: claude.ai, desktop app, mobile.
Plans: paid plans; beta.

MODE 3
Code

Build and run software. Claude Code reads a codebase, edits files, runs commands and integrates with development tools. This is where real build work happens.

Where: terminal (CLI), VS Code and forks, desktop app, and cloud sessions in the browser.

WHICH MODE DOES THIS GUIDE ASSUME?

Each chapter states its surfaces explicitly in the label row. Chapters 5, 6, 9 and 10 (Projects, Instructions, Connectors, Artifacts) are primarily **Chats and Cowork**. Chapters 13–15 (Routines, Agents, Workflows) are primarily **Claude Code**. Chapters 7, 8, 11 and 12 (Skills, MCP, Marketplace, Plugins) exist meaningfully on **both**, and those chapters cover both paths side by side — plugins and marketplaces used to be a Claude Code-only idea and are not any more.

3.3 Why the eleven blocks, and in this order

The eleven building blocks are not a flat list. They answer five different questions, and the chapter order follows that logic: context first, then know-how, then reach, then packaging, then autonomy.

THE QUESTION	THE BLOCKS	WHAT THEY CHANGE
What does Claude know?	Projects (5), Instructions (6)	Give Claude durable context and standing rules so you stop re-explaining yourself every conversation.
How does Claude do it?	Skills (7)	Teach a repeatable procedure once — your test-case format, your PR template — and reuse it forever.
What can Claude reach?	MCP Servers (8), Connectors (9)	Let Claude read and act in your real systems: Azure DevOps, GitHub, Figma, Slack, Drive.
What does Claude produce?	Artifacts (10)	Turn output into something you can open, use, share and iterate on — not a wall of chat text.
How do we share and scale it?	Marketplace (11), Plugins (12)	Package a working setup and distribute it so a whole team installs it with one command.
How does it run without me?	Routines (13), Agents (14), Workflows (15)	Schedule it, delegate it, or orchestrate dozens of parallel workers from a script.

3.4 What each block actually asks of you

The eleven differ enormously in setup cost, and that matters more than capability when you are deciding what to try on a Tuesday afternoon. This is the whole book compressed into one table: what you have to do before each block does anything for you.

BLOCK	EFFORT	WHAT YOU ACTUALLY HAVE TO DO	TERMINAL?
Projects ch 5	START HERE	Click **+ New Project**, name it, upload the documents you keep re-pasting.	**No**
Instructions ch 6	START HERE	Type your standing rules into a settings field. The single highest value-per-minute action in the book.	**No**
Skills ch 7	START HERE	Toggle on the built-in ones; write a `SKILL.md` when you want your own format. Claude will write it for you.	**No**
MCP Servers ch 8	NEEDS SETUP	Point at a server, pick a transport, authenticate. The one genuine configuration task — and the one that unlocks your real systems.	**Usually**
Connectors ch 9	START HERE	Click **Connect**, sign in, then remember to switch it on *in the conversation*. That last step is the one everyone misses.	**No**
Artifacts ch 10	START HERE	Turn on one switch, then ask for something substantial. Mostly it just happens.	**No**
Marketplace ch 11	PREREQ	Add a catalogue so you can install from it. Adding one installs nothing — that trips up almost everybody.	**Either**
Plugins ch 12	ADVANCED	Install someone else's in a click. *Authoring* one means having something worth packaging first.	**Either**
Routines ch 13	ADVANCED	Write a prompt that must succeed with nobody watching, then schedule it. The prompt is the hard part, not the schedule.	**No**

BLOCK	EFFORT	WHAT YOU ACTUALLY HAVE TO DO	TERMINAL?
Agents ch 14	ADVANCED	Write a Markdown file describing a worker and the tools it may touch.	**Yes**
Workflows ch 15	ADVANCED	Write a script that orchestrates many agents. The most powerful and the most rarely needed.	**Yes**

"Terminal?" means Claude Code rather than a browser. Six of the eleven need nothing but a browser.

Read the pattern: the blocks that cost you almost nothing are the ones that pay off immediately, and the four that sound most impressive all assume you have already built something worth automating. That is not a coincidence — Plugins, Routines, Agents and Workflows mostly *package, schedule or parallelise* the earlier blocks. There is nothing for them to carry until you have written a skill or connected a system.

3.5 One family, the right brain for the job

A short aside, because it comes up constantly. Claude is a family of models, not one model. You rarely need to choose manually in the product surfaces — but when you can, this is the trade-off.

MODEL	CHARACTER	CONTEXT WINDOW	REACH FOR IT WHEN
Haiku	Fast & light	200K	Quick lookups, high-volume simple tasks, cheap parallel workers inside a workflow.
Sonnet	The workhorse	1M	Coding, analysis and most day-to-day work — fast and smart.
Opus	The deep thinker	1M	Hard reasoning, planning, complex multi-step problems. The usual default for agentic work.
Fable	The flagship	1M	The most demanding reasoning and long-running agent work.

Model tiers and context windows per Anthropic's model documentation.

> **WHAT "1M CONTEXT" BUYS YOU IN PRACTICE**
>
> A one-million-token context window is why "paste the entire 80-page spec" is a reasonable instruction rather than a joke. It is also why **Projects** (Chapter 5) work the way they do: a project's knowledge base can hold a large amount of reference material and still leave room for the conversation. When a project's knowledge outgrows even that, Claude switches to a retrieval mode automatically — see Chapter 5, Section 3.

CHAPTER 4

Understanding the Core Concepts

One diagram, six confusions cleared up. Read this before any individual building-block chapter.

Most of the difficulty people have with these eleven capabilities is not learning any one of them. It is telling them apart. Six pairs get confused constantly, and every one of the six is resolved by asking the same question: *which layer does this belong to?*

4.1 The five layers

Every building block sits in exactly one of five layers. Once you can place a capability in a layer, you know what it can and cannot do for you.

#	LAYER	BLOCKS	THE LAYER'S JOB
1	**Context** What Claude knows	Projects, Instructions	Facts and standing rules that are true before you type anything. Loaded automatically, every time.
2	**Know-how** How Claude does it	Skills	A procedure Claude follows when the task calls for it. Loaded on demand, not always.
3	**Reach** What Claude can touch	Connectors, MCP Servers	Live read and write access to systems outside the conversation.
4	**Execution** Who drives the work	Agents, Workflows, Routines	How multi-step work gets carried out: delegated, scripted, or scheduled.
5	**Packaging** How it travels	Plugins, Marketplace	Bundling layers 1–4 so a colleague installs your setup instead of rebuilding it.

Artifacts sit outside the five layers because they are not an input to Claude at all — they are the *output*. See Section 4.3.

4.2 The architecture at a glance

How a request actually flows, and where each of the eleven blocks attaches to it.

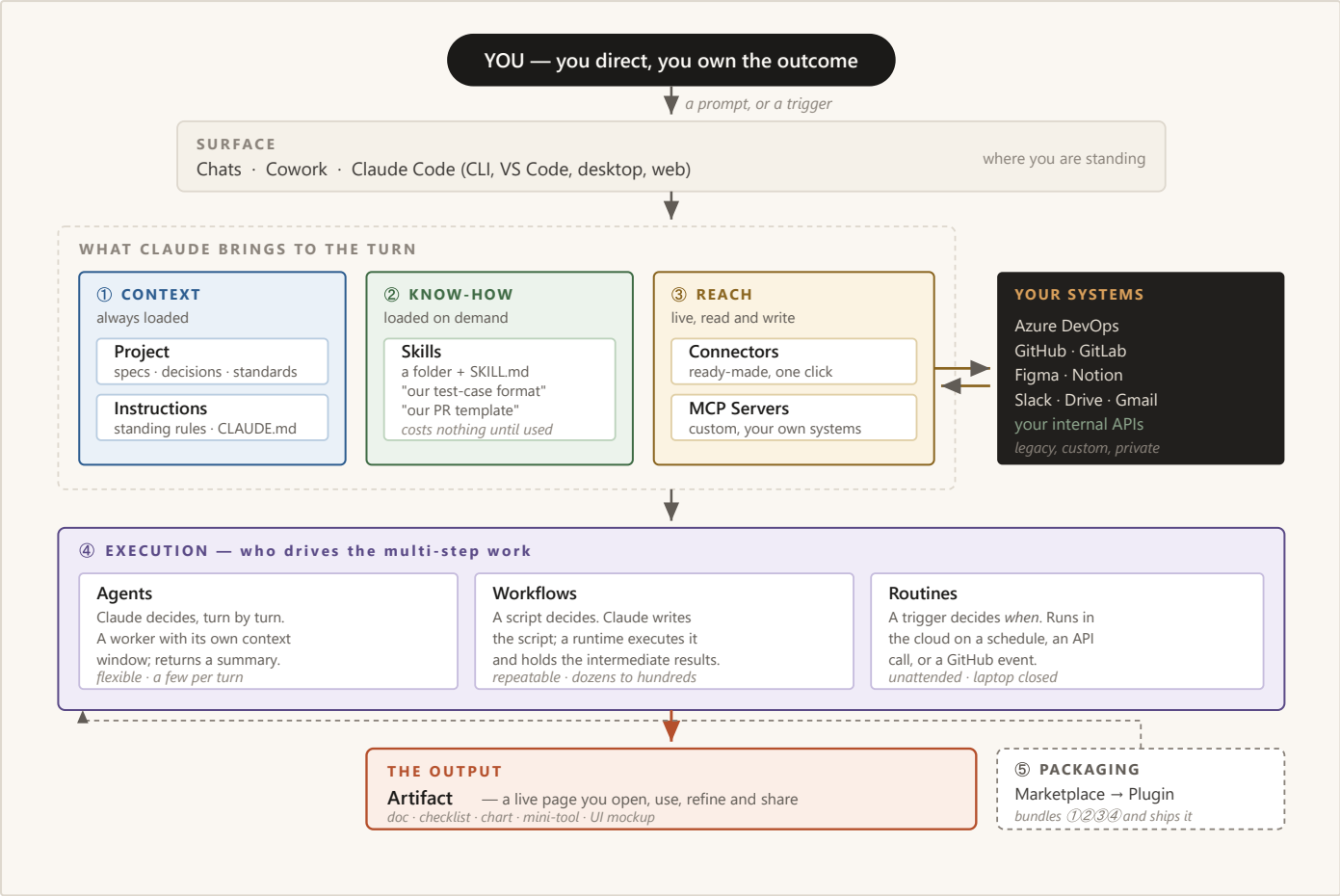


Figure 4.1 How a request flows, and where each building block attaches. Layers ①–③ are what Claude brings *into* a turn; layer ④ decides who drives the work; layer ⑤ packages any of it for distribution. Artifacts are the output, not an input.

Diagram — not a screenshot. Prepared for this guide.

4.3 The six confusions, resolved

These are the distinctions people ask about most. Each one is a "which layer?" question in disguise.

Project vs. Instructions vs. Skills

All three feel like "telling Claude how to behave," which is why they blur. The difference is *what* they carry and *when* it loads.

	PROJECT	INSTRUCTIONS	SKILLS
Carries	Reference *material* — specs, decisions, standards, exports	Standing *rules* — tone, format, constraints	A *procedure* — the steps of a repeatable task
Answers	"What is this work about?"	"How should you always behave?"	"How do we do this specific task?"
Loads	Every chat in that project	Every message, everywhere in scope	Only when the task matches — on demand
Scope	One workspace	Account-wide, project-wide, or repo-wide	Personal, project, or shipped in a plugin
Costs context	Yes, while the project is open	Yes, always	**No** — only its one-line description sits in context until it is used
Typical size	Many documents	A short page	A folder: instructions plus optional scripts and templates

	PROJECT	INSTRUCTIONS	SKILLS
Give it…	The 80-page spec	"Always use British English; never invent ticket IDs"	"Draft a test case: ID, Title, Preconditions, Steps, Expected result"

THE PRACTICAL TEST

If it is *information*, it belongs in a Project. If it is a rule that should apply to *every single message*, it belongs in Instructions. If it is a *procedure* with steps — or if your Instructions have grown past a page and most of it is irrelevant to most messages — it belongs in a Skill.

MCP Server vs. Connector

This one has a clean answer: **a Connector is a ready-made MCP client integration; an MCP server is the thing being connected to.** They are not alternatives at the same level — one is the socket, the other is the standard the socket implements.

CONNECTOR

Ready-made and one-click. You pick a service from a directory, click **Connect**, sign in, done. Anthropic (or the vendor) already built and reviewed the integration.

Use for: Google Drive, GitHub, Slack, Azure DevOps, Notion, Gmail, Figma, Jira — anything on the list.

Under the hood, most connectors *are* remote MCP servers. The word "connector" is the product-facing name for the switched-on, already-plumbed version.

MCP SERVER

The open standard, and what you build against it. The Model Context Protocol is a published specification for how any AI connects to any tool. An MCP server is a program that exposes your system through it.

Use for: what is *not* on the connector list — your internal APIs, a legacy system, a private database, custom tooling.

Anthropic's own analogy: MCP is a USB-C port for AI. Build the port once and every AI can use it; expose any tool through it and every AI can reach it.

The one-line rule: Connectors are for tools someone already integrated. MCP servers are for everything else. See Chapters 8 and 9.

Plugin vs. MCP Server

An MCP server adds *one* thing: reach into a system. A Plugin is a *bundle* that can contain an MCP server plus skills, agents, hooks, commands, settings and more — installable in one step.

	MCP SERVER	PLUGIN
Layer	③ Reach only	⑤ Packaging — can carry ①②③④
Contains	Tools, resources, prompts for one system	Any combination of skills, subagents, MCP servers, hooks, commands, LSP servers, monitors, workflows, default settings
Installed with	`claude mcp add …`, or added as a connector	`/plugin install name@marketplace`
Distributed via	A URL, or a committed `.mcp.json`	A marketplace (Chapter 11)
Versioned	Not by Claude	Yes — `version` in `plugin.json`, with auto-update

A useful way to hold it: *a plugin is how you hand a colleague a working setup. An MCP server is one of the things that setup might include.*

Artifact vs. a normal chat response

NORMAL CHAT RESPONSE

Text in the conversation. You read it, you scroll past it, and to reuse it you copy and paste. It lives only in that conversation.

Right for: answers, explanations, short drafts, anything conversational.

ARTIFACT

A separate, self-contained thing rendered in its own panel or page. You can open it, interact with it, ask for revisions in place, keep versions, download it, and share a link. It persists outside the conversation.

Right for: documents, checklists, charts, mini-tools, UI mockups — anything significant and self-contained you will want to *use* rather than read.

Claude decides on its own when content deserves to be an artifact — roughly, when it is self-contained, more than about fifteen lines, and something you would want to edit, reuse or refer back to. You can also just ask. Chapter 10 covers both.

Routine vs. Workflow vs. Agent

All three run multi-step work without you doing each step. They differ in **who holds the plan** — and that single question tells them apart every time.

	AGENT	WORKFLOW	ROUTINE
Holds the plan	**Claude**, turn by turn	**A script** Claude wrote	**A trigger** — the plan is your saved prompt
Answers	"Do this side task without cluttering my conversation"	"Run this same step across 200 items and cross-check the results"	"Do this every weeknight at 7pm while I'm asleep"
Where results live	Claude's context window	Script variables — only the final answer reaches Claude	A new cloud session per run
Scale	A few delegated tasks per turn	Dozens to hundreds of parallel workers	One session per trigger
You are…	present, in the session	present, but the session stays free	absent — that is the point
Repeatable unit	The worker definition	The orchestration itself	The whole configuration: prompt, repos, connectors, schedule

THEY COMPOSE

These are not competitors. A **Routine** fires on Friday night; the session it starts runs a **Workflow**; that workflow spawns forty **Agents** in parallel; each agent uses a **Skill** and reads Azure DevOps through an **MCP server**; the result is published as an **Artifact** you read on Monday. That is the whole guide in one sentence.

Marketplace vs. Plugin

MARKETPLACE

A *catalogue*. A list of available plugins, published as a `marketplace.json` file in a Git repository, a local folder or at a URL. Adding one registers the catalogue — it installs nothing.

Anthropic runs two: `claude-plugins-official` (curated, added automatically) and `claude-community` (third-party, added by hand). You can host your own, including in a private repo for internal-only plugins.

PLUGIN

An *item in the catalogue*. The actual bundle of capabilities that gets installed into your Claude Code setup.

The two-step model matters: **add the marketplace, then install plugins from it.** Removing a marketplace uninstalls everything you installed from it.

4.4 What loads when — the context budget

One practical consequence of the layer model is worth internalising early, because it explains a lot of otherwise confusing advice about keeping things short.

THING	WHEN IT LOADS	CONSEQUENCE
Instructions / `CLAUDE.md`	Every message	Keep it short and specific. Long, vague instruction files get followed *less* reliably, not more.
Project knowledge	Every chat in the project	Large but bounded. Past a threshold Claude switches to retrieval instead of loading everything.
Skill *description*	Every session	Cheap. This is why you can have many skills available at once.
Skill *body*	Only when used	Effectively free until needed. Put long reference material here, not in Instructions.
Connector / MCP tool definitions	Every session, once connected	Not free. Disconnect what you are not using; large tool sets can be deferred behind tool search.
Plugin components	While the plugin is enabled	Claude Code shows a **Context cost** estimate before you install. Disable plugins you have stopped using.

THE RULE THIS PRODUCES

Facts in Instructions, procedures in Skills. If a section of your instructions has grown into a series of steps, it has outgrown layer 1 and belongs in layer 2. This is the single most common structural mistake, and Chapters 6 and 7 both return to it.

CHAPTER 5

Projects

One shared context, reused by every conversation inside it.

START HERE | CLAUDE.AI | DESKTOP APP | COWORK | ALL PLANS — FREE CAPPED AT 5 PROJECTS

5.1 What is it?

A Project is a workspace that holds a body of context in one place, so that every conversation you start inside it already knows the background. You set the context once instead of re-explaining it at the top of every chat.

The problem it solves

Without a project, every conversation starts from nothing. If you are working on a checkout revamp, each new chat needs the same preamble: here is the spec, here are our coding standards, here is the decision we made about payment providers last week, here is the template we use. You paste it again. And again. And when you inevitably paste a slightly different version, you get slightly different answers — which is worse than starting from nothing, because it looks consistent and is not.

A project makes that context durable. Upload the spec once; every chat in the project reads it.

How it works at a high level

A project has three parts:

1. **A name and description.** For humans, to find it again. Claude does not read these.
2. **Project knowledge.** The files, documents, text and code you upload. This is the shared context every chat in the project can see.
3. **Project instructions.** Standing rules that apply to chats in this project only — see Chapter 6, which covers instructions at all three scopes.

When you start a chat inside a project, Claude reads the knowledge and the instructions before it reads your first message. When project knowledge grows large enough to approach the context window limit, Claude automatically switches to a retrieval mode, expanding the project's practical capacity by searching the knowledge base instead of loading all of it at once.

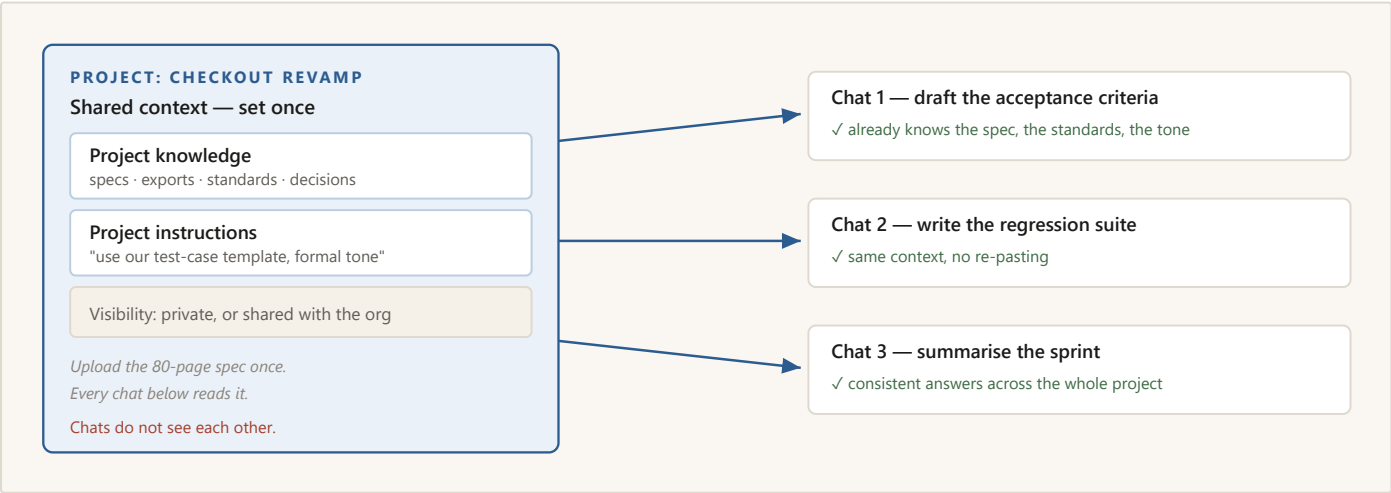


Figure 5.1 One project context, many independent chats. Every chat inherits the knowledge and instructions; no chat can see another chat's history.
Diagram — not a screenshot. Based on Anthropic's projects documentation.

What it is NOT

- **Not shared memory between chats.** This is the single most common misunderstanding. Chats inside a project do *not* see each other. If Chat 1 establishes a useful decision, Chat 2 knows nothing about it unless you add that decision to the project knowledge.
- **Not a folder for organising conversations.** It does organise them, but that is a side effect. The point is the shared context.
- **Not a Claude Code concept.** Claude Code has no "Projects" feature. Its equivalent is the repository itself plus a `CLAUDE.md` file — see Chapter 6.
- **Not a place for secrets.** Project knowledge is readable by anyone the project is shared with, and its contents flow into conversations. Do not upload credentials.
- **Not a substitute for a Skill.** Knowledge is *material*; a Skill is a *procedure*. Uploading your test-case template as a PDF is weaker than teaching it as a Skill, because a Skill can carry the steps and be invoked deliberately.

How it relates to the other blocks

BLOCK	RELATIONSHIP
Instructions	Projects have their own instructions scope, which layers on top of your account-wide instructions. Chapter 6 explains the three scopes and how they combine.
Skills	Skills are account-wide in claude.ai, not per-project. A project provides the material; a skill provides the method. They are complementary and usually used together.
Connectors	Enabled per conversation rather than per project. A project does not automatically grant its chats access to Azure DevOps — you enable the connector in the chat.
Artifacts	Produced inside chats, which live inside projects. A project is the natural home for a series of related artifacts.
Cowork	Cowork has its own projects concept for organising agentic tasks. Same idea, different surface.

5.2 When should I use it?

USE A PROJECT WHEN…

- You will have **more than two or three conversations** about the same body of work.
- There is **reference material** everyone's questions depend on — a spec, a standard, a data export, an API contract.
- You want **consistent answers** across conversations, and across colleagues.
- You are onboarding someone and want them to inherit the context rather than be briefed.
- The work has a clear boundary and a name you could put on a folder.

Do not use a project when…

- It is a one-off question. Just open a chat.
- The "context" is a single short paragraph — that is an instruction, not a knowledge base.
- You need conversations to build on each other's *conclusions*. Projects do not do that; you would have to write the conclusions back into the knowledge base each time.
- You are working in a codebase through Claude Code. Use the repository and a `CLAUDE.md`.

Real-world examples by role

ROLE	PROJECT	WHAT GOES IN THE KNOWLEDGE BASE
QA	"Release 4.2 test coverage"	The requirements doc, the acceptance criteria, your test-case template, the previous release's defect summary, the supported-environment matrix.
Product / BA	"Checkout revamp discovery"	Stakeholder interview notes, the current-state process map, competitor analysis, the decision log, the ADO board export.

ROLE	PROJECT	WHAT GOES IN THE KNOWLEDGE BASE
Design	"Design system v3 handoff"	Component inventory, spacing and colour tokens, accessibility checklist, the handoff spec template, links to the Figma library.
Development	"Payments service rewrite"	The API contract, the target architecture decision record, coding standards, the migration runbook, the legacy schema.

5.3 How do I set it up?

Prerequisites: a Claude account and a browser. Nothing to install.

Step-by-step: create a project

1 **Open the Projects area**

Go to claude.ai/projects

Or, from any chat, hover over the left-hand side of the window to expand the sidebar and click **Projects**.

2 **Create the project**

Click + **New Project** in the upper right. The dialog asks for three things:

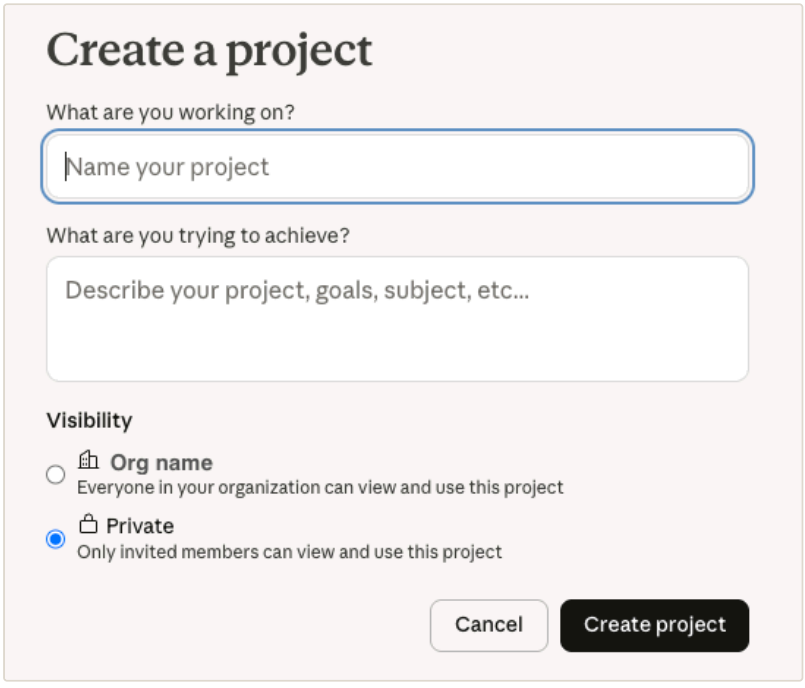


Figure 5.2 The **Create a project** dialog. Note that **Private** is preselected — the safe default. The two text fields are for humans only.
Screenshot: Anthropic, Claude Help Centre — "Manage project visibility and sharing."

3 **Name it and describe it**

Give it a **name** and a **description**. These are for you and your colleagues — Claude does not read them, so write them for a human trying to find the right project six weeks from now.

See Section 5.7 for a naming convention worth adopting before you have thirty projects.

4 **Choose visibility (Team and Enterprise plans)**

On Team and Enterprise plans you choose between **Private** (only you and members you invite) and your **organisation name** — everyone in the organisation can view and use it, where an administrator has enabled that. Private is the safe default and is preselected; you can share later.

> **DECIDE DELIBERATELY**
> Organisation-wide visibility means every colleague can find and read the project's knowledge base. That is exactly what you want for a shared standard, and exactly what you do not want for a knowledge base containing customer data or an unannounced initiative.

5 **Add project knowledge**

On the project's main page, click the + button on the right-hand side and upload documents, text files, or code. Claude processes them into context available to every chat in the project.

A capacity indicator shows how full the knowledge base is. As it approaches the context window limit, Claude automatically enables a retrieval mode to expand capacity.

6 **Set project instructions**

Click **Set project instructions**, write your standing rules for this project, and click **Save instructions**. Claude applies them to every chat in the project.

Chapter 6 covers what to put here in detail. Keep it short.

7 **Start a chat and sanity-check it**

Open a new chat inside the project and ask a question that can only be answered from the knowledge base — not a general question. See Section 5.4 for the verification prompt.

Recommended configuration

SETTING	RECOMMENDATION	WHY
Name	`[Team] Topic – Phase`	Sorts sensibly and survives growth. See 5.7.
Visibility	Private first	Sharing is reversible; an accidental disclosure is not.
Knowledge	The 5–10 documents people actually cite	A knowledge base of everything is a knowledge base of nothing. Curate.
Instructions	Under ~15 lines	Long instruction blocks are followed less reliably. Put procedures in Skills.
Scope	One project per body of work, not per person	Per-person projects fragment the context you were trying to centralise.

Example configuration — a QA release project

Project settings

```
Name:        [QA] Release 4.2 – Test Coverage
Description: Test design and execution tracking for the 4.2 release.
             Owner: QA lead. Source of truth for ACs is the linked ADO epic.
Visibility:  Shared with organization

Knowledge base (6 files):
  01-requirements-r4.2.pdf          the signed-off requirements
  02-acceptance-criteria.md         ACs as agreed, exported from ADO
  03-test-case-template.md          our house format, ID/Title/Pre/Steps/Expected
  04-supported-environments.md      browser + OS + device matrix
  05-r4.1-defect-summary.csv        what broke last time – informs risk areas
  06-regression-scope.md            what is in and out of regression this cycle

Project instructions:
  Write in British English. When drafting test cases, follow
```

```
03-test-case-template.md exactly, including the ID prefix TC-R42-.
Reference acceptance criteria by their AC number from 02.
Never invent an AC number or a ticket ID — if it is not in the knowledge
base, say so and ask.
Cover the negative and boundary cases, not just the happy path.
```

5.4 How do I use it?

How to start

Open the project and start a new chat inside it. That is the whole mechanic — there is no command to invoke and no toggle to remember. If you are in the project, you have the context.

What to provide, and how to structure it

Because the background is already loaded, your prompts get shorter and more specific. The pattern that works:

Prompt structure inside a project

```
[The task]        Draft test cases for AC-14 and AC-15.
[The constraint]  Positive, negative and boundary. Use the house template.
[The output]      Give me a table I can paste into ADO, then flag any AC
                  that is too vague to test as written.
```

Note what is *absent*: no pasted template, no explanation of what AC-14 is, no reminder about British English. That is the project doing its job.

What to expect

- Answers that use your terminology and formats without being asked.
- Claude referring to specific documents in the knowledge base by name.
- Claude pushing back when your request conflicts with something in the knowledge base — which is usually the most valuable thing it does.

How to verify it is actually working

Do not assume. Run this in a new chat inside the project:

Verification prompt

```
Without searching the web: list the documents in this project's knowledge
base, and tell me the ID prefix our test-case template specifies.
Then quote the sentence in the project instructions about inventing IDs.
```

If Claude names your files and quotes your rule, the project is configured correctly. If it answers generically or hedges, either the upload did not complete or you are in a chat outside the project — check the breadcrumb above the chat title, which shows the project name.

Common usage patterns

PATTERN	HOW IT WORKS
One chat per deliverable	Keep each chat scoped to one output — one test suite, one spec, one report. Shorter chats stay coherent and are easier to find later.
Promote decisions into knowledge	When a chat produces a decision that later chats need, write it into a `decisions.md` and upload it. This is the only way to carry conclusions between chats.
The living template	Keep your house format in the knowledge base and update it in place. Every future chat picks up the new version automatically.
Onboarding by project	Hand a new joiner the project instead of a briefing. They read the knowledge base and start asking Claude the questions they would have asked you.

PATTERN	HOW IT WORKS
Adopt an orphan chat	Started a good conversation outside a project? Move it in — see Section 5.5.

5.5 How do I manage it?

CREATE	claude.ai/projects › + New Project Name, description, and (Team/Enterprise) visibility.
VIEW	All projects are listed at `claude.ai/projects`. Archived ones appear at the bottom of that list.
EDIT	Open the project. Knowledge: + to add, or select an item to remove it. Instructions: click **Set project instructions** (or the existing instructions) and **Save instructions**.
SHARE	Team/Enterprise: open the project, click **Share** to the right of the project name, add people by name or email — or bulk-paste addresses — choose **Can view** or **Can edit**, then **Share**.
CHANGE VISIBILITY	Same dialog. Under **General access**, switch between **Only people invited** and **Everyone at [your organisation]**. Works in both directions, at any time — see below.
DISABLE	There is no "disable." **Archive** is the equivalent: the project stops appearing in the main list but keeps its sharing and members.
ARCHIVE	Open the project, click the **…** menu in the upper right, and confirm. To reverse: **…** › **Unarchive**, from either the archived list or inside the project.
DELETE	**…** › **Delete** › **Yes, delete**. *An archived project cannot be deleted — unarchive it first.*
DUPLICATE	There is no one-click duplicate. Create the new project and re-upload the knowledge; keep your standing documents in a source you control (a repo or a shared drive) so this is a copy, not a rewrite.

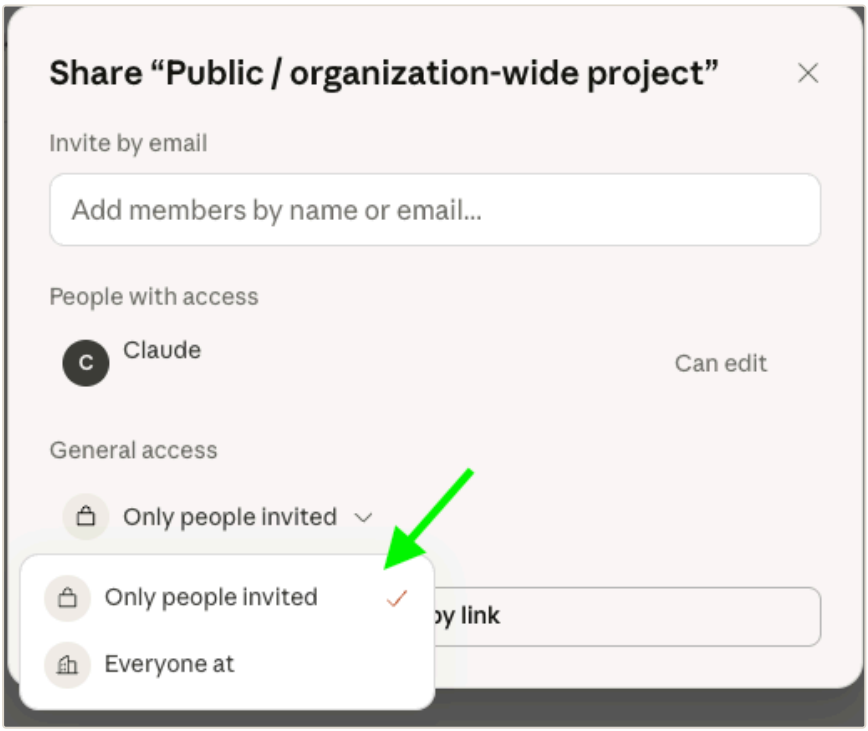


Figure 5.3 Changing a project's visibility after the fact. The **General access** dropdown is the control — distinct from the per-person **Can view** / **Can edit** list above it. Switching to *Only people invited* makes a public project private again.
Screenshot: Anthropic, Claude Help Centre — "Manage project visibility and sharing."

Moving conversations in and out of projects

Chats are not locked to where they started. Anthropic's help centre documents four operations:

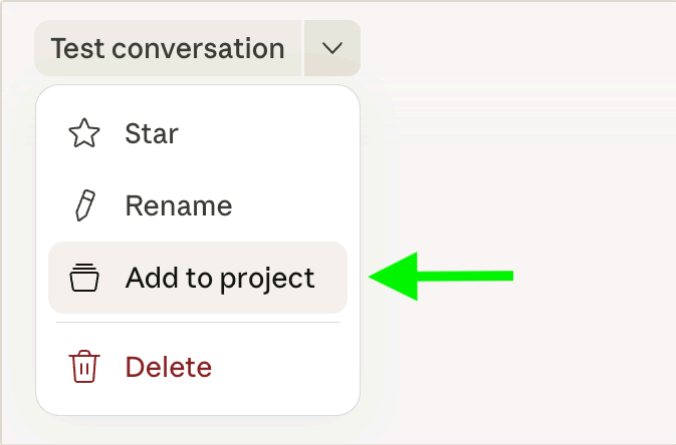


Figure 5.4 Adding an existing chat to a project: click the dropdown arrow beside the chat name and choose **Add to project**.
Screenshot: Anthropic, Claude Help Centre — "How can I create and manage projects?"

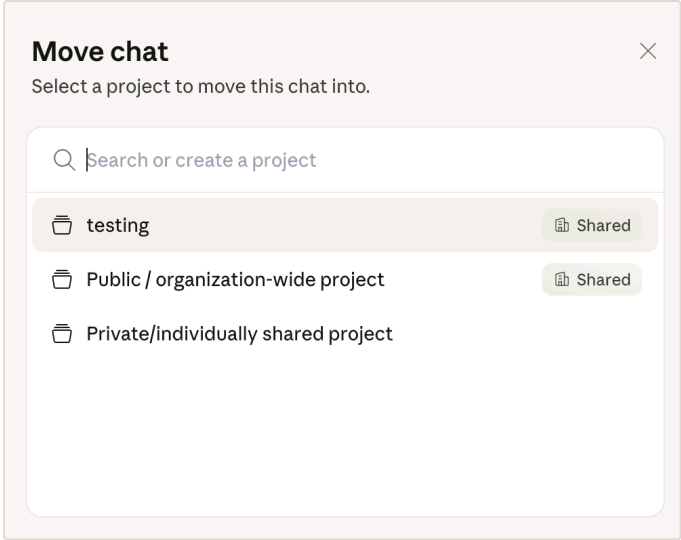


Figure 5.5 The **Move chat** picker. You can search existing projects or create one inline. The **Shared** badge marks projects visible beyond you.
Screenshot: Anthropic, Claude Help Centre — "How can I create and manage projects?"

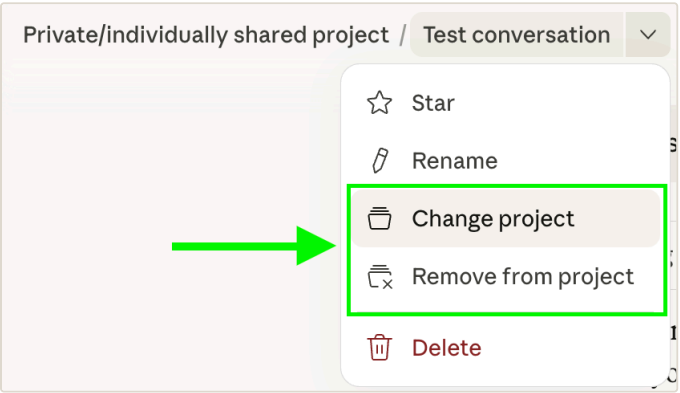


Figure 5.6 Inside a project, the same menu offers **Change project** and **Remove from project**.
Screenshot: Anthropic, Claude Help Centre — "How can I create and manage projects?"

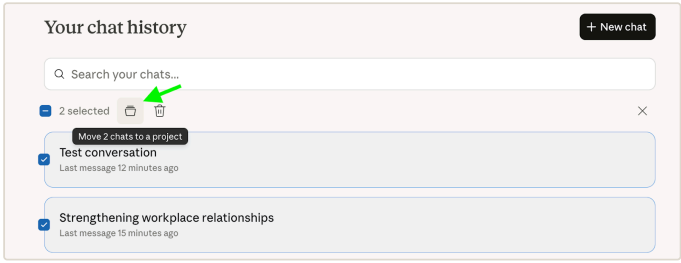


Figure 5.7 Bulk move: select chats on your chat history page and move them together. Useful when adopting projects after the fact.
Screenshot: Anthropic, Claude Help Centre — "How can I create and manage projects?"

Troubleshooting

SYMPTOM	LIKELY CAUSE	FIX
Claude ignores the knowledge base	You are in a chat *outside* the project	Check the breadcrumb above the chat title. If there is no project name, move the chat in (Figure 5.2).
Claude ignores your project instructions	Instructions too long, too vague, or contradicted by account-level instructions	Shorten to specific, checkable rules. Then check your account-wide instructions for a conflicting rule — see Chapter 6, Section 6.5.
Answers in one chat contradict another	Working as designed — chats are isolated	Write the agreed answer into project knowledge. Chats share *context*, never *conclusions*.
Knowledge base "full"	Approaching the context limit	Claude switches to retrieval mode automatically. Also prune: remove superseded documents rather than accumulating versions.
Can't delete a project	It is archived	Unarchive first, then delete. Archived projects cannot be deleted directly.
A colleague can't see the project	Private, or they were never added	Team/Enterprise: **Share project**. Individual plans have no project sharing.

SYMPTOM	LIKELY CAUSE	FIX
Claude cites an outdated fact	The superseded document is still in the knowledge base	Remove it. Two versions of the truth means Claude may pick either.

5.6 Examples

Example 1 — QA: test-case generation from acceptance criteria

The setup from Section 5.3. In a new chat inside the project:

Prompt

```
Draft test cases for AC-14 (guest checkout) and AC-15 (saved card reuse).
Use the house template. Include negative and boundary cases.
Output as a markdown table with columns matching our ADO import format.
Then list any assumption you had to make.
```

Expected: a table using your `TC-R42-` prefix and your column names, covering expired cards, declined payments and session timeout as well as the happy path, plus an explicit assumptions list — which is where you will usually find the real gap in the AC.

Example 2 — Product/BA: requirements analysis

Prompt

```
Compare the signed-off requirements (01) against the current ADO export (06).
List: requirements with no corresponding work item; work items with no
requirement; and any AC whose wording changed between the two.
Present as three short tables. Do not guess at intent — flag ambiguity.
```

Example 3 — Design: handoff spec from a component

Prompt

```
Write the handoff spec for the new Pagination component using our
handoff template from the knowledge base. Cover variants, states,
spacing tokens and accessibility. Where the design system doesn't define
a token for something, say so explicitly rather than inventing a value.
```

Example 4 — Development: onboarding to a legacy service

Prompt

```
I'm new to the payments service. Using the architecture decision record
and the legacy schema in this project, explain: the three main flows,
where the known fragility is, and what the rewrite is trying to change.
Then give me five questions I should ask the current maintainer.
```

Example 5 — Reporting: a recurring status document

Keep the report format in project knowledge as `status-format.md`, then each Friday:

Prompt

```
Here is this week's ADO export (attached). Produce the weekly status
in our standard format: what was done, what's blocked, what's next.
Build it as an artifact so I can adjust the wording before sending.
```

The artifact makes the last-mile edit fast — see Chapter 10.

5.7 Best practices

Recommended practices

- **Curate, don't hoard.** Five well-chosen documents beat fifty. Every document you add is context Claude must weigh; irrelevant material dilutes the relevant material.
- **Keep one source of truth.** When a document is superseded, remove the old one. Two versions in the knowledge base is an invitation to inconsistency.
- **Name the authority.** In the project instructions, say which document wins when two conflict. "The source of truth for acceptance criteria is 02" saves a great deal of confusion.
- **Write the description for a stranger.** Include the owner and what the project is *for*. Future you is a stranger.
- **Promote decisions deliberately.** Maintain a `decisions.md`. It is the only mechanism for carrying conclusions between chats.
- **Prefer Skills for procedures.** If you find yourself uploading a document that is really a set of steps, make it a Skill instead (Chapter 7).

Common mistakes

MISTAKE	WHY IT HURTS
Expecting chats to share memory	They do not. This causes the "but I already told Claude that" frustration more than anything else in this guide.
One project per person	Fragments the shared context. Projects should be per body of work.
Dumping the entire SharePoint folder in	Dilutes the signal and burns capacity. Ask "would a colleague cite this?"
Writing procedures in project instructions	Instructions load every message. Long procedural instructions get followed less reliably. Use a Skill.
Uploading credentials or customer PII	Readable by everyone the project is shared with, and flows into every chat. Never.
Never revisiting the knowledge base	A project whose knowledge is six months stale actively misleads. Review it each cycle.

Naming convention

Adopt one before you have thirty projects. A workable pattern:

Convention

```
[Team] Subject — Phase or period

[QA] Release 4.2 — Test Coverage
[QA] Payments — Regression Baseline
[BA] Checkout Revamp — Discovery
[BA] Checkout Revamp — Delivery
[Design] Design System v3 — Handoff
[Dev] Payments Service — Rewrite
[Shared] House Standards — Templates and Formats
```

The bracketed team prefix groups the list alphabetically. The em-dashed phase lets one subject have several projects without ambiguity. A `[Shared]` prefix marks projects intended as organisation-wide references.

Versioning and updates

- Version *inside* documents, not by uploading `template-v2.md` alongside `template.md`. Keep one file, put a changelog line at the top.
- Where a document has a real home (a repo, Confluence, a drive), treat the project copy as a snapshot and note its source and date in the project instructions.
- Review the knowledge base at the same cadence as the work — each sprint, each release.

Team and organisational considerations

- **Ownership.** Every shared project needs a named owner in its description. Unowned shared projects rot.
- **Permissions.** Grant **Can view** by default; reserve **Can edit** for people accountable for the content. An editor can change the instructions everyone else's chats obey.
- **Shared standards as a project.** A single organisation-wide `[Shared] House Standards` project is a good pattern — but Skills distribute standards better, because they apply everywhere rather than only inside one project.
- **Free-plan limit.** Free accounts cap out at five projects. Plan the taxonomy accordingly if your team is mixed-plan.

> **VERIFY BEFORE YOU RELY ON IT**
>
> Anthropic's help centre indicates that **project instructions require a paid plan**, while projects themselves are available on all plans (with Free limited to five). Plan gating changes; confirm against the official article below before designing a process that assumes Free-plan colleagues can set project instructions.

5.8 Official resources

What are projects? — concept overview
`https://support.claude.com/en/articles/9517075-what-are-projects`

How can I create and manage projects? — the setup and lifecycle reference used throughout this chapter
`https://support.claude.com/en/articles/9519177-how-can-i-create-and-manage-projects`

Understanding Claude's personalization features — how project instructions relate to account-wide instructions and skills
`https://support.claude.com/en/articles/10185728-understanding-claude-s-personalization-features`

Organize your tasks with projects in Claude Cowork
`https://support.claude.com/en/articles/14116274-organize-your-tasks-with-projects-in-claude-cowork`

Collaborate with Claude on Projects — the original announcement
`https://www.anthropic.com/news/projects`

Claude Code equivalent: how Claude remembers your project (`CLAUDE.md`)
`https://code.claude.com/docs/en/memory`

CHAPTER 6

Instructions

Standing rules Claude carries into every message, so you never repeat yourself.

START HERE | CLAUDE.AI | DESKTOP APP | COWORK | CLAUDE CODE | ALL PLANS

6.1 What is it?

Instructions are standing rules Claude follows on every message, without you asking. Write them once — your format, your tone, your constraints — and every subsequent prompt carries them automatically.

The problem it solves

Left to itself Claude writes in a reasonable general-purpose style. But your team does not want a reasonable general-purpose style; it wants *your* style. So people end up appending the same paragraph to every prompt: use British English, don't use bullet points in the summary, always give me the SQL first, never invent a ticket ID. That paragraph is a tax you pay on every single message, and the day you forget it the output is inconsistent with everything before it.

Instructions move that paragraph out of your prompt and into the platform.

How it works at a high level

Instructions exist at several **scopes**, from broad to narrow. All applicable scopes load together — they layer rather than replace.

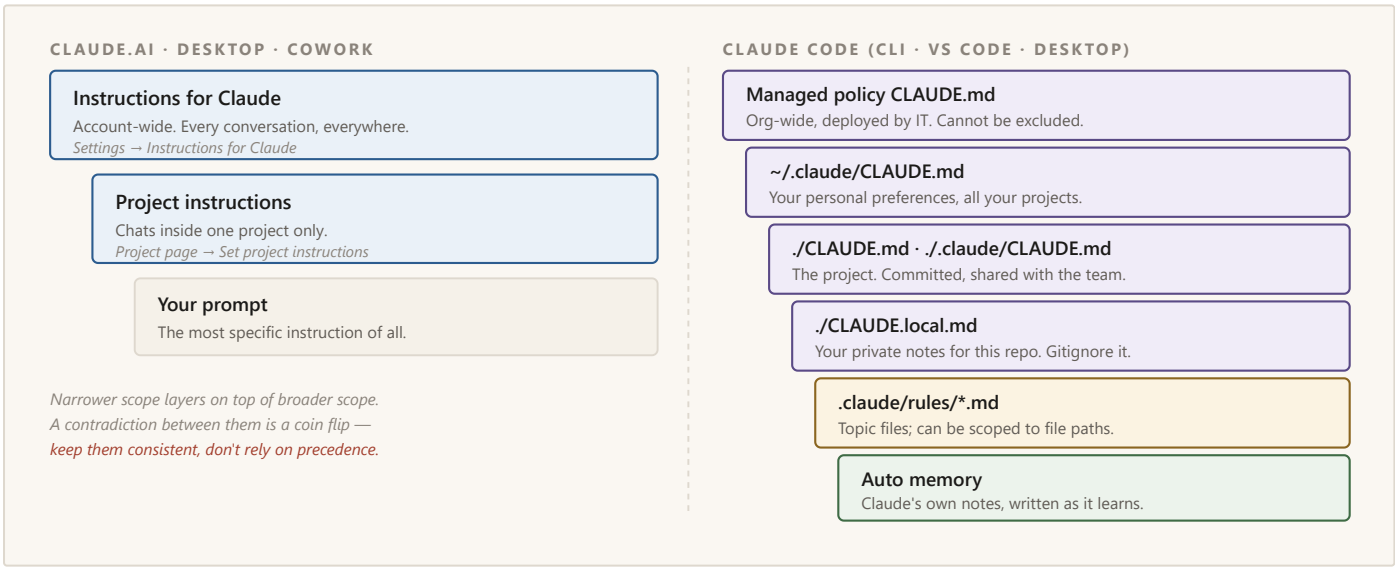


Figure 6.1 Instruction scopes on both surfaces. Everything applicable loads together, ordered broad to narrow, so the most specific instruction is the last thing Claude reads.

Diagram — not a screenshot. Compiled from Anthropic's personalization help article and the Claude Code memory documentation.

What it is NOT

- **Not enforcement.** This is the most important thing to understand. Instructions are *context*, not configuration. Claude reads them and tries to follow them; there is no guarantee of strict compliance, especially for vague or conflicting rules. If something absolutely must happen every time, you need a mechanism that runs regardless of what Claude decides — a hook in Claude Code, or a permission rule.
- **Not a place for procedures.** Instructions load on every message. A twelve-step process in your instructions costs you on every message and gets followed less reliably than the same steps in a Skill.
- **Not knowledge.** Reference material belongs in project knowledge (Chapter 5) or a repository. Instructions are rules *about* behaviour.

- **Not one thing.** "Instructions" is an umbrella. In claude.ai it means two specific settings; in Claude Code it means a family of files. Both are covered here.
- **Not the same as a system prompt.** In Claude Code, `CLAUDE.md` content is delivered as a message after the system prompt, not as part of it — which is precisely why it is persuasive rather than binding.

How it relates to the other blocks

BLOCK	RELATIONSHIP
Projects	A project is one of the scopes at which instructions can be set. Project instructions apply only inside that project and layer on top of your account-wide ones.
Skills	The critical pairing. Facts and always-true rules → Instructions. Procedures and step-by-step methods → Skills. When an instructions file grows past a page, the excess is usually a Skill trying to be born.
Agents	A subagent definition has its own system prompt, which is effectively instructions scoped to that one worker.
Plugins	Can ship rules and default settings, distributing instructions to a whole team.
Hooks (not a chapter here)	The enforcement escalation. When an instruction must be guaranteed — run the linter before every commit — a hook executes it as a shell command at a fixed lifecycle event, regardless of Claude's judgement.

6.2 When should I use it?

WRITE AN INSTRUCTION WHEN…

- You have typed the same correction **twice**. That is the threshold.
- The rule is true for **every** message in its scope — not most, every.
- It is a short, checkable fact or constraint: a format, a tone, a naming rule, a prohibition.
- A new colleague would need to be told the same thing to produce acceptable work.
- In Claude Code: a code review caught something Claude should have known about this codebase.

Do not write an instruction when…

- It is a multi-step procedure → make it a **Skill** (Chapter 7).
- It only applies to some files or some tasks → use a **path-scoped rule** or a Skill with a `paths` field.
- It is reference information → project knowledge or the repository.
- It must be guaranteed → a **hook** or a permission rule, not an instruction.
- You are already at ~200 lines → stop adding and start splitting.

Which scope? A decision table

THE RULE IS TRUE…	CLAUDE.AI / COWORK	CLAUDE CODE
for me, always, everywhere	**Instructions for Claude** (account settings)	`~/.claude/CLAUDE.md`
for this body of work only	**Project instructions**	`./CLAUDE.md` (committed)
for me, in this repo only, privately	*n/a*	`./CLAUDE.local.md` (gitignored)
only when touching certain files	*n/a*	`.claude/rules/<topic>.md` with a `paths:` field
for everyone in the organisation	Org-provisioned skills; admin settings	Managed policy `CLAUDE.md`, deployed by IT

6.3 How do I set it up?

Path A — claude.ai, desktop app and Cowork

Account-wide: Instructions for Claude

1. **Open your settings**

   Click **your initials** in the lower-left corner › **Settings**

2. **Find the instructions field**

   Under **Instructions for Claude**, describe how you want Claude to behave. This applies to every conversation on your account.

3. **Write rules, not an essay**

   Aim for a short list of specific, checkable statements. See the example below and the guidance in Section 6.7.

4. **Save and test in a fresh chat**

   Open a new conversation and give it a task that would expose the rule. Verification prompt in Section 6.4.

Per-project instructions

1. **Open the project**

   claude.ai/projects › your project

2. **Click Set project instructions**

   Write the rules that apply to this body of work only.

3. **Click Save instructions**

   They now apply to every chat in the project, on top of your account-wide instructions.

Cowork: global and folder instructions

Cowork has *its own* instruction scopes, separate from the account-wide field above. If you work in Cowork and wonder why your account instructions do not seem to cover everything, this is why:

1. **Global instructions — every Cowork session**

   Settings › **Cowork** › **Edit** next to **Global instructions** › type › **Save**

   Use it for tone, output format, and background on your role — the things Claude should always know.

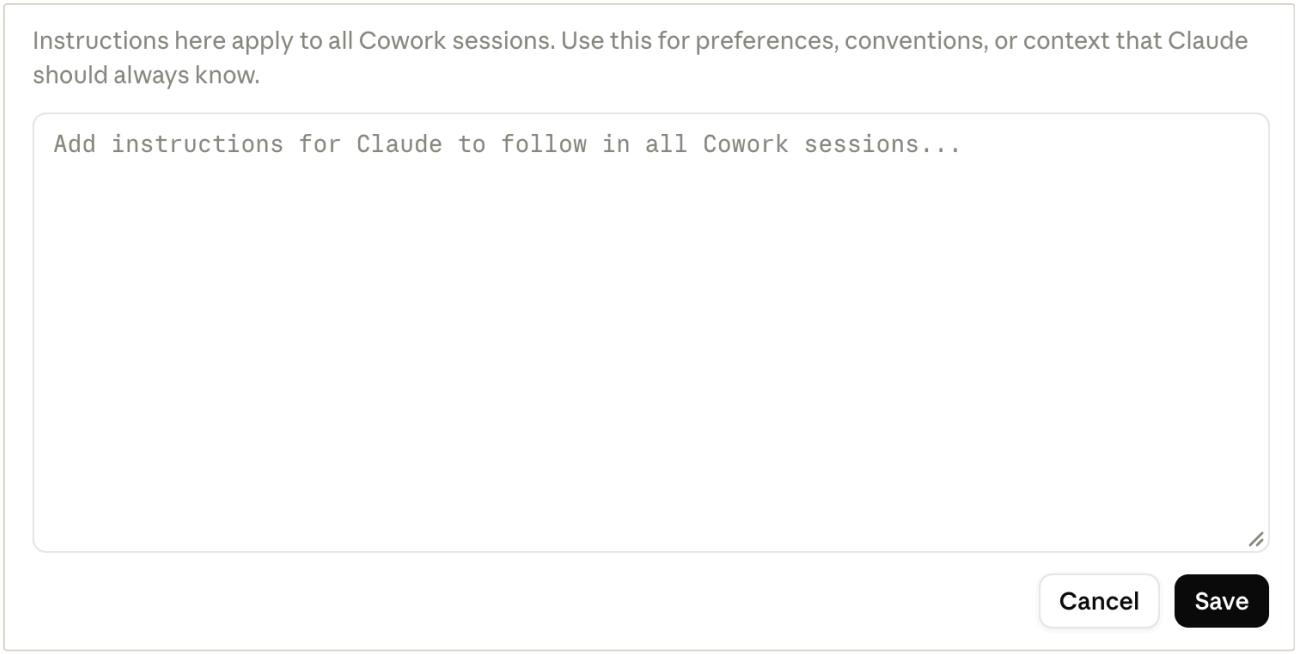


Figure 6.2 Cowork's **Global instructions** editor. Note the wording — *all Cowork sessions*. This is a fourth scope alongside account-wide, per-project and Claude Code's `CLAUDE.md`.
Screenshot: Anthropic, Claude Help Centre — "Get started with Claude Cowork."

2

Folder instructions — one body of work

When you select a local folder on the desktop app, folder instructions add context scoped to that folder — the closest Cowork equivalent to project instructions. **Claude can update these itself during a session**, which is worth knowing: treat the file as something that changes, and read it before assuming what it says.

Example — account-wide "Instructions for Claude"

```
Write in British English. Use "acceptance criteria", not "AC" on first use.

Be direct. No preamble — do not open with "Great question" or restate my
request back to me. If my request is ambiguous in a way that changes the
answer, ask one clarifying question before starting.

For anything factual about our systems: if it is not in the project
knowledge or a connected tool, say you do not know. Never invent ticket
IDs, table names, endpoints, people or dates.

When you produce a document, checklist, chart or table longer than about
15 lines, build it as an artifact so I can edit it in place.

Default to prose for explanations and tables for comparisons. Do not
bullet-point everything.
```

Path B — Claude Code (CLI, VS Code, desktop)

Claude Code reads a family of markdown files. The two you will use constantly are the project `CLAUDE.md` and your personal `~/.claude/CLAUDE.md`.

SCOPE	LOCATION	PURPOSE & SHARING
Managed policy	macOS `/Library/Application Support/ClaudeCode/CLAUDE.md` Linux & WSL `/etc/claude-code/CLAUDE.md` Windows `C:\Program Files\ClaudeCode\CLAUDE.md`	Organisation-wide standards deployed by IT via MDM or Group Policy. Cannot be excluded by individual settings.

SCOPE	LOCATION	PURPOSE & SHARING
User	`~/.claude/CLAUDE.md`	Your personal preferences across every project. Just you.
Project	`./CLAUDE.md` or `./.claude/CLAUDE.md`	Team-shared project standards. Committed to source control.
Local	`./CLAUDE.local.md`	Your private preferences for this one repo. Add it to `.gitignore`.
Rules	`.claude/rules/*.md` and `~/.claude/rules/*.md`	Modular topic files, discovered recursively. Can be limited to matching file paths.

1 **Generate a starting point**

In a repository, run:

```
> /init
```

Claude analyses the codebase and writes a `CLAUDE.md` containing the build commands, test instructions and conventions it can discover. If one already exists, `/init` proposes improvements rather than overwriting it.

`/init` also reads other agents' rule files — Cursor rules in `.cursor/rules/` or `.cursorrules`, and Copilot instructions in `.github/copilot-instructions.md` — and folds the relevant parts in.

2 **Edit it down to what Claude could not discover**

The generated file describes your codebase. The valuable content is what a new joiner would need *told* to them: the pitfalls, the reasons, the conventions that differ from the tool defaults. Delete the directory listings.

3 **Add your personal preferences separately**

```
> /memory
```

This lists every memory file location across user and project scope, including ones that do not exist yet. Select one to open it in your editor; selecting a file that does not exist creates it first.

4 **Confirm what actually loaded**

```
> /context
```

Check the list under **Memory files**. If a file is not listed there, Claude cannot see it. This is the single most useful diagnostic in this chapter.

5 **Split by topic once it grows**

Past roughly 200 lines, move topics into `.claude/rules/`:

```
your-project/
├── CLAUDE.md                # the always-true essentials
└── .claude/
    └── rules/
        ├── code-style.md
        ├── testing.md
        └── api-design.md
```

6 **Scope rules to paths where it helps**

A rule with a `paths:` field loads only when Claude works with matching files, so it costs nothing the rest of the time.

`.claude/rules/api-design.md`

```
---
paths:
  - "src/api/**/*.ts"
---

# API development rules

- Every endpoint validates its input before touching the database.
- Use the standard error envelope from `src/api/errors.ts`.
- Add an OpenAPI documentation comment to every exported handler.
```

The quick capture shortcut

Mid-session, you do not have to open a file. Just tell Claude:

```
> remember that we always use pnpm, never npm
> add this to CLAUDE.md: integration tests need a local Redis on 6379
```

The first form saves to **auto memory** — Claude's own notes. The second writes to `CLAUDE.md`. Say which you want; they behave differently (Section 6.5).

`Example — a project CLAUDE.md worth committing`

```
# Payments Service

## Commands
- Build:            `pnpm build`      (never `npm` — the lockfile is pnpm's)
- Unit tests:       `pnpm test`
- Integration:      `pnpm test:int`   requires local Redis on 6379
- Lint (must pass): `pnpm lint`

## Conventions
- 2-space indentation. Prettier config is authoritative; do not hand-format.
- API handlers live in `src/api/handlers/`. One handler per file.
- Money is always minor units (integer pence). Never a float. This has
  caused two production incidents.
- Dates are ISO-8601 UTC at every boundary. Local time only in the view layer.

## Before you finish
- Run `pnpm lint` and `pnpm test`. Report failures with the output; do not
  describe a change as done if either fails.
- Update `CHANGELOG.md` for anything user-visible.

## Do not
- Do not add a new dependency without saying so explicitly in your summary.
- Do not touch `src/legacy/` — it is being deleted in Q4, patches there are wasted.
```

WHY THAT EXAMPLE WORKS

Every line is either a command, a checkable convention, or a constraint with the *reason* attached. There is no "write clean code," no "be thorough," and no directory listing — Claude can see the directories. The money rule includes why it matters, which is what makes it stick.

6.4 How do I use it?

You do not invoke instructions. The whole point is that they are already there. What changes is what you *stop* typing.

Before and after

WITHOUT INSTRUCTIONS

```
Draft the test cases for the login
flow. Use British English. Follow our
template: ID, Title, Preconditions,
Steps, Expected result. Prefix IDs with
TC-. Include negative cases. Don't
invent ticket numbers. Give me a table.
No preamble.
```

WITH INSTRUCTIONS

```
Draft the test cases for the login flow.
```

Same output. Every time. Including the time you were in a hurry.

How to verify instructions are loaded

claude.ai / Cowork — in a new chat:

Verification prompt

```
Before answering anything else: quote my standing instructions back to me
verbatim, and say which came from my account settings and which from this
project. Then tell me one thing they forbid.
```

Claude Code — do not ask, check:

```
> /context
# look under "Memory files" — every loaded CLAUDE.md and rule is listed there
```

Writing instructions that actually get followed

PRINCIPLE	WEAK	STRONG
Be specific enough to verify	"Format code properly"	"Use 2-space indentation"
Name the command	"Test your changes"	"Run `pnpm test` before saying a change is done"
Name the location	"Keep files organised"	"API handlers live in `src/api/handlers/`"
Say what to do, not only what not to	"Don't be verbose"	"Answer in at most three sentences unless I ask for detail"
Attach the reason to hard rules	"Money is integers"	"Money is minor units — float rounding caused two incidents"
Normal volume	"CRITICAL: You MUST ALWAYS…"	"Use…" — shouting causes over-application, not compliance

DO NOT SHOUT

Emphasis like `CRITICAL:` and `YOU MUST` was written for older, less steerable models. Current Claude models follow the system prompt closely, so that phrasing now causes rules to be applied *too* aggressively — and when several instructions are all marked critical, the marker stops carrying information. State the rule plainly.

6.5 How do I manage it?

CREATE	**claude.ai:** Settings › Instructions for Claude, or a project's **Set project instructions**. **Claude Code:** `/init` to generate, `/memory` to open or create any memory file.

VIEW	**claude.ai:** the same settings field. **Claude Code:** `/memory` lists all locations; `/context` shows which actually loaded this session.
EDIT	Edit the field or the file. Claude Code watches these files — edits apply without restarting.
DISABLE	Clear the field, or delete/rename the file. For auto memory: toggle it in `/memory`, set `"autoMemoryEnabled": false`, or set `CLAUDE_CODE_DISABLE_AUTO_MEMORY=1`.
EXCLUDE	In a monorepo, skip other teams' files with `claudeMdExcludes` in settings, using glob patterns against absolute paths. Managed-policy files cannot be excluded.
DELETE	Delete the file. Nothing else references it.
REUSE	Import shared content with `@path/to/file` from inside a `CLAUDE.md`, symlink a shared rules directory, or ship rules in a plugin.
AUDIT	`/context` for what loaded; `/doctor` proposes trims for an over-long committed `CLAUDE.md`.

Reusing instructions across projects

Three mechanisms, in increasing order of formality:

1. **Imports.** A `CLAUDE.md` can pull in other files with `@path` syntax. Relative paths resolve against the file containing the import; imports can nest up to four levels.

```
See @README for the project overview and @package.json for the npm commands.

# Additional instructions
- git workflow @docs/git-instructions.md
- personal preferences @~/.claude/my-project-instructions.md
```

   Wrap a path in backticks to mention it without importing it. Note that an import in a *project* file whose path resolves outside the working directory triggers a one-time approval dialog — a deliberate guard against files other people commit.

2. **Symlinks.** `.claude/rules/` follows symlinks, so one maintained rule set can serve many repos:

```
$ ln -s ~/shared-claude-rules .claude/rules/shared
$ ln -s ~/company-standards/security.md .claude/rules/security.md
```

3. **Plugins.** The proper answer for a team. Ship rules and settings in a plugin distributed through an internal marketplace — Chapters 11 and 12.

Already using another agent's instruction file?

Claude Code reads `CLAUDE.md`, not `AGENTS.md`. If your repo already has one, do not duplicate — import it, and add Claude-specific notes below:

CLAUDE.md

```
@AGENTS.md

## Claude Code
Use plan mode for changes under `src/billing/`.
```

A symlink also works on macOS and Linux (`ln -s AGENTS.md CLAUDE.md`); on Windows it needs Administrator or Developer Mode, so prefer the import. `/import` can also bring another agent's configuration — instruction files, MCP servers, commands, subagents and skills — across in one go.

Auto memory — the instructions Claude writes itself

Claude Code has a second, complementary system. You write `CLAUDE.md`; Claude writes auto memory.

	CLAUDE.MD	AUTO MEMORY
Author	You	Claude
Contains	Instructions and rules	Learnings and patterns from your corrections
Scope	Project, user, or organisation	Per repository, shared across worktrees
Stored at	The paths in Section 6.3	`~/.claude/projects/<project>/memory/`
Loaded	In full, every session	`MEMORY.md` index only — first 200 lines or 25 KB
Machine-local?	No — committed files travel	Yes. Not shared across machines or cloud sessions.

Auto memory is on by default. Browse or edit it with `/memory` — it is plain markdown you can read, correct, or delete. Messages like "*Saved 2 memories*" in the interface mean Claude just wrote to it.

Troubleshooting

SYMPTOM	LIKELY CAUSE	FIX
Claude ignores an instruction	It is context, not enforcement — and vague instructions get followed least	Make it specific and checkable. If it *must* happen, use a hook instead.
Claude Code never sees the file	Wrong location, or the directory did not exist at session start	Run `/context` and look under **Memory files**. Restart if you created a new top-level directory.
Rules contradict each other	Two files, or account vs project, disagree	Claude may pick either arbitrarily. Reconcile them — do not rely on precedence.
Instructions get followed less as the file grows	Over ~200 lines	Split into `.claude/rules/`, path-scope what you can, and move procedures to Skills. Run `/doctor` for trim suggestions.
An instruction "disappeared" mid-session	It was given in conversation, or lives in a nested/path-scoped file that has not reloaded	Project-root `CLAUDE.md` survives compaction and is re-read. Nested files reload when Claude next touches a matching file. Put anything durable in a file.
Over-application — a tool fires constantly	Emphatic phrasing	Remove `CRITICAL:` / `MUST` / "if in doubt, use X". Dial the language down.
Other teams' rules leak in (monorepo)	Ancestor `CLAUDE.md` files load automatically	Add `claudeMdExcludes` globs to `.claude/settings.local.json`.
Auto memory saved something wrong	Claude inferred a preference from one interaction	`/memory`, open the auto-memory folder, edit or delete the file.

6.6 Examples

QA — project instructions for a test-design project

```
Source of truth for acceptance criteria is 02-acceptance-criteria.md.
Test case IDs use the prefix TC-R42- and increment from the highest existing.
Every AC gets at least one positive, one negative and one boundary case.
If an AC cannot be tested as written, do not invent an interpretation —
list it under "Untestable as written" with the specific ambiguity.
Output test cases as a markdown table matching our ADO import columns.
```

Product/BA — account-wide instructions

```
British English. Write for a mixed audience of engineers and stakeholders:
no unexplained jargon, no marketing tone.

For any analysis, separate what the data shows from what you infer, and
label the second as inference.

Never state a date, ticket number, name or figure that did not come from
material I supplied or a connected tool. Say "not in the provided material".

When I ask for a document, build it as an artifact.
```

Design — path-scoped rule in a front-end repo

`.claude/rules/components.md`

```
---
paths:
  - "src/components/**/*.{tsx,css}"
---

# Component rules

- Use design tokens from `src/tokens.css`. Never a raw hex value.
- Every interactive element needs a visible focus state and an accessible name.
- Support the `prefers-reduced-motion` media query on anything animated.
- New component => add a story in the same folder, plus the states table
  in its README.
```

Development — personal `~/.claude/CLAUDE.md`

```
# Personal preferences

- Show me the plan before large refactors. Small edits, just do them.
- When you finish, lead with the outcome in one sentence, then the detail.
- Do not add comments that restate what the next line does.
- If tests fail, show me the failure output. Do not describe it.
- I prefer `rg` over `grep` and `fd` over `find`; both are installed.
```

Enforcement escalation — when an instruction is not enough

Suppose "*always run the linter before committing*" keeps not happening. That is a signal you have reached the limit of instructions. The escalation path:

LEVEL	MECHANISM	GUARANTEE
1. Persuasion	`CLAUDE.md` instruction	Claude usually complies. No guarantee.
2. Procedure	A Skill for the commit workflow	Reliable when invoked, because the steps are explicit.
3. Enforcement	A `PreToolUse` / `PostToolUse` hook	Executes as a shell command at a fixed event, regardless of what Claude decides.
4. Prohibition	`permissions.deny` in managed settings	The action cannot happen at all.

6.7 Best practices

Recommended practices

- **Target under 200 lines per file.** Anthropic's own guidance. Longer files consume more context and reduce adherence — the opposite of what you wanted.

- **Add on the second occurrence, not the first.** One-off corrections belong in the conversation. Twice means it is a rule.
- **Structure with headers and bullets.** Claude scans structure the way a reader does.
- **Carry the reason for hard rules.** "Money is minor units — float rounding caused two incidents" is followed more reliably than the same rule bare.
- **Review for contradictions periodically.** Instruction files accumulate. Two rules that disagree are worse than neither.
- **Keep `CLAUDE.md` to what Claude cannot discover.** Delete directory layouts and dependency lists; keep pitfalls, rationale and conventions that differ from tool defaults.

Common mistakes

MISTAKE	WHY IT HURTS
Treating instructions as enforcement	They are context. Anything that must happen needs a hook or a permission rule.
Procedures in instructions	Paid for on every message, followed less reliably. Use a Skill.
Shouting (`CRITICAL` , `MUST` , `ALWAYS`)	Causes over-application. When everything is critical, nothing is.
Vague virtues ("be accurate", "be thorough")	Not checkable, so not actionable. Claude is already trying.
Committing `CLAUDE.local.md`	It is meant to be private. Add it to `.gitignore` .
Never pruning	Stale rules actively mislead. A rule about a deleted module is worse than no rule.
Assuming it loaded	Run `/context` . Assumption is the most common root cause of "Claude ignores my instructions."

Naming and organisation

- One topic per rules file, with a descriptive filename: `testing.md` , `api-design.md` , `security.md` . Subdirectories are discovered recursively, so `rules/frontend/` and `rules/backend/` both work.
- In a monorepo, keep rules next to the package they apply to and path-scope them.
- Use HTML comments (`<!-- maintainer note -->`) for notes to humans — block-level HTML comments are stripped before the content reaches Claude, so they cost no context.

Versioning and updates

- Committed `CLAUDE.md` and `.claude/rules/` are versioned by Git. Review changes to them in pull requests — they change behaviour for everyone on the team.
- Treat an instruction change like a config change: state what and why in the commit message.
- Re-audit after a model release. Instructions written to work around an older model's habits can become counterproductive; several patterns that were once necessary now cause over-application.

Security and team considerations

- **Never put secrets in an instruction file.** They load into every session and, if committed, into every clone.
- **Committed instructions are a supply-chain surface.** Anyone who can commit to a repo can change the rules Claude follows for everyone who opens it. Review them.
- **External imports prompt for approval** the first time, precisely because of the above.
- **Managed policy is the enterprise lever.** Organisation-wide behavioural guidance that individuals cannot exclude. Use it for compliance reminders and data-handling rules — and use managed *settings*, not instructions, for anything that must be technically enforced.

6.8 Official resources

Understanding Claude's personalization features — account instructions, project instructions and skills
`https://support.claude.com/en/articles/10185728-understanding-claude-s-personalization-features`

How Claude remembers your project — the complete `CLAUDE.md` , rules and auto-memory reference
`https://code.claude.com/docs/en/memory`

Settings reference — `claudeMdExcludes`, `autoMemoryEnabled`, managed settings, settings precedence
https://code.claude.com/docs/en/settings

Hooks — the enforcement escalation when an instruction is not enough
https://code.claude.com/docs/en/hooks-guide

Managing projects (project instructions field)
https://support.claude.com/en/articles/9519177-how-can-i-create-and-manage-projects

CHAPTER 7

Skills

Packaged know-how. Teach Claude a procedure once; reuse it forever.

START HERE | CLAUDE.AI | DESKTOP APP | COWORK | CLAUDE CODE | ALL PLANS

7.1 What is it?

A Skill is a reusable capability you teach Claude once. It is a folder containing a `SKILL.md` file — instructions for how to do a specific task — plus, optionally, scripts, templates and reference material. Claude loads it when the task calls for it, or you invoke it deliberately.

The problem it solves

Your team already has skills; they are just not written down anywhere Claude can read. A bug report has a shape: Title, Steps to reproduce, Expected vs Actual, Severity, Environment. A test case has a shape. A PR description has a shape. A UI spec has a shape. Everyone knows the shape and nobody has ever written it down as an instruction, so every person explains it to Claude slightly differently and gets slightly different output.

A Skill captures the shape once. It is the difference between "please format this like our test cases, you know, ID and title and steps…" typed forty times by four people, and `/test-case`.

> **YOU ALREADY HAVE SKILLS**
>
> Formats your team almost certainly already uses: **Bug report** — Title · Steps to reproduce · Expected vs Actual · Severity · Environment. **Weekly status** — What was done · What's blocked · What's next. **QA test case** — Test ID · Title · Preconditions · Steps · Expected result. **PR description** — What changed · Why · How to test. **UI spec** — Component · Variants · States · Spacing · Accessibility.
>
> Every one of those is a Skill waiting to be written. If you produce a format more than once a month, it qualifies.

How it works at a high level — progressive disclosure

This is the mechanism that makes Skills cheap, and it is worth understanding because it explains most of the authoring advice later in the chapter. Skills load in three stages:

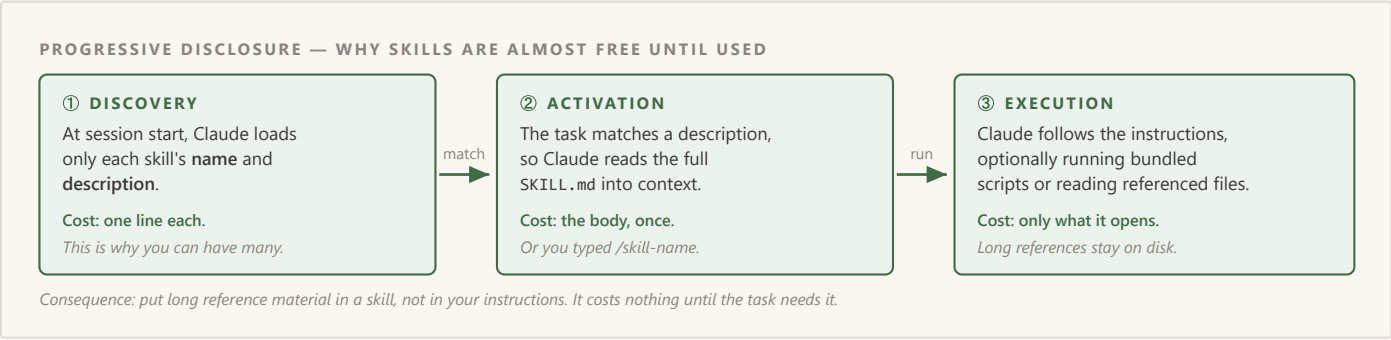


Figure 7.1 The three-stage load. A skill's description is always in context; its body is not.
Diagram — not a screenshot. Based on the Agent Skills open standard and Anthropic's skills documentation.

The anatomy of a skill

A skill is a folder

```
my-skill/
├── SKILL.md          # Required: frontmatter metadata + instructions
├── scripts/          # Optional: executable code Claude can run
```

```
├── references/       # Optional: long documentation, read on demand
├── assets/           # Optional: templates, boilerplate, example files
└── ...               # anything else the skill needs
```

Only `SKILL.md` is required, and it can be as short as five lines:

SKILL.md — the minimum viable skill

```
---
description: Drafts a bug report in our house format. Use when the user
  reports a defect or asks for a bug write-up.
---

Produce a bug report with exactly these sections, in this order:
Title · Steps to reproduce · Expected vs Actual · Severity · Environment.

Severity uses our scale: S1 blocker, S2 major, S3 minor, S4 cosmetic.
If the user has not given you the environment, ask for it — do not guess.
```

SKILLS ARE AN OPEN STANDARD

Claude Code skills follow the **Agent Skills** open format, originally developed by Anthropic and released as an open standard now adopted across many AI tools. A skill you write is portable. Claude Code adds extra frontmatter fields on top of the standard — which matters when you distribute a skill, as Section 7.5 explains.

What it is NOT

- **Not a prompt template you paste.** A skill lives in the platform and Claude can invoke it on its own. Pasting is what a skill replaces.
- **Not a tool or an integration.** A skill is instructions. If Claude needs to *reach* Azure DevOps, that is an MCP server or a connector (Chapters 8–9). A skill can tell Claude how to use those tools, but it does not provide them.
- **Not always loaded.** Unlike instructions, a skill's body is absent from context until needed. Do not put a rule that must always apply into a skill.
- **Not automatically shared.** A skill in `~/.claude/skills/` on your laptop exists only there. Sharing means committing it to a repo, enabling it on your claude.ai account, or shipping it in a plugin.
- **Not a guarantee.** Like instructions, a skill is context. A well-written skill is followed reliably; it is still not enforcement.

How it relates to the other blocks

BLOCK	RELATIONSHIP
Instructions	The key pairing. Facts and always-on rules → Instructions. Procedures → Skills. A skill's body loads on demand; instructions load always.
Projects	A project supplies the material; a skill supplies the method. In claude.ai, skills are account-wide rather than per-project, so one skill serves every project.
MCP / Connectors	A skill often orchestrates them: "read the story from Azure DevOps, then draft cases in our format, then push them to a test suite." That pairing is the workhorse combination.
Agents	A skill can be configured to run in a forked subagent context, and skills can be preloaded into subagents.
Plugins	The distribution answer. A plugin's `skills/` directory ships skills to a whole team, namespaced as `/plugin-name:skill-name`.
Workflows	Both run multi-step work, but a skill is instructions Claude follows in its own context; a workflow is a script a runtime executes across many agents. See Chapter 15.

7.2 When should I use it?

CREATE A SKILL WHEN…

- You keep pasting the **same instructions, checklist or multi-step procedure** into chat.
- You produce a **format more than once a month** — a template, a report shape, a document structure.
- A section of your instructions or `CLAUDE.md` has grown into a **procedure rather than a fact**.
- You have **long reference material** that is only relevant sometimes — a style guide, an API reference, a taxonomy.
- You want **one person's expertise** to become **everyone's default**.
- The task benefits from bundled **scripts or templates**, not just words.

Do not create a Skill when…

- The rule must apply to *every* message → that is an **Instruction**.
- It is reference *material* with no procedure → project knowledge or the repo.
- You need Claude to reach a system it currently cannot → **MCP / Connector** first.
- You have done the task once. Do it twice, then write the skill from what actually worked.
- It needs to fan out across hundreds of items with cross-checking → a **Workflow**.

Typical use cases

ROLE	SKILL	WHAT IT ENCODES
QA	`/test-case`	Your test-case template, ID convention, coverage expectations (positive, negative, boundary), and the rule about never inventing an AC reference.
QA	`/bug-report`	Title · Steps · Expected vs Actual · Severity · Environment, plus your severity scale.
Product / BA	`/user-story`	"As a … I want … so that …", the AC style your team uses, INVEST checks, and the definition of ready.
Product / BA	`/retro-summary`	How to turn a board export into your retro format: went well, didn't, actions with owners.
Design	`/handoff-spec`	Component · Variants · States · Spacing tokens · Accessibility, and where to say "not defined by the design system."
Development	`/pr-description`	What changed · Why · How to test, plus your linking convention for tickets.
Everyone	`/weekly-status`	Done · Blocked · Next, at the length your manager actually reads.

7.3 How do I set it up?

There are two paths, and they are genuinely different. Pick based on where you work.

PATH	USE WHEN	MECHANISM
A — claude.ai / Cowork	You work in chats and Cowork; you want the skill available in the browser, desktop app and mobile	Enable or upload skills to your claude.ai account via **Customize**
B — Claude Code	You work in a repository, terminal or VS Code	A folder on disk: `~/.claude/skills/` or the project's `.claude/skills/`

Path A — claude.ai, desktop and Cowork

1 Open the skills directory

Click **Customize** in your account › **Skills** › **+** › **Browse skills**

This opens the skills directory, where you can see what is available to you.

2 Understand the four kinds you will see

KIND	WHAT IT IS
Anthropic skills	Pre-built for document creation — Excel, Word, PowerPoint, PDF. These invoke automatically when relevant.
Custom skills	Created by you or your organisation for your own workflows.
Organization provisioned	Distributed by a Team or Enterprise Owner. These appear automatically in every member's skills list and can be set enabled or disabled by default.
Partner skills	Professionally built by platforms such as Notion, Figma and Atlassian, integrated with their MCP connectors.

3 Check the prerequisite

Skills require **code execution** to be enabled. If your skills appear greyed out, this is almost always why:

PLAN	WHERE TO ENABLE IT
Free, Pro, Max	**Settings** › **Capabilities** › turn on **Code execution and file creation**.
Team	Enabled at organisation level by default. You only need to toggle the individual skills you want in **Customize** › **Skills**.
Enterprise	An Owner must enable *both* **Code execution and file creation** *and* **Skills** in **Organization settings** › **Skills** before members see anything. Until then the whole area stays inert.

THIS IS NO LONGER THE SAME SWITCH AS ARTIFACTS

It used to be. **Code execution and file creation** once governed both Skills and Artifacts, and older documentation still describes it that way. Artifacts have since been split into their own switches under a **Visuals** section of the same Capabilities tab (Chapter 10, Section 10.3) — but **Skills still depends on code execution**. So: turning on the Visuals switches does not enable Skills, and turning *off* code execution disables Skills even though artifacts keep working.

4 Create a custom skill

Write a `SKILL.md` following the format in Section 7.1, package the skill *folder* as a ZIP, then:

Customize › **Skills** › **+** › **+ Create skill** › **Upload a skill**

The skill appears in your list and can be toggled on or off. Uploaded skills are **private to your account** unless you deliberately share them — see Section 7.5.

FRONTMATTER RESTRICTION ON UPLOAD

A skill uploaded to claude.ai may use only the six fields in the open standard: `name`, `description`, `license`, `compatibility`, `metadata`, `allowed-tools`. Any other field — including Claude Code favourites like `argument-hint` — causes the upload to *fail with a hard error*, not to be ignored:

```
Unexpected key(s) in SKILL.md frontmatter: argument-hint. Allowed
properties are: allowed-tools, compatibility, description, license,
metadata, name
```

5 Test it in a new conversation

Ask for something the description covers and confirm Claude applies the skill.

Path B — Claude Code

Here a skill is just a directory. Where you put it decides who can use it.

SCOPE	PATH	APPLIES TO
Enterprise	Deployed via managed settings	All users in your organisation
Personal	`~/.claude/skills/<skill-name>/SKILL.md`	All your projects, just you
Project	`.claude/skills/<skill-name>/SKILL.md`	This project — commit it and the team gets it
Plugin	`<plugin>/skills/<skill-name>/SKILL.md`	Wherever the plugin is enabled

Precedence: enterprise overrides personal, personal overrides project. A skill at any of these levels also overrides a bundled skill of the same name — so a `code-review` skill in your project replaces the built-in `/code-review`. Plugin skills are namespaced (`/plugin-name:skill-name`) so they never collide.

1 Create the directory

```
# personal — available in every project
$ mkdir -p ~/.claude/skills/test-case

# or project — committed, shared with the team
$ mkdir -p .claude/skills/test-case
```

The **directory name becomes the command you type**: this folder gives you `/test-case`.

2 Write SKILL.md

Two parts: YAML frontmatter between `---` markers telling Claude when to use the skill, and markdown instructions telling Claude what to do.

`.claude/skills/test-case/SKILL.md`

```
---
description: Drafts test cases in our house format from acceptance criteria.
  Use when the user asks for test cases, a test suite, or QA coverage for
  a story, an AC, or a feature.
argument-hint: "[AC number or story ID]"
---

# Test case drafting

## Format — use exactly these columns
| Test ID | Title | Preconditions | Steps | Expected result | AC ref |

- Test IDs use the prefix `TC-` followed by the release, e.g. `TC-R42-001`.
  Continue from the highest existing ID if the user supplies a list.
- One row per test case. Steps are numbered within the cell.
- `AC ref` cites the acceptance criterion. Never invent one — if you cannot
  find it in the provided material, write `UNVERIFIED` and list it at the end.

## Coverage — every AC gets all three
1. **Positive** — the happy path, as written.
2. **Negative** — invalid input, wrong state, missing permission.
3. **Boundary** — the limits: empty, maximum, zero, expiry, timeout.

## Finish with
- A short "Assumptions" list, if you had to make any.
- An "Untestable as written" list for any AC too vague to test, naming the
  specific ambiguity. Do not silently invent an interpretation.
```

3 Test both invocation paths

```
# explicit — you invoke it
> /test-case AC-14

# automatic — Claude matches the description
> write me QA coverage for the guest checkout story
```

Claude Code watches skill directories, so edits apply within the current session without a restart. If you created a top-level skills directory that did not exist when the session started, restart once so Claude can watch it.

4 Add supporting files as it grows

```
.claude/skills/test-case/
├── SKILL.md
├── references/
│   └── severity-scale.md      # read only when severity comes up
└── assets/
    └── ado-import-template.csv
```

Reference these from `SKILL.md`. Because of progressive disclosure, a long reference file costs nothing until Claude actually opens it — which is the whole reason to put it here rather than in `CLAUDE.md`.

Useful frontmatter fields (Claude Code)

All fields are optional; only `description` is genuinely recommended. The ones you will actually reach for:

FIELD	WHAT IT DOES
`description`	What the skill does *and when to use it*. This is how Claude decides to load it. Put the key use case first — the description is truncated at 1,536 characters in the skill listing.
`when_to_use`	Extra trigger phrases or example requests. Appended to the description and counts toward the same cap.
`argument-hint`	Autocomplete hint for expected arguments, e.g. `[issue-number]`.
`disable-model-invocation`	`true` stops Claude loading it automatically — it runs only when you type `/name`. Use for anything slow, expensive or destructive.
`user-invocable`	`false` hides it from the `/` menu. For background knowledge users should not invoke directly.
`allowed-tools`	Tools Claude may use without asking permission during the invoking turn. The grant clears on your next message.
`disallowed-tools`	Tools removed from the pool while the skill is active. Useful for autonomous skills that must never ask a question.
`paths`	Glob patterns limiting when the skill auto-activates — only when working with matching files.
`model` / `effort`	Override the model or reasoning effort while this skill is active.
`context: fork`	Run the skill in a forked subagent context instead of the main conversation. See Chapter 14.

Dynamic context injection — the trick worth knowing

A Claude Code skill body can run a shell command and inline its output *before* Claude reads the skill. This grounds the skill in current reality rather than what Claude can guess:

`~/.claude/skills/summarize-changes/SKILL.md`

```
---
description: Summarizes uncommitted changes and flags anything risky. Use
  when the user asks what changed, wants a commit message, or asks to
  review their diff.
```

```
---

## Current changes

!`git diff HEAD`

## Instructions

Summarize the changes above in two or three bullet points, then list any
risks you notice such as missing error handling, hardcoded values, or
tests that need updating. If the diff is empty, say there are no
uncommitted changes.
```

The `!`git diff HEAD`` line is replaced by the command's output before Claude sees the content. Note this is a Claude Code feature — it does *not* work in claude.ai chat or via the API.

7.4 How do I use it?

Two ways in

YOU INVOKE IT

```
/test-case AC-14 AC-15
```

Deliberate and predictable. Type `/` to see what is available. Use this when you know exactly which procedure you want.

CLAUDE INVOKES IT

```
Draft QA coverage for the
guest checkout story.
```

Claude matches your request against skill descriptions and loads the relevant one. This is why the `description` field matters more than anything else in the file.

What to expect

- Output in your format, without you specifying the format.
- In Claude Code, a visible indication that a skill was used.
- Claude following the skill's *edge-case handling* — the "if you cannot find it, write UNVERIFIED" rules are usually what separate a useful skill from a template.

How to verify it worked

1. **Check the shape.** Does the output have exactly the sections your skill specifies? A missing section means the instruction was ambiguous.
2. **Trigger an edge case deliberately.** Ask for test cases against an AC you did not supply. A good skill says "UNVERIFIED" or asks; a weak one invents an AC number. This single test tells you more than ten happy-path runs.
3. **Check auto-invocation separately.** Phrase a request in your own words without the slash command. If Claude does not reach for the skill, the description is the problem, not the body.
4. **In Claude Code:** confirm it is loaded at all.

```
> /context     # skills appear in the context breakdown
```

Common usage patterns

PATTERN	HOW IT WORKS
Learn the skill from your own history	The highest-yield approach: point Claude at 100+ test cases you have already written and ask it to infer your wording, step granularity and expected-result style, then write the skill from that. Far better than writing it from memory.
Skill + connector	The skill says *how*; the connector provides *reach*. "Read story 4821 from Azure DevOps, draft cases in our format, push them to the test suite."
Manual-only skills	Set `disable-model-invocation: true` for anything slow or destructive, so it never fires on its own.

PATTERN	HOW IT WORKS
Path-scoped skills	A `paths` field means the skill only auto-activates when Claude touches matching files — ideal in a monorepo.
Improve once, everyone benefits	Fix the skill, not the output. A committed project skill improves for the whole team on their next pull.

7.5 How do I manage it?

CREATE	**claude.ai:** Customize › Skills › +. **Claude Code:** create the directory and `SKILL.md`, or just ask Claude to write it for you.
VIEW	**claude.ai:** Customize › Skills. **Claude Code:** type `/` to list invocable skills; `/context` shows what is loaded.
EDIT	Edit `SKILL.md`. Claude Code detects changes live — no restart. (Changes to a skill folder's `hooks/`, `.mcp.json` or `agents/` need `/reload-plugins`.)
DISABLE	**claude.ai:** toggle it off in Customize › Skills. **Claude Code:** a `skillOverrides` entry of `"off"` in settings; `disableBundledSkills` turns off the built-ins; or move the folder out of the skills directory.
DELETE	**claude.ai:** click the … button beside the skill's toggle and choose **Delete**. The same menu offers **Try in chat**, **Download**, **Edit with Claude** and **Replace**. **Claude Code:** delete the directory.
DUPLICATE	Copy the folder and rename it — the new folder name is the new command. Genuinely a one-line operation, which is the nicest thing about the folder-based design. On claude.ai, **Download** from the … menu, edit, and re-upload.
SHARE	Commit to `.claude/skills/` for a project; upload to claude.ai for Cowork and cloud sessions; ship in a plugin for an organisation. On Team and Enterprise you can also share a skill directly — see below. See also the table further down.
VERSION	Git, for project and plugin skills. Put a changelog line at the top of `SKILL.md`; review changes in pull requests. On claude.ai, **Replace** swaps the contents in place and everyone you shared it with receives the update automatically.

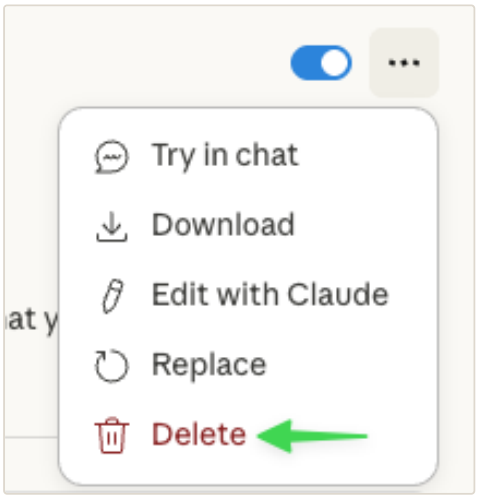


Figure 7.2 The whole per-skill lifecycle sits behind the … button next to the toggle. **Replace** is the one to know: it updates the skill in place, so anyone you shared it with gets the new version without re-installing.
Screenshot: Anthropic, Claude Help Centre — "Use skills in Claude."

Sharing a skill directly (Team and Enterprise)

Sharing is **off by default**, and this catches people out: an Owner must first enable **Share with organization**, **Skill sharing**, or both, under **Organization settings** › **Skills**. Until one of those is on, no **Share** button appears anywhere — so a member concluding "I can't share skills" may simply be waiting on an Owner.

Once enabled, open a skill you created and click **Share**, then choose:

- **Specific people** — enter names or emails. The skill lands in each recipient's list, greyed out until they enable it.
- **Entire organization** — published to your organisation's directory, where anyone can find and install it.

Shared skills are **view-only**: recipients can use them but not edit them, and if you update the skill later they get the new version automatically. Your own list is then split into three sections — **Personal skills**, **Shared skills** and **Organization skills**. Note that you cannot fully delete an organisation-shared skill from your list; only the person who shared it, or an Owner, can remove it.

THE TRAP THAT CATCHES EVERYONE: SKILLS IN COWORK, CLOUD SESSIONS AND ROUTINES

Cowork sessions and Claude Code *cloud* sessions — including every **Routine** run (Chapter 13) — do **not** read `~/.claude/skills/` on your machine. They load the skills enabled for your *claude.ai account*, synced at session start.

So a personal skill that works perfectly in your terminal will report "skill not found" when a routine invokes it, because each routine run starts as a fresh remote session. To make a personal skill available there:

- Upload it to claude.ai and enable it on your account (manage from **Customize** in the desktop app sidebar, or the skills settings on claude.ai); *or*
- For cloud sessions, commit it to the repository's `.claude/skills/` ; *or*
- Ship it in a plugin declared in the repository's `.claude/settings.json` — repo-declared plugins install at session start, whereas plugins enabled only in your user settings do not transfer.

Where to put a skill, by who needs it

AUDIENCE	PUT IT HERE	NOTES
Just me, everywhere (terminal)	`~/.claude/skills/`	Does not reach Cowork, cloud sessions or routines.
Just me, everywhere (all surfaces)	Upload to claude.ai	Six standard frontmatter fields only.
This project's team	`.claude/skills/`, committed	Reviewed in pull requests. The best default for team standards.
Several teams / whole organisation	A plugin in an internal marketplace	Versioned, auto-updating, one command to install. Chapters 11–12.
Whole organisation, mandatory	Organisation-provisioned skills (Team/Enterprise), or enterprise managed settings	Appears in every member's list; can be enabled or disabled by default.

Troubleshooting

SYMPTOM	LIKELY CAUSE	FIX
Claude never invokes it on its own	The `description` does not say *when* to use it	Rewrite it as "Does X. Use when the user asks for Y, mentions Z, or…". Put the key use case first.
Skill does not appear in `/` at all	Wrong directory, missing `SKILL.md`, `user-invocable: false`, or the skills directory did not exist at session start	Check the path against the table in 7.3. Restart once if you just created the top-level directory.
Upload to claude.ai fails	Non-standard frontmatter field	Reduce to the six allowed fields. The error names the offending key.
Works in terminal, "not found" in a routine	Personal skills are not synced to cloud sessions	Upload to claude.ai, commit to the repo, or ship in a repo-declared plugin. See the warning above.
Fires too often	Description too broad, or emphatic language	Narrow the description; remove "always"/"CRITICAL"; consider `disable-model-invocation: true`.

SYMPTOM	LIKELY CAUSE	FIX
Wrong variant runs in a monorepo	Nested skills with the same name	Both stay available: the project-root one answers `/deploy`, the nested one `/apps/web:deploy`. Use the qualified name to be explicit.
Skill body ignored, only format applied	The body is long, unstructured prose	Use headers and short imperative bullets. Structure is read the way a reader reads it.
Plugin skills missing after install	Not activated yet	Run `/reload-plugins`. Note its summary counts only `commands/`, so it can report `0 skills` even when skills did reload.

7.6 Examples

Example 1 — QA: build the skill from your own back catalogue

This is the single highest-value pattern in this chapter.

Prompt

```
Here are 120 test cases I've written over the past year (attached / in
project knowledge). Study them and infer my house style: how I word titles,
how granular my steps are, how I phrase expected results, how I handle
preconditions, and how I reference acceptance criteria.

Then write a SKILL.md that would reproduce that style on new work.
Be explicit about the things I do consistently, and flag anything where
I'm inconsistent so I can decide the rule.
```

Why this works: your real output encodes decisions you have never articulated. Asking Claude to extract the pattern produces a better skill than writing it from memory, and the "where you're inconsistent" list is a genuinely useful audit of your own work.

Example 2 — QA: bug report skill

.claude/skills/bug-report/SKILL.md

```
---
description: Writes a defect report in our house format. Use when the user
  describes something broken, reports a defect, pastes an error, or asks
  for a bug write-up.
---

Produce exactly these sections, in order, as level-2 markdown headings:

## Title
One line, imperative, specific. Pattern: `[Area] What fails, when`.
Good: `[Checkout] Payment fails when saved card has expired`.
Bad: `Checkout broken`.

## Steps to reproduce
Numbered, starting from a known state. Each step is one action.
Include the exact data used — not "enter a card", but which card.

## Expected vs Actual
Two labelled lines. State the expectation as a rule, not an opinion.

## Severity
S1 blocker · S2 major · S3 minor · S4 cosmetic.
Justify the choice in one clause. If you are unsure between two, pick the
higher and say why.

## Environment
```

```
Browser + version, OS, device, build number, environment (dev/UAT/prod).
If the user has not supplied these, list what is missing and ask —
do not guess or write "N/A".
```

Example 3 — Product/BA: user story skill

`.claude/skills/user-story/SKILL.md`

```
---
description: Drafts a user story with acceptance criteria in our format.
  Use when the user asks for a story, a ticket, a backlog item, or ACs.
argument-hint: "[feature or need]"
---

## Story
`As a , I want , so that .`
The role must be a real user type from our personas, not "user".
The outcome must be a benefit, not a restatement of the capability.

## Acceptance criteria
Given / When / Then. Numbered AC-1, AC-2, …
Each AC must be independently testable by someone who did not write it.
Cover the negative path, not only the success path.

## Out of scope
Explicitly list what this story does not cover. This is the section people
skip and the one that prevents scope arguments.

## Open questions
Anything you had to assume. Name the person or role who should answer it.

Finally, run an INVEST check and report any criterion the story fails.
```

Example 4 — Design: handoff spec skill

`.claude/skills/handoff-spec/SKILL.md`

```
---
description: Writes a component handoff spec for engineering. Use when the
  user asks for a handoff, a component spec, or design documentation for
  a UI component.
paths:
  - "src/components/**"
  - "design/**"
---

Sections, in order: Component · Variants · States · Spacing · Accessibility.

- **States** must cover: default, hover, focus-visible, active, disabled,
  loading, error, and empty where applicable. Say "not applicable" rather
  than omitting one.
- **Spacing** cites design-system tokens by name. If the design uses a value
  the token set does not define, do not invent a token name — write
  `NOT IN DESIGN SYSTEM: 14px gap` so it gets resolved before build.
- **Accessibility** covers: accessible name, role, keyboard interaction,
  focus order, contrast ratio of text and of non-text UI, and
  `prefers-reduced-motion` behaviour for anything animated.

Close with an "Open with design" list for every unresolved decision.
```

Example 5 — Development: PR description skill

`.claude/skills/pr-description/SKILL.md`

```
---
description: Writes a pull request description from the current branch's
  changes. Use when the user asks for a PR description, a PR body, or to
  open a pull request.
---

## Current diff against the default branch

!`git diff $(git merge-base HEAD origin/main)...HEAD --stat`

## Instructions

Write three sections and nothing else:

**What changed** — plain prose, not a file list. The reader can see the files.
**Why** — the problem this solves. Link the ticket as `[ABC-123]`.
**How to test** — exact commands and the observable result, so a reviewer
can verify without asking you.

Do not include a "Changes" bullet list that restates the diff.
Do not claim tests pass unless you ran them; if you did not, say so.
```

Example 6 — the Azure DevOps chain from the live demo

Skill plus connector, which is where this becomes genuinely useful:

Prompt (with the Azure DevOps MCP connected — Chapter 8)

```
Read user story 4821 from Azure DevOps. Draft test cases for every
acceptance criterion using /test-case. Then create a test suite under the
Release 4.2 test plan and add the cases to it, linked to story 4821.

Before you push anything, show me the cases and wait for my confirmation.
```

Note the last line. A skill that writes is fine; a skill that *pushes* should pause. See Section 7.7.

7.7 Best practices

Recommended practices

- **Spend your effort on the description.** The body determines quality when the skill runs; the description determines *whether it runs*. State what it does and when to use it, key use case first.
- **Write it after doing the task twice.** The second run tells you which parts are actually the rule and which were incidental.
- **Encode the edge cases, not just the format.** "Never invent an AC reference — write UNVERIFIED" is worth more than the column list. Formats are easy; judgement is what you are really capturing.
- **Match specificity to fragility.** Use prose heuristics for open-ended judgement, and exact commands for narrow steps where only one sequence is safe.
- **Keep `SKILL.md` readable in one sitting.** Move long reference material to `references/`, where it costs nothing until opened.
- **Set `disable-model-invocation: true` on anything destructive.** A skill that pushes to Azure DevOps or opens a PR should fire when you say so, not when Claude infers it.
- **Let Claude write the first draft.** "Write me a skill that does X in our format, based on these examples" is a better starting point than a blank file — then edit down.

Common mistakes

MISTAKE	WHY IT HURTS
A description that only says what, not when	Claude cannot tell when to load it, so it never fires automatically and the skill becomes a slash command you forget exists.
Step-by-step scripts for judgement tasks	Over-prescriptive skills reduce output quality on current models. State the goal and the constraints; let Claude plan the route.
Encoding one session's stumble as a permanent rule	The next session steps around a pothole that is not there. Before adding a rule, ask whether it would have helped most sessions or just that one.
Hardcoding paths, flags and version numbers	Skills rot factually as code ships. Encode architecture and workflow; verify volatile specifics.
A skill that duplicates `CLAUDE.md`	Wasted context and a second place to update. Pick one home per rule.
Non-standard frontmatter on a skill you will upload	Hard upload failure, not a warning.
Assuming a personal skill reaches routines	It does not. This is the single most common Skills surprise.
Shouting in the body	Same as instructions — causes over-application. Say it once, plainly.

Naming conventions

- **Verb-noun, lowercase, hyphenated:** `test-case`, `bug-report`, `handoff-spec`, `retro-summary`, `pr-description`. The folder name is what people type, so make it what they would guess.
- **Name the output, not the technology.** `/weekly-status`, not `/claude-status-helper`.
- **Avoid collisions with bundled skills** unless you intend to replace one. A project `code-review` skill silently overrides the built-in `/code-review`.
- **Plugin skills are namespaced for you** — `/qa-toolkit:test-case`. Keep the plugin name short, because users type the whole thing.

Versioning and updates

- Project and plugin skills are versioned by Git. Review skill changes in pull requests — a skill edit changes everyone's output.
- Add a one-line changelog at the top of `SKILL.md`: what changed and why.
- For plugin distribution, bump `version` in `plugin.json` so installed users receive the update (Chapter 12).
- Re-audit skills after a model release. Scaffolding written for an older model — enumerated steps, emphatic warnings — is often now counterproductive. A/B the skill with the older scaffolding removed.

Security considerations

A SKILL IS INSTRUCTIONS CLAUDE WILL FOLLOW

Treat skills as part of your trust boundary. Specific things to know:

- **Anyone who can commit can change a skill.** A merged external pull request that adds or edits `.claude/skills/` changes the instructions Claude follows — in a session that may have `bash` and web access. Review skill changes as carefully as code.
- **Repository skills load automatically in cloud sessions.** When a session mounts a repository, skills in its root `.claude/skills` are discovered at session start with no review step. Only mount repositories you trust, and audit `.claude/skills/` before mounting one with external contributors.
- **Never put credentials in a skill.** They load into context and, if committed, into every clone.
- **Scope tool grants narrowly.** `allowed-tools` pre-approves tools without asking. Grant the minimum, and prefer `disallowed-tools` to fence off anything an autonomous skill must never touch.
- **Enterprise skill scanning** is available to check uploaded skills and plugins for malicious content. Ask your administrator whether it is on.

Team and organisational considerations

- **Skills are the best mechanism for house standards** — better than a project, because they apply everywhere rather than inside one workspace, and better than instructions, because they cost nothing until used.
- **Start a shared library early.** One repository of committed skills that teams pull from beats twelve people each writing `/test-case`.
- **Assign an owner per skill.** Put the owner in a comment at the top. Unowned skills drift.
- **Organisation-provisioned skills** (Team/Enterprise) appear automatically in every member's list and can default to enabled or disabled — the right lever for a genuinely mandatory format.
- **Publish the catalogue.** People cannot use a skill they do not know exists. A one-page list of available skills and what they do pays for itself immediately.

7.8 Official resources

What are skills? — concept, the four kinds, plan availability, admin controls
`https://support.claude.com/en/articles/12512176-what-are-skills`

How to create custom skills (claude.ai)
`https://support.claude.com/en/articles/12512198-how-to-create-custom-skills`

Use skills in Claude — the enable prerequisites per plan, the upload path, sharing, and the … menu
`https://support.claude.com/en/articles/12512180-use-skills-in-claude`

Browse skills, connectors, and plugins in one directory — the unified directory and the "Your organization" tab
`https://support.claude.com/en/articles/14328846-browse-skills-connectors-and-plugins-in-one-directory`

Provision and manage skills for your organization — for Team/Enterprise Owners: org-wide provisioning and the sharing toggles
`https://support.claude.com/en/articles/13119606-provision-and-manage-skills-for-your-organization`

Extend Claude with skills — the complete Claude Code reference: locations, every frontmatter field, dynamic context injection, Cowork/cloud behaviour
`https://code.claude.com/docs/en/skills`

Agent Skills — the Claude Developer Platform view (API and Managed Agents)
`https://platform.claude.com/docs/en/agents-and-tools/agent-skills`

Agent Skills open standard — the portable format specification
`https://agentskills.io`

anthropics/skills — example skills and the `package_skill.py` packaging script
`https://github.com/anthropics/skills`

Commands reference — the bundled skills that ship with Claude Code
`https://code.claude.com/docs/en/commands`

CHAPTER 8

MCP Servers

A USB-C port for AI. One protocol; any tool, any model.

NEEDS SETUP | CLAUDE CODE | CLAUDE.AI (AS CUSTOM CONNECTORS) | DESKTOP APP

FREE: 1 CUSTOM CONNECTOR · PRO / MAX / TEAM / ENTERPRISE

8.1 What is it?

The **Model Context Protocol** (MCP) is an open standard for how an AI application connects to tools, data sources and systems. An **MCP server** is a program that exposes one of your systems through that standard, so Claude can read from it and act in it.

> **THE ANALOGY WORTH KEEPING**
> Think of MCP as a **USB-C port for AI**. Just as USB-C standardised how any device connects to any charger, MCP standardises how any AI connects to any tool, data source or system. One connector standard: build it once, and every AI can use it — and it can reach any tool.

The problem it solves

Without MCP, Claude only knows what is in the conversation. Your work does not live in the conversation; it lives in Azure DevOps, in GitHub, in Figma, in a database, in an internal API that has no public documentation. So you become a copy-paste courier: export the board, paste it in, read the answer, retype it back into the tool. Every round trip loses fidelity and takes time, and you can only carry what fits in a paste.

An MCP server removes the courier. Claude reads work item 4821 directly, and writes the acceptance criteria back.

A protocol is just agreed rules

Comparing MCP to REST is the fastest way to demystify the whole thing. You already work with protocols every day:

ROLE	A PROTOCOL YOU ALREADY FOLLOW
QA	Test case: Title · Preconditions · Steps · Expected result per step
Development	REST: nouns not verbs (`/customers` , not `/getCustomers`); the HTTP method *is* the action
Product / BA	User story: As a [user] I want [goal] so that [reason]
Design	Handoff: Component · Variants · States · Spacing tokens
Everyone	Commit message: `JIRA-123: verb + what changed`

A useful exercise: name the tools your own systems would expose. A protocol is agreed rules so two sides communicate predictably. MCP is exactly this — between AI and your tools.

How it works: three participants, three primitives

An MCP setup has three participants. The **host** is the AI application (Claude). The host creates one **client** per server. The **server** exposes your system. They exchange JSON-RPC messages.

Every MCP server exposes up to three kinds of thing, and this is the part worth memorising — learn the shape once and every MCP server you ever meet works the same way:

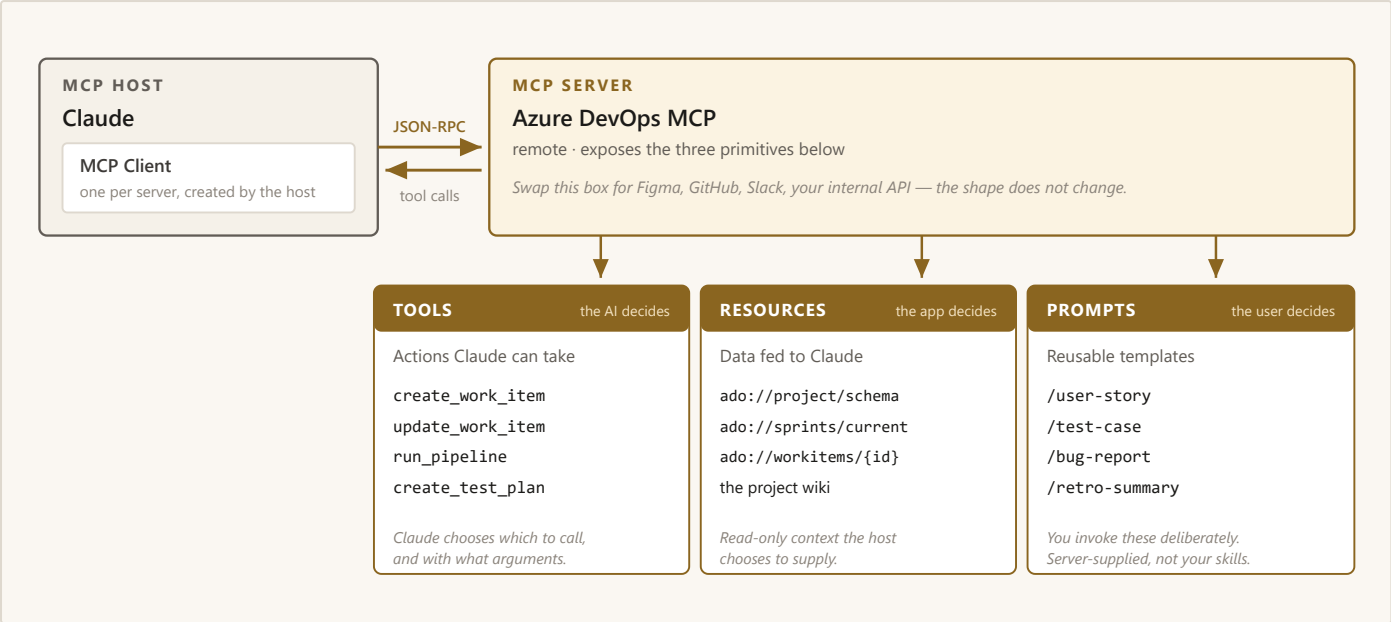


Figure 8.1 Inside an MCP server, using an Azure DevOps integration as the example. Three participants pass messages; the server exposes three kinds of thing, each controlled by a different party.
Diagram — not a screenshot. Based on the Model Context Protocol specification.

What it is NOT

- **Not an Anthropic product.** MCP is an open protocol. Anthropic created it and released it as an open standard; it is implemented by many AI applications, not just Claude.
- **Not a Connector, exactly.** A connector is a *ready-made, reviewed* integration you switch on with a click. An MCP server is the general mechanism. Most connectors are remote MCP servers underneath. See Chapter 9 for the full comparison — the short version is: **connectors for what is on the list, MCP for everything else**.
- **Not instructions.** An MCP server gives Claude *reach*, not know-how. Pair it with a Skill to get both.
- **Not a security boundary you get for free.** A connected server can read and write real data. Section 8.7 is not optional reading.
- **Not something you usually have to build.** Hundreds of servers already exist, and the official Claude Code marketplace ships pre-configured plugins for GitHub, GitLab, Atlassian (Jira/Confluence), Asana, Linear, Notion, Figma, Slack, Sentry, Vercel, Firebase and Supabase.

How it relates to the other blocks

BLOCK	RELATIONSHIP
Connectors	Two names for closely-related things at different levels of convenience. Chapter 9 draws the line precisely.
Skills	The natural pairing: the skill knows the format, the server provides the data. This is the pairing that does most of the real work.
Plugins	A plugin can bundle an MCP server configuration, so installing one plugin connects the tool *and* ships the skills and agents that use it.
Routines	A routine's connectors are the claude.ai account connectors, *not* servers you added locally with `claude mcp add`. This trips people up — see the warning in 8.5.
Agents	A subagent can be restricted to a subset of tools, including MCP tools — a good way to contain what an autonomous worker can touch.
Artifacts	An artifact can call MCP connectors when someone opens it, which is how a published status board fetches fresh data.

8.2 When should I use it?

CONNECT AN MCP SERVER WHEN…

- You find yourself **copying data into chat** from another tool — an issue tracker, a monitoring dashboard, a database.
- You want Claude to **act**, not just advise: create the work item, open the PR, update the AC.
- The system you need is **not in the connectors directory** — an internal API, a legacy system, custom tooling, a private database.
- You need a **local** capability the cloud cannot provide — reading a file on your machine, running a local tool.
- You want the integration **committed and shared** so the whole team gets it from the repo.

Do not reach for a custom MCP server when…

- The tool is already in the **connectors directory** or the official plugin marketplace — use that, it is reviewed and one click.
- You only need the data once. Paste it.
- The problem is that Claude does not know your *format* — that is a Skill.
- You cannot vouch for the server's source. An MCP server can read and modify your data.

What it unlocks, concretely

- **Implement from the tracker:** "Add the feature described in JIRA issue ENG-4521 and create a PR on GitHub."
- **Analyse monitoring data:** "Check Sentry for the usage of the feature described in ENG-4521."
- **Query databases:** "Find emails of 10 users who used feature ENG-4521, from our PostgreSQL database."
- **Integrate designs:** "Update our email template based on the new Figma designs posted in Slack."
- **Automate workflows:** "Create Gmail drafts inviting these 10 users to a feedback session."

8.3 How do I set it up?

Path A — claude.ai, desktop and Cowork: as a custom connector

On the chat surfaces, a remote MCP server is added as a **custom connector**. Claude connects to it from Anthropic's cloud infrastructure — not from your device — and the connection then works across every Claude client: claude.ai, the desktop app, Cowork and mobile.

1. **Open connector settings**

   Go to **Customize** › **Connectors**, or in a chat click **+** › hover **Connectors** › **Manage connectors**

2. **Add a custom connector**

   Click **+** next to Connectors and choose to add a custom connector. Enter your remote MCP server's **URL**.

3. **Add OAuth credentials if the server needs them**

   Click **Advanced settings** to supply an **OAuth Client ID** and **OAuth Client Secret**.

4. **Enable it in a conversation**

   Connectors are enabled *per conversation*. In the chat, click **+** in the lower left, hover **Connectors**, and toggle the ones this conversation should use.

READ THIS BEFORE ADDING A CUSTOM CONNECTOR

Custom connectors let you connect Claude to services that have **not been verified by Anthropic**, granting it the ability to access and potentially modify data in those services. Only connect to remote MCP servers you trust.

Path B — Claude Code: `claude mcp add`

In Claude Code you configure servers from the command line. Three transports matter.

Remote server over HTTP — the common case

```
$ claude mcp add --transport http <name> <url>

# a real one
$ claude mcp add --transport http notion https://mcp.notion.com/mcp

# with an auth header
$ claude mcp add --transport http github https://api.githubcopilot.com/mcp/ \
    --header "Authorization: Bearer YOUR_GITHUB_PAT"
```

`--transport` and `--header` also accept the short forms `-t` and `-H`. In JSON configuration, `streamable-http` is accepted as an alias for `http`, so configs copied from a vendor's documentation work unchanged.

Remote server over SSE — where a service only offers that

```
$ claude mcp add --transport sse asana https://mcp.asana.com/sse
```

Local server over stdio — runs a process on your machine

```
$ claude mcp add [options] <name> -- <command> [args...]

# the -- separates Claude's own flags from the server's command
$ claude mcp add --env AIRTABLE_API_KEY=YOUR_KEY --transport stdio airtable \
    -- npx -y airtable-mcp-server
```

> **THE `--` IS LOAD-BEARING**
>
> For stdio servers, everything after `--` is passed to the server untouched. Without it, Claude Code tries to interpret the server's own arguments as its flags. So `claude mcp add --env KEY=value -t stdio myserver -- python server.py --port 8080` runs `python server.py --port 8080` with `KEY=value` in its environment.

Choose the scope — the decision that matters most

SCOPE	AVAILABLE IN	SHARED?	STORED IN
`local` default	This project, just you	**No**	Your own machine-local configuration
`project`	This project, everyone	**Yes, via version control**	`.mcp.json` in the project root
`user`	All your projects	**No**	Your user configuration

```
# just me, this repo (the default)
$ claude mcp add --transport http stripe --scope local https://mcp.stripe.com

# the whole team, committed to the repo
$ claude mcp add --transport http shared-server --scope project https://example.com/mcp

# me, everywhere
$ claude mcp add --transport http hubspot --scope user https://mcp.hubspot.com/anthropic
```

A project-scoped add creates or updates `.mcp.json`:

`.mcp.json (commit this)`

```
{
  "mcpServers": {
    "shared-server": {
      "type": "http",
      "url": "https://example.com/mcp"
```

```
    }
  }
}
```

For security, Claude Code prompts for approval in interactive sessions before using project-scoped servers from `.mcp.json`. To reset those choices, run `claude mcp reset-project-choices`.

Authenticate

Many servers use OAuth rather than a header token. Authenticate from inside a session:

```
> /mcp
```

This opens the MCP management panel, where you can enable or disable servers, reconnect, and complete OAuth authentication. In the VS Code extension, `/mcp` in the chat panel does the same.

Verify the connection

```
$ claude mcp list
# ✓ Connected            – working
# ! Needs authentication – run /mcp in a session
# ✗ Failed to connect    – with the HTTP status and the server's error text
# ⏸ Pending approval     – a project .mcp.json server awaiting your OK

$ claude mcp get notion       # detail for one server, incl. an Issue: line
$ claude mcp remove notion    # delete it
```

> **`CLAUDE MCP ADD` DOES NOT VALIDATE CREDENTIALS**
> The command saves the configuration without testing it, so a placeholder token is accepted at add time and the server simply fails later. Always confirm with `claude mcp list` or `/mcp` that the server shows **Connected** before you rely on it.

8.4 How do I use it?

How to start

You do not call tools directly. You describe the outcome in plain language and Claude chooses which tools to call. Once a server is connected, simply reference the system:

```
> Read user story 4821 from Azure DevOps and summarise its acceptance criteria.
> Create a bug in the Release 4.2 area path for the defect we just discussed.
> Which items in the current sprint have no acceptance criteria at all?
```

How to structure requests that involve real systems

A pattern that avoids the two common failure modes — Claude guessing at identifiers, and Claude writing before you have checked:

Prompt structure

```
[Read step]     Read story 4821 and the ACs linked to it.
[Think step]    Draft test cases for each AC in our format.
[Confirm step]  Show me the cases. Do not create anything yet.
[Write step]    (after review) Now create them under the R4.2 test plan,
                linked to 4821.
```

Splitting read from write is the single most useful habit in this chapter. It costs one extra turn and prevents the class of mistake that is annoying to undo.

What to expect

- Claude naming the tools it is calling as it works.
- A permission prompt before actions, depending on your permission mode and configuration.
- Real identifiers — actual work-item numbers, actual field names — because it is reading them rather than inferring them.
- Genuine failures surfaced as failures: a 403 from a server you lack permission on will say so.

How to verify it worked

1. **Check the tool status first.** `claude mcp list` or `/mcp`. Most "Claude can't see my Azure DevOps" reports are an unauthenticated server.
2. **Ask for something only the live system knows.** "What is the current state field of work item 4821?" — not something Claude could plausibly invent.
3. **Check the system itself.** After a write, open Azure DevOps and look. Do this the first few times with any new server; you are verifying the integration, not Claude.
4. **Watch for silent tool absence.** If Claude answers from general knowledge instead of calling a tool, the tool may not be loaded — especially in a session where a large tool set is deferred behind tool search.

Large tool sets: tool search

Every connected server's tool definitions consume context on every request. With many servers this adds up. Claude Code can defer tool loading so Claude searches for the tools it needs rather than holding all schemas in context — useful once you are past a few dozen tools. Note that at least one tool must remain non-deferred, and the search tool itself cannot be deferred.

8.5 How do I manage it?

ADD	**Claude Code:** `claude mcp add -t http|sse|stdio <name> <url or -- cmd> [--scope local|project|user]`, or `claude mcp add-json` for full JSON control. **claude.ai:** Customize › Connectors › + › custom connector, with the server URL.
VIEW	`claude mcp list` (with health status), `claude mcp get <name>`, or the `/mcp` panel. claude.ai: Customize › Connectors.
EDIT	Edit `.mcp.json` for project scope; re-run `claude mcp add` to overwrite; or change the connector's settings in claude.ai.
AUTHENTICATE	`/mcp` in a session handles OAuth, reconnection and per-server auth state.
DISABLE	Toggle the server off in the `/mcp` panel, or use the `disabledMcpServers` / `enabledMcpServers` settings. In claude.ai, toggle the connector off per conversation, or disconnect it entirely.
APPROVE	Project `.mcp.json` servers need interactive approval. Reset with `claude mcp reset-project-choices`; control in bulk with `enableAllProjectMcpServers` / `enabledMcpjsonServers`.
REMOVE	`claude mcp remove <name>`, or delete the entry from `.mcp.json`. In claude.ai, disconnect the connector.
REUSE	Commit `.mcp.json` so the team inherits it, or bundle the server in a plugin (Chapter 12) to ship it with the skills that use it.

LOCALLY-ADDED SERVERS DO NOT REACH ROUTINES OR CLOUD SESSIONS

A routine's **Connectors** list is your *claude.ai account* connectors. MCP servers you added locally with `claude mcp add` live on your machine, so they do **not** appear there. To use one in a routine, either add it as a connector at `claude.ai/customize/connectors`, or declare it in a committed `.mcp.json` so it travels with the cloned repository.

Troubleshooting

SYMPTOM	LIKELY CAUSE	FIX
`! Needs authentication`	OAuth not completed	Run `/mcp` in a session and authenticate. Header-based auth needs a valid token in the config, not a placeholder.
`✗ Failed to connect`	Bad URL, bad credentials, network policy	`claude mcp get <name>` shows an `Issue:` line with the HTTP status and the server's error text. Credential-like text is redacted.
`⏸ Pending approval`	A project `.mcp.json` server you have not approved	Run `claude` interactively and approve. A cloned repo cannot approve its own servers — committed `enableAllProjectMcpServers` is ignored in an untrusted folder.
Server missing from `claude mcp list`	Reserved name, or a WebSocket server	Names `workspace`, `claude-in-chrome`, `computer-use`, `Claude Preview` and `Claude Browser` are reserved. WebSocket servers do not appear in `list` — use `claude mcp get` or `/mcp`.
Auth fails with a token that works elsewhere	**MCP auth tokens are not REST API tokens**	Hosted MCP servers typically require OAuth bearer tokens, not the service's native API key. A Notion `ntn_` integration token authenticates against Notion's REST API and will *not* work as an MCP credential.
Whitespace warning on a config value	A pasted token carried a trailing newline	Claude Code names the affected field (e.g. `headers.Authorization`) without echoing the value, and does *not* trim it. Edit the config.
Server shows `not configured`	Empty `url` in the entry	Intentional for plugin placeholders. Set the `url` to connect it.
Works locally, fails in a routine	Locally-scoped server	See the warning above — add it as an account connector or commit `.mcp.json`.
Tool call returns a huge blob	Large tool output	Output over ~100,000 characters is offloaded to a file in the sandbox automatically; Claude gets a preview plus the path and can read the rest.
Context feels tight with many servers	Tool definitions load per request	Disconnect unused servers; enable tool search to defer schemas.

8.6 Examples

Example 1 — QA: the live-demo chain (Azure DevOps)

Setup

```
# connect the Azure DevOps MCP for the whole team
$ claude mcp add --transport http ado <your-ado-mcp-url> --scope project
$ claude mcp list          # confirm ✓ Connected
```

Then, in a session

```
Read user story 4821 and every acceptance criterion linked to it.
Draft test cases with /test-case.
Show me the table and wait — do not create anything in ADO yet.

(after review)
Create these under the "Release 4.2" test plan as a new suite named
"4821 — Guest checkout", and link every case to story 4821.
Report the IDs you created.
```

Example 2 — Product/BA: sprint hygiene audit

```
Using the ADO connector, list every work item in the current sprint that
has any of these problems, as a table with the item ID and a one-line reason:

  - no acceptance criteria
  - assigned to nobody but in an active state
  - no estimate
  - a description under 20 words
  - unchanged for more than 10 days while "In Progress"

Do not change anything. This is a read-only audit.
```

Example 3 — Design: Figma to implementation

```
Read the Pagination component from the Figma file linked in the story,
list its variants and states, and compare them against our implementation
in src/components/Pagination/. Report:

  - states present in Figma but missing in code
  - states in code that Figma does not define
  - any spacing value in code that is not a design token

Table format. Do not edit any files.
```

Example 4 — Development: implement from the tracker

```
Read JIRA issue ENG-4521. Implement it in this repo following our
conventions in CLAUDE.md. Run the tests. Then open a PR with
/pr-description and link the issue.

If the issue's acceptance criteria are ambiguous, stop and ask me
rather than choosing an interpretation.
```

Example 5 — Reporting: the analysis the demo ended on

```
Using the ADO connector, pull every work item I created in the last
six months. Analyse: volume by month, split by type (bug / test case /
story / task), and the change in my mix over the period.

Build it as an artifact with a chart per view and two sentences of
commentary under each. Flag anything that looks like a data quality
problem rather than a real trend.
```

8.7 Best practices

Recommended practices

- **Prefer a reviewed connector or official plugin** over a hand-added server whenever one exists. Less to maintain, and someone else vetted it.
- **Start read-only.** Where a server offers scoped credentials, connect with read access first, use it for a week, then widen. Most of the value is in reading.
- **Split read from write in your prompts.** Confirm before the write step. See 8.4.
- **Commit `.mcp.json` for team servers.** One person configures; everyone inherits; the configuration is reviewed in a pull request like any other change.
- **Name servers for the system, lowercase and short:** `ado`, `github`, `figma`, `sentry`. You will type these.
- **Connect only what the task needs.** Every server costs context and widens the blast radius.

- **Set a per-server timeout** where a server is slow: add `"timeout": 600000` (milliseconds) to that server's `.mcp.json` entry rather than raising the global setting.

Common mistakes

MISTAKE	WHY IT HURTS
Assuming an add succeeded because the command did	`claude mcp add` writes config without validating. Always check `claude mcp list`.
Using a REST API key as an MCP credential	Different auth systems. Hosted MCP servers usually want OAuth.
Adding at `local` scope then expecting the team to have it	`local` is you-only. Use `--scope project` and commit.
Expecting local servers to work in routines	They do not. Account connectors or committed `.mcp.json`.
Granting write access on day one	An unfamiliar server with write scope on your live tracker is an avoidable risk.
Connecting everything permanently	Context cost on every request, and every connected tool is reachable by anything in the session.
Putting secrets in a committed `.mcp.json`	It goes to every clone and every fork. Use environment variables or OAuth.
Trusting an unvetted server URL	It can read and modify your data, and its tool descriptions are instructions Claude reads.

Security considerations

THE THINGS TO GENUINELY WORRY ABOUT

- **An MCP server is code you are trusting.** Anthropic does not control or verify third-party servers. Only connect to servers you or your organisation can vouch for.
- **Write access is real.** A connected server with write scope can create, modify and delete real records. Scope credentials to the minimum the task needs.
- **Tool descriptions are model-visible text.** A malicious server can attempt to influence Claude through its tool descriptions and results. This is a genuine reason to care about provenance, not a theoretical one.
- **Prefer OAuth over long-lived tokens** where the server supports it, and never paste a token into a file that gets committed.
- **Project-scoped servers require approval by design.** Do not blanket-enable `enableAllProjectMcpServers` in a repository with external contributors.
- **Enterprise controls exist.** Enterprise plans can restrict connectors to verified domains and limit action permissions organisation-wide (read-only, write, delete). Ask your administrator what is configured before assuming you can add anything.

Team and organisational considerations

- **Decide who may add custom connectors.** This is an organisational policy question, and it is better answered before someone connects a production database.
- **Standardise names across the org.** If one team calls it `ado` and another `azuredevops`, shared skills and prompts stop being portable.
- **Ship servers with the skills that use them.** A plugin containing the Azure DevOps MCP configuration *and* `/test-case` *and* `/bug-report` is one install instead of three setup steps and a wiki page.
- **Document the approved list.** A short internal page of "these servers are approved, here is how to connect them" prevents both duplication and improvisation.

8.8 Official resources

Connect Claude Code to tools via MCP — the full reference: transports, scopes, every command, tool search, troubleshooting
`https://code.claude.com/docs/en/mcp`

MCP quickstart — step-by-step for your first server

https://code.claude.com/docs/en/mcp-quickstart

Get started with custom connectors using remote MCP (claude.ai)
https://support.claude.com/en/articles/11175166-get-started-with-custom-connectors-using-remote-mcp

Build custom connectors via remote MCP servers
https://support.claude.com/en/articles/11503834-build-custom-connectors-via-remote-mcp-servers

Model Context Protocol — the open specification and server ecosystem
https://modelcontextprotocol.io/docs/getting-started/intro

MCP connector on the Claude Developer Platform (for engineers building applications)
https://platform.claude.com/docs/en/agents-and-tools/mcp-connector

Anthropic Directory — reviewed connectors, all usable from Claude Code with `claude mcp add`
https://claude.ai/directory

CHAPTER 9

Connectors

Ready-made, one-click reach into the tools you already use.

START HERE | CLAUDE.AI | DESKTOP APP | COWORK | MOBILE

WEB CONNECTORS: ALL PLANS · CUSTOM: FREE LIMITED TO 1

9.1 What is it?

A Connector lets Claude access your apps and services, retrieve your data, and take action inside them. Where an MCP server is the general standard, a connector is the switched-on, already-plumbed version: you pick a service from a directory, click **Connect**, sign in, and Claude can use it.

The problem it solves

Chapter 8 explained the copy-paste courier problem, and connectors solve the same problem for the *common* case. The distinct thing they add is that **somebody else did the integration work and reviewed it**. You are not writing a server, hosting anything, or deciding whether to trust an unknown URL. You click a button.

How it works at a high level

Most connectors are remote MCP servers that Anthropic or the vendor has built and listed. When you add one, Claude connects to it from Anthropic's cloud infrastructure — not from your device — which is why the same connector works across claude.ai, the desktop app, Cowork and mobile once you have authenticated.

Enablement happens at two levels, and confusing them is the most common connector question:

1. **Account level** — you connect the service and authenticate once. This is a persistent setting.
2. **Conversation level** — you toggle which connected services *this* conversation may use. Connectors are enabled per conversation.

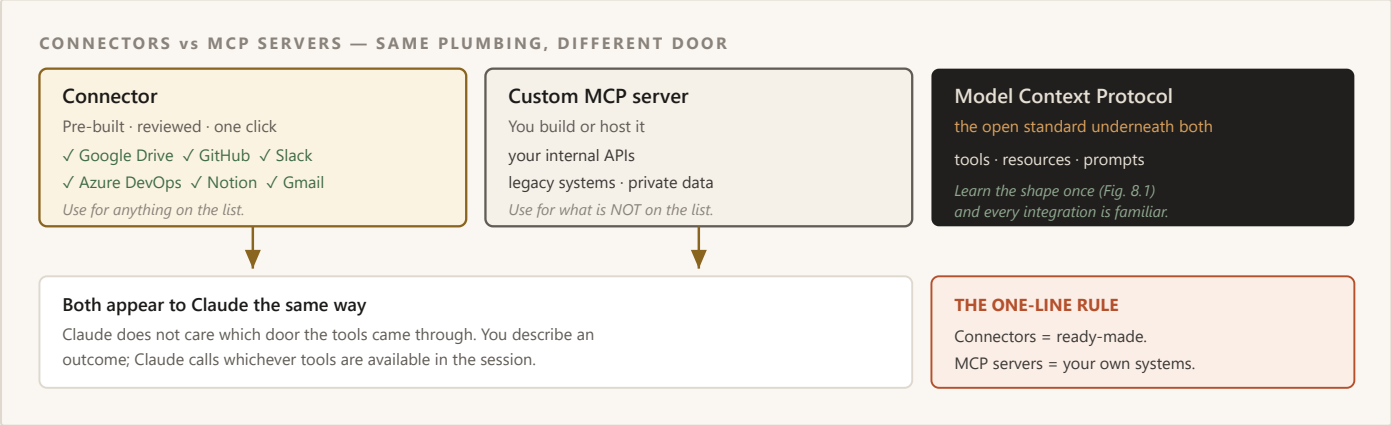


Figure 9.1 Connectors and custom MCP servers are two doors onto the same protocol. Pick by whether an integration already exists.
Diagram — not a screenshot. Based on Anthropic's connectors documentation.

What it is NOT

- **Not a different technology from MCP.** Same protocol, packaged for convenience.
- **Not automatically on.** Connecting a service at account level does not enable it in every conversation. You toggle it per conversation.
- **Not the same as Claude Code's local MCP servers.** Connectors live on your claude.ai account. Servers added with `claude mcp add` live on your machine. They are separate lists — the consequence is in Section 9.5.
- **Not read-only.** Many connectors can write. That is the point, and the reason for the permission controls.

- **Not verified when custom.** A *custom* connector points at an arbitrary remote MCP server. Anthropic has not reviewed it.

How it relates to the other blocks

BLOCK	RELATIONSHIP
MCP Servers	Chapter 8 is the general mechanism; this chapter is the convenient front door. Read them as a pair.
Skills	Skill = method, connector = reach. The demo chain used both.
Projects	Independent. A project does not grant its chats connector access; you enable connectors in the conversation.
Routines	A routine's connector list is exactly your account connectors, and *all of them are included by default* when you create one. Remove what the routine does not need — see 9.7.
Artifacts	An artifact can pull live data through MCP connectors each time someone opens it.
Plugins	In Claude Code, the official marketplace ships plugins that bundle pre-configured MCP servers for the same services — the Claude Code equivalent of a connector.

9.2 When should I use it?

USE A CONNECTOR WHEN…

- The service you need is **in the directory**. Always prefer this over building anything.
- You work primarily in **chats, Cowork or mobile** rather than a terminal.
- You want the integration to **follow you across devices** without reconfiguring.
- You want a reviewed integration rather than one you have to vouch for yourself.

Use something else when…

- The system is internal, legacy or custom → a **custom connector / MCP server** (Chapter 8).
- You work in Claude Code and want the config committed for the team → `--scope project` and `.mcp.json`.
- You need local machine access → a local stdio MCP server, or a desktop extension.
- You only need the data once → paste it.

Desktop vs web — which to use

KIND	AVAILABILITY	USE IT FOR
Web connectors "remote connectors"	All users, free and paid. **Every surface** — web, mobile, Desktop, Cowork *and* Claude Code	Cloud services you sign into: Drive, GitHub, Slack, Notion, Azure DevOps, Gmail. The default choice, and what most connectors are.
Desktop extensions	Claude Desktop and Claude Code **only** — not web, not mobile	Anything that needs your actual machine: local files and folders, a database on `localhost`, a desktop application, or OS-level access such as the filesystem or clipboard. Install via **Claude Desktop › Settings › Extensions**.
Custom connectors	Free (1), Pro, Max, Team, Enterprise	Your own remote MCP servers. Unverified by Anthropic.

Anthropic publishes a dedicated article on choosing between desktop and web connectors — linked in Section 9.8.

The one-line rule: if the thing you want to reach has a URL, use a remote connector and it works everywhere. If it only exists on your computer, you need a desktop extension and you accept that it does not follow you to the browser or your phone. Plugins (Chapter 12) can bundle either kind — which is why a plugin that works on your desktop may appear to do nothing on the web: it bundled a *local* MCP server.

9.3 How do I set it up?

Browse what is available

Two routes to the same directory:

From a chat: + › hover **Connectors** › **Manage connectors** › + next to Connectors

From settings: **Customize** › **Connectors** › + next to Connectors

You can also browse the reviewed catalogue directly at `claude.ai/directory`. Since the introduction of the unified directory, **Customize** is the one place holding your connectors, skills and plugins together (Chapter 11) — some of Anthropic's articles still refer to this as **Settings › Connectors**, which reaches the same list.

Add a connector

1. **Select the connector from the directory**
2. **Review its capabilities**
   The listing states what the connector can read and do. Read this — it is the moment to notice that something has write access.
3. **Click Connect or Install**
4. **Complete authentication**
   You will be taken through the service's own sign-in and consent flow. Authenticate as the account whose permissions you want Claude to have — which is not always your most privileged account.
5. **Enable it in the conversation you want to use it in**
   Click + in the lower left of the chat, hover **Connectors**, and toggle the service on for this conversation.

> **THIS STEP IS THE ONE PEOPLE MISS**
> A connected connector that is toggled off in the current conversation behaves exactly like a connector you never added. If Claude says it cannot see your Drive, check the toggle before anything else.

Add a custom connector

Same place, but you supply the endpoint yourself: enter your remote MCP server's **URL**, and optionally open **Advanced settings** to provide an **OAuth Client ID** and **OAuth Client Secret**. Full guidance is in Chapter 8, Section 8.3, Path A.

Organisation setup (Team and Enterprise)

1. **An Owner enables it for the organisation**
   Organization settings › Connectors › Browse connectors › Add to your team
2. **Each member authenticates individually**
   Making a connector available to the organisation does not sign anyone in. Every user still authenticates with their own account, so each person's access reflects their own permissions in the source system.
3. **Enterprise: restrict and scope**
   Enterprise plans can restrict connectors to verified domains, and limit action permissions organisation-wide — read-only, write, or delete.

Recommended configuration

DECISION	RECOMMENDATION	WHY
Which account to authenticate with	The least-privileged account that can do the job	Claude inherits exactly those permissions. An admin account grants admin reach.
Which connectors to enable per conversation	Only what that conversation needs	Fewer tools in context, narrower blast radius, clearer behaviour.
Write-capable connectors	Enable deliberately, per conversation	Reading is where most of the value is; writing is where the mistakes are.
Custom connectors	Only servers your organisation controls or has vetted	Unverified by Anthropic, with access to your data.

9.4 How do I use it?

Enable the connector, then describe the outcome. You do not name tools.

Examples

```
Summarise the three most recent documents in the "Release 4.2" folder
on Drive, and tell me which ones disagree with each other.

Find every message in #qa-alerts from the last week mentioning "timeout",
group them by service, and tell me which service is getting worse.

Read the Pagination frame in our Figma design-system file and list its
variants and states.

Read work item 4821 in Azure DevOps and draft test cases for its
acceptance criteria. Show me before you create anything.
```

How to verify it worked

1. **Ask for something only the live system knows** — a specific record, a current field value, a file that was added yesterday. Not something Claude could plausibly invent.
2. **Check the connector toggle** in the conversation if the answer is generic or hedged.
3. **Verify the first few writes in the source system.** Open the tool and look. You are testing the integration, not Claude.
4. **Notice the failure mode.** A connector that cannot reach something should say so. An answer that is confidently vague is a signal the tool was not actually called.

9.5 How do I manage it?

ADD	Customize › Connectors › + then **Connect** and authenticate.
VIEW	Customize › Connectors lists everything connected, with its settings and permissions.
EDIT	Open the connector from the same list to modify its settings or review its permissions.
ENABLE / DISABLE PER CHAT	In the conversation: + › hover **Connectors** › toggle. This is the control you will use most often.
DISCONNECT	Customize › Connectors › select the service › disconnect. Revokes Claude's access; re-adding means authenticating again.

RE-AUTHENTICATE	If a token expires or you change the source-system password, disconnect and reconnect.
ORGANISATION	Owners: Organization settings › Connectors. Add for the team, and on Enterprise restrict domains and action permissions.

TWO SEPARATE LISTS — THE THING TO REMEMBER

Your **claude.ai account connectors** and Claude Code's **locally-configured MCP servers** are different lists.

- Account connectors are what a **Routine** can use (Chapter 13), and what Cowork and cloud sessions see.
- Servers added with `claude mcp add` are stored on your machine and do *not* appear in the connectors list.
- To use a local server in a routine: add it as a connector at `claude.ai/customize/connectors`, or declare it in a committed `.mcp.json` so it is part of the cloned repository.

Troubleshooting

SYMPTOM	LIKELY CAUSE	FIX
Claude says it cannot access the service	Connector not toggled on for this conversation	+ › Connectors › toggle it on. Check this first, every time.
Connector missing from the directory	Not enabled for your organisation, or restricted	Team/Enterprise: an Owner must add it under Organization settings. Enterprise may restrict to verified domains.
Authenticated but returns nothing	The account you authenticated with lacks access to that content	Claude inherits your permissions in the source system. Check you can see the item yourself, signed in as that account.
Worked yesterday, fails today	Token expired or credentials changed	Disconnect and reconnect the connector.
Cannot write, only read	Enterprise action-permission restriction, or the OAuth scope granted	Ask your administrator; re-authenticate if the consent screen offered narrower scopes than you need.
Custom connector will not connect	Wrong URL, or OAuth client details missing	Re-check the server URL and supply the Client ID / Secret under **Advanced settings**. See Chapter 8's troubleshooting table — MCP tokens are not REST API keys.
Free plan will not let you add a second custom connector	Free is limited to one	Expected. Upgrade, or use the reviewed directory connectors, which are not limited this way.
Routine cannot reach the tool	It is a local MCP server, not an account connector	See the warning above.

9.6 Examples

QA — triage from Slack into the tracker

```
Read #qa-alerts from the last 7 days. Group the failures by service.
For the three noisiest, draft a bug report each using our format —
severity based on the impact described in the thread.

Show me the drafts. Do not create anything in Azure DevOps yet.
```

Product/BA — reconcile a spec against the board

```
The signed-off spec is in the Drive folder "Checkout Revamp / Signed off".
The delivery board is in Azure DevOps under the Checkout area path.
```

```
Produce three tables: requirements with no work item; work items with no
requirement; and ACs whose wording differs between the two. Cite the
document and item ID for every row.
```

Design — audit the design system against the code

```
Using the Figma connector, list every component in our design-system file
that has been modified in the last 30 days. For each, check whether the
corresponding component in the repo has changed since. Flag drift.
```

Development — from issue to PR

```
Read GitHub issue #482. Implement it following CLAUDE.md, run the tests,
and open a PR that links the issue. If the issue is ambiguous about the
error-handling behaviour, ask me rather than choosing.
```

Reporting — a live status board

```
Build an artifact that shows our current sprint status: items by state,
blocked items with their blocker, and anything unchanged for 5+ days.
Have it pull fresh data from the Azure DevOps connector each time
someone opens it, so I can share the link with the team.
```

Connector-backed artifacts are covered in Chapter 10.

9.7 Best practices

Recommended practices

- **Directory first.** If it is listed, use the listed connector. It is reviewed, maintained and one click.
- **Authenticate with the right account.** Claude gets exactly your permissions. Use a least-privileged account where one exists.
- **Enable per conversation, deliberately.** Treat the toggle as part of setting up the task.
- **Split read from write.** Ask to see the drafts before anything is created. One extra turn, far fewer awkward cleanups.
- **Prune what you stopped using.** Every connected tool costs context and widens exposure.
- **Trim a routine's connector list.** When you create a routine, *all* your connected connectors are included by default, and during a run Claude can use every tool from an included connector — including writes — without asking permission. Remove everything the routine does not need.

Common mistakes

MISTAKE	WHY IT HURTS
Forgetting the per-conversation toggle	The single most common connector support question.
Authenticating with an admin account by habit	Grants Claude far more reach than the task needs.
Leaving every connector enabled everywhere	Context cost, and a much larger surface if a prompt goes wrong.
Adding a custom connector to a server nobody vetted	Unverified, with read and write access to your data.
Assuming account connectors and Claude Code MCP servers are one list	They are not, and routines only see the former.
Assuming a connector is read-only	Many can write. Read the capability description at install time.

Security considerations

WHAT A CONNECTOR ACTUALLY GRANTS

- **Custom connectors are unverified.** They connect Claude to arbitrary services that Anthropic has not reviewed, granting the ability to access and potentially modify data there. Only connect to servers you trust.
- **Permissions are inherited, not granted.** A connector can do what your authenticated account can do. This is a feature — and a reason to think about which account you use.
- **Routines run unattended with no approval prompts.** Claude can use every tool from an included connector, writes included, during a routine run. Scope those lists tightly.
- **Enterprise controls:** restrict connectors to verified domains, and limit action permissions organisation-wide to read-only, write or delete. If you are rolling this out at scale, use them.
- **Data flows through Anthropic's infrastructure** for remote connectors, since Claude connects from the cloud rather than your device. Factor that into any data-residency assessment.

Team and organisational considerations

- **Owners curate.** Decide which connectors are available to the organisation rather than leaving it to individuals. Publish the approved list.
- **Availability is not access.** Adding a connector for the team does not sign anyone in; each member authenticates individually. Say so in your rollout note or you will field the question repeatedly.
- **Mind the free-plan limit** of one custom connector if your team is mixed-plan.
- **Document the authentication account convention** — particularly for shared or service accounts, where the wrong choice quietly grants everyone the reach of one privileged user.

9.8 Official resources

Use connectors to extend Claude's capabilities — the click paths, org controls and plan availability used in this chapter
https://support.claude.com/en/articles/11176164-use-connectors-to-extend-claude-s-capabilities

When to use desktop and web connectors
https://support.claude.com/en/articles/11725091-when-to-use-desktop-and-web-connectors

Anthropic Connectors Directory FAQ
https://support.claude.com/en/articles/11596036-anthropic-connectors-directory-faq

Get started with custom connectors using remote MCP
https://support.claude.com/en/articles/11175166-get-started-with-custom-connectors-using-remote-mcp

Connectors — the full help-centre collection
https://support.claude.com/en/collections/15399129-connectors

The Anthropic Directory — browse reviewed connectors
https://claude.ai/directory

Claude Code equivalent — MCP servers and the official integration plugins
https://code.claude.com/docs/en/mcp

CHAPTER 10

Artifacts

Claude builds you a real thing — not just text about a thing.

START HERE | CLAUDE.AI | DESKTOP APP | COWORK | CLAUDE CODE

CHAT ARTIFACTS: ALL PLANS · CLAUDE CODE ARTIFACTS: PRO / MAX / TEAM / ENTERPRISE

10.1 What is it?

An Artifact is a live, self-contained output rendered in its own window rather than as text in the conversation. You can open it, use it, iterate on it, keep versions of it, download it and share it. A formatted document, an interactive checklist, a chart, a mini-tool — produced immediately.

The problem it solves

Chat is a bad container for a deliverable. If Claude writes you a sprint retro as chat text, you scroll, select, copy, paste into a document, fix the formatting, and then discover a section needs rewording — at which point you go back to the chat and start the cycle again. The output was never a *thing*; it was a message about a thing.

An artifact makes it a thing. It exists in its own right, it can be revised in place, and its versions are tracked.

How Claude decides

Claude creates an artifact on its own when the content meets roughly these criteria:

- **Significant and self-contained** — typically over fifteen lines.
- **Something you are likely to want to edit, iterate on, or reuse** outside the conversation.
- **Complex enough to stand on its own** without the surrounding conversation for context.
- **Something you will want to refer back to** or use later.

You can also simply ask. "Build that as an artifact" always works, and is the reliable way to get one.

What can be an artifact

TYPE	WHAT CLAUDE PRODUCES
Formatted document	Sprint retro, release notes, test plan, meeting summary — markdown or plain text.
Interactive checklist	QA sign-off, sprint planning, launch-readiness — with boxes you can actually tick.
Chart / visualisation	Sprint velocity, bug-trend graph from test results, ticket mix over time.
Mini-tool	A form, a live filter, a calculator — an interactive React component.
UI snippet	An HTML/CSS mockup generated from a component description or a Figma frame.
Code & diagrams	Code snippets, single-page HTML sites, SVG images, flowcharts and diagrams.

Types drawn from Anthropic's artifacts documentation.

What it is NOT

- **Not an application.** An artifact is a capture of work, not a deployed product. It is one self-contained page with no backend: it cannot store form input between visits or serve multiple routes. For a hosted internal tool with a database, build and deploy that properly.
- **Not a file until you download or publish it.** Persistent storage applies to *published* artifacts.
- **Not private once published.** Publishing creates a link. Know which audience you chose.
- **Not the right container for a two-sentence answer.** If the reply is conversational, chat is correct.

- **Not unlimited.** There is a 20 MB storage limit per artifact, and artifact input is text-only — no images, files or binary data passed in.

How it relates to the other blocks

BLOCK	RELATIONSHIP
Everything else	Artifacts are the *output* layer. Projects, Instructions, Skills, MCP and Connectors all shape what goes *into* a turn; an artifact is what comes out of it.
Skills	A skill can specify that its output should be an artifact — e.g. a report skill that always produces one.
Connectors / MCP	An artifact can call MCP connectors when someone opens it, so a published board fetches fresh data on each view.
Projects	Artifacts are created in chats, which live in projects. A project is the natural home for a series of related artifacts.
Routines	A scheduled routine can publish or update an artifact, giving you a status page that refreshes itself.
Workflows	A long workflow can keep an investigation timeline artifact updating as it runs.

10.2 When should I use it?

ASK FOR AN ARTIFACT WHEN…

- The output is **easier to look at than to read** — a chart, a table, a mockup, an annotated diff.
- You will **edit it before using it**. Revising in place beats regenerating.
- You need to **send it to someone** — a link instead of a wall of pasted text.
- It is a **deliverable**: a document, a report, a checklist, a spec.
- You want **version history** of a document as you iterate.
- A long task is running and you want a **timeline that fills in** as it progresses.

Do not use an artifact when…

- The answer is conversational or short. Chat is the right container.
- You need a real application with a backend, authentication or stored state — build and deploy that properly.
- The content is highly sensitive and you might publish it by reflex. Section 10.7.
- You need it in a specific corporate template — generate the content as an artifact, then paste into the template, or have Claude produce the file format directly via the document Skills.

Real-world examples by role

ROLE	ARTIFACT
QA	Release sign-off checklist · test-run summary · bug-trend chart from test results · environment coverage matrix
Product / BA	Sprint retro · velocity chart from an ADO export · stakeholder-ready status page · requirements traceability table
Design	Styled HTML preview of a component · states-and-variants matrix · accessibility audit · side-by-side option comparison
Development	Annotated diff walkthrough for a reviewer · deploy-failure dashboard · architecture diagram · investigation timeline

10.3 How do I set it up?

Artifacts need almost no setup — a switch or two, once. There is nothing to install.

claude.ai, desktop and Cowork

1 **Open your capability settings**

Click your initials or name in the lower-left corner, then:

Free, Pro and Max:

Settings › Capabilities

Team and Enterprise (an Owner does this once, for everyone):

Organization settings › Capabilities

2 **Find the Visuals section and turn on what you need**

The Capabilities tab is divided into named sections. The one that governs artifacts is **Visuals**, and it holds three separate switches rather than one:

TOGGLE	WHAT IT TURNS ON
Artifacts	The artifact panel itself — the dedicated window beside the chat, the version selector, and the **Artifacts** section in your sidebar. *Turn this on first; the other two build on it.*
AI-powered artifacts	Artifacts that call Claude from inside themselves, so the thing you build can answer questions and adapt to input rather than just display. Covered in §10.4 below.
Inline visualizations	Charts and diagrams rendered *in the reply* rather than as a saved artifact. Officially documented as **custom visuals** — useful, but not the same thing as an artifact. See the comparison overleaf.

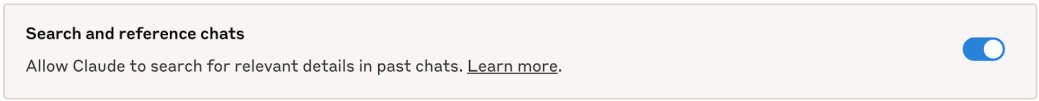


Figure 10.1 Every switch in **Capabilities** looks like this — name, one-line description, toggle on the right. The official example shown is *Search and reference chats*; the Visuals switches follow the same pattern.
Screenshot: Anthropic, Claude Help Centre — "Use Claude's chat search and memory to build on previous context."

IF YOUR CAPABILITIES TAB LOOKS DIFFERENT

This area is actively changing, and not every account is on the same layout. If you do *not* see a **Visuals** section, look for a single switch named **Code execution and file creation** in the same tab and turn that on instead — that is the older, coarser control, and it is still what Anthropic's help-centre article for artifacts describes. Either way you are in **Settings** › **Capabilities**; only the granularity differs.

Do not turn *off* Code execution and file creation if you see it, because **Skills** (Chapter 7) still depends on it — including the built-in Excel, Word, PowerPoint and PDF skills. Artifacts and Skills used to share one switch; artifacts have since been split out into their own, and Skills has not.

3 **That's it**

There is nothing to install and nothing to configure per project. Ask for something substantial and Claude will produce an artifact.

Claude Code

Claude Code can publish artifacts too — as live pages on claude.ai built from your session's work, including your codebase and data pulled through connected tools.

1 **Requirements**

Pro, Max, Team or Enterprise, and a session signed in with `/login`. If Claude writes a local HTML file without producing a link, or says it cannot publish, the tool is not enabled for your session.

2 Ask for one

```
> Make an artifact that walks through this PR with the diff annotated inline.
> Build a dashboard artifact of last week's deploy failures by service
    and keep it updated as you investigate.
```

3 Approve the publish

Claude writes the page to an HTML or Markdown file in your project, then asks permission before publishing a new artifact — something like "*Claude wants to publish 'Deploy failures by service' (deploy-failures.html) to a private page on claude.ai.*" Select **Yes**. Republishing an artifact you have already approved does not prompt again.

4 Open it

Claude prints the URL and your browser opens the page. Press Ctrl+] at any time to reopen the most recent artifact from the terminal. To stop the browser opening automatically, set `CLAUDE_CODE_ARTIFACT_AUTO_OPEN=0`.

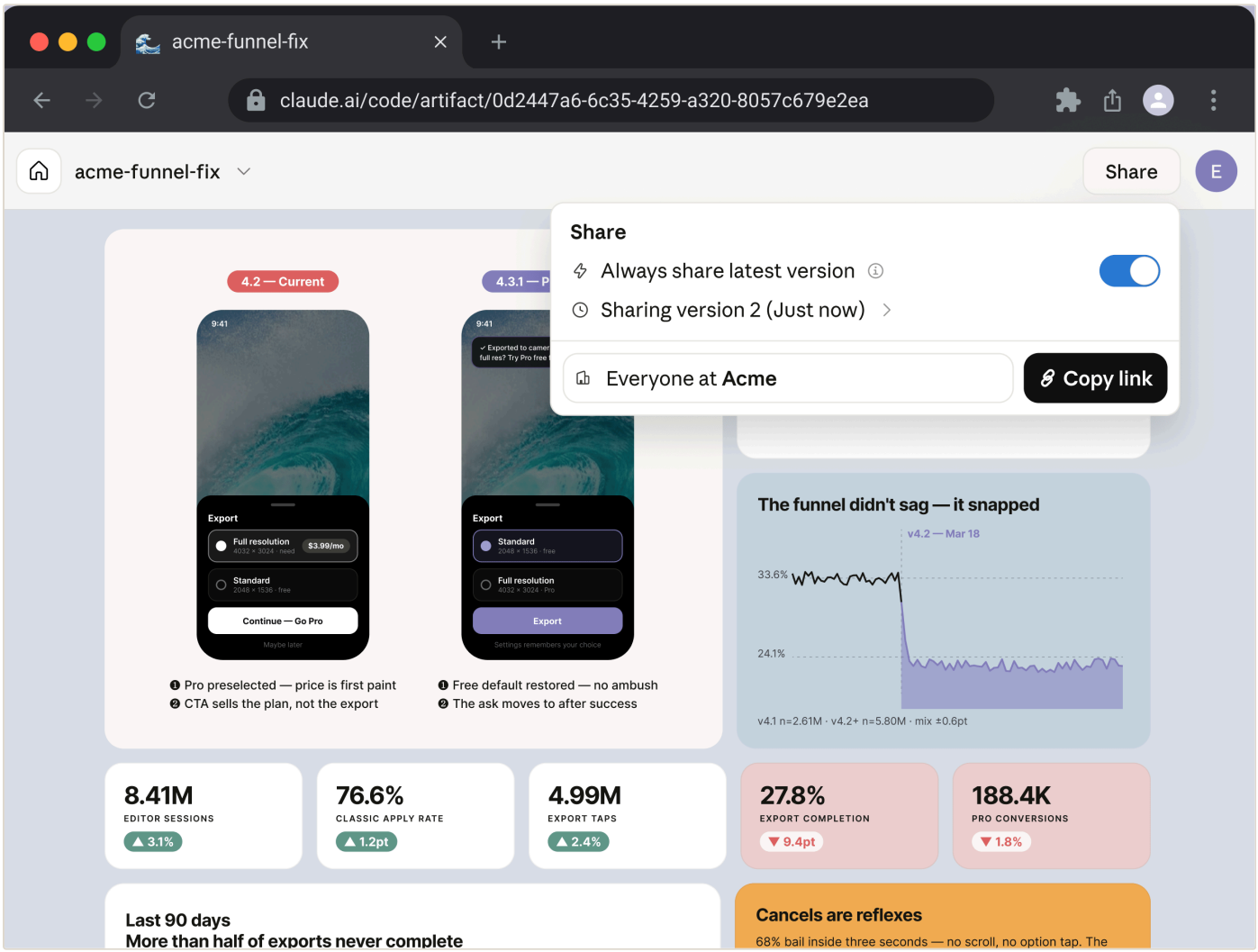


Figure 10.2 A published artifact, with the **Share** menu open. Note the three controls that matter: **Always share latest version**, the version picker (*Sharing version 2*), and the audience selector — here scoped to everyone at the organisation rather than public.
Screenshot: Anthropic, Claude Code documentation — "Share session output as artifacts."

Artifacts vs inline visualisations — the distinction to learn first

Because both switches live in the same **Visuals** section, people conflate them and then report bugs that are not bugs. They are two different features:

	ARTIFACT	INLINE VISUALISATION ("CUSTOM VISUAL")
Where it appears	Its own panel beside the chat	Inside the reply itself
Lifespan	**Persistent.** Saved, versioned, findable later	**Ephemeral.** Not saved when the conversation moves on — "a whiteboard sketch"
Shareable	Yes — publish, share a link, embed	Not directly. Renders for a recipient of a shared chat, but only on web/desktop and only if signed in
How to keep it	Already kept	**Copy as image**, **Download** (.svg / .html), or **Save as artifact**
Status	Generally available	**Beta**, web and desktop only — not on iOS or Android
Use it for	A deliverable someone else will open	Thinking something through in the moment

Sources: Anthropic help centre — "Custom visuals in chat and Cowork" and "Visual and interactive content."

Rule of thumb: if you want to *look* at it, an inline visualisation is fine and faster. If you want to *send* it, ask for an artifact. Anthropic's own guidance: "if you want to build something persistent — a tool, an app, a document to share — ask Claude to create an artifact instead."

Two related behaviours share this section of the settings and surprise people:

- **Data widgets.** Weather and recipe cards are rendered views over live third-party data (Google Maps and Bing Image Search respectively) and need **web search** enabled. They render on web and desktop only; on mobile you get the same information as text.
- **Interactive inputs.** Rather than asking you to type, Claude may offer buttons, multi-select lists or a ranking widget at the bottom of the chat. You can always ignore them and type instead.

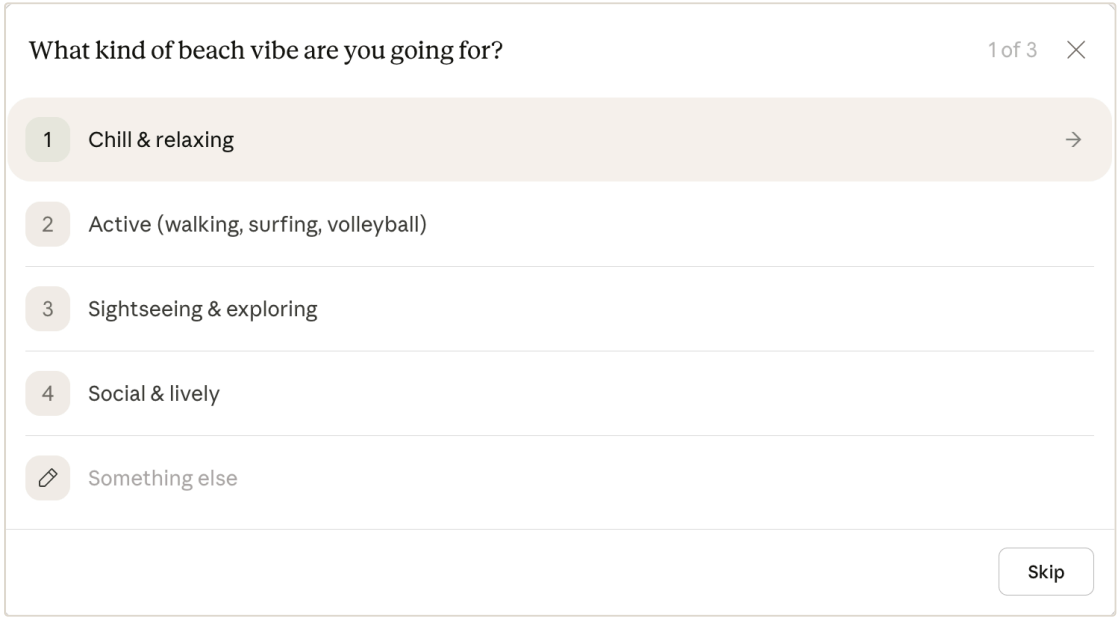


Figure 10.3 **Interactive inputs.** Claude asks a structured question and you click rather than type. This is part of the same visual/interactive family as artifacts and inline visualisations, but it is neither — it is how Claude gathers input.
Screenshot: Anthropic, Claude Help Centre — "Visual and interactive content."

10.4 How do I use it?

How to start

Two ways. Either let Claude decide, or say so:

Asking explicitly

```
Build this as an artifact.
Make me an interactive checklist for the release sign-off.
```

```
Turn this data into a chart artifact with a short commentary under each view.
```

How to structure the request

Artifacts reward specificity about *shape*, because you are asking for a designed object rather than prose:

Prompt structure

```
[What kind]     Build an artifact — an interactive checklist.
[The content]   One section per test area, from the attached test plan.
[The behaviour] Checkboxes I can tick, and a progress count at the top.
[The shape]     Group by priority; put blockers first.
[The limits]    No sign-off section — that lives in ADO.
```

Iterating — the part that makes artifacts worth using

Do not regenerate. Revise:

```
Make the summary section two sentences shorter and move it above the chart.
Add a column for owner.
The severity colours are too similar — make S1 clearly distinct.
Drop the third chart entirely.
```

Claude edits the artifact in place. In Claude Code, it edits the underlying file and republishes to the *same URL*, so a link you have already shared stays current.

Versions, code and download

In the chat surfaces, the artifact appears in a dedicated window to the right of the conversation. Controls in the lower right let you access **versions**, view the underlying **code**, **copy** the content, and **download** files.

What to expect

- The artifact opens in its own panel (chat) or as a page on claude.ai (Claude Code).
- Interactive artifacts genuinely work — buttons click, filters filter, checkboxes tick.
- Each revision creates a new version you can go back to.
- Claude picks a title and an emoji tab icon; ask for specific ones if you care, because they appear in your artifact gallery and in shared links.

How to verify the result

1. **Use it, don't read it.** Tick the boxes. Click the filter. An artifact that looks right and does not work is a common failure and an easy one to catch.
2. **Check the numbers against the source.** For any chart or table built from data, spot-check two or three values. Charts are persuasive; that is exactly why they need checking.
3. **Confirm the version you are sharing.** Before sharing, make sure you are on the version you intend — the share dialog names it.
4. **Open the share link yourself**, ideally in a private window, to confirm the audience is what you expected.

Common usage patterns

PATTERN	HOW IT WORKS
Draft → refine → send	Generate the deliverable, adjust wording in place, then share the link. The most common use, and the one that saves the most time.
Data → chart → commentary	Feed an export, ask for a chart per view plus two sentences under each. The commentary is where the value is; the chart is just the evidence.
Options side by side	"Lay out three approaches as a comparison artifact." Far easier to choose from than three paragraphs.
Live timeline	During a long task, have Claude keep an investigation timeline artifact updating as it works.

PATTERN	HOW IT WORKS
Connector-backed board	An artifact that pulls fresh data through MCP connectors each time someone opens it — a status page that maintains itself.
Reviewer walkthrough	From Claude Code: an annotated diff artifact so a reviewer sees the reasoning, not just the changes.

10.5 How do I manage it?

CREATE	Ask for it, or let Claude produce one when the content warrants it. In Claude Code, approve the publish prompt.
VIEW	In the artifact panel beside the chat. Published artifacts also appear in the **Artifacts** section of your sidebar, so you can find them outside the original conversation.
EDIT	Ask for changes in the conversation. Claude revises in place and creates a new version.
VERSIONS	Use the controls in the lower right of the artifact window. Confirm you are on the intended version before sharing.
DOWNLOAD	Same controls: copy the content, or download the file.
SHARE	Click **Share**, confirm the version, then **Share & copy link**. In Claude Code, share from the page header — including an audience selector and an **Always share latest version** toggle.
PUBLISH	**Publish** on the artifact, then copy the public link. Publishing also adds it to your sidebar gallery.
UNPUBLISH	Open the **Share** menu in the upper right of the artifact and, in the *Artifact shared* dialog, click **Unshare**. One-way — see the warning below.
EMBED	After publishing, a **Get embed code** button appears. It opens a dialog with generated code you can paste into another site.
DISABLE	Turn the **Visuals** switches off in Settings › Capabilities — or, on the older layout, **Code execution and file creation**. On Team and Enterprise an Owner does this in Organization settings › Capabilities. Remember Skills shares the older switch.
DUPLICATE	Ask Claude to create a variant: "make a copy of that artifact with the chart removed." Or download and reuse the file.

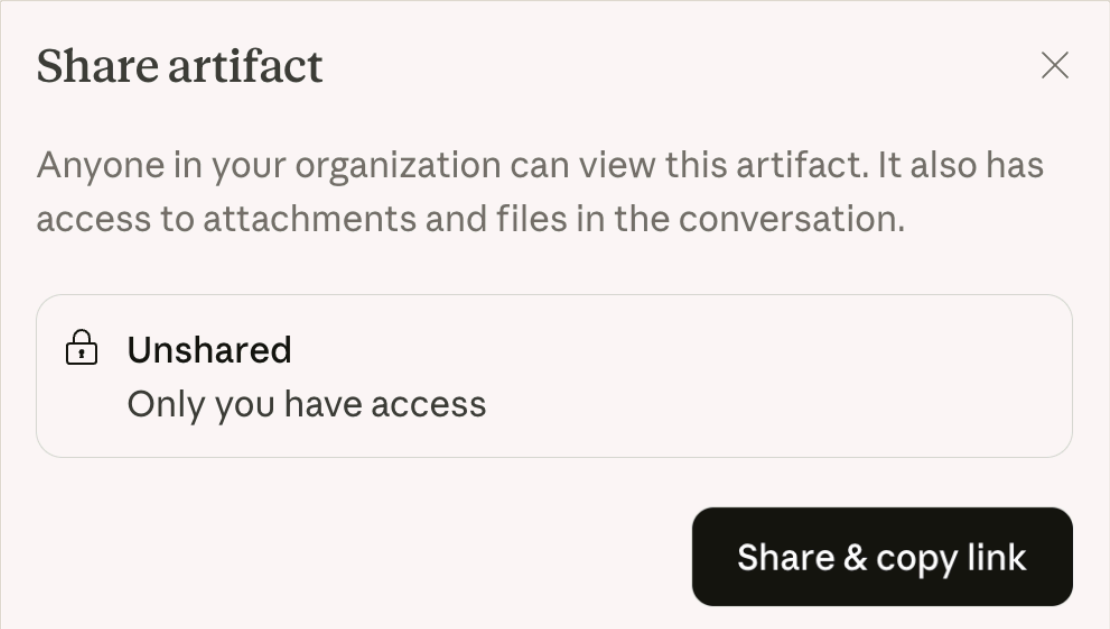


Figure 10.4 The **Share artifact** dialog. Note the two things worth reading before you click: the current state (**Unshared — only you have access**) and the warning that recipients also gain access to *attachments and files in the conversation*, not just the artifact.
Screenshot: Anthropic, Claude Help Centre — "Publish and share artifacts."

UNPUBLISHING IS NOT REVERSIBLE

> Once you unpublish an artifact, **you cannot publish that same artifact again**. You would need to create a new artifact to publish it later. Decide before you unpublish something whose link is already circulating.

Troubleshooting

SYMPTOM	LIKELY CAUSE	FIX
No artifact appears — output stays in chat	Capability off, or content below the threshold	Settings › Capabilities › **Visuals** › turn **Artifacts** on (older layout: **Code execution and file creation**). Then just ask: "build that as an artifact."
You get a chart *inside* the reply, not an artifact panel	That is an inline visualisation (custom visual), not an artifact	Expected behaviour — they are different features. Ask explicitly: "build that as an artifact," or use **Save as artifact** on the visual.
A visual disappeared when the conversation moved on	Inline visualisations are ephemeral by design	Nothing is broken. Keep one with **Copy as image**, **Download**, or **Save as artifact** *before* moving on.
Claude Code writes a local HTML file with no link	The artifact tool is not enabled for the session	Check plan (Pro and above) and that you are signed in with `/login`, not an API key.
Interactive artifact renders but does nothing	A logic error in the generated component	Describe the broken behaviour specifically: "clicking Filter does nothing" beats "it's broken."
Chart numbers look wrong	Ambiguous or mis-parsed source data	Ask Claude to show the intermediate table it derived before charting, and check it. Do not debug the chart; debug the data.
Shared link shows an old version	You shared a specific version	Re-share the current version, or enable **Always share latest version**.
Cannot re-publish after unpublishing	By design	Create a new artifact.
Artifact will not save / too large	20 MB per-artifact storage limit	Split it, or reduce embedded content.
Tried to feed it an image or file	Artifact input is text-only	Describe the content, or supply the data as text. No images, files or binary data as artifact input.
Lost an artifact from an old chat	Only published artifacts persist outside the conversation	Check the **Artifacts** section of the sidebar. Publish anything you want to find again.

10.6 Examples

Example 1 — QA: release sign-off checklist

Prompt

```
Build an interactive checklist artifact for our Release 4.2 sign-off,
from the test plan in project knowledge.

- One section per test area, blockers first
- Each item has a checkbox and a place for the tester's initials
- A progress counter at the top showing completed / total
- A final "Outstanding blockers" area that stays visible

Do not include a formal approval signature block — that lives in ADO.
```

Example 2 — QA: bug-trend chart from test results

```
Here's our defect export for the last six sprints (attached).
Build a chart artifact with three views: defects raised vs closed per
sprint, open defects by severity over time, and mean time to close.
```

```
Under each chart, two sentences: what it shows, and what I should be
worried about. If a trend is likely a data-quality artefact rather than
real, say so instead of interpreting it.
```

Example 3 — Product/BA: the analysis from the live demo

```
Using the Azure DevOps connector, pull every work item I created in the
last six months. Build an analysis artifact showing volume by month,
the split by item type, and how my mix has shifted over the period.

Include a short "what changed and why it might have" commentary.
```

Example 4 — Product/BA: sprint retro

```
Here's the ADO board export for sprint 24. Generate the retro as a
document artifact in our format: what went well, what didn't, and
actions with named owners.

Base every point on something in the data and cite the item IDs.
Where you're inferring rather than observing, mark it as inference.
```

Example 5 — Design: component preview and states matrix

```
Here's the spec for our new Pagination component.

Build an artifact with two parts: a styled HTML preview showing every
variant and state side by side, and below it a states matrix table
(state × variant) marking which combinations are valid.

Use our tokens where the spec names them. Where it doesn't, use a
placeholder and list it under "Needs a token".
```

Example 6 — Development: annotated PR walkthrough (Claude Code)

```
> Make an artifact that walks a reviewer through this PR: the diff
    annotated inline with why each change was made, the two decisions
    I'd like a second opinion on, and how to test it locally.

    Order it by what a reviewer needs to understand first, not by file path.
```

Example 7 — a status board that refreshes itself

```
Build a status-board artifact for the current sprint: items by state,
blocked items with their blocker and age, and anything untouched for
5+ days.

Have it pull fresh data through the Azure DevOps connector each time
someone opens it, and share it with everyone in the organisation —
not publicly.
```

Note the last clause. Audience is a deliberate choice, not a default.

10.7 Best practices

Recommended practices

- **Just ask.** Do not wait for Claude to decide. "Build that as an artifact" is the reliable path.
- **Iterate, don't regenerate.** Small targeted revisions preserve what was already right. Regenerating from scratch loses it.
- **Specify the shape, not just the content.** Order, grouping, what to leave out. Artifacts are designed objects; treat the request like a brief.

- **Ask for commentary with every chart.** A chart tells you what happened; the sentence underneath tells you what to do. The second is the part you were actually after.
- **Check the data, not the picture.** Ask for the intermediate table before the chart when the numbers matter.
- **Choose the audience explicitly** when sharing. Do not accept a default you have not read.
- **Publish what you want to keep.** Only published artifacts persist outside the conversation and appear in your gallery.

Common mistakes

MISTAKE	WHY IT HURTS
Treating an artifact as an application	No backend, no stored state, one page. Building a "tool" that needs to remember things will disappoint.
Regenerating instead of revising	You lose the parts that were right and burn tokens re-deriving them.
Trusting a chart without checking the data	A well-drawn chart of mis-parsed data is more dangerous than no chart.
Sharing without checking the version	Colleagues review something you already fixed.
Publishing sensitive content by reflex	Publishing creates a link. Once shared, treat it as distributed.
Unpublishing casually	One-way. You cannot re-publish that artifact.
Using an artifact for a two-line answer	Adds friction to something chat handled fine.

Security considerations

BEFORE YOU CLICK PUBLISH

- **Publishing distributes.** A published artifact has a link. Assume anyone with the link can read it, and that a link can be forwarded. Prefer an organisation-scoped audience over public whenever the content is internal.
- **Artifacts built from connector data contain that data.** A sprint board artifact embeds the item titles. A defect chart embeds customer-reported symptoms. Check what actually ended up on the page before sharing it outward.
- **Connector-backed artifacts call live tools when opened.** Understand who can trigger those calls and with whose credentials before sharing widely.
- **Never build an artifact that impersonates a real person or organisation**, presents fabricated records as genuine, or collects credentials. This should be obvious; it is worth writing down anyway.
- **Unpublish is one-way.** Treat publication as a decision, not an experiment.

Naming and maintainability

- Ask for a specific **title** when the artifact will be shared — it names the page in the browser tab, your gallery and every shared link.
- Include a date or sprint number in the title for anything periodic: `Sprint 24 Retro`, not `Retro`. You will make another one.
- For a recurring artifact, keep the format in a **Skill** or in project knowledge so each new one is consistent rather than freshly invented.
- Enable **Always share latest version** for a living document; share a pinned version for anything that is a point-in-time record.

Team considerations

- **Agree what artifacts are for.** They are excellent for drafts, reviews, dashboards and one-off analysis. They are not a document management system — final approved documents should end up wherever your organisation keeps records.
- **A link is better than a paste** in Slack or Teams, and much better than a screenshot of a table.
- **Agree the audience default** for internal work — organisation-scoped rather than public — and say so in your rollout note.
- **Team and Enterprise administrators control the capability** organisation-wide via Organization settings › Capabilities. Because the switches are now separate, an organisation can allow plain artifacts while leaving **AI-powered artifacts** off

— worth deciding deliberately rather than by default, since an AI-powered artifact runs on Anthropic infrastructure and each viewer authenticates with their own Claude account.

- **Sharing an artifact shares more than the artifact.** The share dialog states that recipients also get access to attachments and files in the conversation. Publish from a clean chat when the surrounding conversation contains anything you would not circulate.

10.8 Official resources

What are artifacts and how do I use them? — creation criteria, supported types, the settings toggle, limits
https://support.claude.com/en/articles/9487310-what-are-artifacts-and-how-do-i-use-them

Publish and share artifacts — publishing, sharing, versions, and the one-way unpublish
https://support.claude.com/en/articles/9547008-publish-and-share-artifacts

Share session output as artifacts — the Claude Code reference: creating, updating, sharing, page constraints, connector-backed pages
https://code.claude.com/docs/en/artifacts

Use artifacts to visualize and create AI apps without writing code
https://support.claude.com/en/articles/11649427-use-artifacts-to-visualize-and-create-ai-apps-without-ever-writing-a-line-of-code

Prototype AI-powered apps with Claude artifacts
https://support.claude.com/en/articles/11649438-prototype-ai-powered-apps-with-claude-artifacts

Use live artifacts in Claude Cowork
https://support.claude.com/en/articles/14729249-use-live-artifacts-in-claude-cowork

Custom visuals in chat and Cowork — inline visualisations: what they are, how to keep one, why they are not artifacts
https://support.claude.com/en/articles/13979539-custom-visuals-in-chat-and-cowork

Visual and interactive content — the wider family: data widgets, custom visuals, interactive inputs, and their limitations
https://support.claude.com/en/articles/13641943-visual-and-interactive-content

Use visual and interactive content on Team and Enterprise plans — the org-level defaults and third-party data flows
https://support.claude.com/en/articles/13663666-use-visual-and-interactive-content-on-team-and-enterprise-plans

Claude Code now supports artifacts — the announcement
https://claude.com/blog/artifacts-in-claude-code

CHAPTER 11

Marketplace

The catalogue plugins come from — official, community, or your own private one.

PREREQUISITE FOR CHAPTER 12 | CLAUDE CODE CLI | VS CODE EXTENSION | DESKTOP APP | CLAUDE.AI | COWORK

CLAUDE CODE PLANS · CLAUDE.AI DIRECTORY: PRO / MAX / TEAM / ENTERPRISE

WHY THIS CHAPTER EXISTS

A marketplace is not a capability you would set out to learn on its own. This chapter is here because Chapter 12 does not work without it: **you cannot install a plugin without first adding the marketplace it lives in**. Read this chapter as the setup step for the next one.

11.1 What is it?

A marketplace is a *catalogue* of plugins — a list of what is available and where to get it. It is published as a `marketplace.json` file in a Git repository, a local folder, or at a URL. Adding a marketplace registers the catalogue so you can browse it. It installs nothing.

The problem it solves

Suppose your QA lead builds an excellent setup: a `/test-case` skill, a `/bug-report` skill, the Azure DevOps MCP configuration, and a subagent that audits sprint hygiene. How does that reach the other eleven people on the team?

Without a marketplace: a wiki page, a zip file, four manual steps, and a version drift problem within a month. With one: `/plugin install qa-toolkit@acme-internal`, and everyone stays up to date automatically.

How it works — the two-step model

This is the single most important thing to understand, and it is the source of most confusion:

1. **Add the marketplace**
   Registers the catalogue with Claude Code so you can see what is in it. **No plugins are installed at this point.**

2. **Install individual plugins from it**
   Browse the catalogue and install the ones you want.

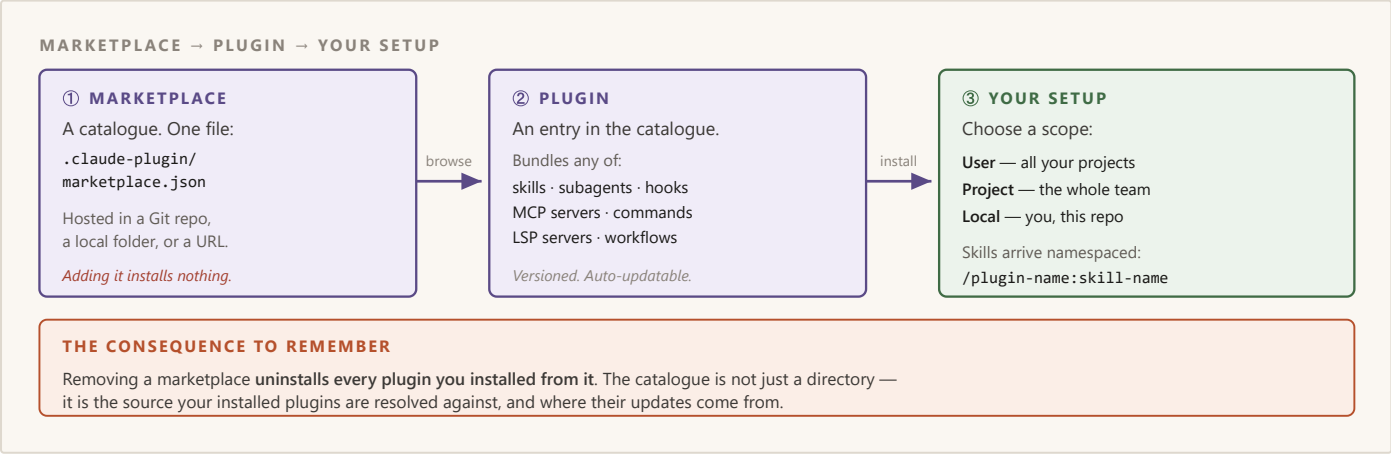


Figure 11.1 The two-step model, and the consequence people discover the hard way.
Diagram — not a screenshot. Based on Anthropic's plugin marketplace documentation.

The marketplaces available to you

MARKETPLACE	ADDED HOW	WHAT IT IS
`claude-plugins-official`	**Automatically**, the first time you start Claude Code interactively	Curated by Anthropic. Inclusion is at Anthropic's discretion. Browse at `claude.com/plugins` or via `/plugin` › **Discover**.
`claude-community`	By hand	Third-party plugins that passed Anthropic's automated validation and safety screening. Each is pinned to a specific commit SHA.
`claude-code-plugins`	By hand	Anthropic's demo marketplace of example plugins, in the `anthropics/claude-code` repository.
Yours	By hand, or via project settings	Your own catalogue — including in a **private repository** for internal-only plugins.

What is in the official marketplace

Worth knowing before you build anything, because a lot is already there:

CATEGORY	PLUGINS
External integrations pre-configured MCP servers	Source control: `github`, `gitlab` · Project management: `atlassian` (Jira/Confluence), `asana`, `linear`, `notion` · Design: `figma` · Infrastructure: `vercel`, `firebase`, `supabase` · Communication: `slack` · Monitoring: `sentry`
Code intelligence language servers	`typescript-lsp`, `pyright-lsp`, `gopls-lsp`, `rust-analyzer-lsp`, `clangd-lsp`, `csharp-lsp`, `jdtls-lsp`, `kotlin-lsp`, `lua-lsp`, `php-lsp`, `swift-lsp`. You must install the language-server binary yourself.
Security	`security-guidance` — reviews each change Claude makes for common vulnerabilities and has Claude fix what it finds in the same session
Development workflows	`commit-commands`, `pr-review-toolkit`, `agent-sdk-dev`, `plugin-dev`
Output styles	`explanatory-output-style`, `learning-output-style`

What it is NOT

- **Not an app store with payments.** It is a JSON catalogue in a repository. There is no billing mechanism.
- **Not Claude Code-only, not any more.** claude.ai has a **unified directory** covering skills, connectors *and* plugins, plus plugin marketplaces you can add yourself — and Team and Enterprise Owners can publish organisation marketplaces from **Organization settings** › **Plugins**. Section 11.3 covers both. What differs is the plumbing: the CLI reads `marketplace.json` from any Git source, while a GitHub-synced *organisation* marketplace accepts a narrower set of source types and requires a private repository.
- **Not curated by default.** Only `claude-plugins-official` is curated by Anthropic, and `claude-community` is validated and safety-screened. Any other marketplace is as trustworthy as whoever hosts it.
- **Not vetted by Anthropic in general.** Anthropic does not control what MCP servers, files or other software a third-party plugin contains, and cannot verify it works as intended.
- **Not a plugin itself.** Marketplace = catalogue. Plugin = item. See Chapter 4, Section 4.3.

11.2 When should I use it?

ADD A MARKETPLACE WHEN…

- You want a plugin that lives in it. That is the only reason to add one.
- Your team has built a shared setup and you want colleagues to install it in one command.
- You want **versioned, auto-updating** distribution rather than "copy this folder."

Do not create your own marketplace when…

- You are the only user → keep skills in `~/.claude/skills/`.
- One project's team needs it → commit to `.claude/` in that repo. Simpler, reviewed in PRs.
- You have one skill and no plans for more → a marketplace is overhead until there is something to catalogue.

When a private marketplace is genuinely the right answer

SITUATION	WHY A MARKETPLACE WINS
Several teams need the same skills	One install command each, instead of copying folders between repos and drifting apart.
The setup spans components	A plugin can carry skills *and* an MCP server *and* a subagent *and* hooks. Copying that by hand is four operations.
You need updates to propagate	Bump the version; installed users get it on their next session, or on auto-update.
The content must stay internal	Host the marketplace in a private repository. Only people with repo access can add it.
You want to standardise across the whole organisation	Combined with project settings, new joiners are prompted to install the standard toolkit on first use.

11.3 How do I set it up?

On claude.ai — the unified directory

In the browser and the desktop app there is no command to run. Everything installable — **skills, connectors and plugins** — is gathered in one directory:

1. **Open it**

   Click **Customize** in the left sidebar › choose the **Skills**, **Connectors** or **Plugins** tab › **+** › **Browse…**

   The tab you pick decides what the directory shows you. Each has an **Install** (or **Connect**) button per entry.

2. **Add a plugin marketplace**

   Anthropic provides built-in marketplaces, and a **Knowledge Work** marketplace is added for you by default. To add another:

   Customize › **Plugins** › in **Personal plugins**, click **+** › **Add marketplace**

CHOICE	WHAT IT DOES
Browse Anthropic sources	Pick from curated marketplaces — Knowledge Work, Life Sciences, Financial Services, Legal. **Add** next to each, then **Done**.
Add from a repository	Sync a marketplace from a GitHub repository or git URL — the same `marketplace.json` idea as the CLI, with the caveats in the box below.

   To remove one — including the default Knowledge Work marketplace — find it in the **Plugins** section, click the menu button in its right corner, and choose **Remove**.

For Team and Enterprise Owners — distributing to everyone

Owners and Primary Owners manage organisation marketplaces in **Organization settings › Plugins**. **Cowork and Skills must both be enabled for the organisation first**, or the option is unavailable. Two ways to fill a marketplace, usable in parallel:

METHOD	PATH AND CONSTRAINTS
Manual upload quick iteration, one-off tools	**Add plugins** › **Upload a file**. First time, *Upload to a new marketplace* is preselected and you name it; later you can *Add to an existing marketplace*. Files must be valid `.zip` under **50 MB**. Uploading a plugin with an existing name overwrites the previous version — no need to delete the old one.
GitHub syncing version-controlled, multiple authors	**Add plugin** › **GitHub**, then the repository in `owner/repo` form. The repository **must be private or internal** — public repos are not allowed for organisation marketplaces. github.com and self-hosted GitHub Enterprise Server both work.

A GITHUB-SYNCED ORG MARKETPLACE SUPPORTS FEWER SOURCE TYPES THAN THE CLI

In `marketplace.json`, **relative paths** to plugin folders inside the connected repository (`"source": "./plugins/my-plugin"`) are fully supported and are what you should use. The `github`, `url` and `git-subdir` source types work *only when the target repository is public* — which an org marketplace repo cannot be. `npm` and `pip` are not supported at all.

Consequence: if your plugins live in separate private repositories, copy those folders into the marketplace repository (a git submodule, subtree, or a CI step all work) and reference them with relative paths.

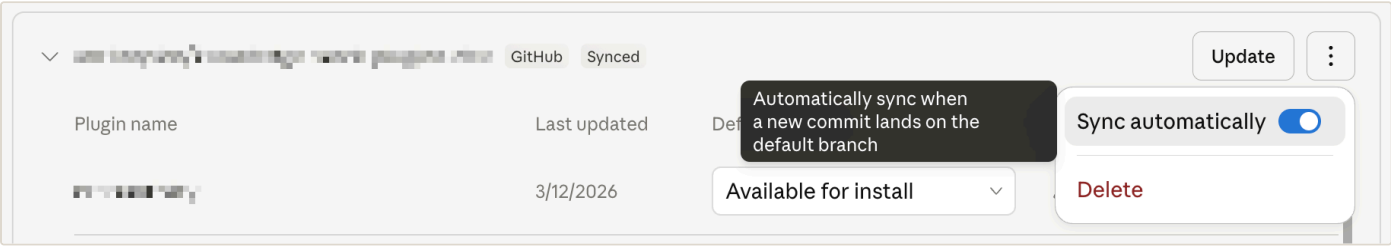


Figure 11.2 A GitHub-synced organisation marketplace. Note the **Synced** badge, the manual **Update** button, and **Sync automatically** — which re-syncs whenever a commit lands on the default branch. The per-plugin dropdown (*Available for install*) is where you decide whether a plugin is merely offered or pushed to everyone.

Screenshot: Anthropic, Claude Help Centre — "Manage plugins for your organization."

In Claude Code — adding an existing marketplace

Four source types are supported. All use the same command.

SOURCE	COMMAND
GitHub (`owner/repo`)	`/plugin marketplace add anthropics/claude-code`
Any Git host	`/plugin marketplace add https://gitlab.com/company/plugins.git`
Local path	`/plugin marketplace add ./my-marketplace`
Remote URL	`/plugin marketplace add https://example.com/marketplace.json`

```
# the community marketplace
> /plugin marketplace add anthropics/claude-plugins-community

# SSH, and pinning to a branch or tag with #ref
> /plugin marketplace add git@gitlab.com:company/plugins.git
> /plugin marketplace add https://gitlab.com/company/plugins.git#v1.0.0

# shortcuts: /plugin market, and rm for remove
> /plugin market list
```

TWO SYNTAX TRAPS ON NON-GITHUB HOSTS

- **Include the `.git` suffix** so Claude Code clones the repository rather than treating the URL as a direct link to a hosted `marketplace.json`.

- **Include** `https://`. A bare host such as `gitlab.com/company/plugins.git` is rejected as an invalid GitHub `owner/repo` shorthand.

Creating your own marketplace

1. **Create a repository for it**

   A private repository if the plugins are internal. This repo holds the catalogue and, typically, the plugins themselves.

2. **Add the catalogue file**

   It goes at `.claude-plugin/marketplace.json` in the repository root.

   `.claude-plugin/marketplace.json`

```
{
  "name": "acme-internal",
  "owner": {
    "name": "Acme Engineering Enablement"
  },
  "plugins": [
    {
      "name": "qa-toolkit",
      "source": "./plugins/qa-toolkit",
      "description": "House QA standards: test-case and bug-report skills, the Azure DevOps MCP
server, and a sprint-hygiene audit agent."
    },
    {
      "name": "ba-toolkit",
      "source": "./plugins/ba-toolkit",
      "description": "User story and retro-summary skills in our format, plus requirements-
traceability checks."
    }
  ]
}
```

   The `name` here is the marketplace name people install against (`qa-toolkit@acme-internal`). Note that a plugin's `name` in the marketplace entry can differ from the `name` in its own `plugin.json` — the marketplace entry name is what `/plugin install`, `enable` and `disable` use.

3. **Add the plugins alongside it**

```
acme-plugins/                          <- your marketplace repository
├── .claude-plugin/
│   └── marketplace.json
└── plugins/
    ├── qa-toolkit/
    │   ├── .claude-plugin/plugin.json
    │   ├── skills/
    │   │   ├── test-case/SKILL.md
    │   │   └── bug-report/SKILL.md
    │   ├── agents/sprint-auditor.md
    │   └── .mcp.json
    └── ba-toolkit/
        ├── .claude-plugin/plugin.json
        └── skills/
            ├── user-story/SKILL.md
            └── retro-summary/SKILL.md
```

   Chapter 12 covers what goes in a plugin and how to build one.

4 **Test it locally before pushing**

```
> /plugin marketplace add ./acme-plugins
> /plugin install qa-toolkit@acme-internal
```

5 **Share it**

Tell colleagues to run `/plugin marketplace add your-org/acme-plugins`, or better, configure it in project settings so they are prompted automatically — next step.

Configure it for a whole team automatically

Rather than asking everyone to run commands, declare the marketplace in the project's settings. When team members trust the repository folder, Claude Code prompts them to install.

`.claude/settings.json (committed)`

```
{
  "extraKnownMarketplaces": {
    "acme-internal": {
      "source": {
        "source": "github",
        "repo": "your-org/acme-plugins"
      }
    }
  },
  "enabledPlugins": {
    "qa-toolkit@acme-internal": true
  }
}
```

Administrators can additionally set `"autoUpdate": true` on an `extraKnownMarketplaces` entry in managed settings, so an organisation marketplace auto-updates without each user toggling it.

> **A PLUGIN DECLARED IN PROJECT SETTINGS STILL NEEDS INSTALLING**
>
> A plugin that only the project's `.claude/settings.json` enables, and that comes from an external source such as a GitHub repository, does not load until the team member installs it. Until then Claude Code reports the plugin as not installed and shows the `claude plugin install` command to run.

11.4 How do I use it?

Browse

```
> /plugin
```

Opens a tabbed panel. Cycle tabs with Tab, or Shift+Tab backwards:

TAB	WHAT IT SHOWS
Discover	Available plugins across all your marketplaces. Plugins relevant to your working directory can be pinned at the top with a **suggested for this directory** label.
Installed	Your installed plugins, grouped by scope, problems first. Also a **Not used recently** group.
Marketplaces	Add, update and remove marketplaces; toggle auto-update.
Errors	Plugin loading errors. The first place to look when something does not appear.

You can also browse the official catalogue in a browser at `claude.com/plugins`. In the VS Code extension, type `/plugins` for a graphical **Manage plugins** dialog with **Plugins** and **Marketplaces** tabs. If `/plugin` is unavailable in your environment, use the desktop app's plugin browser or declare plugins under `enabledPlugins` in `.claude/settings.json`.

Read the details before installing

Selecting a plugin in **Discover** shows a details pane with three things worth reading:

- **Context cost** — an estimate of how many tokens the plugin adds to your context window every turn.
- **Last updated** — staleness signal.
- **Will install** — the plugin's commands, agents, skills, hooks, MCP and LSP servers, so you can see exactly what it adds *before* installing.

Not every plugin supplies this data. For local or custom marketplaces you may see **Components will be discovered at installation** instead of a component list.

Verify the marketplace is registered

```
> /plugin marketplace list
```

11.5 How do I manage it?

ADD	`/plugin marketplace add <owner/repo | git-url | path | url>`, or the **Marketplaces** tab in `/plugin`.
LIST	`/plugin marketplace list`, or the **Marketplaces** tab.
UPDATE	`/plugin marketplace update <name>` refetches the catalogue. The refresh icon in the UI does the same.
AUTO-UPDATE	`/plugin` › **Marketplaces** › select one › **Enable auto-update**. On by default for official Anthropic marketplaces; **off** by default for third-party and local ones.
REMOVE	`/plugin marketplace remove <name>` — **this uninstalls every plugin you installed from it.**
PIN A VERSION	Append `#ref` when adding: `…/plugins.git#v1.0.0`. Useful for a controlled internal rollout.
RESTRICT	Organisations can limit which marketplaces users may add, via managed marketplace restrictions and the `strictKnownMarketplaces` setting.

How auto-update behaves

Claude Code checks for marketplace and plugin updates after your session starts, with a random delay of up to ten minutes, so the running session keeps the versions it launched with. If plugins were updated you are prompted to run `/reload-plugins`, or the new versions load next launch.

```
# turn off ALL auto-updates (Claude Code and plugins)
$ export DISABLE_AUTOUPDATER=1

# keep plugin auto-updates while managing Claude Code updates manually
$ export DISABLE_AUTOUPDATER=1
$ export FORCE_AUTOUPDATE_PLUGINS=1
```

Troubleshooting

SYMPTOM	LIKELY CAUSE	FIX
`Marketplace "claude-plugins-official" not found`	The automatic add failed — blocked network, or a marketplace policy blocked an earlier attempt	`/plugin marketplace add anthropics/claude-plugins-official`, then retry the install.
Plugin not found in the marketplace	Wrong name, or a stale catalogue	Check the name. With auto-update off, run `/plugin marketplace update <name>` and retry — with it on, Claude Code refreshes once and retries automatically.

SYMPTOM	LIKELY CAUSE	FIX
Adding a GitLab/Bitbucket URL fails	Missing `https://` or missing `.git`	Include both. See the caution in 11.3.
`/plugin` not recognised	Old Claude Code version	`claude --version`, then update: `npm install -g @anthropic-ai/claude-code@latest`, `brew upgrade claude-code`, or re-run the native installer. Restart afterwards.
"path not found" installing from a URL marketplace	URL-based marketplaces have limitations with relative plugin paths	Use a Git-based marketplace instead.
Plugin skills do not appear	Stale cache	`rm -rf ~/.claude/plugins/cache`, restart Claude Code, reinstall the plugin.
Removing a marketplace removed people's plugins	Working as designed	Expected behaviour. Announce marketplace removals.
Cannot add a marketplace at all	Organisation restriction	Managed marketplace restrictions may limit which sources are allowed. Ask your administrator.

11.6 Examples

Example 1 — install an integration from the official marketplace

```
> /plugin install github@claude-plugins-official
```

Installs a pre-configured GitHub MCP server — the Claude Code equivalent of switching on a connector.

Example 2 — try the demo marketplace

```
> /plugin marketplace add anthropics/claude-code
> /plugin                                # Discover tab
> /plugin install commit-commands@claude-code-plugins
> /commit-commands:commit                 # stages, writes a message, commits
```

Example 3 — a private internal marketplace

The realistic team scenario end to end:

```
# 1. The enablement owner creates a private repo with the catalogue and plugins
#    (structure as in 11.3)

# 2. They test it locally
> /plugin marketplace add ./acme-plugins
> /plugin install qa-toolkit@acme-internal
> /qa-toolkit:test-case AC-14              # confirm it works

# 3. They push, and commit .claude/settings.json to each team repo
#    so members are prompted on first use

# 4. A colleague clones the repo, trusts the folder, and is prompted:
$ claude plugin install qa-toolkit@acme-internal
> /reload-plugins
```

Example 4 — pin a version for a controlled rollout

```
# pilot team tracks the tip
> /plugin marketplace add your-org/acme-plugins
```

```
# everyone else pins to a known-good tag until the pilot signs off
> /plugin marketplace add https://github.com/your-org/acme-plugins.git#v1.2.0
```

11.7 Best practices

Recommended practices

- **Check the official marketplace before building anything.** GitHub, GitLab, Jira/Confluence, Asana, Linear, Notion, Figma, Slack, Sentry and a dozen language servers are already there.
- **One internal marketplace per organisation, not per team.** Several catalogues fragment discovery. Use one, with clearly-named plugins per team.
- **Read *Will install* before installing** a third-party plugin. It lists exactly what the plugin adds, including MCP servers and hooks.
- **Watch the context cost.** The details pane estimates it. Plugins are not free; several large ones add up on every turn.
- **Use project settings for team rollout** rather than instructions in a wiki. People will not run the command; they will accept a prompt.
- **Enable auto-update for your internal marketplace** so fixes actually propagate — third-party and local marketplaces default to *off*.
- **Announce removals.** Removing a marketplace uninstalls its plugins for everyone who added it.

Common mistakes

MISTAKE	WHY IT HURTS
Expecting `marketplace add` to install something	It registers a catalogue. Nothing is installed until you install a plugin.
Removing a marketplace to "tidy up"	Uninstalls every plugin from it, for you.
Assuming third-party marketplaces auto-update	They default to off. Your fix will not reach anyone.
A public repo for internal plugins	Internal skills often encode internal process. Use a private repository.
Building a plugin that duplicates an official one	Maintenance you did not need to take on.
Installing everything that looks interesting	Context cost every turn, plus startup cost. Check the **Not used recently** group periodically.

Naming conventions

- **Marketplace name:** short, lowercase, organisation-identifying — `acme-internal`. It appears in every install command and every plugin identifier.
- **Plugin names:** role or domain plus `-toolkit` or `-tools` — `qa-toolkit`, `ba-toolkit`, `design-tools`. Keep them short, because skills are namespaced with the plugin name and users type the whole thing.
- **Avoid version numbers in names.** Use the `version` field and Git tags.

Versioning and updates

- Version each plugin in its `plugin.json`. Users receive updates when you bump it.
- Tag releases in the marketplace repository so consumers can pin with `#ref`.
- Keep a `CHANGELOG.md` per plugin. People need to know whether an update changes their output.
- Treat a skill change inside a shipped plugin as a behavioural change to everyone's work. Review it.

Security considerations

> **MARKETPLACES AND PLUGINS ARE HIGHLY TRUSTED COMPONENTS**

> Anthropic's own documentation is direct about this: plugins and marketplaces **can execute arbitrary code on your machine with your user privileges**. Only install plugins and add marketplaces from sources you trust.
>
> - Anthropic does not control what a third-party plugin contains and cannot verify it works as intended.
> - `claude-community` plugins pass automated validation and safety screening and are pinned to a commit SHA — that is a real check, not a guarantee of intent.
> - Review the **Will install** list, and be especially attentive to **hooks** (which run shell commands at lifecycle events) and **MCP servers** (which reach external systems).
> - Organisations can restrict which marketplaces users may add. If you are deploying at scale, use managed marketplace restrictions rather than a policy document.
> - For internal plugins, a private repository is both an access control and an audit trail — changes arrive as reviewable commits.

Team and organisational considerations

- **Name an owner for the internal marketplace.** Someone must merge, tag and announce.
- **Treat it like a shared library.** Pull requests, review, changelog, semantic-ish versioning. It changes how colleagues work.
- **Pilot before broad rollout** — pin the wider audience to a tag while a small group tracks the tip.
- **Publish the catalogue in human form too.** A one-page internal note listing each plugin and what it gives you drives adoption far better than expecting people to browse **Discover**.
- **Submitting to the community marketplace** is possible via in-app forms — claude.ai admin settings (requires a Team or Enterprise organisation with directory management access) or the Console form for individual authors. Run `claude plugin validate ./your-plugin` first.

11.8 Official resources

Discover and install prebuilt plugins through marketplaces — the source for this chapter's commands, tabs and official plugin list
`https://code.claude.com/docs/en/discover-plugins`

Create and distribute a plugin marketplace — `marketplace.json` schema, private repositories, managed restrictions, troubleshooting
`https://code.claude.com/docs/en/plugin-marketplaces`

Browse skills, connectors, and plugins in one directory — the claude.ai unified directory and the "Your organization" tab
`https://support.claude.com/en/articles/14328846-browse-skills-connectors-and-plugins-in-one-directory`

Manage plugins for your organization — Organization settings › Plugins: Anthropic-built marketplaces, manual upload, GitHub syncing and its source-type limits
`https://support.claude.com/en/articles/13837433-manage-plugins-for-your-organization`

The official plugin catalogue, in a browser
`https://claude.com/plugins`

anthropics/claude-plugins-community — the community catalogue
`https://github.com/anthropics/claude-plugins-community`

anthropics/claude-code — the demo marketplace and example plugins
`https://github.com/anthropics/claude-code`

Plugin settings — `extraKnownMarketplaces`, `enabledPlugins`, `strictKnownMarketplaces`
`https://code.claude.com/docs/en/settings`

CHAPTER 12

Plugins

Bundle a working setup — skills, agents, MCP servers, hooks — and hand it to the team in one command.

ADVANCED | CLAUDE CODE CLI | VS CODE EXTENSION | DESKTOP APP | CLAUDE.AI | COWORK

CLAUDE CODE PLANS · CLAUDE.AI PLUGINS: PRO / MAX / TEAM / ENTERPRISE

12.1 What is it?

A plugin is a self-contained directory that bundles capabilities and installs as one unit. It can carry skills, subagents, hooks, MCP servers, commands, language servers, background monitors, workflows and default settings — any combination. Distributed through a marketplace (Chapter 11), it is how one person's working setup becomes everyone's.

The problem it solves

By this point in the guide you can build a genuinely good setup: a `/test-case` skill, a `/bug-report` skill, an Azure DevOps MCP connection, a subagent that audits sprint hygiene, and a hook that runs the linter. That is five separate pieces of configuration in four different places.

Handing that to a colleague without a plugin means a wiki page with five sections, and a guarantee that in six weeks half the team has version A and half has version B. A plugin makes it one install, one version, one update path.

Standalone configuration vs. a plugin

APPROACH	SKILL NAMES	BEST FOR
Standalone the `.claude/` directory	`/hello`	Personal workflows, project-specific customisation, quick experiments.
Plugin a directory with a manifest	`/plugin-name:hello`	Sharing with teammates, distributing to a community, versioned releases, reuse across projects.

The recommended path: start standalone in `.claude/` for quick iteration, then convert to a plugin when you are ready to share. Section 12.5 covers the conversion.

What a plugin can contain

DIRECTORY / FILE	CONTAINS	NOTES
`.claude-plugin/`	`plugin.json` manifest	The *only* thing that goes in here. Optional if components use default locations.
`skills/`	`<name>/SKILL.md` folders	Chapter 7. Namespaced as `/plugin-name:skill-name`.
`commands/`	Flat Markdown files	Legacy layout — use `skills/` for new plugins.
`agents/`	Subagent definitions	Chapter 14.
`hooks/hooks.json`	Event handlers	Shell commands run at lifecycle events. The enforcement mechanism.

DIRECTORY / FILE	CONTAINS	NOTES
`.mcp.json`	MCP server configuration	Chapter 8. This is how integration plugins work.
`.lsp.json`	Language server configuration	Gives Claude real-time diagnostics and code navigation.
`monitors/monitors.json`	Background monitors	Watch logs or files and notify Claude as events arrive.
`workflows/`	Dynamic workflow scripts	Chapter 15. Namespaced the same way as skills.
`bin/`	Executables	Added to the Bash tool's `PATH` while the plugin is enabled.
`settings.json`	Default settings	Applied when enabled. Currently supports `agent` and `subagentStatusLine`.

THE MISTAKE EVERYONE MAKES ONCE

Do **not** put `commands/`, `agents/`, `skills/` or `hooks/` inside `.claude-plugin/`. Only `plugin.json` goes in there. Everything else sits at the **plugin root** — the individual plugin's own directory, the one you pass to `--plugin-dir` or that contains `.claude-plugin/plugin.json`. It is never `~/.claude/`.

What it is NOT

- **Not a marketplace.** A plugin is an item; a marketplace is the catalogue. Chapter 11.
- **Not an MCP server.** A plugin may *contain* one. See Chapter 4, Section 4.3.
- **No longer Claude Code-only** — a change worth flagging if you learned plugins earlier. Plugins install and run on claude.ai, in the Desktop **Chat** tab and in Cowork on all paid plans. What does *not* carry over is the execution components: a plugin's **skills** work on every surface, but its **hooks and subagents run only in Cowork** and appear greyed out in chat. Section 12.3 covers both paths.
- **Not a fully portable unit.** Following from the above: "installed" does not mean "everything in it is active." Check which components a plugin actually contributes on the surface you are using before you rely on it.
- **Not free of cost.** An enabled plugin's components consume context every turn. Claude Code shows a **Context cost** estimate before install, and flags plugins you have stopped using.
- **Not sandboxed.** Plugins can execute arbitrary code with your user privileges. Section 12.7.

How it relates to the other blocks

A plugin is the **packaging layer**. It is the only block whose purpose is to carry other blocks:

What a realistic internal plugin bundles

```
qa-toolkit/
├── .claude-plugin/plugin.json          the manifest — name, version, author
├── skills/                             ← Chapter 7  (know-how)
│   ├── test-case/SKILL.md
│   └── bug-report/SKILL.md
├── agents/sprint-auditor.md            ← Chapter 14 (execution)
├── .mcp.json                           ← Chapter 8  (reach: Azure DevOps)
├── hooks/hooks.json                    ← enforcement (run the linter)
└── README.md                           for humans
```

One `/plugin install` and a colleague has all four layers configured.

12.2 When should I use it?

CREATE A PLUGIN WHEN…

- You want to **share a setup with teammates** rather than describe it.
- The setup spans **more than one component** — skills plus an MCP server plus an agent.
- You need **versioned releases** so people can pin or upgrade deliberately.
- The same setup should apply across **several repositories**.
- You want changes to **propagate automatically** rather than by announcement.

Stay standalone when…

- It is just for you → `~/.claude/skills/`.
- One repository's team needs it → commit to that repo's `.claude/`. Simpler, and reviewed in pull requests.
- You are still iterating → iterate standalone, convert when it settles.
- It is one small skill → a plugin is overhead until there is something to bundle.

Typical use cases by role

PLUGIN	BUNDLES
qa-toolkit	`/test-case`, `/bug-report`, `/regression-scope`, the Azure DevOps MCP server, a `sprint-auditor` agent
ba-toolkit	`/user-story`, `/retro-summary`, `/traceability-check`, a requirements-reviewer agent
design-tools	`/handoff-spec`, `/a11y-audit`, the Figma MCP server, design-token reference material
house-standards	Path-scoped rules, a commit-message skill, a hook that runs the linter, default settings

12.3 How do I set it up?

PLUGINS ARE NO LONGER ONLY A CLAUDE CODE FEATURE

Plugins now install and run on **claude.ai in the browser**, in the **Chat** tab of Claude Desktop, and in **Cowork** — on all paid plans (Pro, Max, Team, Enterprise). Same idea, different front door and one important limitation:

The skills in a plugin work in all three surfaces. Hooks and subagents only run in Cowork, so in chat they appear greyed out. If you install a plugin in chat and its subagents seem missing, that is why. Read Path A below if you work in the browser and Path B if you work in a terminal or repository.

Path A — claude.ai, Desktop chat and Cowork

1 **Open Customize**

Click **Customize** in the left sidebar › **Plugins** tab › **Browse plugins**

Customize is the one place that gathers your plugins, skills and connectors. In Cowork, open the **Cowork** tab first, then **Customize**.

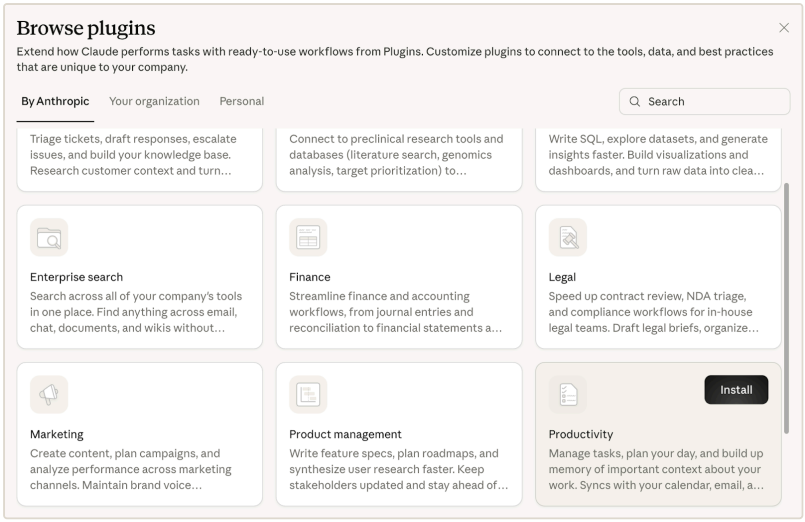


Figure 12.1 The **Browse plugins** catalogue. The three tabs are the thing to notice: **By Anthropic** (ready-made, per function), **Your organization** (whatever your Owners have distributed — Chapter 11) and **Personal**. Install is a single click per card.
Screenshot: Anthropic, Claude Help Centre — "Use plugins in Claude."

2 Install one, or upload your own

Click **Install** on any card. Anthropic ships plugins for sales, finance, legal, marketing, HR, engineering, design, operations and data analysis, each pre-configured with the skills *and connectors* that function needs — which is the real convenience: you are not connecting Drive, Gmail and Slack one at a time.

You can also upload a plugin you built or received from a colleague. On Claude Desktop and in Cowork, plugins you add yourself are stored **locally on your computer**.

3 Use its skills

Type `/` or click **+** to see the skills your installed plugins contribute, in chat and in Cowork alike. Click any skill to read its details before running it.

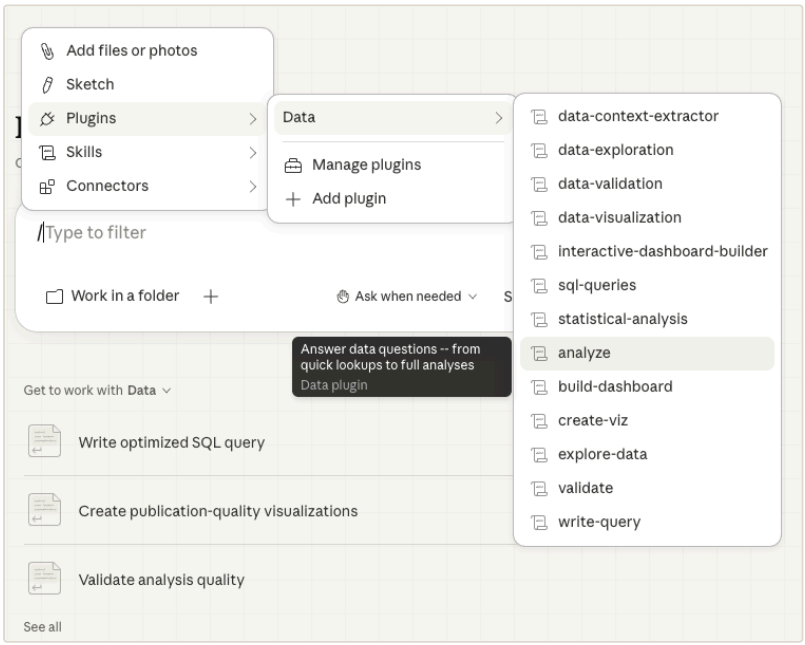


Figure 12.2 The **+** menu is the single entry point for everything you have installed — **Plugins**, **Skills** and **Connectors** sit side by side. Here the *Data* plugin's skills are listed, with **Manage plugins** and **Add plugin** at the top of its submenu.
Screenshot: Anthropic, Claude Help Centre — "Use plugins in Claude."

4

Build one, if you want to

Anthropic ships a plugin called **Plugin Create** whose job is to help you build plugins. You can also start from any Anthropic-built plugin as a template and modify it. In Cowork, open an installed plugin and click **Customize** in the upper right — it opens a new Cowork task pre-loaded with a prompt to adapt that plugin's skills and connectors to how you actually work.

> **WHAT YOU ARE TRUSTING WHEN YOU INSTALL**
>
> A plugin may include **local MCP servers that run on your computer with the same permissions as any other program you run**. Install only from sources you trust. On Enterprise plans an administrator may restrict which plugins you can install, or disable local MCP servers altogether. If your organisation has **skill and plugin scanning** on (Chapter 21), plugins are checked on install and update: malicious content is blocked outright, and anything questionable shows a caution banner.

Organisation-managed plugins behave like any other, with three differences: you cannot edit them, some may be auto-installed or *required* (auto-installed ones you can uninstall, required ones you cannot), and on Enterprise the catalogue you see may differ from a colleague's in another group.

Path B — Claude Code

Installing an existing plugin

Assumes the marketplace is already added — Chapter 11.

```
> /plugin install <plugin-name>@<marketplace-name>

# from the official marketplace
> /plugin install github@claude-plugins-official
```

The command opens the plugin's details, where you choose an **installation scope**:

SCOPE	MEANING
User	Install for yourself across all projects.
Project	Install for all collaborators on this repository — adds the plugin to `.claude/settings.json`.
Local	Install for yourself in this repository only, not shared.
Managed	*You may see this but cannot change it.* Installed by administrators via managed settings.

To install without an interactive step — useful in scripts and setup instructions:

```
$ claude plugin install qa-toolkit@acme-internal            # user scope by default
$ claude plugin install formatter@your-org --scope project
```

Reading the install summary

Claude Code tells you whether the plugin is active in your current session:

MESSAGE	MEANING
`Plugin is now active.`	Nothing more to do.
`Run /reload-plugins to activate.`	Not active yet — either because activating would invalidate the prompt cache, or the attempt failed. Run the command.
A failure report	Check the `/plugin` **Errors** tab for detail.

If `/reload-plugins` warns that reloading would re-read the conversation, rerun it as `/reload-plugins --force`. The `claude plugin install` shell command does not run inside a session, so its plugins load next launch or on `/reload-plugins`.

Building your own plugin

1 Create the plugin directory

```
$ mkdir my-first-plugin
$ mkdir my-first-plugin/.claude-plugin
```

Location does not matter yet — you will point Claude Code at it with `--plugin-dir`.

2 Write the manifest

`my-first-plugin/.claude-plugin/plugin.json`

```
{
  "name": "my-first-plugin",
  "description": "A greeting plugin to learn the basics",
  "version": "1.0.0",
  "author": {
    "name": "Your Name"
  }
}
```

FIELD	PURPOSE
`name`	Unique identifier *and skill namespace*. Skills become `/my-first-plugin:hello`.
`description`	Shown in the plugin manager when browsing or installing.
`version`	Optional. If set, users receive updates only when you bump it.
`author`	Optional. Helpful for attribution — and for knowing who to ask.

Additional fields such as `homepage`, `repository` and `license` are available — see the full manifest schema in the plugins reference.

3 Add a skill

```
$ mkdir -p my-first-plugin/skills/hello
```

`my-first-plugin/skills/hello/SKILL.md`

```
---
description: Greet the user with a friendly message
disable-model-invocation: true
---

Greet the user warmly and ask how you can help them today.
```

The folder name becomes the skill name, prefixed with the plugin's namespace.

4 Test it locally

```
$ claude --plugin-dir ./my-first-plugin
> /my-first-plugin:hello
```

Run `/help` and open the **Custom commands** tab to see your skill listed under the plugin namespace.

Load several at once by repeating the flag. The flag also accepts a `.zip` archive. When a `--plugin-dir` plugin shares a name with an installed one, the local copy wins for that session — handy for testing a change to something you already have installed.

5 **Add arguments**

SKILL.md — with $ARGUMENTS

```
---
description: Greet the user with a personalized message
---

# Hello Skill

Greet the user named "$ARGUMENTS" warmly and ask how you can help them
today. Make the greeting personal and encouraging.
```

```
> /reload-plugins
> /my-first-plugin:hello Alex
```

The `/reload-plugins` skills count covers only `commands/` directories, so it can report `0 skills` even though the skill you just edited did reload. Do not be alarmed by that.

6 **Iterate without relaunching**

Run `/reload-plugins` after changes. It reloads plugins, skills, agents, hooks, plugin MCP servers and plugin LSP servers.

7 **Validate before you distribute**

```
$ claude plugin validate ./my-first-plugin
# ✓ Validation passed          (or "passed with warnings")
# add --strict to treat warnings as errors
```

8 **Publish it in a marketplace**

Add an entry to your `marketplace.json` — Chapter 11, Section 11.3.

Develop in your skills directory instead of using `--plugin-dir`

If you would rather not pass a flag every launch:

```
$ claude plugin init my-tool
```

This scaffolds `~/.claude/skills/my-tool/` with a `.claude-plugin/plugin.json` manifest and a starter `SKILL.md`. On the next session it loads as `my-tool@skills-dir` with no marketplace or install step. In a project's `.claude/skills/` this requires accepting the workspace trust dialog first.

Adding the other component types

hooks/hooks.json — run the formatter after every edit

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Write|Edit",
        "hooks": [
          {
            "type": "command",
            "command": "jq -r '.tool_input.file_path' | xargs npm run lint:fix"
          }
        ]
      }
```

```
      ]
    }
  }
```

Hook input arrives as JSON on stdin, which is why the example pipes through `jq` to extract the file path. This is the mechanism that turns an instruction into a guarantee — see Chapter 6, Section 6.6.

`.lsp.json — a language server for a language without an official plugin`

```
{
  "go": {
    "command": "gopls",
    "args": ["serve"],
    "extensionToLanguage": { ".go": "go" }
  }
}
```

Users must have the language-server binary installed. If it is missing, the `/plugin` **Errors** tab shows `Executable not found in $PATH`. An entry with invalid configuration is skipped silently instead — run `claude --debug` to see why. For common languages, install the official LSP plugins rather than writing your own.

`monitors/monitors.json — watch a log in the background`

```
[
  {
    "name": "error-log",
    "command": "tail -F ./logs/error.log",
    "description": "Application error log"
  }
]
```

Each stdout line is delivered to Claude as a notification during the session. Claude Code starts monitors automatically when the plugin is active — you do not have to ask Claude to start watching.

`settings.json — change default behaviour when enabled`

```
{
  "agent": "security-reviewer"
}
```

Setting `agent` activates one of the plugin's custom agents as the main thread, applying its system prompt, tool restrictions and model. This lets a plugin change how Claude Code behaves by default. Only `agent` and `subagentStatusLine` are supported; unknown keys are silently ignored.

12.4 How do I use it?

Using an installed plugin

You mostly do not think about the plugin — you use its components:

```
# its skills, namespaced
> /qa-toolkit:test-case AC-14

# its agents, by scoped name or automatically
> Use the sprint-auditor agent to check this sprint's hygiene

# its MCP tools, just by describing the outcome
> Read work item 4821 from Azure DevOps
```

How to verify a plugin installed correctly

1. **Check the install summary** — active, or needs `/reload-plugins`.
2. **List what it contributed:**

```
$ claude plugin details qa-toolkit@acme-internal
```

Or open the plugin's detail view in `/plugin` › **Installed**. Both show the commands, skills, agents, hooks, MCP servers and LSP servers it contributes.

3. **Test each component type separately:**
   - Skills: invoke `/plugin-name:skill-name`.
   - Agents: check they appear in `/context` under **Custom Agents**, or @-mention one by its scoped name.
   - MCP servers: `/mcp` and confirm **connected**.
   - Hooks: trigger the matching event — ask Claude to edit a file for a `PostToolUse` hook — and confirm the effect. Claude Code records which hooks matched, their exit codes and output in the debug log.
4. **Check the Errors tab** in `/plugin` if anything is missing.

12.5 How do I manage it?

INSTALL	`/plugin install <name>@<marketplace>`, or `/plugin` › **Discover** › Enter. Non-interactive: `claude plugin install`.
LIST	`/plugin list` (add `--enabled` or `--disabled`), or the **Installed** tab.
INSPECT	`claude plugin details <name>@<marketplace>`, or the detail view in the **Installed** tab.
ENABLE	`/plugin enable <name>@<marketplace>`
DISABLE	`/plugin disable <name>@<marketplace>` — keeps it installed but inactive. The right move for something you might want back.
UNINSTALL	`/plugin uninstall <name>@<marketplace>`. If a project's settings enable it, Claude Code asks whether you mean just you (writes an override to `.claude/settings.local.json`) or everyone (removes it from the shared `.claude/settings.json`).
UPDATE	Automatic if auto-update is on for its marketplace; otherwise `/plugin marketplace update <marketplace>`.
APPLY CHANGES	`/reload-plugins` (or `--force` if it warns about the prompt cache).
FAVOURITE	Press f on a plugin in the **Installed** list to pin it to the top.
AUDIT	The **Not used recently** group lists marketplace plugins you installed but have not used in at least two weeks across at least ten sessions, with a **Last used** line. Your prompt to disable or uninstall.
SCRIPTING	Use the `claude plugin` shell commands rather than the slash commands — the slash versions open the panel and leave it open (press Esc to close before typing another).

Converting standalone configuration into a plugin

The recommended path: iterate in `.claude/`, convert when it settles.

1 Create the plugin structure alongside your existing `.claude/`

```
$ mkdir -p my-plugin/.claude-plugin
```

`my-plugin/.claude-plugin/plugin.json`

```
{
  "name": "my-plugin",
  "description": "Migrated from standalone configuration",
  "version": "1.0.0"
}
```

2 Copy the directories you have

```
$ cp -r .claude/commands my-plugin/
$ cp -r .claude/agents   my-plugin/
$ cp -r .claude/skills   my-plugin/
```

You may not have all three; a missing one just prints *No such file or directory*. Run `ls my-plugin` to confirm what came across.

3 Move hooks out of settings into the plugin

```
$ mkdir my-plugin/hooks
```

Copy the `hooks` object from your `.claude/settings.json` into `my-plugin/hooks/hooks.json` — the format is the same.

4 Test the migrated plugin

```
$ claude --plugin-dir ./my-plugin
```

Test each component: run the commands, check agents appear in `/context`, trigger each hook's event.

5 Remove the originals

TWO DIFFERENT OVERRIDE BEHAVIOURS — KNOW WHICH YOU ARE HITTING

- **Agents:** project and user `.claude/agents/` definitions *override* same-named plugin agents. The plugin version only takes effect once you delete the originals.
- **Skills:** plugin skills are namespaced, so `/skill-name` and `/plugin-name:skill-name` both remain available — neither overrides the other. You will have two copies until you remove one.

What changes when you migrate

STANDALONE (`.CLAUDE/`)	PLUGIN
Only available in one project	Shareable via marketplaces
Files in `.claude/commands/`	Files in `plugin-name/commands/`
Hooks in `settings.json`	Hooks in `hooks/hooks.json`
Manual copy to share	`/plugin install`

Troubleshooting

SYMPTOM	LIKELY CAUSE	FIX
Components missing after install	Directories nested inside `.claude-plugin/`	Only `plugin.json` goes there. Everything else at the plugin root.

SYMPTOM	LIKELY CAUSE	FIX
Plugin installed but nothing works	Not activated in this session	`/reload-plugins`, or `--force` if warned about the prompt cache.
Plugin skills never appear	Stale cache	`rm -rf ~/.claude/plugins/cache`, restart, reinstall.
Language server not starting	Binary not installed or not on `$PATH`	Install it. Check the `/plugin` **Errors** tab for `Executable not found in $PATH`.
LSP entry silently skipped	Invalid configuration	Run `claude --debug` to see the reason.
High memory use	Language servers on a big project	`rust-analyzer` and `pyright` can be heavy. `/plugin disable <name>` and rely on Claude's built-in search.
Enable/Disable does nothing	The marketplace-entry name differs from the `plugin.json` name, on an older version	Update Claude Code. From v2.1.195, both names are accepted; earlier versions report `already disabled` and leave it enabled.
Plugin declared in project settings but not installed	External-source plugins need an explicit install	Run the `claude plugin install` command Claude Code reports.
Cannot override a force-enabled plugin with `--plugin-dir`	Managed settings force it	By design. Managed force-enable and force-disable cannot be overridden locally.
Context feels bloated	Too many enabled plugins	Check **Not used recently** and the per-plugin **Context cost**; disable what you do not use.

12.6 Examples

Example 1 — install the integrations your team actually needs

```
> /plugin install github@claude-plugins-official
> /plugin install atlassian@claude-plugins-official
> /plugin install figma@claude-plugins-official
> /plugin install typescript-lsp@claude-plugins-official
> /plugin install security-guidance@claude-plugins-official
```

Remember to install the language-server binary for `typescript-lsp` (`typescript-language-server`).

Example 2 — the QA toolkit, end to end

Structure

```
qa-toolkit/
├── .claude-plugin/plugin.json
├── skills/
│   ├── test-case/SKILL.md          # Chapter 7, Example 1
│   └── bug-report/SKILL.md         # Chapter 7, Example 2
├── agents/
│   └── sprint-auditor.md           # Chapter 14
├── .mcp.json                       # the Azure DevOps server
└── README.md
```

.claude-plugin/plugin.json

```
{
  "name": "qa-toolkit",
  "description": "House QA standards: test-case and bug-report skills in our format, the Azure DevOps MCP
server, and a sprint-hygiene audit agent.",
  "version": "1.3.0",
```

```
  "author": { "name": "Acme QA Enablement" },
  "repository": "https://github.com/your-org/acme-plugins"
}
```

`.mcp.json`

```
{
  "mcpServers": {
    "ado": {
      "type": "http",
      "url": "https://your-ado-mcp-endpoint/mcp"
    }
  }
}
```

A colleague's entire setup process

```
> /plugin marketplace add your-org/acme-plugins
> /plugin install qa-toolkit@acme-internal
> /mcp                                    # authenticate ADO once
> /qa-toolkit:test-case AC-14              # working
```

Example 3 — house standards as an enforced plugin

Where a plugin does something instructions cannot: guarantee behaviour.

`house-standards/hooks/hooks.json`

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Write|Edit",
        "hooks": [
          { "type": "command", "command": "jq -r '.tool_input.file_path' | xargs npx prettier --write" }
        ]
      }
    ]
  }
}
```

Every file Claude writes or edits is formatted, regardless of whether Claude remembered to. That is the difference between an instruction and a hook — Chapter 6, Section 6.6.

Example 4 — a plugin that changes Claude's default persona

`security-review-mode/settings.json`

```
{
  "agent": "security-reviewer"
}
```

With this plugin enabled, the `security-reviewer` agent defined in the plugin's `agents/` directory becomes the main thread — its system prompt, tool restrictions and model apply by default. Disable the plugin to go back to normal. A neat way to ship a "mode."

12.7 Best practices

Recommended practices

- **Start standalone, convert when it settles.** Iterating on a plugin is slower than iterating on a folder.

- **One plugin per audience, not per component.** `qa-toolkit` beats four separate plugins the same people always install together.
- **Ship a** `README.md`. Installation, what it provides, who owns it. People will read this before they read any skill.
- **Validate before distributing:** `claude plugin validate ./your-plugin`. The community review pipeline runs the same check.
- **Set** `version` **and bump it.** Without it, users may not receive your updates in the way you expect.
- **Keep the namespace short.** Users type `/qa-toolkit:test-case` in full.
- **Mind the context cost.** Check what your plugin adds; a plugin whose MCP tools are not deferred behind tool search costs more on every turn.
- **Test all four component types** before release — skills, agents, hooks and MCP servers each fail differently.

Common mistakes

MISTAKE	WHY IT HURTS
Directories inside `.claude-plugin/`	Components silently do not load. The most common structural error.
Assuming the plugin root is `~/.claude/`	It is the plugin's own directory. Claude Code does not read `~/.claude/.mcp.json`.
Forgetting `/reload-plugins`	You conclude the plugin is broken when it simply is not active yet.
Leaving duplicate standalone copies after migrating	Agents: the standalone one wins. Skills: you get both. Either way, confusion.
No version field, then expecting updates to land	Version drives the update path.
Shipping hooks without documenting them	Hooks run shell commands. A colleague deserves to know before installing.
One giant plugin for the whole organisation	Everyone pays the context cost of everyone else's tools. Split by audience.

Naming conventions

- **Plugin name:** lowercase, hyphenated, audience-or-domain based — `qa-toolkit`, `design-tools`, `house-standards`. This is the skill namespace, so short matters.
- **Skills inside:** verb-noun, as in Chapter 7 — `test-case`, `bug-report`.
- **Agents inside:** role nouns — `sprint-auditor`, `security-reviewer`.
- **Keep the marketplace-entry name and the** `plugin.json` **name identical** unless you have a specific reason. They are allowed to differ, and it confuses everyone when they do.

Versioning and updates

- Semantic-ish versioning is enough: patch for fixes, minor for new components, major when output changes shape.
- A `CHANGELOG.md` per plugin. When a skill changes, people's output changes; they need to know.
- For a controlled rollout, pin the wider audience to a marketplace tag while a pilot group tracks the tip — Chapter 11, Example 4.
- Approved community plugins are pinned to a specific commit SHA in the catalogue, and CI bumps the pin as you push. The public catalogue syncs nightly, so allow a delay between approval and installability.

Security considerations

> **PLUGINS CAN EXECUTE ARBITRARY CODE WITH YOUR PRIVILEGES**
>
> - **Only install from sources you trust.** Anthropic does not control what MCP servers, files or other software a plugin includes, and cannot verify that it works as intended.
> - **Hooks are the highest-risk component.** They run shell commands at lifecycle events. Read a plugin's hooks before installing it.

- `bin/` **adds executables to the Bash tool's** `PATH` while the plugin is enabled. Know what is in there.
- **Review the *Will install* list** in the details pane — it enumerates commands, agents, skills, hooks, MCP and LSP servers before you commit.
- `--plugin-url` **fetches a remote archive at startup** and loads it for that session. Same trust considerations as any plugin source: point it only at archives you control.
- **Organisations should use managed settings** to force-enable an approved set and force-disable the rest, plus managed marketplace restrictions. These cannot be overridden with `--plugin-dir`.
- **Enterprise skill and plugin scanning** for malicious content is available — ask your administrator whether it is enabled.

Team and organisational considerations

- **Plugins are the right unit for organisation-wide standards** — versioned, auto-updating, one-command install, and they can carry all five layers at once.
- **Name an owner per plugin** in `plugin.json` and the README.
- **Use project settings for rollout,** so new joiners get prompted rather than briefed — Chapter 11, Section 11.3.
- **Split by audience so people pay only for what they use.** QA should not carry the design plugin's context cost.
- **Review plugin changes like code.** A skill edit inside a shipped plugin changes everyone's output.

12.8 Official resources

Use plugins in Claude — the claude.ai / Desktop / Cowork side: where plugins work, installing from Customize, Plugin Create, org-managed plugins
`https://support.claude.com/en/articles/13837440-use-plugins-in-claude`

Manage plugins for your organization — for Owners: Organization settings › Plugins, manual and GitHub-synced marketplaces
`https://support.claude.com/en/articles/13837433-manage-plugins-for-your-organization`

Create plugins — the quickstart, structure overview, component types, migration and debugging guidance used in this chapter
`https://code.claude.com/docs/en/plugins`

Discover and install prebuilt plugins — installing, scopes, managing, the official plugin list
`https://code.claude.com/docs/en/discover-plugins`

Plugins reference — the complete manifest schema, component path fields, version management, skills-directory plugins, debugging tools
`https://code.claude.com/docs/en/plugins-reference`

Create and distribute a plugin marketplace
`https://code.claude.com/docs/en/plugin-marketplaces`

Hooks — the event handlers a plugin can ship
`https://code.claude.com/docs/en/hooks`

Settings — `enabledPlugins`, configuration scopes, managed settings
`https://code.claude.com/docs/en/settings`

CHAPTER 13

Routines

Put Claude on autopilot: work that runs on a schedule, an API call, or a GitHub event — with your laptop closed.

ADVANCED | CLAUDE.AI/CODE | DESKTOP APP | CLAUDE CODE CLI

PRO / MAX / TEAM / ENTERPRISE, WITH CLAUDE CODE ON THE WEB ENABLED

RESEARCH PREVIEW

Anthropic labels Routines a **research preview**: behaviour, limits and the API surface may change. Build with that in mind, and re-check the official documentation before making one load-bearing.

13.1 What is it?

A routine is a *saved Claude Code configuration* — a prompt, one or more repositories, an environment, and a set of connectors — packaged once and run automatically. Routines execute on Anthropic-managed cloud infrastructure, so they keep working when your laptop is closed.

The problem it solves

Some work is valuable, repeatable and completely unrewarding to do at the moment it needs doing. Grooming the backlog every weeknight. Checking whether the docs still match the API after this week's merges. Reviewing every PR against your own checklist. Verifying a deploy at 2am.

These are tasks where the human contribution is the *specification*, not the execution. A routine lets you write the specification once and stop being present for it.

How it works: three trigger types

A routine can have one or more triggers, and a single routine can combine them. A PR-review routine can run nightly, be triggered from a deploy script, *and* react to every new pull request.

TRIGGER	FIRES WHEN	USE IT FOR
Schedule	A recurring cadence — hourly, daily, weekdays, weekly — or once at a specific future time	Backlog grooming, docs-drift checks, weekly summaries, a one-off reminder to yourself next Tuesday.
API	You POST to a per-routine endpoint with a bearer token	Wiring Claude into alerting, deploy pipelines and internal tools. Anything that can make an authenticated HTTP request.
GitHub	A repository event — pull request or release — optionally filtered	Bespoke code review, backports, porting a change to a parallel SDK.

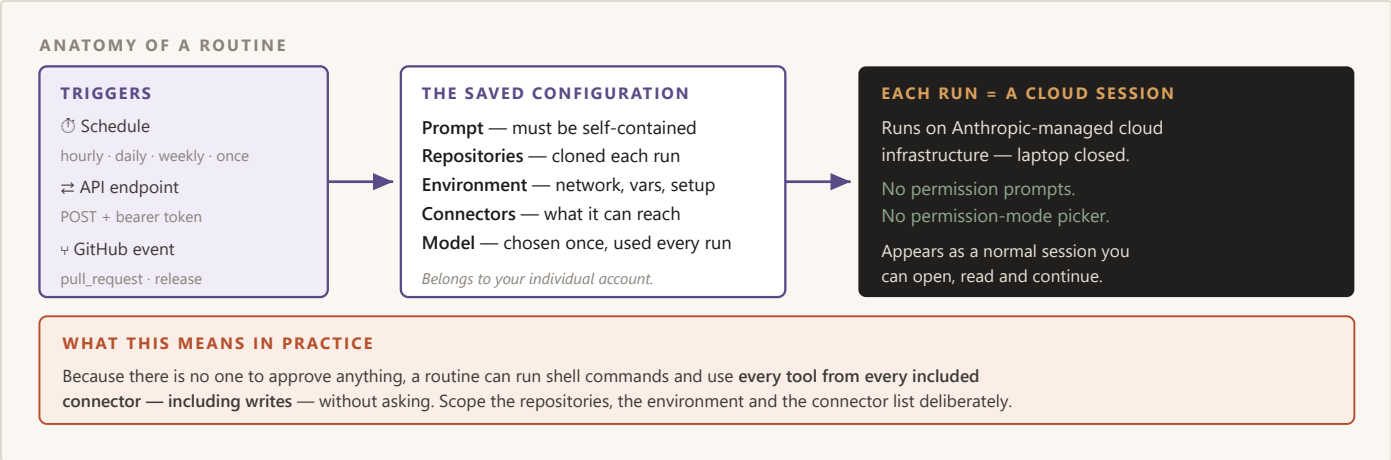


Figure 13.1 A routine is a saved configuration plus triggers. Each run is a full, autonomous cloud session.
Diagram — not a screenshot. Based on Anthropic's routines documentation.

What it is NOT

- **Not a background process on your machine.** Routines run in the cloud. The local equivalents are **Desktop scheduled tasks** (which run on your machine with access to local files) and `/loop` for in-session scheduling.
- **Not shared with teammates.** Routines belong to your individual claude.ai account and count against your own daily allowance. There is no team routine.
- **Not anonymous.** Anything a routine does through your connected GitHub identity or connectors appears *as you*: commits and PRs carry your GitHub user; Slack messages and Linear tickets use your linked accounts.
- **Not interactive.** There is no permission-mode picker and no approval prompts during a run. The prompt must be self-contained.
- **Not a Workflow.** A routine decides *when* work happens; a workflow decides *how* multi-agent work is orchestrated. A routine can start a session that runs a workflow. Chapter 15.
- **Not a guarantee of success.** A green status in the run list means the session started and exited without an *infrastructure* error. It does not mean your task succeeded. Section 13.4.

IF YOU DO NOT USE CLAUDE CODE: COWORK *SCHEDULED TASKS* ARE THE EQUIVALENT

Routines are a Claude Code feature and this chapter describes them. The chat-side counterpart is **scheduled tasks in Cowork** — available on all paid plans (Pro, Max, Team, Enterprise) — which schedules an ordinary Cowork session rather than a coding session. Same principle, different subject matter:

	ROUTINE (CLAUDE CODE)	SCHEDULED TASK (COWORK)
Works on	A repository — code, PRs, CI	Your connected tools and the files in your Claude account
Create it	`claude.ai/code/routines`, or Desktop sidebar › Routines	**Scheduled** in the left sidebar › **New task** › *Create with Claude* or set it up manually
Local folders	Cloud environment, not your disk	**Cannot** be tied to a folder on your computer
Runs when your machine is off	Yes — cloud	Yes — runs remotely
Has access to	Repo, plugins declared in `.claude/settings.json`, account skills	The same connectors, skills and installed plugins as a normal Cowork task
Org control	Routines toggle at `claude.ai/admin-settings/claude-code`	The Cowork admin toggle for Team and Enterprise

Typical Cowork uses: a daily briefing over Slack and calendar, a weekly report compiled from Drive, recurring competitor research, periodic file tidying, standup summaries from a tracker. Each run becomes its own Cowork session you can open and review.

How it relates to the other blocks

BLOCK	RELATIONSHIP
Connectors	A routine's connector list is your *claude.ai account* connectors. All of them are included by default when you create one — remove what it does not need.
MCP Servers	Servers added locally with `claude mcp add` are *not* available. Add them as account connectors or declare them in a committed `.mcp.json`.
Skills	Each run is a fresh remote session, so `~/.claude/skills/` is not read. Upload the skill to claude.ai, commit it to the repo, or ship it in a repo-declared plugin.
Plugins	Plugins declared in the repository's `.claude/settings.json` install at session start. Plugins enabled only in your user settings do not transfer.
Agents / Workflows	A routine's session can spawn subagents and run workflows like any other session.
Artifacts	A routine can publish or update an artifact, giving you a status page that maintains itself.

13.2 When should I use it?

CREATE A ROUTINE WHEN…

- The work is **unattended, repeatable and tied to a clear outcome**. That is Anthropic's own framing and it is the right test.
- It should happen **on a cadence**, or in **reaction to an event** you do not want to watch for.
- You can write a prompt that is **completely self-contained** — no clarifying questions possible.
- The output is reviewable afterwards: a PR, a summary, a posted message, a draft.
- You want a **one-off reminder** in the future — a cleanup PR after a rollout, a follow-up next week.

Do not create a Routine when…

- The task needs judgement calls mid-run. Nobody is there to answer.
- It performs irreversible actions you would not approve unseen.
- You need it to run on your machine with local files → a **Desktop scheduled task**.
- You want it inside an open session → `/loop`.
- The value is in the conversation, not the artefact. Just have the conversation.

Example use cases — each pairing a trigger with the right kind of work

ROUTINE	TRIGGER	WHAT IT DOES
Backlog maintenance	Schedule	Every weeknight, reads issues opened since the last run via a connector, applies labels, assigns owners based on the area of code referenced, and posts a summary to Slack so the team starts the day with a groomed queue.
Alert triage	API	Your monitoring tool calls the endpoint when an error threshold is crossed, passing the alert body. The routine pulls the stack trace, correlates it with recent commits, and opens a draft PR with a proposed fix linked back to the alert. On-call reviews a PR instead of starting from a blank terminal.
Bespoke code review	GitHub	On `pull_request.opened`, applies your team's own review checklist, leaves inline comments for security, performance and style, and adds a summary so human reviewers focus on design.
Deploy verification	API	Your CD pipeline calls the endpoint after each production deploy. Runs smoke checks, scans error logs for regressions, and posts go / no-go to the release channel before the window closes.
Docs drift	Schedule	Weekly, scans merged PRs since the last run, flags documentation referencing changed APIs, and opens update PRs against the docs repository.

ROUTINE	TRIGGER	WHAT IT DOES
Library port	GitHub	On merged PRs in one SDK repository, ports the change to a parallel SDK in another language and opens a matching PR — keeping two libraries in step without a human re-implementing each change.

Availability and limits

PLAN	ROUTINE RUNS PER DAY	NOTES
Pro	Up to 5	Routines draw down subscription usage like interactive sessions, *plus* this daily cap on how many runs can start per account. See your consumption at `claude.ai/code/routines` or `claude.ai/settings/usage`.
Max	Up to 15	
Team / Enterprise	Up to 25	

One-off runs do not count against the daily cap — they draw regular subscription usage like any session. When you hit the cap or your usage limit, organisations with usage credits enabled can continue on metered overage; without them, further runs are rejected until the window resets.

13.3 How do I set it up?

Prerequisite: Claude Code on the web enabled, and GitHub access granted so repositories can be cloned. If you create from the CLI and GitHub is not connected, Claude prompts you to run `/web-setup`.

Where to create one

Three surfaces, one cloud account — a routine created in any of them appears immediately in the others:

SURFACE	HOW
Web	`claude.ai/code/routines` › **New routine**. The full form — all three trigger types.
Desktop app	**Routines** in the sidebar › **New routine** › choose **Cloud**. Choosing **Local** instead creates a Desktop scheduled task, which runs on your machine.
CLI	`/schedule` (alias `/routines`). Creates *scheduled* routines only — add API or GitHub triggers on the web.

Create from the web — the full walkthrough

1. **Open the creation form**

   Visit **claude.ai/code/routines** › **New routine**

2. **Name it and write the prompt**

   The prompt is the most important part. The routine runs autonomously, so it must be **self-contained and explicit** about what to do and what success looks like.

   The prompt input includes a **model selector**. Claude uses the selected model on every run.

   > **HOW THE FIRED PROMPT IS TREATED**
   > When a trigger fires, the session receives the saved prompt as its *assigned task* and carries it out. The trigger attests only that the prompt was stored ahead of time by an authorised session on your account — so the fired prompt is not live user input and cannot act as approval or consent for actions during the run. Content the session *fetches* during the run keeps its normal handling.

3. **Select repositories**

   Add one or more GitHub repositories. Each is cloned at the start of every run, starting from the default branch. Claude creates `claude/`-prefixed branches for its changes.

4 **Select an environment**

A cloud environment controls what the session can access:

- **Network access** — the level of internet access available during each run.
- **Environment variables** — values Claude can use. Note these are visible to anyone who uses the environment; add credentials with that in mind.
- **Setup script** — install dependencies and tools. The result is cached, so it does not re-run every session.

A **Default** environment is provided with **Trusted** network access, which allows only a default allowlist of package registries, cloud provider APIs, container registries and common development domains. *Connectors you add reach their services through Anthropic's servers, so they do not need allowlist changes.* If your routine must reach your own services or a domain outside that list, edit the environment's network access first.

5 **Select a trigger**

Under **Select a trigger**, choose one or combine several. Details for each in Section 13.4.

6 **Review connectors — do not skip this**

Under **Connectors** at the bottom of the form, **all of your connected MCP connectors are included by default**. Remove any the routine does not need: *Claude can use every tool from an included connector, including writes, without asking for permission during a run.*

7 **Create, then test immediately**

Click **Create**. To avoid waiting for the first scheduled fire, open the routine's detail page and click **Run now**. Each run creates a session alongside your others, where you can see what Claude did, review changes and create a pull request.

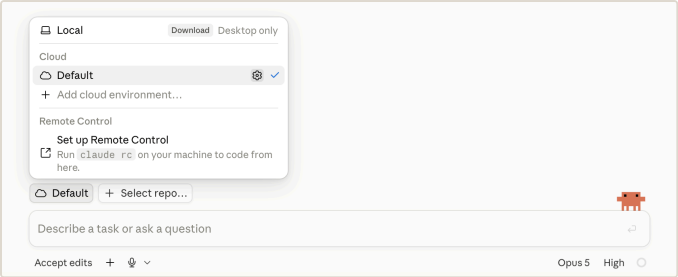


Figure 13.2 The environment selector. Hover an environment and click the settings icon on the right to edit it.
Screenshot: Anthropic, Claude Code documentation — "Cloud environments."

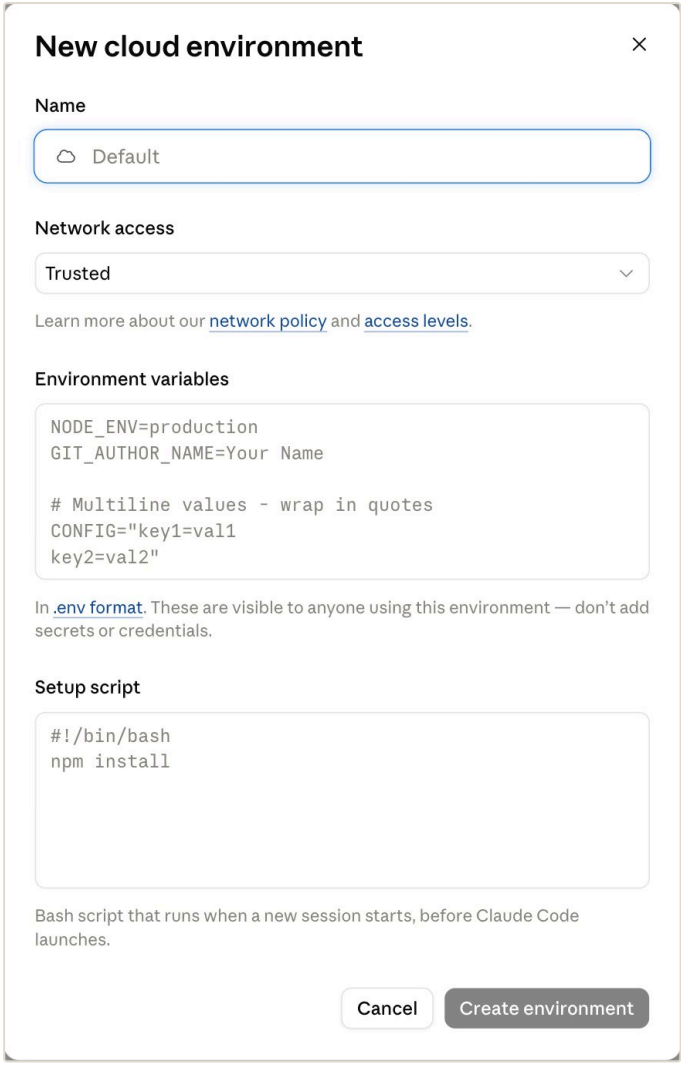


Figure 13.3 The **Update cloud environment** dialog, where **Network access** and **Allowed domains** are set. This is what a routine's network policy comes from.
Screenshot: Anthropic, Claude Code documentation — "Cloud environments."

Create from the CLI

```
# conversational — Claude asks about schedule, repos and prompt
> /schedule

# or describe it directly
> /schedule daily PR review at 9am
> /schedule clean up feature flag in one week

# manage existing ones
> /schedule list
> /schedule update
> /schedule run
```

A successful start looks like a conversation. Claude asks follow-up questions before saving. If it instead replies that you need to authenticate or that it cannot connect to your remote claude.ai account, *no routine was created* — see Section 13.5.

Recommended configuration

SETTING	RECOMMENDATION	WHY
Connectors	Only what the routine needs	No approval prompts during a run. This is your primary containment control.

SETTING	RECOMMENDATION	WHY
Repositories	Only what it must read or change	Same reasoning. Each is cloned every run.
Network access	Leave on **Trusted** unless you have a specific need	Connectors do not need the allowlist widened. Widen only for your own services.
First trigger	A one-off, or **Run now**	Test before committing to a cadence. One-off runs do not count against the daily cap.
Output	A draft PR or a message, not a direct push to a shared branch	Keeps a human in the loop on anything consequential.
Prompt	State the success criterion explicitly	Nobody can clarify mid-run.

13.4 How do I use it?

Configuring each trigger type

Schedule trigger

Pick a preset frequency — hourly, daily, weekdays, weekly — in the **Select a trigger** section. Times are entered in your local zone and converted automatically, so the routine runs at that wall-clock time regardless of where the infrastructure is.

- **Runs may start a few minutes late** due to stagger. The offset is consistent for each routine.
- **Custom intervals:** pick the closest preset in the form, then run `/schedule update` in the CLI to set a specific cron expression. **The minimum interval is one hour**; anything more frequent is rejected.
- **One-off runs:** fire once at a specific timestamp. After firing, the routine auto-disables and the web UI marks it **Ran**. To reuse it, edit and set a new time.

```
# one-off, described in natural language — Claude confirms the absolute timestamp
> /schedule tomorrow at 9am, summarize yesterday's merged PRs
> /schedule in 2 weeks, open a cleanup PR that removes the feature flag
```

One-off scheduling from the CLI is rolling out gradually. If `/schedule` only offers recurring schedules, create the one-off from the web.

API trigger

Gives the routine a dedicated HTTP endpoint. Added from the web only — the CLI cannot create or revoke tokens.

1. **Open the routine for editing**

   claude.ai/code/routines › click the routine › the **pencil** icon › **Edit routine**

2. **Add the trigger**

   Scroll to **Select a trigger** below the **Instructions** box, click **Add another trigger**, and choose **API**.

3. **Copy the URL and generate a token**

   The modal shows the URL and a sample curl command. Copy the URL, click **Generate token**, and copy the token **immediately** — it is shown once and cannot be retrieved later. Store it in your alerting tool's secret store.

4 **Call it**

```
$ curl -X POST https://api.anthropic.com/v1/claude_code/routines/trig_01ABC.../fire \
    -H "Authorization: Bearer sk-ant-oat01-xxxxx" \
    -H "anthropic-beta: experimental-cc-routine-2026-04-01" \
    -H "anthropic-version: 2023-06-01" \
    -H "Content-Type: application/json" \
    -d '{"text": "Sentry alert SEN-4521 fired in prod. Stack trace attached."}'
```

A successful request returns the new session ID and URL, so you can watch the run:

```
{
  "type": "routine_fire",
  "claude_code_session_id": "session_01HJK...",
  "claude_code_session_url": "https://claude.ai/code/session_01HJK..."
}
```

THE `TEXT` FIELD IS TREATED AS UNTRUSTED DATA — AND YOUR PROMPT MUST OPT IN

The optional `text` field carries run-specific context such as an alert body. It does **not** reach the routine as a bare message: it arrives wrapped in a `<routine-fire-payload>` block that labels it untrusted and tells Claude not to follow instructions inside it unless the routine's own prompt says to.

So your saved prompt must reference it explicitly — for example "*Investigate the alert described in the routine-fire-payload block*" — or the routine treats the text as inert context. Anyone holding the bearer token can send `text`, which is precisely why the wrapper exists. The same wrapping applies to text supplied with **Run now**.

The value is freeform and unparsed: send JSON and the routine receives it as a literal string. Each routine has its own token, scoped to triggering that routine only; **Regenerate** or **Revoke** from the same modal.

GitHub trigger

Starts a new session when a matching repository event occurs. Sessions are never reused across events, so two PR updates produce two independent sessions. Configured from the web only.

The **Claude GitHub App must be installed** on the repository. The trigger setup prompts you if it is not. Note that running `/web-setup` in the CLI grants repository access for *cloning* but does **not** install the GitHub App and does not enable webhook delivery.

EVENT	TRIGGERS WHEN
Pull request	A PR is opened, closed, assigned, labeled, synchronized, or otherwise updated
Release	A release is created, published, edited, or deleted

Within each category you can pick a specific action such as `pull_request.opened`, or react to all actions. Pull requests can be filtered on Author, Title, Body, Base branch, Head branch, Labels, Is draft and Is merged — each paired with an operator: equals, contains, starts with, is one of, is not one of, or matches regex. **All conditions must match.**

THE REGEX FILTER MATCHES THE WHOLE FIELD

`matches regex` tests the *entire* field value, not a substring. To match any title containing `hotfix`, write `.*hotfix.*` — without the surrounding `.*` it matches only a title that is exactly `hotfix`. For literal substring matching, use `contains` instead.

During the research preview, GitHub webhook events are subject to per-routine and per-account hourly caps; events beyond the limit are dropped until the window resets. Current limits are shown at `claude.ai/code/routines`.

Verifying a run actually worked

GREEN DOES NOT MEAN SUCCESS

A green status in the run list means the session **started and exited without an infrastructure error**. It does **not** mean the task in your prompt succeeded. Open the run and read the transcript. Blocked network requests, missing connector tools and task-level

failures all surface there rather than in the status indicator.

1. **Click into the run.** It opens as a full session: see what Claude did, review changes, create a PR, or continue the conversation.
2. **Check the artefact, not the status.** Did the PR appear? Was the message posted? Were the labels applied?
3. **Look for blocked requests** if something is missing — a `403` with `x-deny-reason: host_not_allowed` means the network policy blocked it.
4. **Build verification into the prompt.** End it with an instruction to state plainly what it did and did not manage to do. That turns the transcript into a report.

13.5 How do I manage it?

CREATE	Web: `claude.ai/code/routines` › **New routine**. Desktop: **Routines** › **New routine** › **Cloud**. CLI: `/schedule`.
VIEW	The routines list. Click one for its detail page: repositories, connectors, prompt, schedule, API tokens, GitHub triggers and past runs. CLI: `/schedule list`.
RUN NOW	**Run now** on the detail page — optionally with run-specific text, handled exactly like the API trigger's `text`. CLI: `/schedule run`.
PAUSE	The toggle in the **Repeats** section pauses or resumes the schedule. A paused routine keeps its configuration but does not run.
EDIT	The **pencil** icon › **Edit routine**: name, prompt, repositories, environment, connectors, or any trigger. CLI: `/schedule update`.
DELETE	The delete icon on the detail page. Past sessions the routine created remain in your session list.
RUNS	Each run is a normal session — use the dropdown beside the session title to rename, archive or delete it.
TOKENS	Per-routine API tokens: **Generate**, **Regenerate**, **Revoke** from the API trigger modal. Shown once at generation.
ORG CONTROL	Team and Enterprise Owners can disable routines for all members with the **Routines** toggle at `claude.ai/admin-settings/claude-code`. When disabled, existing routines stop running and members cannot create new ones.

Branch permissions

Claude pushes to branches prefixed `claude/`, which are always accepted. When your prompt directs Claude to push elsewhere, Claude Code checks first and **rejects the push** if any of the following is true:

- The branch is protected on GitHub.
- Someone else has an open pull request from that branch.
- The branch carries commits authored by someone other than you.

Widening network access

1. **Open the routine for editing**
   Detail page › the **pencil** icon.
2. **Open the environment selector**
   Below the **Instructions** box, select the cloud icon showing your environment's name, such as **Default** (Figure 13.2).
3. **Open its settings**
   Hover the environment in the list and click the settings icon that appears on the right.

4 **Change the access level**

In **Update cloud environment** (Figure 13.3), set **Network access** to **Custom** and enter your domains under **Allowed domains**. Tick **Also include default list of common package managers** to keep the default allowlist alongside yours. Or choose **Full** for unrestricted access.

5 **Save**

Save changes. The new policy applies from the next run.

Troubleshooting

SYMPTOM	CAUSE	FIX
`/schedule` returns `Unknown command`	A requirement is unmet — the CLI hides the command	Most often: you are authenticated with a Console API key or a cloud provider (Bedrock, Google Cloud, Foundry) rather than a claude.ai subscription login. Unset `ANTHROPIC_API_KEY` / `ANTHROPIC_AUTH_TOKEN` and any `apiKeyHelper`, then `/login`. Also caused by `DISABLE_TELEMETRY`, `DO_NOT_TRACK`, `CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC` or `DISABLE_GROWTHBOOK` being set, since the command depends on feature-flag fetching. Also hidden inside a Claude Code on the web session — use the web UI.
`/schedule` asks you to authenticate	No stored claude.ai login; API accounts are not supported	Run `/login`, sign in with your claude.ai account, retry. **No routine was created** in the meantime.
"Routines are disabled by your organization's policy"	An Owner turned off the **Routines** toggle	Server-side setting — it cannot be overridden locally. Ask an Owner.
Green status, nothing happened	Status reflects infrastructure, not your task	Open the run and read the transcript.
Skill not found during a run	Personal skills are not synced to cloud sessions	Upload to claude.ai, commit to the repo, or ship in a repo-declared plugin. Chapter 7, Section 7.5.
Connector tool unavailable	It is a locally-added MCP server, not an account connector	Add it at `claude.ai/customize/connectors`, or declare it in a committed `.mcp.json`.
`403` with `x-deny-reason: host_not_allowed`	Network policy blocked the request	Widen **Allowed domains** as above. Note connector traffic does not need this.
Push rejected	Protected branch, someone else's open PR, or another author's commits	Let Claude use a `claude/` branch and open a PR instead.
Routine ignores the `text` payload	The prompt does not reference the payload	Mention the `routine_fire_payload` block explicitly in the saved prompt.
GitHub trigger never fires	The Claude GitHub App is not installed, or filters are too narrow	Install the App via the trigger setup. Check your regex filters — they match the whole field.
CLI shows no next run time	The routine has no schedule trigger	Expected for API-only or GitHub-only routines.
Runs rejected	Daily cap or usage limit	Check `claude.ai/settings/usage`. Enable usage credits to continue on metered overage (Team/Enterprise: an admin enables it at `claude.ai/admin-settings/usage`).

13.6 Examples

Example 1 — QA: nightly test-failure triage

Prompt (schedule: weekdays, 06:00)

```
Every weekday morning, review the CI test results from the last 24 hours
in this repository.

1. Identify tests that failed. For each, classify it as: a genuine
   regression, a flaky test, or an environment problem. Justify the
   classification with evidence from the logs.
2. For genuine regressions, find the likely commit by correlating the
   failure with the commits merged since the last green run.
3. Post a summary to the #qa-alerts Slack channel: one line per failure,
   grouped by classification, with the suspect commit linked.
4. If a test has failed intermittently on three or more of the last ten
   runs, open an issue tagged "flaky" — one issue per test, and check for
   an existing open issue first so you do not duplicate.

Do not modify any test files. This is analysis only.

Finish your response with: what you found, what you posted, what you
opened, and anything you could not determine.
```

Note the last paragraph — it turns the transcript into a report you can skim.

Example 2 — Product/BA: weekly backlog grooming

Prompt (schedule: weekly, Friday 17:00)

```
Using the Azure DevOps connector, review every work item created or
modified in the last 7 days in the Checkout area path.

For each item, check: does it have acceptance criteria; is it assigned;
does it have an estimate; is the description more than 20 words.

Then:
- Apply the label "needs-refinement" to items failing two or more checks.
- Do NOT change assignees, estimates, states or descriptions.
- Post a summary to #product-planning: counts by problem type, and the
  five worst items by ID with a one-line reason each.

Success means: labels applied correctly and one summary message posted.
If the connector is unavailable, post nothing and say so clearly.
```

Example 3 — Development: bespoke PR review (GitHub trigger)

Trigger: pull_request.opened, filter: Is draft = false

```
Review this pull request against our house checklist:

- Money handled as minor-unit integers, never floats
- All dates ISO-8601 UTC at boundaries
- Input validated before any database call
- No new dependency without a note in the PR body explaining why
- Tests added or updated for changed behaviour
- Nothing added under src/legacy/

Leave inline comments for each finding, citing the file and line.
Then add one summary comment: what you checked, what you found, and
what a human reviewer should focus on.

Report every issue you find, including low-confidence ones, and mark your
```

```
confidence. Do not filter for importance — a human will triage.

Do not approve or request changes. Do not push commits.
```

The "report everything, mark confidence" instruction matters: told to be conservative, Claude filters faithfully and you see fewer findings than it actually noticed.

Example 4 — Development: alert triage (API trigger)

Prompt

```
An alert has fired. Its details are in the routine-fire-payload block.

Investigate the alert described there:
1. Extract the error signature and the affected service.
2. Find the stack trace and locate the responsible code in this repository.
3. Correlate with commits merged in the last 48 hours.
4. If you can identify a safe fix, open a DRAFT pull request on a
   claude/ branch with the change and an explanation linking back to
   the alert. If you cannot, open an issue with your analysis instead.

Never push to main. Never mark the PR ready for review.
End with a one-paragraph summary for the on-call engineer.
```

Wiring it up from a monitoring tool

```
$ curl -X POST https://api.anthropic.com/v1/claude_code/routines/<id>/fire \
    -H "Authorization: Bearer $ROUTINE_TOKEN" \
    -H "anthropic-beta: experimental-cc-routine-2026-04-01" \
    -H "anthropic-version: 2023-06-01" \
    -H "Content-Type: application/json" \
    -d "{\"text\": \"$ALERT_BODY\"}"
```

Example 5 — Design/Docs: weekly drift check

Prompt (schedule: weekly, Monday 08:00)

```
Scan the pull requests merged in the last 7 days in this repository.

Identify any that changed a public API signature, a component prop, or a
configuration option. For each, check whether our documentation in docs/
still describes it correctly.

Where documentation is now wrong, open ONE pull request against docs/ on a
claude/ branch containing all the corrections, with a description listing
each change and the PR that caused it.

If nothing drifted, do not open a PR — just say so.
```

Example 6 — a one-off note to your future self

```
> /schedule in 3 weeks, check whether the checkout-v2 feature flag is
    still referenced anywhere. If it is not, open a PR removing the flag
    and its dead branches. If it is, list where and say why it can't go yet.
```

One-off runs do not count against the daily cap, which makes this pattern essentially free.

13.7 Best practices

Recommended practices

- **Write the prompt as if for a competent stranger with no way to ask questions.** That is exactly the situation.

- **State the success criterion explicitly,** and ask for a closing summary of what was and was not done.
- **Test with Run now or a one-off** before committing to a cadence.
- **Trim the connector list.** It defaults to everything, and there are no approval prompts. This is the single most important configuration step.
- **Prefer proposing to doing.** Draft PRs, issues and messages are reviewable; direct pushes and deletions are not.
- **Include "check for an existing one first"** in anything that creates records, or a daily routine will create the same issue thirty times.
- **Give the routine a narrow repository set.** Every repository is cloned each run.
- **Read the first three runs properly.** After that, skim. Never assume a green tick.

Common mistakes

MISTAKE	WHY IT HURTS
Trusting the green status	It means "no infrastructure error," not "your task worked."
Leaving every connector included	All tools, including writes, usable without approval, unattended.
A prompt that assumes clarification is possible	Nobody is there. Ambiguity becomes an invented decision.
No duplicate check on record creation	A nightly routine that opens an issue opens it every night.
Expecting local skills or MCP servers to be available	Fresh remote session every run. Neither is present.
Forgetting the routine acts as you	Commits, PRs, Slack messages and tickets all carry your identity.
Assuming `text` is instructions	It is wrapped as untrusted data. Your prompt must opt in to using it.
A regex filter without `.*`	Matches the whole field. Your GitHub trigger silently never fires.

Security considerations

ROUTINES ARE THE HIGHEST-TRUST CONFIGURATION IN THIS GUIDE

An autonomous session, in the cloud, with your identity and your connectors, and no human to approve anything. The controls that matter:

- **Scope the connector list.** Claude can use every tool from an included connector, writes included, without asking. Remove everything unnecessary.
- **Scope the repositories** to what the routine must touch.
- **Environment variables are visible to anyone who uses the environment.** Add credentials with that in mind, and prefer a dedicated environment for anything sensitive.
- **Keep network access on Trusted** unless you have a specific reason. Connectors do not require it to be widened.
- **Guard the API token.** It is shown once. Anyone holding it can trigger the routine and supply `text`. Store it in a secret store; revoke and regenerate if exposed.
- **Fired prompts are not consent.** The design deliberately treats the trigger as attesting only that the prompt was stored earlier by an authorised session — it cannot serve as approval for actions during the run. Do not architect around that; architect with it.
- **Prefer draft output.** The branch-protection rules are a safety net, not a policy. Your prompt is the policy.
- **Organisation control:** Owners can disable routines entirely at `claude.ai/admin-settings/claude-code`.

Naming, versioning and team considerations

- **Name by cadence and outcome:** `Nightly — CI failure triage`, `Weekly — backlog grooming (Checkout)`, `On PR — house checklist review`. You will have several.
- **Routines are personal, so document them.** A routine that posts to a team channel is team-visible infrastructure owned by one account. Write down who owns which, or people will wonder where the Monday message comes from when that person is on leave.

- **Keep the prompt in version control too.** Paste it into a file in the repo it operates on. It is the specification, and it deserves review and history.
- **Watch the daily cap** when planning: 5 on Pro, 15 on Max, 25 on Team/Enterprise. An hourly routine consumes a lot of that.
- **Review routines quarterly.** A routine that operates on a deleted area path or a renamed channel fails quietly and forever.

13.8 Official resources

Automate work with routines — the complete reference used throughout this chapter: creation, all three triggers, management, limits, troubleshooting
`https://code.claude.com/docs/en/routines`

Trigger a routine via API — full API reference, error responses, validation rules, field limits
`https://platform.claude.com/docs/en/api/claude-code/routines-fire`

Cloud environments — network access levels, the default allowlist, environment variables, setup scripts, caching
`https://code.claude.com/docs/en/cloud-environments`

Claude Code on the web — cloud sessions and GitHub authentication options
`https://code.claude.com/docs/en/claude-code-on-the-web`

Desktop scheduled tasks — the local alternative, with access to local files
`https://code.claude.com/docs/en/desktop-scheduled-tasks`

`/loop` and in-session scheduling — recurring tasks inside an open session
`https://code.claude.com/docs/en/scheduled-tasks`

GitHub Actions — running Claude in your CI pipeline on repository events
`https://code.claude.com/docs/en/github-actions`

Schedule recurring tasks in Claude Cowork — the Cowork equivalent
`https://support.claude.com/en/articles/13854387-schedule-recurring-tasks-in-claude-cowork`

Introducing routines in Claude Code — the announcement
`https://claude.com/blog/introducing-routines-in-claude-code`

CHAPTER 14

Agents

Delegated workers. Give one a goal and it drives itself, in its own context, and reports back.

ADVANCED | CLAUDE CODE CLI | VS CODE EXTENSION | DESKTOP APP | CLAUDE CODE PLANS

14.1 What is it?

An agent — a **subagent**, in Claude Code's terminology — is a specialised worker Claude spawns to handle a specific kind of task. It runs in *its own context window*, with its own system prompt, its own tool access and its own permissions. It works independently and returns only its results.

The problem it solves

Two problems, and the first is the one people underestimate.

Context pollution. Suppose you ask Claude to find where a behaviour is implemented across a large codebase. It greps, reads twenty files, follows three false trails, and eventually answers. Those twenty files are now in your conversation forever, crowding out the actual work you were doing. You needed the *conclusion*; you got the conclusion plus all the evidence.

A subagent does that searching in its own context window and hands back a summary. Your conversation stays focused.

Repetition and constraint. If you keep spawning the same kind of worker with the same instructions — a code reviewer, a test writer, a documentation checker — you can define it once. And because a subagent has its own tool list, you can make one *structurally* unable to do damage: a reviewer with only `Read`, `Grep` and `Glob` cannot modify anything, regardless of what it decides.

How it works at a high level

A subagent is a markdown file with YAML frontmatter. The frontmatter tells Claude when to delegate to it and what it may use; the body is its system prompt. When Claude encounters a task matching the description, it delegates; the subagent works and returns results to the conversation that spawned it.

`~/.claude/agents/code-improver.md`

```
---
name: code-improver
description: Scans files and suggests improvements for readability, performance,
  and best practices. Use after writing or modifying code.
tools: Read, Grep, Glob
model: sonnet
---

You are a code improvement specialist. For each issue you find, explain
the problem, show the current code, and provide an improved version.
```

Four lines of frontmatter and two of prompt. Note `tools: Read, Grep, Glob` — this agent is read-only by construction.

What subagents give you

BENEFIT	HOW
Preserve context	Exploration and implementation detail stay out of your main conversation.
Enforce constraints	Limit which tools a subagent can use. A read-only reviewer is read-only by construction, not by instruction.
Reuse configurations	User-level subagents work across all your projects; project ones are committed and shared.
Specialise behaviour	A focused system prompt for one domain beats a general-purpose one carrying every concern.
Control costs	Route mechanical tasks to a faster, cheaper model like Haiku while your main session stays on Opus.

The wider agent family

"Agents" is a family, not one feature. Knowing which member you want saves a lot of confusion — and Chapter 15 is one of them.

APPROACH	WHAT IT GIVES YOU	USE IT WHEN
Subagents this chapter	Delegated workers inside one session that do a side task in their own context and return a summary.	A side task would flood your conversation with search results, logs or file contents you will not reference again.
Agent view research preview	One screen to dispatch and monitor background sessions, opened with `claude agents`.	You have several independent tasks, want to hand them off, check status at a glance, and step in only when one needs you.
Agent teams experimental, off by default	Multiple coordinated sessions with a shared task list and inter-agent messaging, managed by a lead.	You want Claude to split a project into pieces, assign them, and keep workers in sync.
Dynamic workflows Chapter 15	A script that runs many subagents and cross-checks their results.	The job outgrows a handful of subagents, or findings need verifying against each other.

In every approach the workers are Claude sessions. To involve a different tool, expose it to Claude as an MCP server (Chapter 8). Note also that running several sessions or subagents at once **multiplies token usage**.

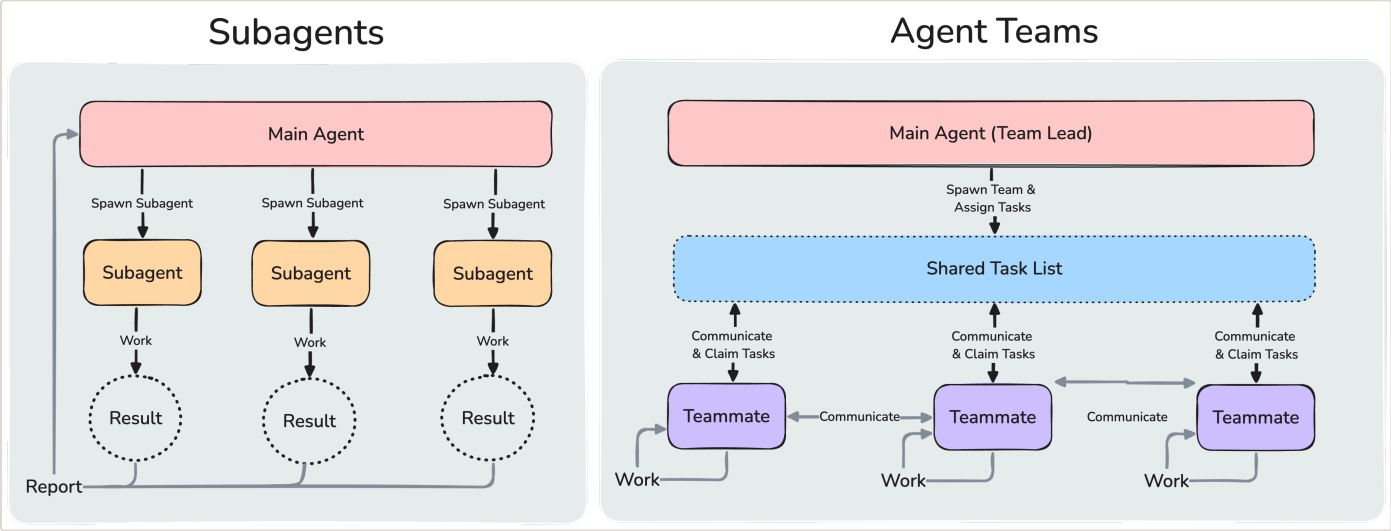


Figure 14.1 Subagents report back to the conversation that spawned them. Agent-team members are peer sessions sharing a task list and messaging each other directly. Choose by whether the workers need to talk to each other.

Diagram: Anthropic, Claude Code documentation — "Create custom subagents."

What it is NOT

- **Not a Routine.** A subagent runs inside a session you are in. A routine runs a session on a schedule in the cloud (Chapter 13).
- **Not a Workflow.** With a subagent, *Claude* decides turn by turn what to spawn. With a workflow, a *script* holds the plan (Chapter 15).
- **Not a Skill.** A skill is instructions Claude follows in its own context. A subagent is a separate worker with a separate context. A subagent can be given skills; the reverse is not true.
- **Not something you author in chat.** Subagents are defined as files in Claude Code, and plain claude.ai chat has no equivalent. The one exception worth knowing: a **plugin** can bundle subagents, and those *do* run in **Cowork** — though they stay greyed out in chat. So the honest statement is that Cowork can *run* subagents shipped to it, not that you can write one there. Chapter 12, Section 12.3.
- **Not free.** Each subagent has its own context and its own token cost. Delegating trivial work is a net loss.

- **Not able to see your conversation** — unless it is a *forked* subagent. A normal subagent starts fresh, which is precisely why it preserves your context, and precisely why its task prompt must be self-contained.

How it relates to the other blocks

BLOCK	RELATIONSHIP
Skills	A subagent can have skills injected at startup via its `skills` field, giving it domain knowledge without discovery. A skill can also be configured with `context: fork` to run inside a subagent.
Instructions	A subagent's body *is* its instructions, scoped to that worker. Note it does not inherit the main conversation's auto memory.
MCP / Connectors	A subagent's `tools` and `mcpServers` fields control which systems it can reach — a good containment mechanism.
Plugins	A plugin's `agents/` directory ships subagents to a team. Plugin subagents appear in the @-mention typeahead under their scoped name.
Workflows	Workflows orchestrate subagents at scale. The subagent is the worker primitive a workflow drives.
Worktrees	Subagents can each get a separate git checkout so parallel workers never edit the same files.

14.2 When should I use it?

USE A SUBAGENT WHEN…

- A side task would **flood your conversation** with output you will not reference again — searches, logs, file contents.
- You keep **spawning the same kind of worker** with the same instructions. Define it once.
- You want a worker that is **structurally unable** to do something — a reviewer with no write tools.
- A task suits a **cheaper or faster model** than your main session is using.
- Several independent pieces can run **in parallel**.

Do not use a Subagent when…

- You could finish it yourself in a handful of tool calls. Delegation has real overhead: the subagent re-establishes context, re-explores, reports back, and you re-read the report.
- The work needs your conversation's context → use a *forked* subagent (`/subtask`), which inherits it.
- You need it verified — verification usually belongs in your main loop, not delegated.
- The job needs dozens of workers with cross-checking → a **Workflow** (Chapter 15).
- You want it to run while you are asleep → a **Routine** (Chapter 13).

Choosing within the family

QUESTION	ANSWER
Claude delegates and collects results inside one conversation	**Subagents**
You hand off independent tasks and check back later	**Agent view** (`claude agents`)
Claude plans, assigns and supervises a group of workers who talk to each other	**Agent teams** (experimental)
A script should hold the plan instead of Claude's turn-by-turn judgement	**Dynamic workflows**
The tasks touch the same files	Isolate with **worktrees**

Typical subagents by role

SUBAGENT	PURPOSE & TYPICAL TOOLS
sprint-auditor	QA/BA. Audits a sprint's work items for missing ACs, estimates and owners. Read-only plus the tracker's MCP tools.

SUBAGENT	PURPOSE & TYPICAL TOOLS
test-writer	QA/Dev. Writes and runs tests for a named module. Read, Write, Edit, Bash.
code-reviewer	Dev. Reviews a diff against a checklist. `Read, Grep, Glob` only — cannot change anything.
a11y-auditor	Design/Dev. Checks components against your accessibility checklist. Read-only.
doc-checker	Any. Verifies documentation still matches the code. Read-only.
explorer	Any. Answers "where is X implemented?" without dragging twenty files into your conversation. Read-only, cheap model.

14.3 How do I set it up?

> **THERE IS NO CREATION WIZARD ANY MORE**
>
> As of Claude Code v2.1.198, `/agents` no longer opens an interactive creation panel — running it prints a notice pointing to the subagent file locations. Create subagents by **asking Claude** or by editing files directly. (On v2.1.197 and earlier, `/agents` did open a wizard with **Running** and **Library** tabs.) Note also that `/agents` is a different thing from `claude agents`, which opens agent view.

The easiest path: ask Claude

1 **Describe the subagent you want and where to save it**

```
> Create a personal code-improver subagent in ~/.claude/agents/ that scans
    files and suggests improvements for readability, performance, and best
    practices. It should explain each issue, show the current code, and
    provide an improved version. Make it read-only and have it use Sonnet.
```

Claude writes the file with a `name`, a `description`, a `tools` list, a `model` and a system prompt.

2 **Review what it wrote**

Open the file and check the frontmatter matches what you asked — particularly the `tools` list. This is your containment boundary; do not accept it unread.

3 **Try it**

```
> Use the code-improver agent to suggest improvements in this project
```

If Claude cannot find the new subagent, restart Claude Code. This happens only when `~/.claude/agents/` did not exist before the session started, because a running session does not detect a newly created `agents` directory.

Where to put it — scope and precedence

LOCATION	SCOPE	PRIORITY	HOW TO CREATE
Managed settings	Organisation-wide	1 (highest)	Deployed via managed settings
`--agents` CLI flag	Current session	2	Pass JSON when launching
`.claude/agents/`	Current project	3	Ask Claude, or create the file
`~/.claude/agents/`	All your projects	4	Ask Claude, or create the file
A plugin's `agents/`	Where the plugin is enabled	5 (lowest)	Installed with plugins

Project subagents are ideal for anything codebase-specific — commit them so the team can use and improve them collaboratively. They are discovered by walking up from the working directory, so every `.claude/agents/` between there and the repository root is scanned; when more than one defines the same name, the definition closest to the working directory wins. Directories added with `--add-dir` are also scanned.

The frontmatter fields that matter

FIELD	REQUIRED	WHAT IT DOES
`name`	Yes	Unique identifier, lowercase and hyphens. Cannot contain `:` — that is reserved for plugin-scoped identifiers. The filename does not have to match.
`description`	Yes	When Claude should delegate to this subagent. As with skills, this determines whether it is ever used automatically.
`tools`	No	The tools it may use. **Omitting it inherits every tool available to subagents.** If no entry resolves to a real tool the subagent usually fails to launch with an error naming the entries.
`model`	No	`sonnet`, `opus`, `haiku`, `fable`, a full model ID, or `inherit`. Defaults to `inherit`.
`skills`	No	Skills injected into the subagent's context at startup — domain knowledge without it having to discover them. Prefer this over listing `Skill` in `tools`.
`mcpServers`	No	MCP servers to give this subagent, including ones not available in the main conversation. Inline servers connect when it starts and disconnect when it finishes; string references share the parent session's connection.
`memory`	No	`user`, `project` or `local` — gives the subagent its own persistent memory for cross-session learning.
`color`	No	Display colour in the task list and transcript: `red`, `blue`, `green`, `yellow`, `purple`, `orange`, `pink`, `cyan`.
`hooks`	No	Hooks scoped to this agent's lifecycle.

Example configurations

`.claude/agents/code-reviewer.md — read-only by construction`

```
---
name: code-reviewer
description: Reviews a diff or a set of files against our house checklist.
  Use when asked to review code, check a PR, or assess a change before merge.
tools: Read, Grep, Glob
model: opus
color: blue
---

You review code against this checklist, in this order:

1. Correctness — logic errors, off-by-one, unhandled nulls, wrong operator.
2. Money and dates — money must be minor-unit integers, never floats;
   dates must be ISO-8601 UTC at every boundary.
3. Input validation at system boundaries (user input, external APIs).
   Do not flag missing validation on internal calls.
4. Error handling that could hide a failure.
5. Test coverage for changed behaviour.

Report every issue you find, including low-confidence ones, with a
confidence level and an estimated severity. Do not filter for importance —
a separate step ranks them.

For each finding give: the file and line, what is wrong, the concrete
failure it would cause, and a suggested fix. If you find nothing in a
category, say so explicitly rather than omitting it.

You cannot modify files. Do not propose to.
```

`.claude/agents/explorer.md — cheap, fast, context-preserving`

```
---
name: explorer
description: Finds where something is implemented in this codebase and
  reports back concisely. Use for "where is X handled?", "which files
  touch Y?", and any search that would otherwise fill the conversation
  with file contents.
tools: Read, Grep, Glob
model: haiku
color: cyan
---

You locate code. You do not review, refactor or explain design.

Search the codebase and return:
- The specific files and line ranges that answer the question
- One sentence per location saying what it does
- Anything relevant you found that the question did not ask about

Be brief. Do not paste file contents unless a specific snippet is the
answer. Do not speculate about code you did not read.
```

`.claude/agents/sprint-auditor.md — with a skill and an MCP server`

```
---
name: sprint-auditor
description: Audits the current sprint's work items for hygiene problems —
  missing acceptance criteria, no owner, no estimate, stale in-progress
  items. Use when asked to check sprint or backlog health.
tools: Read, Grep, Glob
mcpServers: ado
skills: test-case
model: sonnet
color: orange
---

You audit sprint hygiene. You are read-only: report problems, never fix them.

For every work item in the named sprint, check:
- Acceptance criteria present and testable as written
- An assignee, if the item is in an active state
- An estimate
- A description of more than 20 words
- Last-changed date, flagging anything "In Progress" and untouched 10+ days

Return one table: item ID, problem, and a one-line reason. Sort worst first.
Then a two-sentence summary of the sprint's overall health.

Never modify a work item. Never invent an item ID.
```

14.4 How do I use it?

Three ways a subagent runs

METHOD	BEHAVIOUR
Automatic delegation	Claude matches your request against subagent descriptions and delegates on its own. Convenient; not guaranteed.
Naming it in your prompt	`Use the test-runner subagent to fix failing tests` — strongly steers Claude to that worker.
@-mention	**Guarantees** that subagent runs for the task. Type `@` and pick from the typeahead, exactly as you @-mention a file.

```
# guaranteed: the @-mention controls WHICH subagent runs
> @agent-code-reviewer please review the changes on this branch

# plugin subagents use their scoped name
> @agent-qa-toolkit:sprint-auditor audit sprint 24
```

You can type the mention manually as `@agent-<name>` without the picker — while you type that form the typeahead shows file matches rather than agents, but the mention still resolves on submit. Importantly: **your full message still goes to Claude**, which writes the subagent's task prompt from what you asked. The @-mention controls *which* subagent is invoked, not *what prompt it receives*.

Forking: when the subagent needs your context

A normal subagent starts fresh. A *forked* subagent inherits your full conversation context:

```
> /subtask     # fork a subagent that inherits this conversation
> /fork        # copy the whole session into a new background session
```

With agent view turned off, the forked-subagent command is `/fork` instead and `/subtask` is unavailable.

What to expect

- An indication that a subagent was spawned, with its name and colour.
- Your conversation receives the *summary*, not the intermediate work. That is the point.
- Longer latency than doing it inline — the subagent re-establishes context first.
- Named background subagents appear in the @-mention typeahead with their status.

Checking on running work

COMMAND	SHOWS
`/tasks`	Everything running in the background of the current session — check on, attach to, or stop each. Includes finished subagents.
`/context`	Custom Agents available in this session. Use this to confirm a new subagent loaded.
`claude agents`	Agent view: every background session, its state, and which need your input.
`/workflows`	Dynamic workflow runs and their phases (Chapter 15).
`/agents`	A notice pointing to the subagent file locations. Not a panel any more.

How to verify a subagent is working properly

1. **Confirm it loaded:** `/context` and look under **Custom Agents**, or check it appears in the `@` typeahead.
2. **Force it with an @-mention** for the first few runs, so you are testing the subagent rather than Claude's delegation judgement.
3. **Test the tool restriction deliberately.** Ask a read-only reviewer to fix something. It should decline or report that it cannot — not attempt it. This verifies your containment boundary actually exists.
4. **Check the summary is a summary.** If the subagent returns twenty files of content, its prompt needs to say "be brief" — you have lost the context benefit that motivated the delegation.
5. **Then remove the @-mention** and see whether Claude delegates on its own. If not, the `description` is the problem.

14.5 How do I manage it?

CREATE	Ask Claude ("create a … subagent in `~/.claude/agents/` that …"), or write the markdown file yourself.
VIEW	`/context` lists available Custom Agents. The `@` typeahead lists them for invocation. Files live in the locations in 14.3.
EDIT	Edit the markdown file. Keep the `tools` list under review — it is the security boundary.

DISABLE	Add `Agent(subagent-name)` to the `deny` array in your permission settings to stop Claude using a specific subagent. Or move the file out of the agents directory.
DELETE	Delete the file.
DUPLICATE	Copy the file, change `name`, adjust the prompt. A "strict" and a "lenient" reviewer is a common and useful pair.
SHARE	Commit to `.claude/agents/` for one project; ship in a plugin's `agents/` directory for an organisation (Chapter 12).
VERSION	Git. Review agent changes in pull requests — they change delegated behaviour for the whole team.

PRECEDENCE SURPRISE WHEN MIGRATING TO A PLUGIN

Project and user `.claude/agents/` definitions **override** same-named plugin agents. So after you move an agent into a plugin, the plugin version does not take effect until you delete the original. (Skills behave differently — plugin skills are namespaced, so both remain available. See Chapter 12, Section 12.5.)

Troubleshooting

SYMPTOM	LIKELY CAUSE	FIX
Claude cannot find the new subagent	The `agents` directory did not exist when the session started	Restart Claude Code. A running session does not detect a newly created agents directory.
Subagent fails to launch, error names tool entries	No entry in `tools` resolves to a real tool	Fix the tool names. Check spelling and capitalisation against the tools reference.
Claude never delegates automatically	The `description` does not say *when* to use it	Rewrite as "Does X. Use when …". Meanwhile force it with an @-mention.
Wrong subagent runs	Two descriptions overlap	Narrow both descriptions, or @-mention to guarantee the choice.
Subagent returns everything, not a summary	Its prompt does not ask for brevity	Say so explicitly: what to return, how long, and what to leave out.
It modified files you did not expect	`tools` omitted, so it inherited everything	Omitting `tools` is not "default safe" — it grants every tool available to subagents. List them explicitly.
Plugin agent ignored after migration	A same-named local definition overrides it	Delete the original from `.claude/agents/`.
Parallel subagents conflict on files	They edited the same paths	Give each a worktree, or partition the work so each owns distinct files.
Token usage much higher than expected	Several subagents at once	Running several sessions or subagents multiplies usage. Delegate less, or route workers to a cheaper model.
Subagent does not know something the conversation established	Normal subagents start fresh	Working as designed. Put the context in the task prompt, or use `/subtask` to fork.

14.6 Examples

Example 1 — QA: sprint hygiene audit

Using the `sprint-auditor` from Section 14.3:

```
> @agent-sprint-auditor audit sprint 24 in the Checkout area path
```

Your conversation gets one table and a two-sentence summary — not the forty work items the subagent read to produce them.

Example 2 — QA/Dev: parallel test writing

```
> Write tests for the three modules I changed on this branch. Use a
    separate test-writer subagent per module, each in its own worktree so
    they don't conflict, and run them concurrently.
```

Three independent workers, no file conflicts, one summary each. This is the shape where subagents genuinely pay for themselves.

Example 3 — Development: review with a guaranteed read-only worker

```
> @agent-code-reviewer review every file changed against origin/main
```

Because `tools: Read, Grep, Glob`, this reviewer cannot modify your working tree even if it decides a fix is obvious. That is a stronger guarantee than any instruction.

Example 4 — Development: context-preserving exploration

```
> @agent-explorer where is the retry logic for failed payments, and what
    decides the backoff interval?
```

The `explorer` agent runs on Haiku, reads what it needs, and returns file and line references. Your main session — which might be mid-refactor on Opus — never sees the twelve files it opened.

Example 5 — Design: accessibility audit

.claude/agents/a11y-auditor.md

```
---
name: a11y-auditor
description: Audits components against our accessibility checklist. Use
  when asked about accessibility, a11y, WCAG, keyboard support, screen
  readers, or contrast.
tools: Read, Grep, Glob
model: sonnet
color: purple
---

Audit each component named, against this checklist:

- Accessible name on every interactive element
- Correct role and, where used, correct ARIA
- Full keyboard operation, and a visible focus-visible state
- Sensible focus order; focus trapped correctly in modals and released on close
- Text contrast and non-text UI contrast
- prefers-reduced-motion respected by anything animated
- Status changes announced, not only shown

For each component return a table: check, pass/fail/not-applicable, and the
evidence (file and line) for any failure. Never mark something as passing
that you did not verify in the code — say "not verified" instead.

You are read-only. Report; do not fix.
```

```
> @agent-a11y-auditor audit every component under src/components/forms/
```

Example 6 — a plugin that ships a whole team of agents

Structure

```
qa-toolkit/
├── .claude-plugin/plugin.json
├── skills/
│   ├── test-case/SKILL.md
```

```
    └── bug-report/SKILL.md
└── agents/
    ├── sprint-auditor.md
    ├── test-writer.md
    └── regression-scoper.md
```

```
> /plugin install qa-toolkit@acme-internal
> @agent-qa-toolkit:sprint-auditor audit sprint 24
```

14.7 Best practices

Recommended practices

- **Always set `tools` explicitly.** Omitting it inherits everything. For any reviewer, auditor or explorer, `Read, Grep, Glob` is the right list — and it turns a hope into a guarantee.
- **Write the `description` for delegation.** Same discipline as skills: what it does *and* when to use it.
- **Tell it what to return.** A subagent's value is its summary. Specify the shape, the length, and what to omit.
- **Route mechanical work to a cheaper model.** An explorer on Haiku is dramatically cheaper than the same search in an Opus main session, and just as good at grep.
- **Delegate rarely and deliberately.** Each subagent re-establishes context, re-explores and reports back, and then you re-read the report. That overhead has to be worth it.
- **Brief precisely the first time.** Launching, waiting, and re-briefing costs more than one good prompt.
- **Commit to the delegation.** If you delegate, do not redo the work or re-derive the findings when it reports back.
- **Send parallel work in one message** so the subagents actually run concurrently.
- **Use worktrees for parallel writers** so they cannot conflict.

Common mistakes

MISTAKE	WHY IT HURTS
Omitting `tools` and assuming it is restricted	It inherits every tool available to subagents. This is the most consequential mistake in the chapter.
Delegating trivial work	A subagent for two file reads costs more time and tokens than doing it inline.
Using a subagent to verify the main session's work	Verification generally belongs in your main loop. A verifier that shares the same blind spot adds cost, not confidence.
Expecting it to know the conversation	It starts fresh. Use `/subtask` if it genuinely needs the context.
A vague `description`	Claude never delegates automatically, and the subagent becomes an @-mention you forget exists.
Parallel writers without worktrees	They overwrite each other.
Spawning many subagents "to be thorough"	Token usage multiplies. More workers is not more rigour.
Leaving a duplicate local definition after moving to a plugin	The local one silently wins.

Naming conventions

- **Role nouns, lowercase, hyphenated:** `code-reviewer`, `sprint-auditor`, `a11y-auditor`, `test-writer`, `explorer`. You will @-mention these.
- **Name the job, not the model or the mechanism.** `explorer`, not `haiku-search-helper`.
- **No colons** — reserved for plugin scoping. A name containing one is not loaded, and the error goes to the debug log.
- **Use `color`** to make agents distinguishable at a glance in the transcript, especially when several run in parallel.

Versioning and team considerations

- **Commit project agents.** They encode team standards; they belong in version control and in code review.
- **Treat an agent prompt change as a behavioural change** for everyone who uses it.
- **Ship shared agents in a plugin** for cross-repository reuse, and remember the precedence rule when migrating.
- **Document the team's agents** alongside the skills catalogue. An agent nobody knows about is an agent nobody uses.
- **Agent teams are experimental and disabled by default**, and teammates are not isolated in worktrees — partition the work so each owns different files. Treat that surface as exploratory rather than something to standardise on.

Security considerations

THE TOOLS LIST IS A REAL SECURITY BOUNDARY — USE IT

- **Restrict by default.** A subagent with an explicit read-only tool list cannot write, regardless of what it concludes. This is the most useful containment mechanism in Claude Code.
- **Be careful with** `mcpServers`. It can give a subagent access to systems the main conversation does not have. That is powerful and worth being deliberate about.
- **Deny specific agents where needed:** `Agent(subagent-name)` in the permission `deny` array stops Claude using it at all.
- **Committed agents are a supply-chain surface.** Anyone who can commit can change a delegated worker's prompt and tool list. Review those diffs.
- **Subagent** `hooks` **run shell commands** at that agent's lifecycle events. Same trust considerations as any hook.
- **A subagent's own auto memory is separate** from the main conversation's, and persists across sessions when `memory` is set. Know what you are letting it remember.

14.8 Official resources

Create custom subagents — the complete reference: file format, every frontmatter field, scopes and precedence, invocation, forking, memory
https://code.claude.com/docs/en/sub-agents

Run agents in parallel — the comparison of subagents, agent view, agent teams and workflows used in Section 14.1
https://code.claude.com/docs/en/agents

Agent view — dispatch and monitor background sessions with `claude agents` **(research preview)**
https://code.claude.com/docs/en/agent-view

Agent teams — coordinated peer sessions with a shared task list (experimental, disabled by default)
https://code.claude.com/docs/en/agent-teams

Manage costs — how multi-agent runs count toward usage and rate limits
https://code.claude.com/docs/en/costs

Permission settings — the `deny` **array and** `Agent(name)` **rules**
https://code.claude.com/docs/en/permissions

Managed Agents on the Claude Developer Platform — server-managed stateful agents, for engineers building applications
https://platform.claude.com/docs/en/managed-agents/overview

CHAPTER 15

Workflows

Move the plan into code. A script Claude writes, orchestrating dozens of agents, that you can read and rerun.

ADVANCED | CLAUDE CODE CLI | VS CODE EXTENSION | DESKTOP APP

ALL PAID PLANS · REQUIRES CLAUDE CODE V2.1.154+

15.1 What is it?

A dynamic workflow is a JavaScript script that orchestrates subagents at scale. **Claude writes the script** for the task you describe, and a runtime executes it in the background while your session stays responsive. You did not have to write any code — but you can read the script, edit it, and rerun it.

The problem it solves

Chapter 14 gave you delegated workers. But delegation has a ceiling: Claude decides turn by turn what to spawn next, and every result lands in a context window. Ask for a codebase-wide audit of 300 route handlers and the coordination itself becomes the bottleneck — Claude spends its context managing the work rather than doing it.

A workflow moves the plan out of Claude's head and into a script. The script holds the loop, the branching and the intermediate results, so **Claude's context holds only the final answer**.

There is a second, subtler benefit. Because the plan is code, a workflow can apply a repeatable *quality* pattern rather than just running more agents: have independent agents adversarially review each other's findings before they are reported, or draft a plan from several angles and weigh them against each other. You get a more trustworthy result than a single pass, reliably, every time.

Who holds the plan — the distinction that matters

	SUBAGENTS	SKILLS	AGENT TEAMS	WORKFLOWS
What it is	A worker Claude spawns	Instructions Claude follows	A lead agent supervising peer sessions	**A script the runtime executes**
Who decides what runs next	Claude, turn by turn	Claude, following the prompt	The lead agent, turn by turn	**The script**
Where intermediate results live	Claude's context window	Claude's context window	A shared task list	**Script variables**
What's repeatable	The worker definition	The instructions	The team definition	**The orchestration itself**
Scale	A few delegated tasks per turn	Same as subagents	A handful of long-running peers	**Dozens to hundreds of agents per run**
Interruption	Restarts the turn	Restarts the turn	Teammates keep running	**Resumable in the same session**

Reproduced from Anthropic's workflows documentation. The single most useful table in this chapter.

What a workflow script looks like

You do not write these — Claude does. But you should be able to recognise one, because you can read and edit the script:

`.claude/workflows/audit-routes.js`

```
export const meta = {
  name: 'audit-routes',
  description: 'Audit every route handler for missing auth checks',
}

const found = await agent('List every .ts file under src/routes/.', {
  schema: { type: 'object', required: ['files'],
            properties: { files: { type: 'array', items: { type: 'string' } } } },
})

const audits = await pipeline(found.files, file =>
  agent(`Audit ${file} for missing authentication checks.`, { label: file }),
)

return audits.filter(Boolean)
```

Plain JavaScript with top-level `await`. `agent()` spawns one subagent; `pipeline()` runs one per item in a list. An `agent()` call resolves to `null` if you stop it mid-run or it hits an unrecoverable API error — `pipeline()` keeps that `null` in the results, which is why the example ends with `.filter(Boolean)`.

What it is NOT

- **Not something you write.** Claude writes the script. Your job is to describe the task well.
- **Not a Routine.** A workflow decides *how* work is orchestrated; a routine decides *when* a session starts. A routine's session can run a workflow.
- **Not interactive.** There is no mid-run user input. Only agent permission prompts can pause a run. For sign-off between stages, run each stage as its own workflow.
- **Not able to touch the filesystem itself.** The script coordinates; the *agents* read, write and run commands. A script containing `import()` fails before the run starts.
- **Not unbounded.** Up to 16 concurrent agents (fewer on machines with limited CPU cores) and 1,000 agents total per run.
- **Not cheap.** A single run can use meaningfully more tokens than working through the same task in conversation. Section 15.7.

How it relates to the other blocks

BLOCK	RELATIONSHIP
Agents	Subagents are the worker primitive a workflow orchestrates. Chapter 14 is the prerequisite for this one.
Skills	Both run multi-step work, but a skill is instructions Claude follows in its own context; a workflow is a script a runtime executes. If you already have a skill that fans work out, you can point Claude at it and ask for a workflow that does the same thing.
Plugins	Put a script in a plugin's `workflows/` directory to distribute it. Namespaced as `/plugin-name:workflow-name`.
Routines	A routine's cloud session can run a workflow — scheduled, large-scale, unattended work.
Artifacts	A long workflow can keep an investigation-timeline artifact updating as it runs.
MCP / Connectors	Workflow agents can reach session-connected MCP tools, with schemas loaded on demand per agent.

15.2 When should I use it?

USE A WORKFLOW WHEN…

- The task needs **more agents than one conversation can coordinate**.
- The same step must run **across many items** — 300 files, 50 endpoints, every changed file in a PR.
- You want findings **verified against each other** rather than reported on first pass.
- You want the orchestration **codified as a script you can read and rerun**.
- A hard plan is worth **drafting from several independent angles** before you commit.
- The job is **larger than one agent can hold in context**.

Do not use a Workflow when…

- A handful of subagents would do → Chapter 14. Workflows have real overhead.
- You need to approve something mid-run. Split it into stages instead.
- The task is conversational or exploratory.
- You are cost-sensitive and have not scoped it. Run on a slice first.

The shapes that fit

SHAPE	EXAMPLE
Audit many files for the same issue	Every route handler checked for missing auth, with each finding adversarially verified before it is reported.
Keep fixing until a check passes	Run the type checker, fix what failed, repeat until clean or two rounds make no progress.
Migrate many files in parallel	Discover the files, transform each in an isolated copy so edits cannot conflict, verify each result.
Review, then synthesise	One reviewer per changed file, then one agent that ranks and deduplicates all the findings.
Research across many sources	Fan readers out across changelogs, issues and docs, then compare and synthesise.
Find issues until the list stops growing	Search in rounds; stop when two consecutive rounds turn up nothing new.

15.3 How do I set it up?

Prerequisites: Claude Code v2.1.154 or later, and any paid plan. Available with Anthropic API access and on Amazon Bedrock, Google Cloud's Agent Platform and Microsoft Foundry. **On Pro, turn them on** from the **Dynamic workflows** row in `/config`.

The fastest way to see one work

Claude Code ships one built-in workflow, `/deep-research`. Run it before building anything of your own.

1. **Run it with a real question**

```
> /deep-research What changed in the Node.js permission model between v20 and v22?
```

It fans web searches across several angles, fetches and cross-checks the sources it finds, votes on each claim, and returns a cited report with claims that did not survive cross-checking filtered out.

2. **Allow the workflow**

Claude Code asks whether to allow it. Select **Yes**. The exact prompt depends on your permission mode — see Section 15.4.

3 **Watch progress**

```
> /workflows
```

Arrow keys to select the run, Enter to open its progress view: each phase with its agent count, token total and elapsed time. You can also watch from the task panel below the input box — press the down arrow to focus it, then Enter to expand.

4 **Read the report**

When the run finishes, the report lands in your session, citing the sources each claim came from. Claims the verifiers could not check are listed as *unverified* rather than counted as refuted.

`/deep-research` runs only when you invoke it. It requires the WebSearch tool to be available.

Getting a workflow for your own task

Three routes:

ROUTE	HOW
Ask in your prompt	Include the keyword `ultracode`, or just ask in your own words — "use a workflow", "run a workflow". Claude treats a direct request as the same opt-in.
Let Claude decide	`/effort ultracode` — Claude then plans a workflow for every substantive task in the session.
Run an existing one	A bundled workflow like `/deep-research`, or one you saved.

```
# one task as a workflow, without changing the session's effort level
> ultracode: audit every API endpoint under src/routes/ for missing auth checks

# or in plain words — same effect
> use a workflow to audit every route handler for missing authentication checks,
    and adversarially verify each finding before reporting it
```

Claude Code highlights the keyword in your input. Pressing Option+W (macOS) or Alt+W (Windows/Linux) dismisses the highlight for that prompt; backspace right after the highlighted keyword also works. To stop it triggering at all, turn off **Ultracode keyword trigger** in `/config`.

> **WHERE THE KEYWORD WORKS — AND DELIBERATELY DOES NOT**
>
> The keyword is an opt-in *only in a prompt you type yourself*: the interactive prompt, an IDE extension panel, a Remote Control client, or an Agent SDK application that stamps your keyboard input as human. It does **not** start a workflow when it arrives another way: a prompt passed with `-p`, an Agent SDK prompt not marked as human input, a scheduled-task prompt, or a webhook payload or pull-request comment relayed into the conversation. That is a security property, not a limitation.

Ultracode as a session mode

```
> /effort ultracode                # this session
$ claude --effort ultracode         # start a session with it on (v2.1.203+)
> /effort high                     # drop back for routine work
```

Ultracode combines `xhigh` reasoning effort with automatic workflow orchestration. With it on, a single request can turn into several workflows in a row — one to understand the code, one to make the change, one to verify it. **Every task in the session uses more tokens and takes longer.** It lasts for the current session and resets when you start a new one.

Set a size guideline — do this before your first large run

A size guideline tells Claude how many agents to aim for. It is sent as advice, not a cap, so a prompt that calls for a different scale still overrides it.

VALUE	AGENT COUNT CLAUDE AIMS FOR
`unrestricted`	No guideline — Claude sizes the workflow to the task

VALUE	AGENT COUNT CLAUDE AIMS FOR
`small`	Fewer than 5 agents
`medium` (default)	Fewer than 15 agents
`large`	Fewer than 50 agents

```
> /config workflowSizeGuideline=small
```

Or pick a value in the **Dynamic workflow size** row of `/config`. On v2.1.219+ you can also set `workflowSizeGuideline` in any settings file, which takes precedence over `/config`. Changes take effect on the next prompt; the runtime agent caps still apply regardless.

15.4 How do I use it?

Approving a run

The prompt shows the planned phases and these options: **Yes, run it**; **Yes, and don't ask again for <name> in <path>**; **View raw script**; **No**. Ctrl+G opens the script in your editor; Tab lets you adjust the prompt before the run starts.

PERMISSION MODE	WHEN YOU ARE PROMPTED
Default, accept edits	Every run, unless you chose **Yes, and don't ask again** for that workflow in this project
Auto	First launch only. Any **Yes** records consent in your user settings. Skipped entirely when ultracode is on
Bypass permissions, `claude -p`, Agent SDK	Never — the run starts immediately

> **YOUR PERMISSION MODE CONTROLS ONLY THE LAUNCH PROMPT**
>
> The subagents a workflow spawns **always run in `acceptEdits` mode** and inherit your tool allowlist, regardless of your session's mode. **File edits are auto-approved.**
>
> Shell commands, web fetches and MCP tools that are not in your allowlist can still prompt you mid-run. On a long run that is disruptive — add the commands the agents will need to your allowlist *before* starting.

Watching a run

```
> /workflows
```

KEY	ACTION
↑ / ↓	Select a phase or agent
Enter / →	Drill into a phase, then into an agent to read its prompt, recent tool calls and result
Esc / ←	Back out one level
j / k	Scroll within the agent detail when it overflows
f	Filter the agent list in the selected phase by status; press again to cycle
p	Pause or resume the run
x	Stop the selected agent, or the whole workflow when focus is on the run
r	Restart the selected running agent
s	**Save** the run's script as a command

In the Desktop app, an approval card shows the workflow name, phase list and a token-usage caution with **Once**, **Always** and **Deny** actions; progress appears in the Background tasks side pane.

How to verify the result

1. **Read what the phases actually did.** Drill into a phase and then an agent in `/workflows` to see its prompt, tool calls and result. A workflow that produced a confident report from agents that all failed to read anything is worth catching.
2. **Spot-check findings against the code.** Open two or three of the reported issues and confirm they are real. This is the whole point of the adversarial-verification pattern, and it is also how you check the verification worked.
3. **Look for what is missing.** If a workflow bounded coverage — top-N, sampling, no retry — a well-written one logs what it dropped. Silent truncation reads as "covered everything."
4. **Check the agent count and token total** in the progress view against what you expected.

Saving a workflow for reuse

When a run does what you wanted, keep it. Run `/workflows`, select the run, press s. In the save dialog, Tab toggles between two locations:

LOCATION	SCOPE
`.claude/workflows/`	In your project — shared with everyone who clones the repo
`~/.claude/workflows/`	Your home directory — every project, visible only to you

Press Enter to save. The workflow then runs as `/<name>` in future sessions and appears in `/` autocomplete alongside the bundled ones. If a project workflow and a personal workflow share a name, the project one runs.

In a monorepo, saving to the project location writes to the closest existing `.claude/workflows/` between your working directory and the repository root. Project workflows load from every such directory along that path, and the one closest to the working directory wins on a name clash.

Passing input to a saved workflow

```
> Run /triage-issues on issues 1024, 1025, and 1030
```

Claude passes the list as structured data, which the script reads as a global named `args` — so it can call array and object methods directly without parsing. If `args` is omitted, the global is `undefined`.

Distributing a workflow in a plugin

Place the script in a `workflows/` directory at the plugin root. Plugin workflows are namespaced by the plugin name: a plugin called `acme-tools` containing a script whose `meta.name` is `release-audit` runs as `/acme-tools:release-audit`.

15.5 How do I manage it?

START	Include `ultracode` or ask for a workflow in your own words; or `/effort ultracode` for the session; or run a bundled or saved workflow command.
WATCH	`/workflows`, or the progress line in the task panel below the input box.
PAUSE / RESUME	p in the `/workflows` view.
STOP	x on an agent, or on the run to stop the whole workflow.
RESTART AN AGENT	r on the selected running agent.
SAVE	s in `/workflows`, then choose project or personal location.
EDIT	Every run writes its script to a file under your session's directory in `~/.claude/projects/`. Claude receives the path when the run starts, so you can ask for it — then read it, diff it against a previous run, or edit it and ask Claude to relaunch from the edited version.

SIZE	`/config workflowSizeGuideline=small|medium|large|unrestricted`, or the **Dynamic workflow size** row in `/config`.
TURN OFF	Toggle **Dynamic workflows** off in `/config`; set `"disableWorkflows": true` in `~/.claude/settings.json`; or set `CLAUDE_CODE_DISABLE_WORKFLOWS=1`. Organisation-wide: `"disableWorkflows": true` in managed settings, or the toggle on the Claude Code admin settings page.

When workflows are disabled, the bundled workflow commands are unavailable, the `ultracode` keyword no longer triggers a run, and `ultracode` is removed from the `/effort` menu.

Resuming after a pause — and why stopping is expensive

You can resume a stopped run: agents that already completed usually return cached results, and the rest run live. Two rules decide which results survive:

1. An agent that was **still running** when you stopped is not saved, so it starts over.
2. Replay follows the order agents *started*. Cached results stop at the first agent that did not finish, and **every agent that started after that one runs again** — even if it completed.

> **WORKED EXAMPLE, BECAUSE RULE 2 IS COUNTER-INTUITIVE**
>
> A script starts four agents A, B, C, D in that order. You stop the run while B is still going. On resume: A returns from cache. B runs again because it never finished. **C and D run again too** — because they started after B, even though both had completed.
>
> The consequence: a workflow that fans work out across many small agents preserves far more progress than one with a few long agents. Resume works within the same session; exiting Claude Code means the next session starts the workflow fresh.

Behaviour and limits

CONSTRAINT	WHY
No mid-run user input	Only agent permission prompts can pause a run. For sign-off between stages, run each stage as its own workflow.
No direct filesystem or shell access from the workflow itself	Agents read, write and run commands. The script coordinates the agents.
No module loading — a script containing `import()` fails before the run starts	The script body is plain JavaScript. Work needing a library goes in an agent's task.
Up to 16 concurrent agents, fewer on limited CPU cores	Bounds local resource use.
1,000 agents total per run	Prevents runaway loops.

Troubleshooting

SYMPTOM	CAUSE	FIX
The keyword does nothing	Workflows disabled, or on Pro without turning them on	Enable **Dynamic workflows** in `/config`. Check `disableWorkflows` and `CLAUDE_CODE_DISABLE_WORKFLOWS`, and whether managed settings disable it org-wide.
`ultracode` not in the `/effort` menu	Workflows disabled, or the model does not support `xhigh` effort	Ultracode is available only on models supporting `xhigh`. Check `/model`.
Keyword ignored from a script or webhook	By design	The keyword only opts in from a prompt you type. Invoke a saved workflow command instead.
Run keeps stopping for permission prompts	Shell commands, web fetches or MCP tools not in your allowlist	Add them to your tool allowlist before starting a long run.

SYMPTOM	CAUSE	FIX
Resume re-ran agents that had finished	Replay follows start order	Expected — see the worked example above. Fan out across more, smaller agents.
Resume did nothing after restarting Claude Code	Resume works within the same session only	The next session starts the workflow fresh.
`Large workflow` warning	More than 25 agents scheduled, or projected tokens above 1.5M	Advisory only — it does not pause the run. Open `/workflows` and stop it if unintended. A chosen size guideline replaces the 25-agent threshold; ultracode sessions do not show the warning.
Agents ran on a different model than requested	Your organisation's `availableModels` allowlist blocked it	Expected substitution. The progress view names both the requested and substituted models.
Script fails immediately	`import()` in the script body	Not supported. Move library work into an agent's task.
Empty or unexpected result	Agents returned `null`, or coverage was silently bounded	Drill into the phases in `/workflows` and read the agent results. Ask Claude for the script path and inspect it.

15.6 Examples

Each of these asks Claude to write and run a workflow. You do not write the script.

Example 1 — QA: audit many files for the same issue

```
> use a workflow to audit every route handler under src/routes/ for missing
    authentication checks, and adversarially verify each finding before
    reporting it
```

One agent per file, then independent verifiers that try to *refute* each finding. What survives is reported. This is the pattern that makes a workflow more trustworthy than a single pass, not just faster.

Example 2 — QA: hunt flaky tests until the list stops growing

```
> use a workflow to find flaky tests in this repo: run the suite repeatedly,
    record which tests fail intermittently, and stop once two rounds in a row
    find nothing new
```

The "stop when two rounds find nothing new" shape matters — a simple "find 10" target misses the tail, and an unbounded loop never converges.

Example 3 — Development: keep fixing until a check passes

```
> use a workflow to run npx tsc --noEmit and keep fixing the reported errors
    until the type check passes or two rounds in a row make no progress
```

Example 4 — Development: a large parallel migration

```
> use a workflow to migrate every component under src/components/ from
    styled-components to Tailwind, working on each file in its own isolated
    copy, and verify each result compiles and its tests still pass
```

"Its own isolated copy" is the important clause — it makes the agents work in separate worktrees so parallel edits cannot conflict.

Example 5 — Development: review every changed file, one ranked summary

```
> use a workflow to review every file changed in this PR for correctness
    issues, then merge the per-file findings into one ranked, deduplicated
```

```
    summary
```

Example 6 — Product/BA: cross-checked competitive research

```
> use a workflow to research how our three competitors handle rate limiting:
    read their public docs and recent changelog entries in parallel, then
    compare the approaches and flag anything you could not verify
```

Example 7 — save it, then run it every sprint

```
# the run did what you wanted, so keep it
> /workflows          # select the run, press s, choose .claude/workflows/

# from then on, in any session on this repo
> /audit-routes

# with input
> Run /triage-issues on issues 1024, 1025, and 1030
```

Example 8 — scope the cost before committing

```
# 1. cap the scale
> /config workflowSizeGuideline=small

# 2. run on one directory, not the whole repo
> use a workflow to audit src/routes/admin/ for missing auth checks

# 3. check the token total in /workflows, then widen if it was worth it
```

15.7 Best practices

Recommended practices

- **Run `/deep-research` first.** It costs one prompt and teaches you what the progress view, phases and cross-checking actually look like.
- **Scope on a slice before committing.** One directory instead of the repo; a narrow question instead of a broad one. Then check the token total in `/workflows`.
- **Set a size guideline** before your first large run. `medium` (under 15 agents) is the default and a sensible ceiling for most work.
- **Ask for verification explicitly.** "Adversarially verify each finding before reporting it" is what turns volume into confidence.
- **Ask for the quality pattern that fits the task:** adversarial verification for findings; several independent attempts plus a judge for a hard plan; loop-until-dry for open-ended discovery.
- **Prefer many small agents over few long ones** — both for concurrency and because resume preserves far more progress.
- **Allowlist the commands the agents will need** before a long run, so it does not stall on prompts.
- **Save runs that worked.** The orchestration is the reusable asset, and it is one keypress.
- **Ask Claude for the script path** when you want to understand or adjust what happened. Reading it is genuinely informative.
- **Check `/model` before a large run** if you habitually switch to a smaller model — every agent uses your session's model unless the script routes a stage elsewhere.

Common mistakes

MISTAKE	WHY IT HURTS
Reaching for a workflow when subagents would do	Real overhead in tokens and latency. Workflows earn their cost at scale, not at three tasks.
Running on the whole repo first	You discover the cost after paying it. Slice first.
Expecting to approve things mid-run	There is no mid-run input. Split into stages.
Trusting the report without spot-checking	Volume is persuasive. Open two findings and confirm they are real.
Stopping a run with a few long agents	Replay order means everything after the unfinished agent re-runs.
Leaving ultracode on for routine work	Every task in the session costs more and takes longer. `/effort high` when you are done.
Assuming your permission mode protects the agents	Workflow subagents always run in `acceptEdits`; file edits are auto-approved.
Not noticing bounded coverage	Ask what was dropped. Silent truncation reads as completeness.

Cost management

Worth its own heading, because a workflow is the most expensive thing in this guide.

- Runs count toward your plan's usage and rate limits like any session.
- The `/workflows` view shows each agent's token usage as the run progresses, and you can stop it at any time — usually without losing completed work.
- Claude Code flags a run that grows unusually large: more than 25 agents scheduled, or projected tokens above 1.5 million, shows a **Large workflow** warning in the task panel. It is advisory and does not pause the run. A size guideline you chose replaces the 25-agent threshold, and ultracode sessions do not show it.
- Choose `small` as a size guideline to keep runs to fewer agents.
- Ask Claude to use a smaller model for stages that do not need the strongest one.

Naming, versioning and team considerations

- **Name saved workflows after the job:** `audit-routes`, `triage-issues`, `migrate-styles`, `release-audit`.
- **Commit useful workflows** to `.claude/workflows/` so the team shares the orchestration, and review changes to them — a workflow script spawns agents with file-edit auto-approval.
- **Distribute across repositories in a plugin** (Chapter 12), namespaced as `/plugin-name:workflow-name`.
- **Agree an organisational size guideline** so nobody's first workflow is a 200-agent run. Managed settings can also disable workflows entirely.
- **Note the symlink safety check:** Claude Code refuses to write a saved workflow through a symlink — for the project location it refuses if `.claude`, `.claude/workflows` or the target file is a symlink; for the personal location only if the target file itself is.

Security considerations

> **WHAT A WORKFLOW ACTUALLY AUTHORISES**
>
> - **Workflow subagents run in `acceptEdits` mode regardless of your session's mode.** File edits are auto-approved. Approving a workflow is approving a lot of file writes.
> - **They inherit your tool allowlist** — so widening the allowlist to avoid mid-run prompts also widens what dozens of parallel agents may do without asking.
> - **Read the script before approving an unfamiliar workflow. View raw script** and Ctrl+G exist for exactly this.
> - **"Don't ask again" is per workflow, per project** — a reasonable scope, but know that you granted it.
> - **The keyword deliberately does not fire from untrusted routes** — webhook payloads, PR comments, scheduled prompts, `-p`. Do not build a mechanism that works around that.
> - **In `claude -p` and the Agent SDK there is nobody to prompt**, so tool calls follow your configured rules without interactive confirmation. Be deliberate about running workflows there.

- **Organisations can disable workflows** via managed settings or the Claude Code admin settings page.

15.8 Official resources

Orchestrate subagents at scale with dynamic workflows — the complete reference used throughout this chapter: bundled workflows, the ultracode keyword, approval, the progress view, saving, resume rules, limits, cost
`https://code.claude.com/docs/en/workflows`

Run agents in parallel — how workflows compare with subagents, agent view and agent teams
`https://code.claude.com/docs/en/agents`

Create custom subagents — the worker primitive workflows orchestrate
`https://code.claude.com/docs/en/sub-agents`

Manage costs — how multi-agent runs count toward usage limits
`https://code.claude.com/docs/en/costs`

Permission modes — what the launch prompt depends on
`https://code.claude.com/docs/en/permission-modes`

Introducing dynamic workflows — the announcement
`https://claude.com/blog/introducing-dynamic-workflows-in-claude-code`

A harness for every task: dynamic workflows in Claude Code
`https://claude.com/blog/a-harness-for-every-task-dynamic-workflows-in-claude-code`

CHAPTER 16

How Everything Fits Together

Eleven blocks, one system. The relationship map, and the live demo traced through it step by step.

Chapter 4 introduced the five layers so you could tell the blocks apart. This chapter shows them working as one system — first as a map of what depends on what, then as one worked end-to-end pipeline, traced block by block.

16.1 The dependency map

Some blocks stand alone. Others only make sense on top of something else. This is the map that tells you what you need before you can use a given capability.

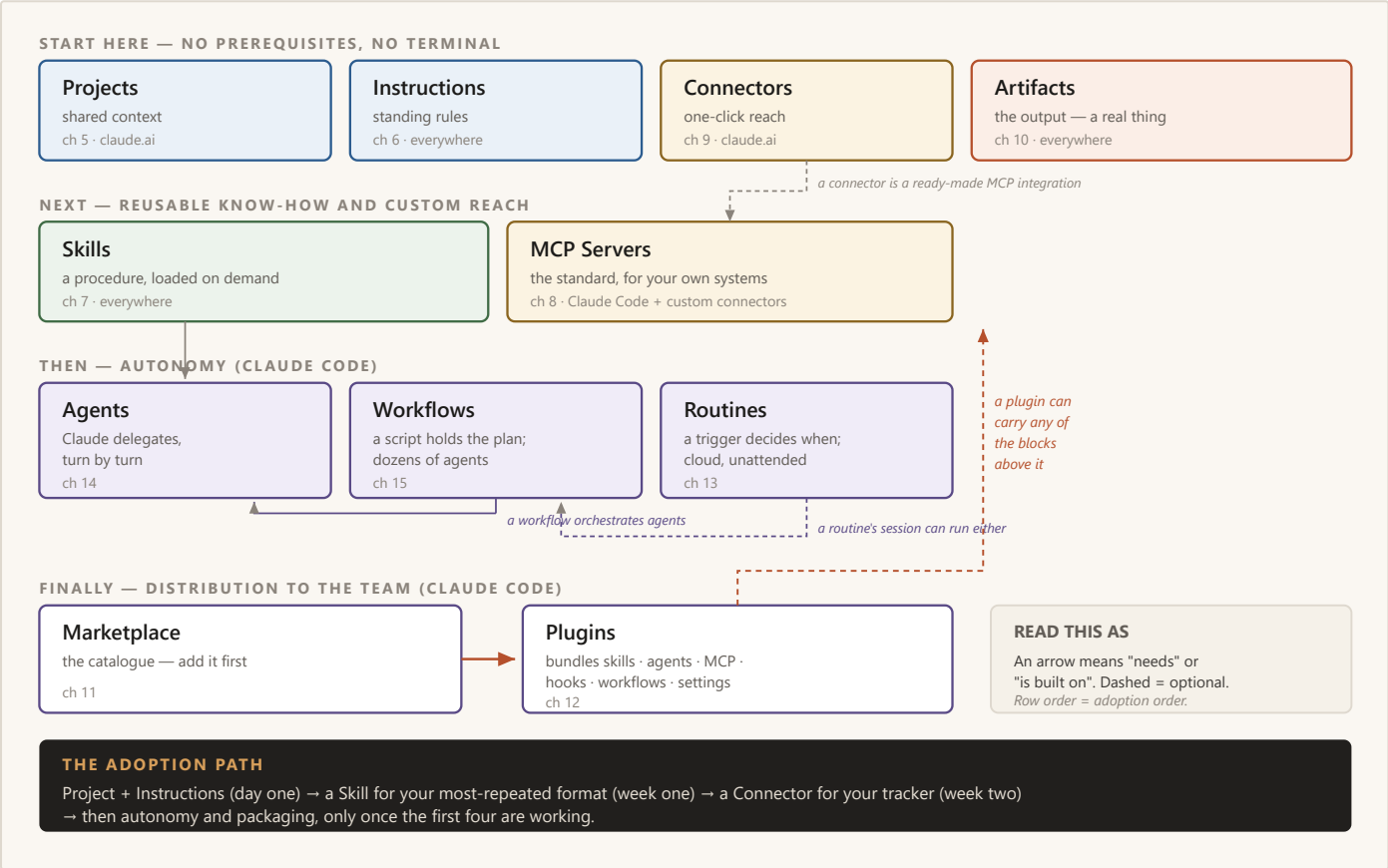


Figure 16.1 What depends on what. Row order is also a sensible adoption order — nothing below row 2 is worth attempting before rows 1 and 2 are working for you.

Diagram — not a screenshot. Prepared for this guide.

What each block needs before it is useful

BLOCK	HARD PREREQUISITES	MUCH BETTER WITH
Projects	None	Instructions, so the shared context comes with shared rules.
Instructions	None	Skills, once the rules turn into procedures.
Artifacts	One capability toggle	Skills, so a recurring deliverable has a consistent format.
Connectors	An account on the target system	Skills, so the fetched data lands in your format.
Skills	None	Connectors or MCP, so the procedure operates on real data.
MCP Servers	A server to connect to	Skills; and a Plugin, to distribute the configuration.

BLOCK	HARD PREREQUISITES	MUCH BETTER WITH
Agents	Claude Code	Skills injected at startup; worktrees for parallel writers.
Workflows	Claude Code v2.1.154+	Agents (they are the workers); a size guideline set in advance.
Routines	Claude Code on the web; GitHub access	*Account* connectors and committed skills — local ones do not travel.
Marketplace	A repository to host it	Something worth cataloguing. Do not start here.
Plugins	A marketplace, to distribute	A settled standalone setup to convert.

16.2 A worked pipeline, traced through the system

The clearest way to watch the blocks cooperate is a single real Azure DevOps workflow that chains five of them together against a tester's own back catalogue. It is the best single illustration of how these pieces compose, so here it is with the mechanics of each step spelled out and cross-referenced.

#	DEMO STEP	BLOCK	WHAT IS ACTUALLY HAPPENING
1	**Start a Project in Cowork** "One workspace to hold the whole demo's context."	Projects ch 5	A workspace is created so every subsequent chat inherits the same context. The knowledge base holds the material the rest of the demo depends on — nothing has to be re-pasted.
2	**Add Instructions & Memory** "My rules and the running context Claude keeps."	Instructions ch 6	Standing rules are attached at project scope, layering on top of account-wide instructions. From here on every message carries them without being asked.
3	**Connect Azure DevOps MCP** "Authenticate — Claude can now reach ADO directly."	MCP Servers ch 8	Reach. Claude gains the server's *tools* (create/update work item, run pipeline, create test plan), its *resources* (project schema, current sprint, work items, the wiki) and its *prompts*. The copy-paste courier is gone.
4	**Learn my test-case style** "Pull 100+ test cases I've written; build a Skill that matches my wording, step-by-step approach and expected results."	Skills ch 7	The most instructive step. Rather than writing the skill from memory, the existing corpus is read *through the connector from step 3* and the skill is derived from real output. Know-how, extracted rather than invented.
5	**Link a story & push a suite** "Attach to a user story and push the generated cases into a test suite."	Skills + MCP ch 7 + 8	The composition that makes it real: the *skill* supplies the format, the *server* supplies the write. Note this is the step where a read-then-confirm-then-write habit matters (Chapter 8, Section 8.4).
6	**Build an analysis artifact** "Trend & stats of tickets by type I created — how my activity changed over the last 6 months."	Artifacts ch 10	Output. Six months of real work-item data becomes something you open, read, refine in place and share as a link — rather than a wall of chat text you would have to reformat.

A representative end-to-end pipeline, with the mechanics of each step.

WHY THAT ORDERING IS THE RIGHT ORDERING

Notice the dependency chain. You cannot derive a skill from your real test cases (step 4) until Claude can *read* them (step 3). You cannot push cases into a suite (step 5) until you have both the format and the reach. And the analysis (step 6) is only interesting because it runs against real data rather than a pasted sample.

Every step earns the next one. That is the general pattern: **context, then know-how, then reach, then output** — and only then autonomy.

What the demo did not cover, and where it would slot in

BLOCK	WHERE IT WOULD EXTEND THE SAME FLOW
Connectors ch 9	If Azure DevOps is available as a reviewed connector for your organisation, step 3 becomes a single click rather than a server configuration.
Agents ch 14	A read-only `sprint-auditor` subagent runs the hygiene check without dragging forty work items into your conversation.
Workflows ch 15	Instead of one story, do all ninety: one agent per story, findings cross-checked, one ranked summary.
Routines ch 13	Run the whole hygiene audit every Friday at 17:00 and post the summary to the team channel. Nobody has to remember.
Plugins + Marketplace ch 11–12	Bundle the skill, the agent and the server configuration into `qa-toolkit`, publish it internally, and every QA colleague installs the whole demo in one command.

16.3 Three composition patterns worth knowing

Beyond the demo, three combinations come up repeatedly. Each is more than the sum of its parts.

Pattern 1 — The house standard

Instructions + Skills + Plugin. The problem: twelve people produce twelve variants of the same document.

1. Facts and always-true rules go in **Instructions** — tone, terminology, prohibitions.
2. Each repeated format becomes a **Skill** — `/test-case`, `/bug-report`, `/user-story`.
3. Both ship in a **Plugin** from an internal **Marketplace**, so the standard is installed rather than described — and improving it once improves it for everyone.

Why it works: the split respects the context budget. Rules load always because they are short; procedures load on demand because they are long. And distribution means the standard cannot drift into twelve local copies.

Pattern 2 — The connected specialist

MCP/Connector + Skill + Agent. The problem: an audit that needs live data and would otherwise flood your conversation.

1. A **Connector or MCP server** gives reach into the tracker.
2. A **Skill** defines what "good" looks like — the checklist, the output shape.
3. A read-only **Agent** with `tools: Read, Grep, Glob` plus that server does the work in its own context and returns a summary.

Why it works: the agent's tool list makes it *structurally* unable to modify anything, which is a stronger guarantee than any instruction. And your main conversation gets the conclusion without the evidence.

Pattern 3 — The unattended report

Routine + Skill + Workflow + Artifact. The problem: valuable analysis nobody has time to run.

1. A **Routine** fires on Friday at 17:00 — laptop closed.
2. Its session runs a saved **Workflow** that fans out across every item, cross-checking findings.
3. Each worker follows a committed **Skill** so the output is consistent.
4. The result is published as an **Artifact** — a link in the team channel on Monday morning.

> **THE THREE THINGS THAT BREAK THIS PATTERN**
>
> Pattern 3 is the one people get wrong, and always in the same three ways:
>
> - **Local skills do not travel.** A routine runs a fresh remote session; `~/.claude/skills/` is not read. Upload to claude.ai, commit to the repo, or ship in a repo-declared plugin.
> - **Local MCP servers do not travel either.** A routine sees your *account connectors*, not servers added with `claude mcp add`.

- **Nobody is there to approve anything.** Every included connector's tools, writes included, are usable without a prompt. Trim that list, and prefer draft output over direct writes.

16.4 Reading the whole system as one sentence

If you remember nothing else from this chapter:

> A **Routine** fires on a schedule. The session it starts inherits your **Instructions** and, in Claude Code, the repository's committed context. It runs a **Workflow**, which spawns dozens of **Agents**. Each agent follows a **Skill** and reads your real systems through a **Connector** or an **MCP Server**. Their findings are cross-checked and synthesised, and the result is published as an **Artifact** — a link you open on Monday. All of it — skills, agents, server configuration, hooks — was installed as one **Plugin** from your organisation's **Marketplace**, and the reference material it works against lives in a **Project**.
>
> *You did not do any of the steps. You specified the outcome, and you own the result.*

That is the whole guide. Chapter 17 turns it into decision tables you can use without re-reading any of it.

CHAPTER 17

"Which One Should I Use?"

The bookmark chapter. Decision tables that answer the question without re-reading anything.

17.1 The master table — "If I need to..."

Start here. One row per intent.

IF I NEED TO...	USE	WHERE
Give Claude persistent background material for a body of work	**Project** ch 5	claude.ai, Cowork
Make Claude follow a rule on *every* message	**Instructions** ch 6	Everywhere
Teach Claude a reusable process or format	**Skill** ch 7	Everywhere
Connect Claude to a tool that is already on the list	**Connector** ch 9	claude.ai, Cowork
Connect Claude to an internal, legacy or custom system	**MCP Server** ch 8	Claude Code; claude.ai as a custom connector
Get an output I can open, use, refine and share	**Artifact** ch 10	Everywhere
Do a side task without cluttering my conversation	**Agent** ch 14	Claude Code
Run the same step across dozens or hundreds of items	**Workflow** ch 15	Claude Code
Have something happen while I'm asleep	**Routine** ch 13	claude.ai/code, Desktop, CLI
Give my whole team my working setup in one command	**Plugin** ch 12	Claude Code; claude.ai, Desktop chat and Cowork on paid plans
Distribute plugins across the whole organisation	**Marketplace** ch 11	Claude Code; Organization settings › Plugins for Team/Enterprise
Guarantee something happens, not merely ask for it	**Hook** ch 6.6, ch 12	Claude Code
Stop Claude from being *able* to do something	**Tool restriction** on an agent, or a permission `deny` rule ch 14, ch 21	Claude Code

17.2 Decision tree — "Claude doesn't behave the way I want"

The most common category of question, and the four answers are genuinely different.

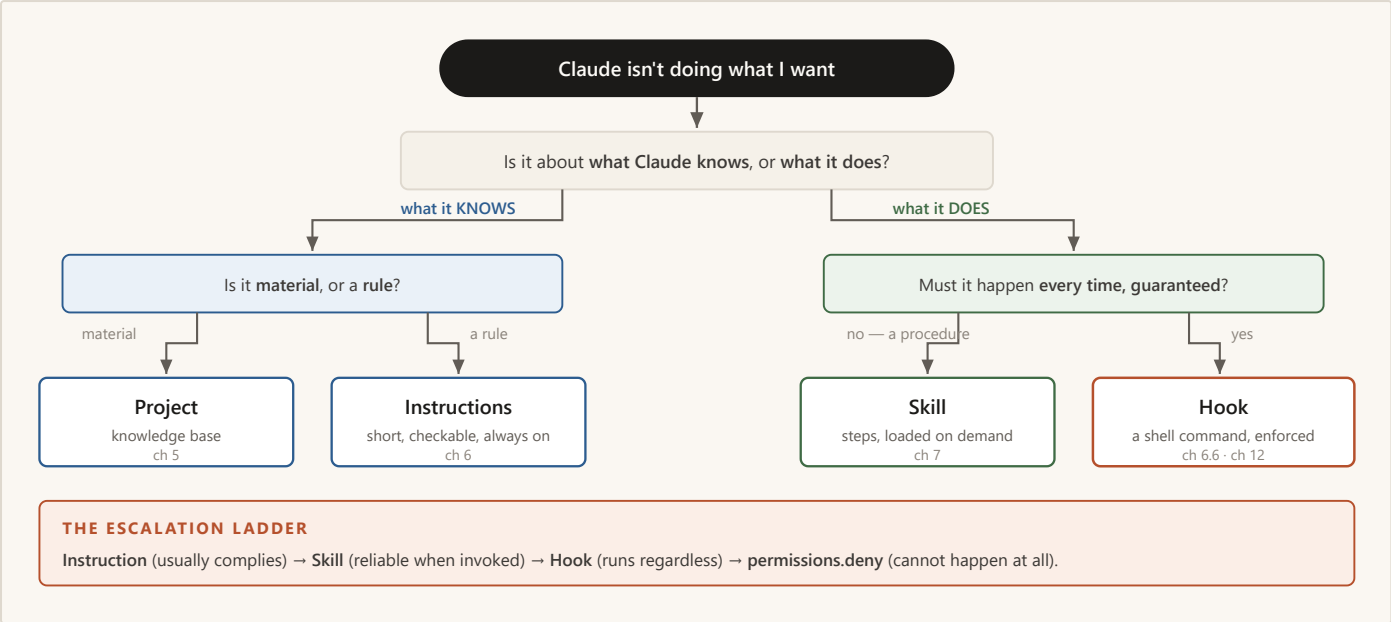


Figure 17.1 Two questions resolve almost every "how do I make Claude…" problem. The bottom bar is the escalation path when persuasion is not enough.
Diagram — not a screenshot. Prepared for this guide.

17.3 Decision tree — "I want Claude to reach one of our systems"

QUESTION	ANSWER	THEN
Is it in the connectors directory, or the official plugin marketplace?	**Yes**	Use the **Connector** (claude.ai) or the official **Plugin** (Claude Code). Reviewed, one click, nothing to maintain. ch 9, ch 11
	No	Continue ↓
Do you work mainly in chats/Cowork, or in a repository?	**Chats**	Add it as a **custom connector** with the server URL. ch 8.3 Path A
	Repo	`claude mcp add`. Continue ↓ for scope. ch 8.3 Path B
Who needs it?	**Just me, here**	`--scope local` (the default)
	The team	`--scope project` → commit `.mcp.json`
	Me, everywhere	`--scope user`
Will a **Routine** need it?	—	Then it must be an **account connector** or a committed `.mcp.json`. Locally-added servers do not travel. ch 13

17.4 Decision tree — "This is multi-step work"

Four options, and the deciding question is *who holds the plan*.

IF…	USE	BECAUSE
It is one procedure with known steps, and you are present	**Skill**	Claude follows the instructions in its own context. No orchestration needed. ch 7
A side task would flood your conversation with output you will not reuse	**Agent**	It works in its own context window and hands back a summary. ch 14
You have several independent tasks and want to check back later	**Agent view** `claude agents`	One screen to dispatch and monitor background sessions. ch 14.1

IF...	USE	BECAUSE
The same step must run across dozens or hundreds of items	**Workflow**	A script holds the loop and the intermediate results; Claude's context holds only the answer. ch 15
Findings need verifying against each other before you trust them	**Workflow**	Adversarial verification is a repeatable pattern once the plan is code. ch 15
It should happen on a cadence, or on an event, with nobody watching	**Routine**	A trigger starts a cloud session. Laptop closed. ch 13
It needs your approval partway through	**Several stages**, run separately	Workflows have no mid-run input. Routines have no approval prompts at all.

START SMALLER THAN YOU THINK

The ordering above is roughly cheapest-first. A great many tasks people reach for a workflow for are three subagents; a great many tasks people reach for subagents for are one good skill. Escalate when the smaller option visibly fails, not in anticipation.

17.5 Decision tree — "I want to share this with my team"

WHAT YOU ARE SHARING	TO WHOM	MECHANISM
A skill	One project's team	Commit to `.claude/skills/`. Reviewed in pull requests. *The best default.*
	Yourself, on every surface	Upload to claude.ai (six standard frontmatter fields only)
	Several teams	A **Plugin** in an internal **Marketplace**
Reference material	Anyone	A shared **Project** (Team/Enterprise), or commit it to the repo
Instructions / rules	One project	Committed `CLAUDE.md` or `.claude/rules/`
Instructions / rules	The whole organisation	Managed policy `CLAUDE.md`, deployed by IT
A tool connection	One project's team	`--scope project` and commit `.mcp.json`
A tool connection	The organisation	Owners add the **Connector** for the team; each member authenticates individually
An agent	One project	Commit to `.claude/agents/`
A workflow	One project	Save to `.claude/workflows/` and commit
Everything at once	The whole organisation	One **Plugin** carrying skills, agents, MCP config, hooks and workflows, from your internal **Marketplace**
A deliverable or report	Anyone, including outside the team	A published **Artifact** link — check the audience

17.6 The confusable pairs, in one place

Chapter 4 explained these properly. This is the lookup version.

PAIR	THE ONE-LINE DISTINCTION
Project vs **Instructions**	Project = *material* (the spec). Instructions = *rules* (always use British English).
Instructions vs **Skills**	Instructions load *always* and should be short. Skills load *on demand* and can be long. Facts vs procedures.

PAIR	THE ONE-LINE DISTINCTION
Project vs **Skills**	A project is one workspace's context. A skill applies everywhere, whenever the task matches.
Connector vs **MCP Server**	A connector is a ready-made, reviewed MCP integration. An MCP server is the general standard — for what is not on the list.
Plugin vs **MCP Server**	An MCP server adds *one* thing: reach. A plugin is a *bundle* that may contain one, plus skills, agents and hooks.
Marketplace vs **Plugin**	Marketplace = the catalogue. Plugin = an item in it. Add the marketplace, then install plugins.
Artifact vs **chat response**	A chat response is text you read. An artifact is a thing you open, use, version and share.
Artifact vs **an application**	An artifact is one self-contained page with no backend. It cannot store state between visits.
Agent vs **Workflow**	Agent: *Claude* holds the plan, turn by turn. Workflow: a *script* holds the plan.
Agent vs **Routine**	An agent runs inside a session you are in. A routine *is* a session, started by a trigger, in the cloud.
Workflow vs **Routine**	Workflow = *how* the work is orchestrated. Routine = *when* it starts. A routine can run a workflow.
Agent vs **Agent team**	Subagents report back to the conversation that spawned them. Agent-team members are peers with a shared task list who message each other.
Instruction vs **Hook**	An instruction asks. A hook *runs*, as a shell command, at a fixed lifecycle event, regardless of what Claude decides.
Auto memory vs `CLAUDE.md`	You write `CLAUDE.md`. Claude writes auto memory. The first travels with the repo; the second is machine-local.

17.7 "Where does this live?" — surface matrix

Before you promise a colleague a capability, check they are on a surface that has it.

BLOCK	CLAUDE.AI & DESKTOP	COWORK	CLAUDE CODE CLI / VS CODE	CLOUD SESSIONS & ROUTINES	NOTE
Projects	✓	✓	✗	✗	Claude Code's analogue is the repo + `CLAUDE.md`.
Instructions	✓	✓	✓	✓	Different mechanisms per surface; same purpose.
Skills	✓	✓	✓	◐	Cloud/Cowork read *account* skills or repo-committed ones — not `~/.claude/skills/`.
MCP Servers	◐	◐	✓	◐	On chat surfaces, as custom connectors. In routines, account connectors or committed `.mcp.json`.
Connectors	✓	✓	◐	✓	Claude Code uses MCP directly, or official integration plugins.
Artifacts	✓	✓	✓	✓	Claude Code publishes to a claude.ai page; Pro and above.
Marketplace	◐	✗	✓	◐	Desktop has a plugin browser. Cloud sessions install repo-declared plugins.
Plugins	◐	✗	✓	◐	As above.
Routines	✓	◐	✓	✓	Cowork has its own scheduled tasks. CLI creates schedule triggers only.
Agents	✗	◐	✓	✓	Cowork is agentic but does not expose user-defined subagents.
Workflows	◐	✗	✓	◐	Available in the Desktop app, IDE extensions, headless mode and the Agent SDK.

✓ available ◐ available with caveats (read the note) ✗ not available. Verify against the chapter's official links before relying on a row.

17.8 The five-minute triage

For when someone asks you in a corridor and you have no book to hand.

THEY SAY…	YOU SAY…
"I keep re-pasting the same spec."	Project.
"It won't use British English."	Instructions — account-wide.
"It won't use our test-case template."	A Skill. Build it from your existing test cases, not from memory.
"I want it to read our Azure DevOps."	Connector if it is listed; MCP server if not.
"I want a proper document, not chat text."	Artifact. Just ask for one.
"It filled my conversation with grep results."	A read-only Agent for the search.
"I need this checked across 300 files."	Workflow. Scope it on one directory first.
"Nobody remembers to do the Friday report."	Routine. Draft output, not direct writes.
"My team all do this differently."	Skill, then a Plugin from an internal Marketplace.
"It *sometimes* forgets to run the linter."	That is a Hook, not an instruction.
"I don't want it touching production."	Tool restrictions on the agent, and `permissions.deny`. Not a polite request.
"Which of these is right for me?"	Chapter 17, Section 17.1. Then start with the smallest one that fits.

CHAPTER 18

Practical Examples by Role

Complete scenarios, each chaining several blocks, with the prompts you would actually type.

Every chapter has its own examples. This chapter is different: each scenario below is an *end-to-end* piece of work that chains several blocks together, in the order you would actually build it. Use them as templates — substitute your own tracker, repository and formats.

> **A CONVENTION USED THROUGHOUT**
> Each scenario opens with a **Blocks used** line so you can see the composition at a glance, and closes with **What to check** — because the verification step is the part people skip, and it is the part that determines whether any of this is trustworthy.

18.1 QA

Scenario 1 — Turn a release's acceptance criteria into a reviewed test suite

Blocks used: Project (5) → Instructions (6) → Skill (7) → MCP/Connector (8–9) → Artifact (10)

Setup, once per release. Create a project as in Chapter 5, Section 5.3, with the requirements, the ACs, your test-case template and the previous release's defect summary. Add project instructions naming the source of truth. Connect your tracker.

Step 1 — derive the skill from your own back catalogue (do this once, ever)

```
Read the 120 test cases I've written for releases 4.0 and 4.1 from the
tracker. Infer my house style: how I word titles, how granular my steps
are, how I phrase expected results, how I handle preconditions, and how I
reference acceptance criteria.

Write a SKILL.md that reproduces that style on new work. Flag anywhere I'm
inconsistent so I can decide the rule rather than you guessing it.
```

Step 2 — generate coverage, per story

```
Read user story 4821 and every acceptance criterion linked to it.
Draft test cases with /test-case. Positive, negative and boundary for each AC.

Show me the table and wait. Do not create anything in the tracker yet.
List separately: any assumption you made, and any AC too vague to test
as written.
```

Step 3 — push, after review

```
Create these under the "Release 4.2" test plan as a new suite named
"4821 — Guest checkout", linked to story 4821. Report the IDs you created.
```

Step 4 — the coverage report your lead actually wants

```
Build an artifact showing coverage for release 4.2: one row per AC, with
the test cases covering it, and a clear list of ACs with no coverage at all.
Highlight the untestable-as-written ones separately — those are a
requirements problem, not a testing gap.
```

What to check: pick two generated cases and confirm the AC reference is real. Ask for coverage against an AC you did not supply — a good skill returns `UNVERIFIED`; a weak one invents a number.

Scenario 2 — A defect report that does not come back for more information

Blocks used: Skill (7) → Connector (9)

Prompt

```
Here's what happened: the payment step spins forever when I use the saved
Visa ending 4242 on the UAT environment, Chrome 141 on Windows 11, build
4.2.0-rc3. The console shows a 504 from /api/payments/authorise.

Write this up with /bug-report. Then check the tracker for an existing
open defect describing the same symptom before creating a new one —
if there is one, tell me its ID instead.
```

What to check: that it did not fill the Environment section with plausible guesses. A good bug-report skill asks rather than invents (Chapter 7, Example 2).

Scenario 3 — Nightly failure triage that runs itself

Blocks used: Routine (13) → Skill (7) → Connector (9) → Artifact (10)

The full prompt is in Chapter 13, Example 1. Three things make the difference between a routine that helps and one people mute:

- **Classify, don't just list.** "Genuine regression / flaky / environment" with justification is actionable; a list of red tests is not.
- **Check for an existing issue before opening one.** Without this, a nightly routine opens the same issue every night.
- **End with a self-report.** "State what you found, what you posted, and anything you could not determine" turns the transcript into something skimmable.

What to check: read the first three runs in full. Never trust the green tick — it means the session did not crash, not that the triage was right (Chapter 13, Section 13.4).

Scenario 4 — Regression scope decided from evidence

Blocks used: Agent (14) → Connector (9) → Project (5)

Prompt

```
@agent-sprint-auditor For release 4.2, work out what regression scope is
actually justified.

Use: the changed areas from the merged PRs this cycle, the defect history
in 05-r4.1-defect-summary.csv from project knowledge, and the current
test suite structure in the tracker.

Give me: areas that must be regressed and why; areas that can be skipped
and the risk of skipping them; and any area where the evidence is too thin
to make a call. Do not modify anything.
```

What to check: that the "must regress" reasons cite specific changes, not general caution. And that the agent is genuinely read-only — ask it to update a suite and confirm it declines.

18.2 Product & Business Analysis

Scenario 5 — Requirements-to-delivery reconciliation

Blocks used: Project (5) → Connector (9) → Artifact (10)

Prompt

```
The signed-off requirements are in project knowledge (01). The delivery
board is in the tracker under the Checkout area path.
```

```
Produce three tables:
  1. Requirements with no corresponding work item
  2. Work items with no corresponding requirement
  3. ACs whose wording differs between the two, showing both versions

Cite the document section and the item ID for every row. Where a match is
arguable rather than clear, put it in a fourth table called "Needs a human
decision" rather than deciding for me.

Build it as an artifact so I can take it into the review meeting.
```

What to check: the fourth table. If it is empty on a real project, the analysis was too confident — ask again with "be stricter about what counts as a clear match."

Scenario 6 — A retro grounded in data rather than memory

Blocks used: Skill (7) → Connector (9) → Artifact (10)

Prompt

```
Generate the sprint 24 retro with /retro-summary, using the board data
from the tracker.

Rules:
- Base every point on something in the data and cite the item IDs.
- Where you are inferring rather than observing, label it as inference.
- Include what went well — do not produce a list of only problems.
- Actions need a named owner and must be specific enough to be done.

Build it as an artifact so I can adjust the wording before circulating.
```

What to check: the inference labels. A retro that presents inference as observation is worse than no retro, because people argue about the conclusion rather than the evidence.

Scenario 7 — Weekly backlog hygiene, unattended

Blocks used: Routine (13) → Connector (9)

Full prompt in Chapter 13, Example 2. The design decision worth copying: the routine **applies labels but changes nothing else**. Labels are cheap to reverse and easy to filter on; reassigning owners or editing descriptions unattended is not.

What to check: the connector list on the routine. It defaults to *everything you have connected*, and there are no approval prompts during a run (Chapter 13, Section 13.3).

Scenario 8 — Competitive research you can defend

Blocks used: Workflow (15)

Prompt

```
use a workflow to research how our three named competitors handle rate
limiting: read their public documentation and recent changelog entries in
parallel, cross-check each claim against a second source, then compare
the approaches.

Flag anything you could not verify from two independent sources as
unverified rather than dropping it or asserting it.
```

What to check: open two citations. Cross-checked research is only as good as the sources, and the unverified list is the honest part of the output — if it is empty, be suspicious.

18.3 Design

Scenario 9 — Component handoff that engineering does not send back

Blocks used: Skill (7) → Connector (9, Figma) → Artifact (10)

Prompt

```
Read the Pagination component from our design-system Figma file.

Write the handoff spec with /handoff-spec. Cover variants, states, spacing
and accessibility.

Two rules:
- Cite design tokens by name. Where the design uses a value the token set
  does not define, write NOT IN DESIGN SYSTEM:  rather than
  inventing a token name.
- Every state must be listed, including the ones that do not apply — say
  "not applicable" rather than omitting them.

Then build an artifact with a styled HTML preview of every variant and
state side by side, so I can sanity-check the spec against something
visual.
```

What to check: the `NOT IN DESIGN SYSTEM` lines. Those are the real output — they are the decisions that would otherwise be made silently by whoever implements it.

Scenario 10 — Design-system drift audit

Blocks used: Connector (9) → Agent (14) → Workflow (15) for scale

Small version — one agent

```
@agent-a11y-auditor audit every component under src/components/forms/
```

Large version — a workflow across the whole library

```
use a workflow to compare every component in our design-system Figma file
against its implementation in src/components/.

For each: states in Figma but missing in code; states in code that Figma
does not define; and any hardcoded value in code where a token exists.

Verify each finding by re-reading the specific file before reporting it,
then merge everything into one ranked summary.
```

What to check: scope it on one directory first and look at the token total in `/workflows` before running it across the library (Chapter 15, Example 8).

Scenario 11 — Break out of a default aesthetic

Blocks used: Artifact (10) → Instructions (6)

When you ask for a UI mockup without direction, you tend to get a consistent house style that may be wrong for your product. Two techniques work; the second is better:

Technique 1 — specify concretely

```
Build the dashboard mockup as an artifact using exactly: background
#0F1115, surface #171A20, accent #4F9CF9, text #E6E9EF, font stack
"IBM Plex Sans", 8px spacing grid, 4px radii. Dense data table, no cards.
```

Technique 2 — propose, then pick

```
Before building anything, propose 4 distinct visual directions for this
brief — each as: background hex / accent hex / typeface, plus one line of
rationale. Wait for me to pick one, then implement only that direction
as an artifact.
```

What to check: that the four directions are genuinely distinct rather than four shades of the same idea. If they are not, name the constraint you want varied.

18.4 Development

Scenario 12 — From tracker item to reviewed pull request

Blocks used: Instructions (6, `CLAUDE.md`) → Connector (9) → Skill (7) → Agent (14)

Prompt

```
Read issue ENG-4521. Implement it in this repo following CLAUDE.md.
Run the tests.

If the acceptance criteria are ambiguous about the error-handling
behaviour, stop and ask me rather than choosing an interpretation.

When it's working, open a PR with /pr-description and link the issue.
```

Then, before you ask a human

```
@agent-code-reviewer review every file changed against origin/main.

Report every issue including low-confidence ones, with a confidence level
and estimated severity. Do not filter for importance — I will triage.
```

What to check: that the reviewer is genuinely read-only (Chapter 14, Example 3). And note the "report everything" instruction — told to be conservative, Claude filters faithfully and you see fewer findings than it actually noticed.

Scenario 13 — Onboarding onto an unfamiliar service

Blocks used: Instructions (6, `/init`) → Agent (14)

Step 1 — generate the shared context, then edit it down

```
> /init
```

Then delete the directory listings it produced and keep only what a new joiner would have to be *told*: the pitfalls, the reasons, the conventions that differ from the tool defaults (Chapter 6, Section 6.3).

Step 2 — orient without flooding your session

```
@agent-explorer explain the three main request flows through this service,
where the known fragility is, and which files I should read first.

Give me file and line references. Do not paste file contents.
```

Step 3 — the questions worth asking a human

```
Based on what you've read, give me five questions I should ask the current
maintainer — the ones where the code cannot tell me the answer.
```

What to check: the five questions. If they are answerable from the code, the exploration was shallow.

Scenario 14 — A migration too large for one conversation

Blocks used: Workflow (15) → Agent (14) → worktrees

Prompt

```
use a workflow to migrate every component under src/components/ from
styled-components to Tailwind, working on each file in its own isolated
copy, and verify each result compiles and its tests still pass.

Report at the end: files migrated, files that failed verification and why,
and anything you skipped.
```

Two details that matter. "Its own isolated copy" puts each agent in a separate worktree so parallel edits cannot conflict. "Anything you skipped" surfaces bounded coverage — silent truncation reads as completeness (Chapter 15, Section 15.4).

What to check: set `/config workflowSizeGuideline=medium` and run it on one subdirectory first. Then check the token total before widening.

Scenario 15 — Alert to draft fix, while you sleep

Blocks used: Routine (13, API trigger) → Connector (9) → Agent (14)

Full prompt and the curl wiring are in Chapter 13, Example 4. The design constraints that make this safe:

- **Draft PR only, never main.** Stated explicitly in the prompt, not left to inference.
- **The prompt must opt in to the payload.** The `text` field arrives wrapped as untrusted data; unless your prompt references the `routine-fire-payload` block, it is inert context.
- **A fallback path.** "If you cannot identify a safe fix, open an issue with your analysis instead" — otherwise you get a bad PR instead of a good issue.

18.5 Cross-role: reporting and analysis

Scenario 16 — The self-maintaining status board

Blocks used: Artifact (10) → Connector (9)

Prompt

```
Build a status-board artifact for the current sprint: items by state,
blocked items with their blocker and how long they have been blocked, and
anything untouched for 5 or more days.

Have it pull fresh data through the tracker connector each time someone
opens it. Share it with everyone in the organisation — not publicly.
```

What to check: open the share link in a private window and confirm the audience is what you intended. Then spot-check three numbers against the tracker — a well-drawn chart of mis-parsed data is more dangerous than no chart (Chapter 10, Section 10.4).

Scenario 17 — The activity analysis from the live demo

Blocks used: Connector (9) → Artifact (10)

Prompt

```
Pull every work item I created in the last six months from the tracker.

Build an analysis artifact: volume by month, split by item type, and how
my mix has shifted over the period. Two sentences of commentary under
each view.
```

```
Where a trend is more likely a data-quality artefact than a real change,
say so instead of interpreting it.
```

Scenario 18 — Weekly status, in the format your manager reads

Blocks used: Skill (7) → Project (5) → Artifact (10)

Prompt

```
Here's this week's board export. Produce the weekly status with
/weekly-status: what was done, what's blocked, what's next.

Build it as an artifact so I can adjust the wording before sending.
Keep it to one screen — if it does not fit, cut detail, not sections.
```

"Cut detail, not sections" is worth stealing. It prevents the useful-but-inconvenient section from being the one that disappears.

18.6 The pattern behind all eighteen

Look back at the scenarios and the same five instructions recur. They are the transferable part:

INSTRUCTION	WHY IT EARNS ITS PLACE
"Show me before you write anything."	Splits read from write. One extra turn; far fewer awkward cleanups in a real system.
"Never invent an ID / a token / a date."	Converts a plausible fabrication into an explicit gap you can act on.
"Label inference as inference."	Lets the reader argue with the evidence rather than the conclusion.
"Report everything; I'll triage."	Told to be conservative, Claude filters faithfully — and you never see what it filtered.
"Tell me what you skipped or could not determine."	Surfaces bounded coverage. Silence reads as completeness.

IF YOU TAKE ONE THING FROM THIS CHAPTER

Put those five into your **Instructions** (Chapter 6) and you get them on every message, in every scenario above, without typing them again. That is the whole thesis of this guide applied to the guide itself.

CHAPTER 19

Troubleshooting

Start with the five checks. Then find your symptom in the index.

19.1 The five checks, in order

A large majority of "Claude isn't working" reports are one of five things, and they take about two minutes to rule out. Do these before reading any further.

#	CHECK	HOW
1	**Am I where I think I am?**	Is this chat actually inside the project? Look for the project name in the breadcrumb above the chat title. Am I in the right repository? Claude Code loads configuration relative to where you launched it.
2	**Is the connector switched on *for this conversation*?**	Connecting at account level is not enough. In the chat: **+** › hover **Connectors** › toggle. This is the single most common connector problem.
3	**Did the configuration actually load?**	Claude Code: `/context` and read the **Memory files** and **Custom Agents** sections. If a file is not listed there, Claude cannot see it. Do not assume — look.
4	**Is the capability enabled at all?**	Everything lives in **Settings** › **Capabilities** (or **Organization settings** on Team/Enterprise), but they are *separate* switches now: **Skills** needs **Code execution and file creation**; **Artifacts** has its own switches under the **Visuals** section. Turning one on does not turn the other on. Workflows need enabling on Pro. Routines can be disabled org-wide.
5	**Is the tool connected, or just configured?**	`claude mcp list` or `/mcp`. `claude mcp add` writes configuration *without validating credentials*, so a server can look configured and be unreachable.

> **THE SIXTH CHECK, FOR CLAUDE CODE SPECIFICALLY**
>
> `/doctor` runs a setup checkup and will tell you about several classes of problem at once, including proposing trims for an over-long `CLAUDE.md`. If you are stuck and out of ideas, run it.

19.2 Symptom index

Each row points at the chapter section with the full explanation and fix.

Context and instructions

SYMPTOM	MOST LIKELY CAUSE	GO TO
Claude ignores the project's knowledge base	The chat is outside the project	5.5
Two chats in one project give contradictory answers	Working as designed — chats never share conclusions	5.1, 5.5
Claude cites a fact that is no longer true	A superseded document is still in project knowledge	5.5, 5.7
Claude ignores an instruction	Instructions are context, not enforcement — and vague ones fare worst	6.5, 6.6
Instructions followed *less* as the file grew	Over ~200 lines	6.5, 6.7
A rule applies too aggressively	Emphatic phrasing (`CRITICAL`, `MUST`)	6.4

SYMPTOM	MOST LIKELY CAUSE	GO TO
An instruction disappeared mid-session	Given only in conversation, or in a nested/path-scoped file not yet reloaded	6.5
Other teams' rules leak in (monorepo)	Ancestor `CLAUDE.md` files load automatically	6.5
Claude Code never sees my `CLAUDE.md`	Wrong location, or the directory did not exist at session start	6.5
Claude "remembered" something wrong	Auto memory inferred a preference from one interaction	6.5

Skills

SYMPTOM	MOST LIKELY CAUSE	GO TO
Claude never invokes the skill on its own	The `description` says *what* but not *when*	7.5, 7.7
Skill does not appear in the `/` menu	Wrong directory, missing `SKILL.md`, `user-invocable: false`, or directory created after session start	7.5
Upload to claude.ai fails with "Unexpected key(s)"	A non-standard frontmatter field	7.3
Works in the terminal, "not found" in a Routine	Personal skills are not synced to cloud sessions	**7.5**
Skill fires too often	Description too broad, or emphatic language	7.5
Wrong variant runs in a monorepo	Nested skills with the same name; use the qualified name	7.3, 7.5
Only the format is applied, the rest ignored	Long unstructured prose in the body	7.5
Plugin skills missing after install	Not activated yet — `/reload-plugins`	7.5, 12.3

Tools: MCP and connectors

SYMPTOM	MOST LIKELY CAUSE	GO TO
"I can't access that service"	Connector not toggled on for this conversation	**9.5**
`! Needs authentication`	OAuth not completed — run `/mcp`	8.5
`✗ Failed to connect`	Bad URL, bad credentials, or network policy; `claude mcp get` shows the detail	8.5
`⏸ Pending approval`	A project `.mcp.json` server you have not approved interactively	8.5
A token that works elsewhere is rejected	**MCP tokens are not REST API keys** — hosted servers usually want OAuth	**8.5**
Authenticated but returns nothing	The account you authenticated with cannot see that content	9.5
Worked yesterday, fails today	Token expired — disconnect and reconnect	9.5

SYMPTOM	MOST LIKELY CAUSE	GO TO
Can read but not write	Enterprise action-permission restriction, or narrow OAuth scope	9.5
Server missing from `claude mcp list`	Reserved name, or a WebSocket server	8.5
Whitespace warning on a config value	A pasted token carried a trailing newline; Claude Code does not trim it	8.5
Free plan will not add a second custom connector	Free is limited to one	9.5
Routine cannot reach the tool	It is a locally-added MCP server, not an account connector	**9.5, 13.5**
Context feels tight with many servers connected	Tool definitions load per request; use tool search or disconnect	8.4, 8.5

Artifacts

SYMPTOM	MOST LIKELY CAUSE	GO TO
No artifact appears — output stays in chat	Capability off, or below the threshold. Just ask for one.	10.3, 10.5
Claude Code writes a local HTML file with no link	The artifact tool is not enabled for that session	10.3, 10.5
Interactive artifact renders but does nothing	A logic error — describe the broken behaviour specifically	10.5
Chart numbers look wrong	Source data mis-parsed. Debug the data, not the chart.	10.5
Shared link shows an old version	You shared a pinned version	10.5
Cannot re-publish after unpublishing	By design — unpublish is one-way	**10.5**
Artifact will not save	20 MB per-artifact limit	10.5
Lost an artifact from an old chat	Only published artifacts persist outside the conversation	10.5

Plugins and marketplaces

SYMPTOM	MOST LIKELY CAUSE	GO TO
Plugin installed but nothing works	Not activated in this session — `/reload-plugins`	12.3, 12.5
Components missing after install	Directories nested inside `.claude-plugin/`	**12.1, 12.5**
`Marketplace "..." not found`	The automatic add failed; add it manually	11.5
Plugin not found in the marketplace	Stale catalogue — `/plugin marketplace update`	11.5
Adding a GitLab/Bitbucket URL fails	Missing `https://` or missing `.git`	11.3
`/plugin` not recognised	Old Claude Code version	11.5
Plugin skills never appear	Stale cache — clear `~/.claude/plugins/cache`	11.5, 12.5

SYMPTOM	MOST LIKELY CAUSE	GO TO
Removing a marketplace removed my plugins	Working as designed	**11.5**
Language server not starting	The binary is not installed or not on `$PATH`	11.5, 12.5
Enable/Disable appears to do nothing	Marketplace-entry name differs from `plugin.json` name, on an older version	12.5
Plugin declared in project settings but not installed	External-source plugins need an explicit install	11.3, 12.5
Plugin agent ignored after migrating	A same-named local definition overrides it	**12.5, 14.5**

Autonomy: routines, agents, workflows

SYMPTOM	MOST LIKELY CAUSE	GO TO
Green run status, but nothing happened	Status reflects infrastructure, not your task	**13.4, 13.5**
`/schedule` returns `Unknown command`	Usually a Console API key or cloud-provider login instead of a claude.ai subscription login	**13.5**
`/schedule` asks you to authenticate	No stored claude.ai login — and no routine was created	13.5
"Routines are disabled by your organization's policy"	An Owner turned the toggle off	13.5
Routine ignores the `text` payload	The prompt does not reference the `routine-fire-payload` block	**13.4**
GitHub trigger never fires	Claude GitHub App not installed, or a regex filter without `.*`	13.4, 13.5
`403` with `x-deny-reason: host_not_allowed`	Cloud environment network policy	13.5
Push rejected during a routine	Protected branch, someone else's open PR, or another author's commits	13.5
Claude cannot find a new subagent	The `agents` directory did not exist at session start — restart	14.5
Subagent modified files unexpectedly	`tools` omitted, so it inherited every tool	**14.5, 14.7**
Subagent fails to launch, error names tools	No entry in `tools` resolves to a real tool	14.5
Subagent returns everything, not a summary	Its prompt does not ask for brevity	14.5
Parallel subagents conflicted on files	No worktree isolation	14.5
The `ultracode` keyword does nothing	Workflows disabled, or on Pro without enabling them	15.5
Keyword ignored from a script or webhook	By design — it only opts in from a prompt you type	15.3
Workflow keeps stopping for permission prompts	Commands not in your tool allowlist	15.4

SYMPTOM	MOST LIKELY CAUSE	GO TO
Resume re-ran agents that had already finished	Replay follows start order	**15.5**
`Large workflow` warning	Over 25 agents or 1.5M projected tokens — advisory only	15.5, 15.7
Token usage far higher than expected	Several agents or a workflow — usage multiplies	14.5, 15.7

19.3 The traps worth memorising

Six behaviours that are working exactly as designed, cost people the most time, and are not guessable.

THE TRAP	WHAT IS ACTUALLY HAPPENING
Chats in a project do not share memory	They share *context*, never *conclusions*. If Chat 1 decides something, Chat 2 knows nothing about it. The only fix is writing the decision into project knowledge. ch 5
Personal skills and local MCP servers do not reach cloud sessions	Cowork, cloud sessions and every Routine run start as fresh remote sessions. They read your *claude.ai account* skills and connectors, plus anything committed to the repository. `~/.claude/skills/` and `claude mcp add` servers are invisible to them. ch 7.5, 8.5, 13.5
A green routine run means "no crash"	Not "your task succeeded." Blocked network requests, missing tools and task-level failures all surface in the transcript, not the status indicator. Open the run. ch 13.4
Omitting `tools` on a subagent grants everything	It is not "default safe" — it inherits every tool available to subagents. An explicit `tools: Read, Grep, Glob` is what makes a reviewer read-only. ch 14.3
Workflow subagents always run in `acceptEdits`	Regardless of your session's permission mode. File edits are auto-approved. Approving a workflow is approving a lot of file writes. ch 15.4
Removing a marketplace uninstalls its plugins	The catalogue is not just a directory — it is where your installed plugins are resolved from. ch 11.5

19.4 When the output is wrong rather than broken

A different class of problem: everything is configured correctly and the answer is still not what you wanted.

SYMPTOM	DIAGNOSIS AND FIX
It invented an identifier, a date or a name	Add the prohibition explicitly, in Instructions so it applies everywhere: "Never state a ticket ID, date, name or figure that did not come from material I supplied or a connected tool. Say 'not in the provided material'." ch 6, 18.6
It found fewer issues than you expected	Almost always a filtering instruction being followed faithfully. If your prompt or skill says "only report high-severity" or "be conservative", Claude investigates just as thoroughly and then declines to report what it judges below the bar. Ask for everything with a confidence level and triage yourself. ch 14.6, 18.6
It presented a guess as a fact	Require the distinction: "Separate what the data shows from what you infer, and label the second as inference." ch 6.6, 18.6
It went beyond what you asked	Add a scope instruction: deliver what was asked at the scope intended; raise a concern in a sentence and continue rather than silently widening the work.
It is too verbose	Ask for the shape you want, positively: "Answer in at most three sentences unless I ask for detail." Positive examples of the desired concision work better than a list of prohibitions.
It stopped short and claimed completion	Require the report: "If you could not complete something, do the rest and state plainly what is missing and why." Then check for it.

SYMPTOM	DIAGNOSIS AND FIX
A skill's output is right in shape, wrong in judgement	The skill encodes the format but not the edge cases. Add the judgement rules — "if you cannot find the AC, write UNVERIFIED" — which is where a skill's real value lives. ch 7.7
It was better last month	Consider whether your prompts and skills carry scaffolding written for an older model. Enumerated steps and emphatic warnings that were once necessary can now reduce quality. A/B with the scaffolding removed. ch 6.7, 7.7

19.5 Diagnostic commands worth knowing

COMMAND	TELLS YOU
`/context`	What actually loaded this session: memory files, custom agents, skills, and the context breakdown. **The first thing to run.**
`/doctor`	A setup checkup, including proposed trims for an over-long committed `CLAUDE.md`.
`/memory`	Every memory file location across user and project scope, including ones that do not exist yet; toggles auto memory.
`/mcp`	Per-server status, enable/disable, reconnect, and OAuth authentication.
`claude mcp list` / `get`	Health status per server, with the failure detail on an `Issue:` line.
`/plugin` › **Errors**	Plugin load errors, including missing language-server binaries.
`claude plugin details`	Exactly what a plugin contributed: commands, skills, agents, hooks, MCP and LSP servers.
`/tasks`	Everything running in the background of this session, including finished subagents.
`/workflows`	Workflow runs, their phases, per-agent token usage, and the controls to pause, stop or save.
`/usage`	Plan, session and weekly usage, plus attribution by skill, subagent, plugin and MCP server.
`claude --debug`	Why a component was skipped — e.g. an invalid LSP configuration.
`claude --version`	The answer to a surprising number of "that command doesn't exist" reports.

19.6 Escalating

1. **Reproduce it minimally.** A fresh chat or session, with the smallest configuration that still shows the problem. Half the time this identifies the cause.
2. **Check the official documentation** for that block — Section 8 of its chapter, or Chapter 22. Several features in this guide are labelled research preview or beta, and behaviour genuinely changes.
3. **Check your version and update.** A meaningful proportion of documented behaviour is version-gated, and this guide notes version numbers where Anthropic does.
4. **Ask internally with specifics:** the surface, the plan, the exact command or prompt, the exact message, and what `/context` showed. "It doesn't work" cannot be diagnosed.
5. **Report a genuine bug** at `github.com/anthropics/claude-code/issues` with those details.

> **BEFORE ESCALATING A PLAN OR AVAILABILITY QUESTION**
>
> Plan gating, beta status and daily limits change. If your question is "should this work on our plan?", the answer is in the official article linked in that chapter's Section 8, not in this guide — which is a snapshot dated on the cover.

CHAPTER 20

Best Practices

The cross-cutting habits — adoption order, context economy, review culture, and the ten rules that carry across every block.

Every chapter has its own Section 7. This chapter is the part that does not belong to any single block: how to adopt these capabilities in a sensible order, how to spend context, how to name and version things consistently, and what a team needs to agree on.

20.1 The ten rules

If this chapter gets reduced to a poster, this is the poster.

#	RULE	WHY
1	**Facts in Instructions, procedures in Skills.**	Instructions load on every message, so they must be short. Skills load on demand, so they can be long. The most common structural mistake in this guide is a twelve-step process living in an instructions file.
2	**Do it twice before you automate it.**	The second run tells you which parts are the actual rule and which were incidental to the first attempt.
3	**Derive the standard from your real output, not from memory.**	Your existing test cases and PR descriptions encode decisions you have never articulated. Have Claude extract the pattern — the "where you're inconsistent" list alone is worth the exercise.
4	**Split read from write.**	"Show me before you create anything" costs one turn and prevents the class of mistake that is tedious to undo in a live system.
5	**Encode the edge cases, not just the format.**	"Never invent an AC reference — write UNVERIFIED" is worth more than the column list. Formats are easy; judgement is what you are capturing.
6	**Restrict tools deliberately.**	An explicit read-only tool list makes a reviewer read-only by construction. That is a stronger guarantee than any instruction, and omitting the list grants everything.
7	**Verify the artefact, not the status.**	A green tick means no crash. Open the PR, check the numbers, read two findings. Then trust it.
8	**Ask what was skipped.**	Silence reads as completeness. "Tell me what you could not determine" turns a confident answer into an honest one.
9	**Fix the configuration, not the output.**	Correcting a response helps once. Correcting the skill or the instruction helps every time, for everyone who has it.
10	**Start smaller than you think.**	Many tasks people reach for a workflow for are three subagents; many they reach for subagents for are one good skill. Escalate when the smaller option visibly fails.

20.2 The adoption sequence

An order that produces value quickly and does not create maintenance you cannot sustain. Each stage should be *working* before you start the next.

STAGE	DO THIS	YOU KNOW YOU ARE READY TO MOVE ON WHEN…
Day one no setup	Account-wide **Instructions**. A **Project** for your current body of work. Ask for **Artifacts**.	Your prompts have got noticeably shorter, and you have stopped re-pasting the same background.

STAGE	DO THIS	YOU KNOW YOU ARE READY TO MOVE ON WHEN…
Week one	One **Skill** — whichever format you produce most often. Derive it from your existing output.	You have invoked it without thinking about it, and the output needed no reformatting.
Week two	A **Connector** for your tracker. Combine it with the skill from week one.	You have stopped exporting and pasting board data.
Month one	Two or three more skills. A committed `CLAUDE.md` if you work in a repository. Share the skills with your immediate team by committing them.	A colleague has used one of your skills and it worked without explanation.
Month two	A read-only **Agent** for the audit or search you do most. One **Routine** for the recurring report nobody enjoys.	The routine has run unattended three times and you have read all three transcripts.
Quarter one	A **Plugin** from an internal **Marketplace**, bundling what has proven itself. A **Workflow** for the one job that is genuinely too big for a conversation.	Someone outside your team installed the plugin and did not need a walkthrough.

THE FAILURE MODE THIS SEQUENCE AVOIDS

The common rollout mistake is starting at the bottom: building a plugin and an internal marketplace before anyone has a skill worth distributing. You end up maintaining infrastructure for content that does not exist, and the people you built it for never adopt it because there is nothing in it they need.

Build the content first. Package it once it has earned distribution.

20.3 Context economy

A single mental model explains most of the "keep it short" advice scattered through this guide. Everything that loads costs something, and the cost is paid at different frequencies.

WHAT	PAID	CONSEQUENCE FOR HOW YOU WRITE IT
Instructions, `CLAUDE.md`, unscoped rules	**Every message**	Under ~200 lines. Specific and checkable. Anything longer gets followed *less*, not more.
Project knowledge	Every chat in the project	Curate to the documents people actually cite. Remove superseded versions rather than accumulating them.
Skill descriptions	Every session	Cheap — which is why you can have many skills. Put the key use case first; the listing truncates.
Skill bodies and reference files	**Only when used**	Effectively free until needed. This is where long reference material belongs.

WHAT	PAID	CONSEQUENCE FOR HOW YOU WRITE IT
Connector / MCP tool definitions	Every request, once connected	Disconnect what you are not using. Defer large tool sets behind tool search.
Plugin components	While enabled	Check the **Context cost** estimate before installing; review the **Not used recently** group periodically.
Path-scoped rules and skills	Only when touching matching files	The best of both: full detail, zero cost when irrelevant. Underused.

THREE MOVES THAT RECLAIM THE MOST CONTEXT

1. Move any procedure out of Instructions into a Skill.
2. Add `paths:` to rules and skills that only matter for part of the codebase.
3. Disconnect connectors and disable plugins you have stopped using. Run `/usage` — it attributes usage by skill, subagent, plugin and MCP server, which usually names the culprit immediately.

20.4 Naming, consistently, across every block

Individually trivial; collectively the difference between a shared setup and twelve local ones. Pick these conventions once, organisation-wide.

THING	CONVENTION	EXAMPLES
Project	`[Team] Subject — Phase`	`[QA] Release 4.2 — Test Coverage`, `[BA] Checkout Revamp — Discovery`
Skill	verb-noun, lowercase, hyphenated. Name the *output*.	`test-case`, `bug-report`, `handoff-spec`, `retro-summary`
Agent	role noun. Name the *job*, not the model.	`code-reviewer`, `sprint-auditor`, `a11y-auditor`, `explorer`
MCP server	the system, short and lowercase	`ado`, `github`, `figma`, `sentry`
Plugin	audience or domain, short (it is the skill namespace)	`qa-toolkit`, `ba-toolkit`, `design-tools`
Marketplace	organisation-identifying, one per organisation	`acme-internal`
Routine	cadence, then outcome	`Nightly — CI failure triage`, `On PR — house checklist review`
Workflow	the job	`audit-routes`, `triage-issues`, `migrate-styles`
Artifact	include the period for anything recurring	`Sprint 24 Retro`, not `Retro`

Two rules behind all of them: the name is what someone will *type* or *search for*, so make it what they would guess; and never put a version number in a name — that is what the `version` field and Git tags are for.

20.5 Versioning and change management

The thing to internalise: **a skill edit is a behavioural change for everyone who has it.** Treat these artefacts with the same care as code, because they change what code gets written.

ARTEFACT	VERSIONED BY	REVIEW EXPECTATION
Committed `CLAUDE.md`, `.claude/rules/`	Git	Review in pull requests. State what and why in the commit message — it is a config change.
Project skills, agents, workflows	Git	Review in pull requests. A one-line changelog at the top of the file.
Plugins	`version` in `plugin.json` + Git tags	Bump the version, keep a `CHANGELOG.md`, pilot before broad rollout.
Uploaded claude.ai skills	Manually	Keep the canonical copy in a repository and treat the upload as a deployment.
Project knowledge	Inside the documents	One file per topic with a changelog line. Never `template-v2.md` alongside `template.md`.
Routines	Not versioned	Keep the prompt in the repository the routine operates on. It is the specification.

Re-audit after a model release

> **THE MAINTENANCE TASK NOBODY SCHEDULES**
>
> Prompts, skills and tool descriptions accumulate scaffolding written for older models: enumerated step-by-step scripts, emphatic `CRITICAL:` warnings, instructions to "think step by step", prohibitions against failures that no longer occur.
>
> On current models this scaffolding is not neutral — over-prescriptive instructions *reduce* output quality, and emphatic language causes over-application rather than compliance. After a model release, A/B your most-used skills with the older scaffolding removed. Keep what measurably helps; delete the rest.
>
> The counterpart is also true: re-baselining sometimes means *adding* guidance for a new model's habits. The task is fit, in both directions.

20.6 Prompting habits that pay off everywhere

Not a prompt-engineering course — just the handful of patterns that recur across all eighteen scenarios in Chapter 18.

HABIT	WHAT IT LOOKS LIKE
Say what "done" is	Especially for anything unattended. "Success means: labels applied and one summary posted."
Name the authority	"The source of truth for acceptance criteria is document 02." Resolves conflicts before they happen.
Give the reason for hard rules	"Money is minor units — float rounding caused two incidents" is followed more reliably than the bare rule.
Ask for the shape, not just the content	Order, grouping, what to leave out. Especially for artifacts, which are designed objects.
Require the distinction	"Separate what the data shows from what you infer, and label the second."
Ask for coverage, then filter yourself	"Report everything with a confidence level" beats "only report what matters" — the second is followed faithfully and you never see what was dropped.
State the prohibition, positively where possible	"Say 'not in the provided material'" is more actionable than "don't hallucinate."
Normal volume	Drop `CRITICAL:`, `YOU MUST`, "if in doubt, always". They cause over-application, and when everything is critical nothing is.
One clarifying question, not zero	"If my request is ambiguous in a way that changes the answer, ask one question before starting." Cheaper than a wrong deliverable.

20.7 Team practices

What a team needs to agree on

DECISION	A REASONABLE DEFAULT
Who owns each shared artefact	A named person per project, skill, agent, plugin and marketplace, written in the description or a header comment. Unowned shared things rot.
Where shared skills live	One repository, committed, reviewed in pull requests. One internal marketplace once there is enough to catalogue.
Who may add custom connectors	Answer this *before* someone connects a production database. Publish the approved list.
The artifact-sharing default	Organisation-scoped, not public, for internal work.
Which authentication account	The least-privileged account that can do the job. Claude inherits exactly those permissions.
Review expectations	Skills, agents, rules, hooks and workflows are reviewed like code, because they change what code gets written.
The workflow size guideline	`medium` (under 15 agents), so nobody's first workflow is a 200-agent run.

Publish the catalogue

The highest-leverage, lowest-effort thing an enablement owner can do. People cannot use a skill they do not know exists, and nobody browses a plugin marketplace speculatively. A single internal page — one line per skill, agent and plugin, saying what it gives you and who owns it — drives more adoption than any amount of tooling.

Make improvement the default response

When output is wrong, there are two possible responses: correct the output, or correct the configuration. The first is faster once. The second compounds — and in a team, it compounds across everyone.

Build the habit explicitly: "*if you had to correct Claude twice on the same thing, the skill or the instruction needs an edit.*" That single norm is what turns a collection of individual usage into a shared standard that gets better over time.

20.8 Measuring whether any of this is working

Avoid vanity metrics. The useful signals are behavioural:

SIGNAL	WHAT IT TELLS YOU
Prompts are getting shorter	Context and standards are working. This is the clearest early indicator.
Output needs less reformatting	Your skills encode the format properly.
People invoke each other's skills	Distribution is working, and the naming is guessable.
The same correction stops recurring	Someone fixed the configuration rather than the output.
Routine transcripts get skimmed, not read	Trust has been earned — assuming the first few *were* read.
`/usage` attribution looks reasonable	Nothing is quietly burning budget. If one plugin or MCP server dominates, investigate.
The **Not used recently** list is short	Your installed set reflects what you actually use.

> **THE ONE THAT MATTERS MOST**

> Ask people what they have *stopped doing by hand*. That is the whole point of the shift from doing to directing — and it is a much better question than asking how often they use Claude.

CHAPTER 21

Security & Organizational Considerations

What each capability actually grants, which controls exist, and the four questions to answer before a organisation-wide rollout.

WHAT THIS CHAPTER IS AND IS NOT

This chapter explains the security *model* — what a given capability grants, and which controls exist to constrain it. It is not your organisation's policy, and it is not a compliance assessment. Those are separate documents owned by different people. Read this so that when you write those documents you are constraining the right things.

21.1 The trust ladder

The eleven blocks are not equally consequential. Ranked by what they can actually do if misconfigured or misused, they form a clear ladder — and your attention should be distributed accordingly.

LEVEL	BLOCK	WHAT IT GRANTS, AND THE CONTROL THAT MATTERS
LOW	**Projects**, **Instructions**, **Artifacts**	Read and shape context; produce output. The risk is *disclosure*, not action: project knowledge is readable by everyone it is shared with, and publishing an artifact creates a link. **Control:** visibility settings and what you choose to upload.
LOW	**Skills**	Instructions Claude will follow, loaded on demand. Low direct risk — but a skill in a repository is instructions anyone with commit access can change. **Control:** code review, and `allowed-tools` scoping.
MEDIUM	**Connectors**, **MCP Servers**	Live read *and write* access to real systems, with your permissions. **Controls:** which account you authenticate as; per-conversation toggles; Enterprise domain restrictions and read-only/write/delete action limits.
MEDIUM	**Agents**	An autonomous worker — but the one block with a genuinely strong containment mechanism. **Control:** the `tools` list. Explicitly read-only means read-only by construction. Omitting it grants everything.
HIGH	**Marketplace**, **Plugins**	Arbitrary code execution with your user privileges. Hooks run shell commands; `bin/` adds executables to the Bash tool's `PATH`. **Controls:** only trusted sources; read the **Will install** list; managed marketplace restrictions.
HIGH	**Workflows**	Dozens of agents, and **their subagents always run in `acceptEdits` mode regardless of your session's permission mode** — file edits are auto-approved. **Controls:** read the script before approving; size guidelines; your tool allowlist.
HIGHEST	**Routines**	An autonomous cloud session, with your identity and your connectors, and **no human to approve anything**. **Controls:** the connector list, the repository list, the environment's network policy, draft-only output, and the organisation-wide off switch.

WHERE TO SPEND YOUR REVIEW EFFORT

Reviewing a project's knowledge base carefully while approving plugins and routines casually is exactly the wrong allocation. The top three rows of that table are where an error costs you something; the bottom four are where it can cost you something you did not authorise.

21.2 Data: what goes where

SURFACE	WHAT TO KNOW
Project knowledge	Readable by everyone the project is shared with, and flows into every chat in it. Organisation-wide visibility means every colleague can read it. **Never upload credentials.**
Instructions and `CLAUDE.md`	Load into every session; committed files reach every clone and every fork. Never put secrets in one.
Remote connectors	Claude connects from Anthropic's cloud infrastructure rather than your device. Factor that into any data-residency assessment.
Cloud environments (routines, cloud sessions)	Environment variables are **visible to anyone who uses the environment**. Use a dedicated environment for anything sensitive.
Published artifacts	Have a link, and links get forwarded. An artifact built from connector data *contains* that data. Check the audience, and check the page before sharing outward.
Auto memory	Machine-local, per repository. Not shared across machines or into cloud sessions. Auditable and editable as plain markdown via `/memory`.
Routine identity	Anything a routine does through your GitHub identity or connectors appears **as you** — commits, PRs, Slack messages, tickets.
Code and training	Anthropic's documented position for Claude Code is that your code is processed to provide assistance and is not used to train models; see the data-and-privacy link in Section 21.7.

The four things never to put anywhere in this system

1. **Credentials** — in project knowledge, an instructions file, a skill, a committed `.mcp.json`, a system prompt, or a routine's prompt. Use OAuth, or a secret store, or keep the credential host-side.
2. **Customer PII you do not need** in a knowledge base that is shared more widely than the data classification allows.
3. **Anything you would not want in a link** in an artifact you might publish by reflex.
4. **Secrets in an environment variable on a shared cloud environment**, which is visible to everyone who uses it.

21.3 Prompt injection, in plain terms

Once Claude can read from external systems, some of what it reads was written by someone else. A work-item description, a PR comment, a web page, an MCP server's tool description — all of it is text that reaches the model.

This is why several behaviours in this guide exist, and knowing the reason makes them easier to respect:

THE DESIGN	WHAT IT PROTECTS AGAINST
A routine's `text` payload arrives wrapped in a `<routine-fire-payload>` block labelled untrusted, and your prompt must explicitly opt in to acting on it	Anyone holding the bearer token can send `text`. The wrapper means a leaked token delivers *data*, not *instructions*.
A fired routine prompt cannot serve as approval or consent for actions during the run	The trigger attests only that the prompt was stored earlier by an authorised session. It is not live user input.
The `ultracode` keyword only opts in from a prompt *you type* — never from `-p`, a scheduled prompt, a webhook payload, or a relayed PR comment	Otherwise a comment on a public pull request could launch a large multi-agent run.

THE DESIGN	WHAT IT PROTECTS AGAINST
External imports in a project memory file trigger a one-time approval dialog	Files other people commit to a shared project could otherwise silently pull in instructions from elsewhere.
Project `.mcp.json` servers require interactive approval, and a cloned repo cannot approve its own	A repository could otherwise ship a server configuration that connects automatically.
Repository skills load at session start with no review step	This one is a *warning*, not a protection. Anyone who can commit to a mounted repository can add instructions Claude will follow — in a session with `bash` and web access.

THE PRACTICAL RULES THAT FOLLOW

- **Only mount repositories you trust**, and audit `.claude/skills/` before mounting one with external contributors.
- **Review skill, agent, rule and hook changes like code** — they are instructions, in a session with real capability.
- **Do not build around the guardrails.** If you find yourself wanting the `ultracode` keyword to fire from a webhook, invoke a saved workflow command instead — the restriction is deliberate.
- **Be sceptical of unvetted MCP servers.** Their tool descriptions and results are text the model reads, which is a genuine reason to care about provenance.

21.4 Administrative controls that exist

What an Owner or administrator can actually configure. Use these rather than a policy document where a technical control exists — a setting is enforced; a wiki page is not.

Organisation settings (Team and Enterprise, claude.ai)

CONTROL	EFFECT
Capabilities › **Code execution and file creation**	Gates **Skills** organisation-wide, including the built-in Excel, Word, PowerPoint and PDF skills. On Enterprise, Skills additionally needs its own toggle under Organization settings › Skills.
Capabilities › **Visuals**	Artifacts have their own switches, separate from code execution: **Artifacts**, **AI-powered artifacts** and **inline visualizations**. You can allow plain artifacts while withholding AI-powered ones. See Chapter 10, Section 10.3.
Capabilities › **Preferences**	Memory-related switches, including **Search and reference chats** and **Generate memory from chat history**. Relevant to any data-handling review, since both change what persists between conversations.
Visual / interactive content	Maps and image-search results are **on by default on Team** and **off by default on Enterprise**. When on, generic (not verbatim) search requests go to Google Maps and Bing Image Search; prompts are not sent, but IP addresses are.
Connectors	Choose which connectors are available to the organisation. Members still authenticate individually.
Connector restrictions (Enterprise)	Restrict to verified-domain connectors, and limit action permissions org-wide: read-only, write, or delete.
Organisation-provisioned skills	Distribute skills to every member's list, enabled or disabled by default.
Skill sharing (Team / Enterprise)	**Off by default.** Two separate toggles under Organization settings › Skills: **Share with organization** (publish to the org directory) and **Skill sharing** (share with named colleagues). Until one is on, no Share button exists.
Organisation plugin marketplaces	Organization settings › Plugins. Distribute a curated set; plugins can be *available*, *auto-installed* (members may uninstall) or *required* (they may not). Requires Cowork and Skills enabled first.
Skill and plugin scanning (Enterprise, beta)	Organization settings › Skills › **Skill and plugin security scanning**. **Off by default.** See the detail below — the exclusions matter.
Project visibility	Private vs organisation-wide, with **Can view** / **Can edit** per member.

CONTROL	EFFECT
Routines toggle	At `claude.ai/admin-settings/claude-code`: disable routines for all members. Existing routines stop running.
Usage credits	At `claude.ai/admin-settings/usage`: whether the organisation may continue on metered overage past its limits.

Skill and plugin scanning — what it does and does not cover

Worth a paragraph of its own, because it is the one control that inspects *content* rather than restricting access, and because its exclusions are easy to misread as coverage. When a member uploads or edits a third-party skill or plugin, Claude reviews the contents and returns **pass**, **warn** or **fail**. Failures are blocked; warnings show a caution banner. Scans usually take one to two minutes and results are cached, so re-uploading the same file returns almost instantly. There is no extra cost.

SCANNED	NOT SCANNED
Third-party skills a member uploads or installs; plugins; and the skills bundled *inside* a plugin.	Skills and plugins **already present before you turned scanning on** — nothing you currently use stops working, but nothing already installed is retro-checked either. Skills you create *with* Claude. Skills exposed through a connected MCP server (same trust model as any MCP tool). **MCP servers and hooks are not scanned at all.** Organisations using CMEK, zero data retention or HIPAA configurations install without the scan.

Practical reading for a rollout: turn it on *before* you invite the whole organisation to start uploading skills, not after. And note that "we have scanning on" is not an answer to MCP server risk — that is governed by the connector controls above and by Chapter 8.

Managed settings (Claude Code, deployed by IT)

CONTROL	EFFECT
Managed policy `CLAUDE.md`	Organisation-wide behavioural guidance that individuals **cannot exclude**. Deployed via MDM, Group Policy or Ansible to the platform-specific path (Chapter 6, Section 6.3), or inline via the `claudeMd` key.
`permissions.deny`	**Blocks** specific tools, commands or file paths. This is enforcement, not persuasion — the action cannot happen.
`sandbox.enabled`	Enforce sandbox isolation.
`Agent(name)` deny rules	Prevent Claude using a specific subagent at all.
Managed marketplace restrictions, `strictKnownMarketplaces`	Restrict which plugin marketplaces users may add.
Force-enabled / force-disabled plugins	Cannot be overridden locally, including with `--plugin-dir`.
`disableWorkflows`	Turn dynamic workflows off organisation-wide.
`availableModels`	Restrict which models may be selected. Workflow agents requesting a blocked model get a substitution, named in the progress view.
`forceLoginMethod`, `forceLoginOrgUUID`	Pin the authentication method and organisation.
`enabledPlugins`, `extraKnownMarketplaces`	Declare the standard toolkit so members are prompted to install it.

SETTINGS FOR ENFORCEMENT, CLAUDE.MD FOR GUIDANCE

Anthropic's own framing, and it is the right division: use **managed settings** to block tools, enforce sandboxing, pin authentication and restrict models — those are enforced by the client regardless of what Claude decides. Use a **managed** `CLAUDE.md` for code-style guidance, data-handling reminders and behavioural instructions — which shape behaviour but are not a hard enforcement layer.

21.5 The four questions to answer before a organisation-wide rollout

Not a long checklist. Four decisions that, left unmade, cause the problems.

#	QUESTION	WHY IT MATTERS, AND A REASONABLE DEFAULT
1	**Who may connect Claude to which systems, with what access?**	This is the question that matters most, because it determines what Claude can change in your real systems. *Default:* Owners curate the organisation connector list; individuals may not add custom connectors to production systems; authenticate with the least-privileged account that does the job; start read-only where the system supports scoped credentials.
2	**Who may install plugins, and from where?**	Plugins execute arbitrary code with the user's privileges. *Default:* the official Anthropic marketplace plus one internal marketplace in a private repository. Restrict the rest with managed marketplace restrictions rather than a policy note.
3	**Are routines allowed, and with what output?**	Unattended, cloud, your identity, no approval prompts. *Default:* allowed, with three conventions — trim the connector list to what the routine needs, produce *draft* PRs and messages rather than direct writes, and read the first three runs of every new routine in full.
4	**What is the artifact sharing default?**	Publishing creates a forwardable link, and artifacts built from connector data contain that data. *Default:* organisation-scoped, not public, for anything internal. Remember unpublishing is one-way.

Rollout sequence for an enablement owner

1. **Decide the four questions above**
   Write the answers down in one short page. Longer than a page and nobody reads it.
2. **Configure what can be configured**
   Organisation capabilities, the connector list, Enterprise action permissions, and any managed settings for Claude Code. Prefer a setting to a sentence.
3. **Publish the approved list and the catalogue**
   Which connectors are approved and how to connect them; which skills, agents and plugins exist and what they give you; who owns each. This single page drives more adoption than any tooling.
4. **Pilot with one team**
   Follow the adoption sequence in Chapter 20, Section 20.2. Pin the wider audience to a marketplace tag while the pilot tracks the tip.
5. **Establish the review norm**
   Skills, agents, rules, hooks and workflows are reviewed in pull requests, because they change what code gets written and what actions get taken.
6. **Re-audit each quarter, and after model releases**
   Prune stale project knowledge, disable unused plugins and connectors, remove routines that operate on renamed resources, and A/B skills with older scaffolding removed.

21.6 Cost and usage governance

Not a security matter, but the same shape of problem: a small configuration decision with a compounding effect.

WHAT MULTIPLIES USAGE	THE CONTROL
Several subagents or sessions at once	Delegate deliberately; route mechanical work to a cheaper model with the `model` field.

WHAT MULTIPLIES USAGE	THE CONTROL
Workflows	Set an organisational **size guideline** (`medium` is the default); scope on a slice first; watch the **Large workflow** warning at 25 agents or 1.5M projected tokens.
Ultracode left on	It applies to *every* task in the session. `/effort high` when you return to routine work.
Routines	Daily caps apply per account — 5 on Pro, 15 on Max, 25 on Team/Enterprise. An hourly routine consumes most of that.
Long instructions and many connected tools	Paid on *every* message or request. See Chapter 20, Section 20.3.
Unused plugins	Context cost every turn. Check the **Not used recently** group.

The diagnostic: `/usage` attributes recent usage by skill, subagent, plugin and MCP server, and flags behaviours accounting for 10% or more — cache misses, long context, subagent-heavy or highly parallel sessions. If costs surprise you, start there rather than guessing.

21.7 Official resources

Get started with skill and plugin scanning — what is scanned, what is not, and how to turn it on
`https://support.claude.com/en/articles/15927065-get-started-with-skill-and-plugin-scanning`

Provision and manage skills for your organization — org provisioning and the two sharing toggles
`https://support.claude.com/en/articles/13119606-provision-and-manage-skills-for-your-organization`

Use visual and interactive content on Team and Enterprise plans — the plan defaults and third-party data flows
`https://support.claude.com/en/articles/13663666-use-visual-and-interactive-content-on-team-and-enterprise-plans`

Claude Code security — the security model and safeguards
`https://code.claude.com/docs/en/security`

Data and privacy — how data is handled, and how to opt out of logging
`https://code.claude.com/docs/en/data-usage`

Permissions — allow and deny rules, additional directories, managed settings
`https://code.claude.com/docs/en/permissions`

Permission modes — what each mode approves automatically
`https://code.claude.com/docs/en/permission-modes`

Settings — every setting key, configuration scopes and precedence
`https://code.claude.com/docs/en/settings`

Server-managed settings — organisation-wide policy deployment
`https://code.claude.com/docs/en/server-managed-settings`

Sandboxing — isolating what Claude Code can reach
`https://code.claude.com/docs/en/sandboxing`

Hooks — the enforcement mechanism, and what installing one grants
`https://code.claude.com/docs/en/hooks`

Cloud environments — network access levels and environment-variable visibility
`https://code.claude.com/docs/en/cloud-environments`

Manage costs — tracking and reducing usage
`https://code.claude.com/docs/en/costs`

Use Claude Cowork safely
`https://support.claude.com/en/articles/13364135-use-claude-cowork-safely`

Use Claude Cowork on Team and Enterprise plans
`https://support.claude.com/en/articles/13455879-use-claude-cowork-on-team-and-enterprise-plans`

CHAPTER 22

Official Documentation & Resources

Every link in this guide, in one place. Official Anthropic sources only.

ABOUT THESE LINKS

Every URL in this guide points at documentation published by Anthropic. Each was checked before publication and resolved successfully. Nothing here is invented or inferred.

Three caveats. First, links under `claude.ai` are *application* URLs requiring sign-in, not documentation. Second, documentation moves — if a link fails, start from the relevant index in Section 22.1. Third, several capabilities in this guide are labelled *research preview* or *beta* by Anthropic; the official page is authoritative about their current state, and this guide is a snapshot dated on the cover.

22.1 Start here — the four indexes

INDEX	URL	COVERS
Claude Code documentation	`code.claude.com/docs/en/overview`	Everything Claude Code: skills, MCP, plugins, agents, workflows, routines, settings, hooks, IDE integrations.
Claude Help Centre	`support.claude.com/en/`	claude.ai and Cowork: projects, artifacts, connectors, skills, personalisation, plans.
Claude Developer Platform	`platform.claude.com/docs`	The API, models, Agent Skills, MCP connector, Managed Agents. For engineers building applications.
Claude Code docs, machine-readable	`code.claude.com/docs/llms.txt`	The full page index. Useful for finding a page whose URL has changed.

A TRICK WORTH KNOWING

Most Claude Code documentation pages have a plain-markdown twin: append `.md` to the URL. So `code.claude.com/docs/en/skills.md` returns the raw page. That is handy for reading offline, diffing against this guide when something changes, or feeding a page to Claude.

22.2 By building block

Chapter 5 — Projects

What are projects?
`support.claude.com/en/articles/9517075-what-are-projects`

How can I create and manage projects? — setup, sharing, archiving, deleting
`support.claude.com/en/articles/9519177-how-can-i-create-and-manage-projects`

Organize your tasks with projects in Claude Cowork
`support.claude.com/en/articles/14116274-organize-your-tasks-with-projects-in-claude-cowork`

Collaborate with Claude on Projects — announcement
`www.anthropic.com/news/projects`

Chapter 6 — Instructions

Get started with Claude Cowork — global and folder instructions
`support.claude.com/en/articles/13345190-get-started-with-claude-cowork`

Understanding Claude's personalization features
support.claude.com/en/articles/10185728-understanding-claude-s-personalization-features

How Claude remembers your project — `CLAUDE.md`, rules, auto memory
code.claude.com/docs/en/memory

Settings reference
code.claude.com/docs/en/settings

Hooks — the enforcement escalation
code.claude.com/docs/en/hooks-guide

Chapter 7 — Skills

What are skills?
support.claude.com/en/articles/12512176-what-are-skills

How to create custom skills
support.claude.com/en/articles/12512198-how-to-create-custom-skills

Use skills in Claude — per-plan prerequisites, upload path, sharing
support.claude.com/en/articles/12512180-use-skills-in-claude

Browse skills, connectors, and plugins in one directory
support.claude.com/en/articles/14328846-browse-skills-connectors-and-plugins-in-one-directory

Provision and manage skills for your organization
support.claude.com/en/articles/13119606-provision-and-manage-skills-for-your-organization

Get started with skill and plugin scanning — Enterprise, beta
support.claude.com/en/articles/15927065-get-started-with-skill-and-plugin-scanning

Extend Claude with skills — the complete Claude Code reference
code.claude.com/docs/en/skills

Agent Skills on the Developer Platform
platform.claude.com/docs/en/agents-and-tools/agent-skills

Agent Skills open standard
agentskills.io

anthropics/skills — examples and the packaging script
github.com/anthropics/skills

Commands reference — the bundled skills
code.claude.com/docs/en/commands

Chapter 8 — MCP Servers

Connect Claude Code to tools via MCP — the full reference
code.claude.com/docs/en/mcp

MCP quickstart
code.claude.com/docs/en/mcp-quickstart

Get started with custom connectors using remote MCP
support.claude.com/en/articles/11175166-get-started-with-custom-connectors-using-remote-mcp

Build custom connectors via remote MCP servers
support.claude.com/en/articles/11503834-build-custom-connectors-via-remote-mcp-servers

Model Context Protocol — the open specification
modelcontextprotocol.io/docs/getting-started/intro

MCP connector on the Developer Platform
platform.claude.com/docs/en/agents-and-tools/mcp-connector

Chapter 9 — Connectors

Use connectors to extend Claude's capabilities
support.claude.com/en/articles/11176164-use-connectors-to-extend-claude-s-capabilities

When to use desktop and web connectors

support.claude.com/en/articles/11725091-when-to-use-desktop-and-web-connectors

Anthropic Connectors Directory FAQ
support.claude.com/en/articles/11596036-anthropic-connectors-directory-faq

Connectors — full help-centre collection
support.claude.com/en/collections/15399129-connectors

Chapter 10 — Artifacts

What are artifacts and how do I use them?
support.claude.com/en/articles/9487310-what-are-artifacts-and-how-do-i-use-them

Publish and share artifacts
support.claude.com/en/articles/9547008-publish-and-share-artifacts

Share session output as artifacts — Claude Code
code.claude.com/docs/en/artifacts

Use artifacts to visualize and create AI apps without writing code
support.claude.com/en/articles/11649427-use-artifacts-to-visualize-and-create-ai-apps-without-ever-writing-a-line-of-code

Prototype AI-powered apps with Claude artifacts
support.claude.com/en/articles/11649438-prototype-ai-powered-apps-with-claude-artifacts

Use live artifacts in Claude Cowork
support.claude.com/en/articles/14729249-use-live-artifacts-in-claude-cowork

Custom visuals in chat and Cowork — inline visualisations
support.claude.com/en/articles/13979539-custom-visuals-in-chat-and-cowork

Visual and interactive content — data widgets, custom visuals, interactive inputs
support.claude.com/en/articles/13641943-visual-and-interactive-content

Use visual and interactive content on Team and Enterprise plans
support.claude.com/en/articles/13663666-use-visual-and-interactive-content-on-team-and-enterprise-plans

Claude Code now supports artifacts — announcement
claude.com/blog/artifacts-in-claude-code

Chapters 11–12 — Marketplace and Plugins

Discover and install prebuilt plugins through marketplaces
code.claude.com/docs/en/discover-plugins

Use plugins in Claude — the claude.ai / Desktop / Cowork side
support.claude.com/en/articles/13837440-use-plugins-in-claude

Manage plugins for your organization — org marketplaces for Owners
support.claude.com/en/articles/13837433-manage-plugins-for-your-organization

Create plugins
code.claude.com/docs/en/plugins

Create and distribute a plugin marketplace
code.claude.com/docs/en/plugin-marketplaces

Plugins reference — complete manifest schema and component specs
code.claude.com/docs/en/plugins-reference

The official plugin catalogue, in a browser
claude.com/plugins

anthropics/claude-plugins-community
github.com/anthropics/claude-plugins-community

anthropics/claude-code — demo marketplace and example plugins
github.com/anthropics/claude-code

Chapter 13 — Routines

Schedule recurring tasks in Claude Cowork — the chat-side equivalent

support.claude.com/en/articles/13854387-schedule-recurring-tasks-in-claude-cowork

Automate work with routines — the complete reference
code.claude.com/docs/en/routines

Trigger a routine via API
platform.claude.com/docs/en/api/claude-code/routines-fire

Cloud environments
code.claude.com/docs/en/cloud-environments

Claude Code on the web
code.claude.com/docs/en/claude-code-on-the-web

Desktop scheduled tasks — the local alternative
code.claude.com/docs/en/desktop-scheduled-tasks

`/loop` and in-session scheduling
code.claude.com/docs/en/scheduled-tasks

GitHub Actions
code.claude.com/docs/en/github-actions

Schedule recurring tasks in Claude Cowork
support.claude.com/en/articles/13854387-schedule-recurring-tasks-in-claude-cowork

Introducing routines in Claude Code — announcement
claude.com/blog/introducing-routines-in-claude-code

Chapter 14 — Agents

Create custom subagents — the complete reference
code.claude.com/docs/en/sub-agents

Run agents in parallel — subagents vs agent view vs teams vs workflows
code.claude.com/docs/en/agents

Agent view (research preview)
code.claude.com/docs/en/agent-view

Agent teams (experimental, disabled by default)
code.claude.com/docs/en/agent-teams

Managed Agents on the Developer Platform
platform.claude.com/docs/en/managed-agents/overview

Chapter 15 — Workflows

Orchestrate subagents at scale with dynamic workflows
code.claude.com/docs/en/workflows

Introducing dynamic workflows — announcement
claude.com/blog/introducing-dynamic-workflows-in-claude-code

A harness for every task: dynamic workflows in Claude Code
claude.com/blog/a-harness-for-every-task-dynamic-workflows-in-claude-code

22.3 Getting set up

Claude Code quickstart
code.claude.com/docs/en/quickstart

Claude Code setup and installation
code.claude.com/docs/en/setup

Use Claude Code in VS Code — installation, shortcuts, settings, troubleshooting
code.claude.com/docs/en/vs-code

Claude Code VS Code extension on the Marketplace
marketplace.visualstudio.com/items?itemName=Anthropic.claude-code

The extension on Open VSX (for VS Code forks)
`open-vsx.org/extension/Anthropic/claude-code`

Claude Desktop app
`code.claude.com/docs/en/desktop`

Claude Cowork — product page
`claude.com/product/cowork`

Get started with Claude Cowork
`support.claude.com/en/articles/13345190-get-started-with-claude-cowork`

Use Claude Cowork on web, desktop, and mobile
`support.claude.com/en/articles/15520349-use-claude-cowork-on-web-desktop-and-mobile`

Claude Code — product page
`claude.com/product/claude-code`

22.4 Reference and operations

All commands and bundled skills
`code.claude.com/docs/en/commands`

Settings — every key, scope and precedence
`code.claude.com/docs/en/settings`

Permissions
`code.claude.com/docs/en/permissions`

Permission modes
`code.claude.com/docs/en/permission-modes`

Model configuration and effort levels
`code.claude.com/docs/en/model-config`

Checkpointing — rewind Claude's file edits
`code.claude.com/docs/en/checkpointing`

Manage costs
`code.claude.com/docs/en/costs`

Troubleshooting
`code.claude.com/docs/en/troubleshooting`

Security
`code.claude.com/docs/en/security`

Sandboxing
`code.claude.com/docs/en/sandboxing`

Data and privacy
`code.claude.com/docs/en/data-usage`

Server-managed settings
`code.claude.com/docs/en/server-managed-settings`

Hooks reference
`code.claude.com/docs/en/hooks`

Models overview — IDs, context windows, capabilities
`platform.claude.com/docs/en/about-claude/models/overview`

22.5 Application URLs (sign-in required)

Not documentation — the places in the product this guide sends you.

URL	WHAT IT IS
`claude.ai/projects`	Your projects (Chapter 5)

URL	WHAT IT IS
`claude.ai/customize/connectors`	Manage connectors (Chapter 9)
`claude.ai/directory`	The reviewed connectors directory
`claude.ai/code`	Claude Code on the web
`claude.ai/code/routines`	Create and manage routines (Chapter 13)
`claude.ai/settings/usage`	Your usage and remaining routine runs
`claude.ai/admin-settings/claude-code`	Owner controls: the Routines toggle, workflow settings
`claude.ai/admin-settings/usage`	Owner control: usage credits
`github.com/anthropics/claude-code/issues`	Report a Claude Code bug

22.6 Source note for this guide

So the next person to update this document knows what it was built from.

CONTENT	SOURCE
All product behaviour: click paths, setting names, commands, file formats, frontmatter fields, limits, plan availability, error messages, version notes	The Anthropic documentation linked in Sections 22.1–22.4, read at the time of writing. Claude Code content comes from the documentation pages themselves; claude.ai content was checked against the raw help-centre articles rather than a summary, which is the only reliable way to get an exact UI label.
The photographic screenshots	Anthropic's own published documentation and help centre — no third-party or reconstructed captures. Each figure caption names its source article.
All diagrams	Prepared for this guide and labelled *Diagram — not a screenshot*. Where a diagram reproduces documented structure, the caption says so.
Model names and context windows	Anthropic's model documentation.
Example identifiers — `acme-internal`, `acme-plugins`, `qa-toolkit`, endpoint placeholders	Invented for illustration. They are deliberately obvious placeholders; substitute your own before running anything.

THREE THINGS TO KNOW BEFORE YOU RELY ON IT

- **Plan availability moves.** Where this guide states a plan requirement or a limit, it reflects the linked official article at the time of writing. Confirm before designing a process around it — particularly for anything Anthropic labels research preview or beta: Routines, Agent view, Agent teams, custom visuals, skill and plugin scanning.
- **The product moves faster than its documentation.** Where the two disagreed, this guide describes what is in the product and says so explicitly. Section 10.3 is the clearest example: it names both the current **Visuals** layout and the older single-switch one, because accounts differ.
- **Settings get reorganised more often than they get renamed.** If a path here does not match your screen, the switch has almost certainly moved rather than disappeared — check the same tab under a different section heading, and on Team or Enterprise check whether an Owner has enabled it at organisation level first.

CHAPTER 23

Quick Reference Cheat Sheet

The last four pages. Print them, pin them, ignore the rest of the book.

23.1 The eleven blocks in one line each

BLOCK	CH	WHAT IT IS	REACH FOR IT WHEN
Projects	5	A workspace holding shared context for many chats	You keep re-pasting the same background
Instructions	6	Standing rules applied to every message	You keep typing the same correction
Skills	7	A folder + `SKILL.md`: a procedure, loaded on demand	You produce a format more than once a month
MCP Servers	8	The open standard for connecting AI to any tool	The system is internal, legacy or custom
Connectors	9	Ready-made, one-click MCP integrations	The tool is already in the directory
Artifacts	10	A live output you open, use, version and share	The output is a deliverable, not an answer
Marketplace	11	A catalogue of plugins	You need to install or distribute plugins
Plugins	12	A bundle of skills, agents, MCP servers, hooks, workflows	You want your setup to become the team's
Routines	13	A saved configuration run by a trigger, in the cloud	It should happen while you are asleep
Agents	14	A delegated worker with its own context and tools	A side task would flood your conversation
Workflows	15	A script Claude writes, orchestrating many agents	The same step must run across hundreds of items

23.2 Where things live on disk (Claude Code)

Project — committed, shared with the team

```
your-repo/
├── CLAUDE.md                       project instructions (or .claude/CLAUDE.md)
├── CLAUDE.local.md                 your private notes — gitignore this
├── .mcp.json                       project-scoped MCP servers
└── .claude/
    ├── settings.json               shared settings, enabledPlugins, marketplaces
    ├── settings.local.json         your overrides — gitignore this
    ├── rules/*.md                  topic rules, optionally paths:-scoped
    ├── skills/<name>/SKILL.md      project skills          → /name
    ├── agents/<name>.md            project subagents       → @agent-name
    └── workflows/<name>.js         saved workflows         → /name
```

Personal — all your projects, just you

```
~/.claude/
├── CLAUDE.md                       your preferences everywhere
├── settings.json                   your settings
├── rules/*.md                      your personal rules
├── skills/<name>/SKILL.md          personal skills
├── agents/<name>.md                personal subagents
├── workflows/<name>.js             personal workflows
├── plugins/cache/                  delete this if plugin skills vanish
└── projects/<project>/memory/      auto memory — MEMORY.md + topic files
```

Plugin — the root is the plugin's own directory, never ~/.claude/

```
my-plugin/
├── .claude-plugin/plugin.json      ← ONLY plugin.json goes in here
├── skills/<name>/SKILL.md          → /my-plugin:name
├── agents/<name>.md                → @agent-my-plugin:name
├── workflows/<name>.js             → /my-plugin:name
├── hooks/hooks.json                event handlers (shell commands)
├── .mcp.json                       MCP servers
├── .lsp.json                       language servers
├── monitors/monitors.json          background monitors
├── bin/                            executables added to Bash PATH
└── settings.json                   default settings (agent, subagentStatusLine)
```

Managed policy — organisation-wide, deployed by IT, cannot be excluded

```
macOS    /Library/Application Support/ClaudeCode/CLAUDE.md
Linux    /etc/claude-code/CLAUDE.md
Windows  C:\Program Files\ClaudeCode\CLAUDE.md
```

23.3 Commands you will actually use

Diagnostics — start here

```
> /context      what loaded this session
> /doctor       setup checkup
> /usage        plan + attribution
> /tasks        background work
$ claude --version
$ claude --debug
```

Instructions & memory

```
> /init         generate CLAUDE.md
> /memory       open any memory file
> /import       bring in another agent's config
```

Skills

```
> /             list invocable skills
> /skill-name
> /apps/web:deploy  nested variant
```

MCP

```
$ claude mcp add -t http <name> <url>
$ claude mcp add -t sse <name> <url>
$ claude mcp add <name> -- <cmd>
$ claude mcp list       health status
$ claude mcp get <name>
$ claude mcp remove <name>
$ claude mcp reset-project-choices
> /mcp                  auth + enable/disable
```

Plugins & marketplaces

```
> /plugin               the manager
> /plugin marketplace add <src>
> /plugin marketplace list
> /plugin marketplace update <m>
> /plugin marketplace remove <m>
> /plugin install <p>@<m>
> /plugin list [--enabled]
> /plugin disable <p>@<m>
> /plugin uninstall <p>@<m>
> /reload-plugins [--force]
$ claude plugin init <name>
$ claude plugin validate ./p
$ claude plugin details <p>@<m>
$ claude --plugin-dir ./my-plugin
```

Agents

```
> @agent-code-reviewer  guaranteed
> @agent-plugin:name    from a plugin
> /subtask              fork with context
$ claude agents         agent view
$ claude --worktree <name>
```

Workflows

```
> ultracode: <task>
> use a workflow to <task>
> /deep-research <question>
> /workflows            watch · s to save
> /effort ultracode
> /config workflowSizeGuideline=small
```

Routines

```
> /schedule              create
> /schedule list | update | run
> /web-setup             GitHub access
```

23.4 Click paths (claude.ai and Desktop)

New project	`claude.ai/projects` › **+ New Project**
Project knowledge	Open the project › **+** on the right
Project instructions	Open the project › **Set project instructions** › **Save instructions**
Archive / delete a project	Open the project › **…** (upper right). *Unarchive before you can delete.*
Move a chat into a project	Chat title dropdown › **Add to project**
Account instructions	Your initials (lower left) › **Settings** › **Instructions for Claude**
Skills	**Customize** › **Skills** › **+** › **Browse skills**
Connectors	**Customize** › **Connectors** › **+**
Enable a connector in a chat	**+** (lower left) › hover **Connectors** › toggle
Plugins	**Customize** › **Plugins** › **Browse plugins** · add a marketplace: **+** › **Add marketplace**
Enable *artifacts*	**Settings** › **Capabilities** › **Visuals** › **Artifacts / AI-powered artifacts / inline visualizations**
Enable *skills*	**Settings** › **Capabilities** › **Code execution and file creation** — a *different* switch from artifacts
Routines	`claude.ai/code/routines`, or Desktop sidebar › **Routines** › **New routine** › **Cloud**
Org capabilities / connectors	**Organization settings** › **Capabilities / Connectors / Skills / Plugins**

23.5 VS Code extension essentials

Install	Extensions view › search **Claude Code** › **Install**. Needs VS Code 1.94+.
Open it	The Spark icon in the editor toolbar, the Activity Bar, or **✱ Claude Code** in the status bar
Focus input	Ctrl+Esc / Cmd+Esc
New tab	Ctrl+Shift+Esc
@-mention selection	Alt+K / Option+K
Focus view	Ctrl+Alt+F
Plugins UI	Type `/plugins` in the prompt box
Reference a terminal	`@terminal:name`

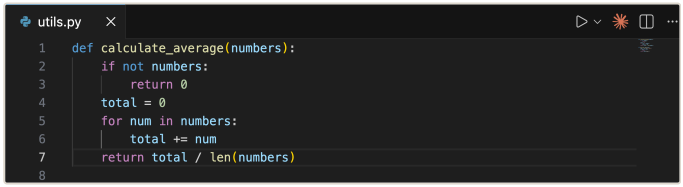


Figure 23.1 The Spark icon in the **Editor Toolbar** — the quickest way to open Claude. It appears only when a file is open.

Screenshot: Anthropic, Claude Code documentation — "Use Claude Code in VS Code."

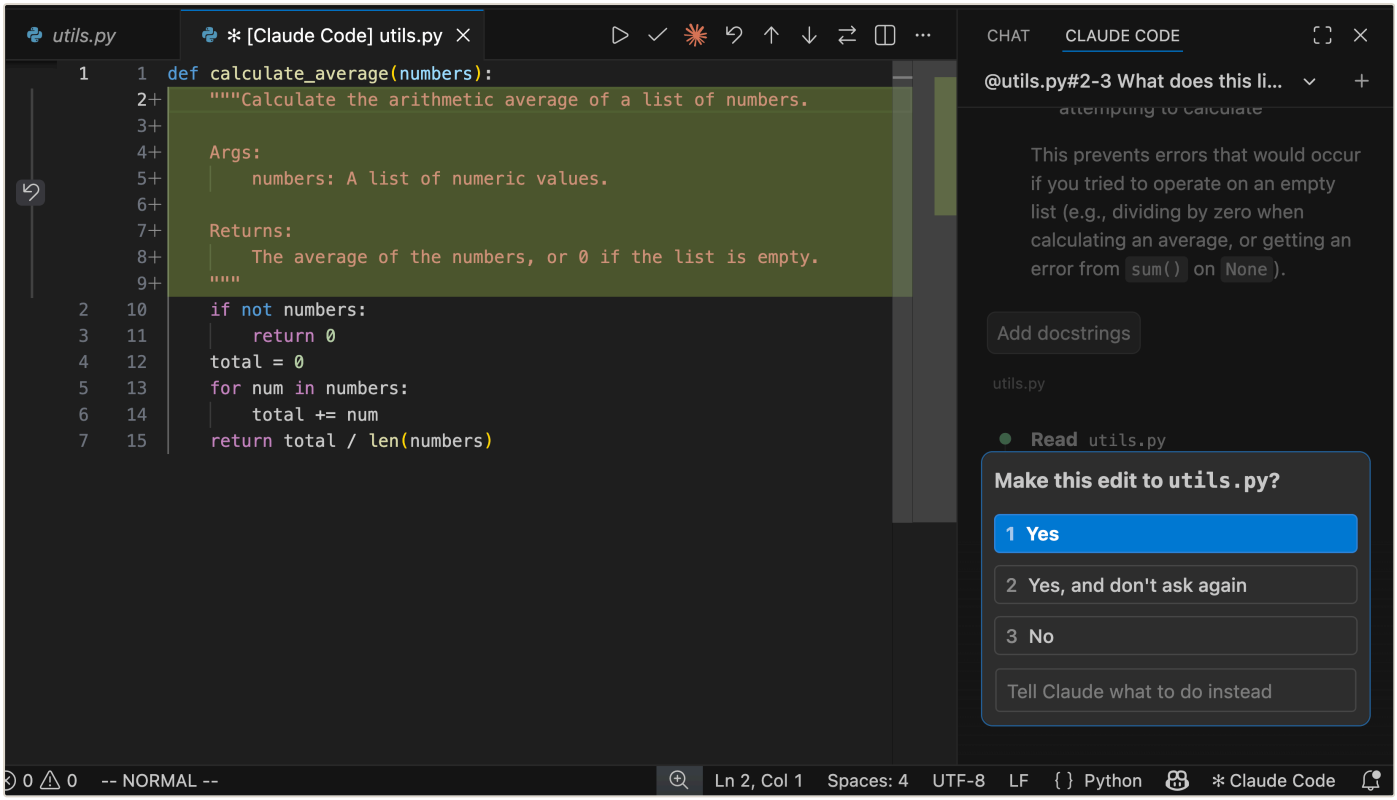


Figure 23.2 Before editing a file, Claude shows a side-by-side diff and asks permission. You can accept, reject, or edit the proposed content directly in the diff before accepting — Claude is told you modified it, so it does not assume the file matches its original proposal.

Screenshot: Anthropic, Claude Code documentation — "Use Claude Code in VS Code."

23.6 Frontmatter quick reference

`SKILL.md — the fields you will actually use`

```
---
description: What it does. Use when <trigger>.   # the field that matters most
when_to_use: extra trigger phrases
argument-hint: "[issue-number]"
disable-model-invocation: true      # manual only — use for destructive skills
user-invocable: false               # hide from the / menu
allowed-tools: Read, Grep           # pre-approved for the invoking turn
disallowed-tools: AskUserQuestion   # removed while active
paths: ["src/api/**/*.ts"]          # only auto-activate for matching files
model: sonnet                       # or opus, haiku, fable, inherit
effort: high                        # low | medium | high | xhigh | max
context: fork                       # run in a forked subagent
---
```

Uploading to claude.ai? Only six fields are allowed: `name`, `description`, `license`, `compatibility`, `metadata`, `allowed-tools`. Anything else is a *hard upload failure*.

`agents/<name>.md — subagent`

```
---
name: code-reviewer                 # required · lowercase-hyphens · no colons
description: When Claude should delegate to this.   # required
tools: Read, Grep, Glob             # OMITTING THIS GRANTS EVERYTHING
model: sonnet                       # or opus, haiku, fable, inherit
skills: test-case                   # injected at startup
mcpServers: ado                     # servers for this agent
memory: project                     # user | project | local
```

```
color: blue
---
The system prompt goes here.
```

`.claude/rules/<topic>.md — path-scoped rule`

```
---
paths:
  - "src/api/**/*.ts"
  - "lib/**/*.{ts,tsx}"
---
```

23.7 Limits and numbers worth remembering

THING	NUMBER	NOTE
Projects on Free	5 max	Projects themselves are on all plans
Custom connectors on Free	1	Directory connectors are not limited this way
`CLAUDE.md` target size	< 200 lines	Longer files get followed *less*
Auto memory index loaded	200 lines / 25 KB	First limit hit wins; only `MEMORY.md`
Skill description in the listing	1,536 chars	Truncated — put the key use case first
Artifact storage	20 MB each	Input is text-only
Artifact creation threshold	~15 lines	Roughly when Claude decides on its own
Routine runs per day	5 / 15 / 25	Pro / Max / Team & Enterprise. One-off runs do not count.
Routine minimum interval	1 hour	More frequent cron expressions are rejected
Workflow concurrent agents	16 max	Fewer on limited CPU cores
Workflow total agents	1,000 per run	Runaway-loop backstop
Workflow size guidelines	<5 / <15 / <50	`small` / `medium` (default) / `large`
Large-workflow warning	25 agents · 1.5M tokens	Advisory — does not pause the run
MCP tool output offload	~100,000 chars	Written to a file; Claude gets a preview + path
Context windows	200K / 1M	Haiku / Sonnet, Opus, Fable

23.8 The six traps

TRAP	REMEMBER
Chats in a project share memory	**They do not.** Shared *context*, never *conclusions*. Write decisions into project knowledge.
Local skills and MCP servers work in routines	**They do not.** Cloud sessions read *account* skills and connectors, or repo-committed ones.
A green routine run means it worked	**It means no crash.** Open the transcript.
Omitting `tools` is safe	**It grants everything.** List them explicitly.
My permission mode protects workflow agents	**It does not.** They always run in `acceptEdits`; file edits are auto-approved.
Removing a marketplace is tidying up	**It uninstalls its plugins.**

23.9 The five instructions to paste into your settings today

From Chapter 18, Section 18.6. Put these in **Instructions for Claude** (or your `CLAUDE.md`) and you get them on every message, forever.

Copy this

```
Never state a ticket ID, date, name, endpoint or figure that did not come
from material I supplied or a connected tool. Say "not in the provided
material" instead.

Separate what the data shows from what you infer, and label inference
as inference.

Before creating, updating or deleting anything in a connected system,
show me what you intend to do and wait for confirmation.

When reporting findings, report everything you noticed with a confidence
level — do not filter for importance. I will triage.

If you could not complete part of a task, finish the rest and state
plainly what is missing and why. If my request is ambiguous in a way
that changes the answer, ask one clarifying question before starting.
```

AND THE ONE-LINE SUMMARY OF ALL TWENTY-THREE CHAPTERS

You are the mind. AI is the hands.

Give it context (Projects, Instructions), teach it your methods (Skills), connect it to your real systems (Connectors, MCP), let it produce real things (Artifacts), package what works (Plugins, Marketplace), and hand off the repetitive parts (Agents, Workflows, Routines). You own the outcome; AI accelerates the doing.